

// ZMAX //

zmax

v0.4.36

THE COMPANION

modes · keys · the engine underneath

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

Contents

Before You Begin	6
Chapter 1 — The Pitch	8
Chapter 2 — First Contact	11
Chapter 3 — The Selection Engine	13
Chapter 4 — Three Modes	15
Chapter 5 — Operator-Pending, Emulated	17
Chapter 6 — The Vim Grammar	19
Chapter 7 — The g, z, [, and] Prefixes	21
Chapter 8 — Windows and Buffers	23
Chapter 9 — The Emacs Layer	24
Chapter 10 — The Spacemacs Leader	25
Chapter 11 — The Command Line	26
Chapter 12 — Pickers and Fuzzy Finding	28
Chapter 13 — Tree-sitter in Daily Use	30
Chapter 14 — Language Servers	32
Chapter 15 — The Debugger	34
Chapter 16 — Version Control	37
Chapter 17 — Registers, Marks, and the Jumplist	39
Chapter 18 — The Five Tongues	40
Chapter 19 — fusevm: One VM to Run Them All	42
Chapter 20 — elisp in zmax	45
Chapter 21 — The Other Four Tongues	47
Chapter 22 — The Embedding Architecture	49
Chapter 23 — The Rope	53
Chapter 24 — How a Keystroke Becomes an Edit	55

Chapter 25 — The Syntax Layer	57
Chapter 26 — The Event System	59
Chapter 27 — The Main Loop and Rendering	63
Chapter 28 — The Crate Map	65
Chapter 29 — Measuring the Port	67
Chapter 30 — Multiple Cursors in Anger	70
Chapter 31 — A Day with the Language Server	72
Chapter 32 — Text Surgery with AWK	73
Chapter 33 — Text Surgery with stryke	74
Chapter 34 — Text Surgery with Zsh	75
Chapter 35 — Configuring zmax from Scratch	76
Chapter 36 — Bringing Your Config With You	78
Chapter 37 — A Debugging Session	80
Chapter 38 — First Programs in Five Tongues	81
Chapter 39 — Snippets and Completion	83
Chapter 40 — Macros and Repetition	85
Chapter 41 — Working Across Many Files	86
Chapter 42 — Search and Replace at Scale	87
Chapter 43 — Making a Theme	88
Chapter 44 — The Single Binary in Practice	90
Chapter 45 — Trust and Safety	91
Chapter 46 — A Complete elisp Extension	94
Chapter 47 — A Complete Configuration, Annotated	96
Chapter 48 — Porting a Real .vimrc	99
Chapter 49 — A Data-Wrangling Session	101
Chapter 50 — A Complete AWK Report	103
Chapter 51 — A Refactor, Start to Finish	104
Chapter 52 — A New Command in AWK	105
Chapter 53 — A Zsh Build-and-Test Loop	107
Chapter 54 — A Snippet Library	108

Chapter 55 — The Workbench	110
Chapter 56 — Org Mode	112
Chapter 57 — The Transform Library	113
Chapter 58 — The Long Tail	115
Chapter 59 — The JetBrains Layer	117
Chapter 60 — Emacs, In Earnest	122
Chapter 61 — Narrowing and Indirect Buffers	125
Chapter 62 — Paredit and Structural Editing	127
Chapter 63 — Layouts, Workspaces, and Winner Mode	129
Chapter 64 — The Help System and Discoverability	131
Chapter 65 — ediff and the Diff Tools	134
Chapter 66 — Four Traditions, Four Presets, One Engine	136
Chapter 67 — A Refactoring Session	138
Chapter 68 — Rectangle and Column Editing	141
Chapter 69 — Reshaping a Lisp with Paredit	143
Chapter 70 — A Day of Layouts and Narrowing	145
Chapter 71 — Vimscript That Drives the Editor	147
Chapter 72 — Resolving a Merge Conflict	149
Chapter 73 — The Macro Ring and Counters	151
Chapter 74 — A Transform-Library Session	153
Chapter 75 — Binary Files and the Hex Editor	156
Chapter 76 — Navigation at Speed	158
Chapter 77 — The Regex Workbench	160
Chapter 78 — A Data-Wrangling Workday	161
Chapter 79 — Snippets and Live Templates in Practice	163
Chapter 80 — The Start Screen, the Mouse, and Coming Back	165
Chapter 81 — A Multi-Tradition Workday	167
Chapter 82 — The Calendar, and the Calendars Inside It	169
Chapter 83 — The Diary	172
Chapter 84 — Dired	175

Chapter 85 — Comint and the Inferior Process	178
Chapter 86 — Calc	181
Chapter 87 — The Error List: Compilation, Tags, and Occur	184
Chapter 88 — Spelling	187
Chapter 89 — The File Configures Itself	190
Chapter 90 — Bytes: Coding Systems and Line Endings	193
Chapter 91 — Sessions, Snapshots, and Swap Files	196
Chapter 92 — Watchers: the Tree, the Theme, and the Build	199
Chapter 93 — The Property Store	202
Chapter 94 — Faces, the Face Menu, and Enriched Text	205
Chapter 95 — Abbreviations: Two Traditions, Two Modules	208
Chapter 96 — Cross-References Without an Index	211
Chapter 97 — Filtering, Sorting, and Counting Lines	213
Chapter 98 — Addresses in Text	216
Chapter 99 — The Bookkeeping Modes	218
Chapter 100 — The Grid Modes	221
Chapter 101 — Two Consumers of the Overlay Mechanism	223
Chapter 102 — The Files You Were Just In	225
Chapter 103 — Two Lists: Arguments and Processes	227
Chapter 104 — Packages: an ELPA Client Without a Byte Compiler	229
Chapter 105 — Small Operations That Are Not Small	231
Appendix A — Configuration Schema	234
Appendix B — Keymap Cheat Sheet	237
Appendix C — Languages and Themes	240
Appendix D — Glossary	242
Appendix E — From Helix and From Vim	244
Appendix F — Selected Typable Commands	245
Appendix G — The Leader Map	248
Appendix H — Embedded Scripting Reference	249
Appendix I — Capability Index	251

Appendix J — The Typable Command Catalogue	253
Appendix K — The Static Command Surface	257
Appendix L — The Language Support Matrix	260
Appendix M — The Configuration Reference	263
Appendix N — Remapping and the [keys] Config	268
Appendix O — Registers in Full	270
Appendix P — The fusevm Machine, in Reference	272
Appendix Q — The Five Tongues at a Glance	274
Appendix R — The Embedding Seam, in Reference	276
Appendix S — The Operator-Pending Table	278
Appendix T — The Crate Workspace, in Reference	280
Appendix U — The Port Report, by the Numbers	281
Chapter 106 — Rmail: Reading Mail Out of a Flat File	283
Chapter 107 — Diff-Mode as a Text Transformation Suite	289
Chapter 108 — merge_ops: A Module That Is Almost Entirely Tests	291
Chapter 109 — cc-mode Without a Parser	294
Chapter 110 — Four Rings and a Store	297
Chapter 111 — Major-Mode Overlays: Making One Keymap Behave Like Many	300
Chapter 112 — The Word Index: Debouncing an Eventually-Consistent Set	302
Chapter 113 — Amusements, Corpora, and a Module That Cannot Compile	305
Chapter 114 — Vim Display Options Inside a Grapheme Layer	309

Before You Begin

This is not the reference manual. The reference enumerates every one of `zmax`'s commands, every default key, every configuration option, and every language and theme that ships in the box — and you reach for it when you need to know exactly which key runs `goto_word` or what `[editor.lsp]` keys are legal. This book is the other thing: the one you read in a chair, front to back, to understand not just *what* `zmax` does but *why* it is shaped the way it is.

`zmax` is a modal text editor written in Rust, forked from Helix. That sentence is true and almost completely fails to describe it. A fuller description: `zmax` keeps the engine that makes Helix remarkable — the rope-backed buffers, the tree-sitter syntax layer, the language-server client, and above all the multiple-selection model where every edit acts on a set of cursors at once — and throws away the one thing about Helix that stops most people at the door: its keymap. Helix asks you to relearn the verbs you have typed for twenty years, to think selection-first, to press the object before the action. `zmax` does not. The keys you press are the keys vim binds — `dd`, `dw`, `cw`, `yy`, the operator-pending grammar in your fingers — emulated faithfully on top of a selection engine that was never designed to speak them. The default keymap (`DEFAULT_KEYMAP` in `zmax-term/src/config.rs`) is `spacemacs`: that vim base, plus a `SPC` leader menu and the emacs `C-x` prefix layered over it. Pure vim is one command away, as a preset; so are helix and emacs. The three great editing traditions meet on one engine, and the default is the one that serves all three.

The book is in eight movements. The first is the editor itself: the model you are actually driving — buffers, selections, ranges, modes — and why a selection engine wearing vim keys behaves the way it does. The second is the keymaps: the vim grammar, the emacs bindings, the Spacemacs leader menus, and the seam where operator-pending edits are translated into selection operations. The third is the working editor: language servers, the debugger, version control, pickers, the command line. The fourth is the thing no other editor has ever shipped: five complete programming languages — elisp, vimscript, stryke, AWK, and Zsh — baked into the editor as native code, every one of them compiled ahead of time to a single shared bytecode machine, none of them living in a separate process or reached across a foreign-function boundary. The fifth is the machine underneath: the rope, the tree-sitter integration, the event system, the rendering path, and the crate boundaries that hold it together. The sixth is a long stretch of practice — real tasks worked end to end with the keys and tools the earlier movements describe. The seventh is the one that has grown fastest since the first edition: the *four* editing traditions `zmax` now wears at once — vim, emacs, Spacemacs, and a JetBrains/IntelliJ personality — the deep emacs machinery (kill-ring, registers, rectangles, narrowing, paredit), and the IDE workbench that grows out of all of them. The eighth is reference appendices — a keymap cheat sheet, the configuration schema, a glossary.

Everything here is grounded in the actual repository: keys, command names, configuration keys and architectural claims are written against specific files in the `zmax` tree, and cite them where the citation helps you check. Two honest caveats about that. The tree moves faster than a book does, so any exact count printed here is a snapshot — the generated artifacts under `docs/` are the authority, and this book points you at them instead of asking you to trust a number on a page. And where a behavior is still provisional, or where a tongue reaches less far than another, the text says so rather than rounding up. `zmax` is the work of MenkeTechnologies, built on the foundation laid by Blaž Hrašnik and the Helix contributors, and this book is grounded in that code.

A word on this edition. `zmax` moves fast, and since the first edition the source advanced by more than a hundred commits — enough to change the pitch in two concrete ways this book now reflects. First, a *fourth* editing tradition arrived: a JetBrains/IntelliJ personality, with semantic refactorings, real column selection, IDE tool windows, and Run To Cursor, joining vim, emacs, and Spacemacs. Second, the machinery that makes those traditions *deep* — an emacs kill-ring and registers and true rectangles, narrowing and indirect buffers, the paredit suite, window layouts and winner mode — grew in from sketch to substance, and the embedded

vimscrip^t matured from “evaluates” to “runs real control flow and most of the builtin library.” The final two movements of this book are devoted to that newer material. Where the first edition’s caution has since been overtaken by the code, the relevant chapter says so in the place it comes up, because updating a claim that the source has outgrown is the same discipline as marking a provisional one — both are the book refusing to let its account drift from the tree it describes.

Let’s start.

Chapter 1 — The Pitch

Every editor is an argument about how a person should spend their attention. The argument `zmax` makes has two halves, and they are easier to state than to believe. The first half: you should not have to relearn the verbs you already know. The second half: the editor should be programmable in the languages you already write, with no wall between the editor and the program — no subprocess, no plugin protocol, no foreign-function boundary, no second language invented solely to configure the first. Both halves are unusual. The second half has never been done before at all. Taken together they describe an editor that is conservative about your fingers and radical about its own internals, and the tension between those two commitments is the whole story.

Start with what `zmax` is made of, because the claims that follow are only credible if the foundation is. `zmax` is a modal text editor written in Rust, forked from Helix at v25.7.1. From Helix it inherits an engine that is, by any honest accounting, one of the best editor cores in existence: text stored in a rope so that an edit in a ten-megabyte file touches a handful of tree nodes rather than shifting the whole buffer; a tree-sitter syntax layer that reparses incrementally as you type; a language-server client and a debug-adapter client built in, not bolted on; and, at the center, the multiple-selection model that Helix took from Kakoune, in which the fundamental object you manipulate is not a cursor but a set of selections, and every motion and edit acts on all of them at once. That engine is the part of Helix nobody disputes. It is fast, it is correct, and it is the reason `zmax` did not start from a blank file.

The keymap problem

What people dispute about Helix is the keymap, and the dispute is not aesthetic. Helix is selection-first: you select the object and *then* press the verb, so “delete a word” is `w` then `d` — noun, then verb. Vim is the opposite order, verb then noun: `d` then `w`. This sounds like a trivial transposition until you remember that the order is wired into twenty years of muscle memory for millions of people, that the entire vim grammar — `ci`, `dt`, `2dd`, `y$` — is built on verb-before-noun, and that no amount of agreeing Helix is well-designed will make an expert’s hands stop typing `dd` when they mean “delete this line.” The cost of Helix’s keymap is paid not once, at learning time, but every day, by everyone who already fluently speaks vim and now has to think. `zmax` treats that cost as unacceptable and removes it.

Its keys are vim’s — not vim-flavored, not vim-inspired, but the vim grammar with verb-before-noun operator-pending edits: `dd`, `dw`, `cw`, `yy`, `d$`, `c^`, `>j`, `gUw`, the whole vocabulary. This is the part that should not work, because the engine underneath is selection-first and the vim grammar is selection-last. The trick — and Chapter 5 is devoted to exactly how it is pulled off — is that each vim operator is emulated as a short sequence of native selection commands. When you type `dw`, `zmax` does not run a monolithic “delete word” command; it runs `[collapse_selection, subword_extend_w, delete_selection]` — collapse to a bare cursor, extend a selection forward over the word, delete the selection. The verb-noun surface is sugar; the select-then-act engine is the substance; and the seam between them is invisible from the keyboard. On top of the vim layer `zmax` adds an emacs personality (`M-x`, `M-f`, `C-space` to set the mark, the readline editing keys) and a Spacemacs- style `SPC` leader tree of discoverable submenus, so that the three great editing traditions — vim, emacs, Spacemacs — are reachable on one engine without choosing sides. The keymap that ships as the default is the one that combines them (`spacemacs`); the pure-vim base is the `vim` preset. That is the first half of the argument, and on its own it would make `zmax` a comfortable place for a vim exile to live.

The five tongues

The second half is the one with no precedent. `zmax` embeds five complete programming languages as its extension and scripting layer: **elisp**, **vimscript**, **stryke**, **AWK**, and **Zsh**. Not one scripting language, the way `emacs` has `elisp` and nothing else. Not a host language plus an FFI escape hatch, the way `Neovim` has `vimscript` and reaches `Lua`. Five full language runtimes, first-class, all present in the same binary.

The word “embeds” is doing precise work, and it is worth slowing down on, because every other editor that advertises multiple languages means something weaker by it. When `Neovim` runs `Lua`, it calls across a C ABI into `LuaJIT`, a separately-built library. When `VS Code` runs an extension, the extension lives in a *separate operating-system process* — a `Node` host — and every call between the editor and the extension is serialized to JSON and shipped over a pipe. When `Vim` runs a `Python` plugin, it loads a dynamic library and marshals values across the C boundary. In every one of these designs there is a seam: a process boundary, a socket, an ABI, a serialization step — a place where the editor stops and the foreign thing begins, with marshalling and latency and a translation layer in between.

`zmax` has no such seam. All five interpreters are native `Rust` crates compiled directly into the editor: `elisp` via `elisp.rs`, `vimscript` via `vimlrs`, `stryke` via `strykelang`, `AWK` via `awkrs`, `Zsh` via `zshrs`. They share the editor’s own address space. A script that moves the cursor is not sending a message to the editor; it is *calling a Rust function in the same process*, with the editor’s own command context reachable through a thread-local pointer, no JSON in between, no pipe, no ABI, no subprocess to spawn or keep alive. The host callback a language uses to reach the editor carries no state, because it does not have to: the editor is a pointer read away. (There is one place a copy still happens — `elisp`’s buffer text is mirrored in and out per `eval`, for a reason Chapter 22 sets out — and the book flags it there rather than pretending the picture is uniformly seamless.) One binary, one address space, five tongues — and the reason that sentence is even possible is the four words the rest of this section unpacks: it is all `Rust`.

One bytecode machine underneath

Five interpreters could be five separate piles of code that happen to be linked together. They are not. All five compile down to a single shared bytecode virtual machine called **fusevm** — “one VM to run them all” — a language-agnostic machine with 224 opcodes across 21 sections, eleven fused superinstructions that collapse common opcode pairs into one dispatch, and a three-stage `Cranefly` JIT. A language frontend’s only job is to lower its source to `fusevm` opcodes; in exchange it gets fused hot-loop dispatch, slot-indexed fast paths, and native-code compilation, all for free, all shared with the other four tongues. `elisp` and `AWK` and `Zsh` are not three interpreters in `zmax`; they are three front ends onto one machine.

This is where the words *ahead-of-time* and *just-in-time* both earn their place, and the distinction between them matters. The machine itself — every interpreter, the VM dispatch loop, the JIT, the entire editor — is compiled ahead of time, statically, into one native binary. There is no runtime compilation of the engine, no warm-up of the interpreter, no toolchain the user must have installed: the thing that runs your `elisp` was turned into machine code by `rustc` and `cargo` long before you launched the editor. The just-in-time compiler enters at exactly one place, and it is the place it must: dynamic `eval`. `zmax`, like `emacs`, lets you evaluate code interactively at runtime — type an `elisp` form, or a line of `stryke`, and run it against the live editor. Code that arrives at runtime cannot have been compiled ahead of time, by definition; so `fusevm` compiles it to bytecode on the fly and its `Cranefly` JIT promotes the hot paths to native code as they run. The engine is AOT and static; the `eval` you type is JIT’d and fast. Nothing in the design is interpreted in the slow, tree-walking sense that makes stock `vimscript` a punchline — which is why an embedded tongue here can outrun its own reference implementation.

Why this is the shape

Hold the two halves together and the design has a single logic. An editor named `zmax` — `z` plus `emacs` — is making an explicit promise about extensibility, because the one thing everyone knows about `emacs` is that it

is programmable all the way down in elisp. zmax keeps that promise and then breaks the monopoly: where emacs gives you one language to extend it, zmax gives you five, and where emacs's elisp is a bespoke C interpreter, zmax's five tongues are AOT- compiled Rust on a JIT-backed VM in the editor's own address space. And it does all of this without asking you to give up the vim keys your hands already know, because the editor underneath the languages is the Helix engine wearing vim's grammar. Conservative about your fingers, radical about its internals: the rest of this book is the long version of that sentence, and every claim in it is traceable to a file in the source tree.

A note on honesty before we go further. zmax is young — its own status badge says *early* — and this book does not pretend otherwise. Some vim edits are not yet wired (`df<c>`, `ciw`, the `.` repeat); the five-language embedding is being built out crate by crate as this is written; numbers and coverage shift between releases. Where something is provisional, the book says so in the place it comes up, rather than rounding a work-in-progress up to a finished claim. The pitch is big enough to survive being stated accurately.

Chapter 2 — First Contact

The fastest way to stop arguing about an editor is to open it, so this chapter is the first hour: getting `zmax` onto your machine, launching it, and making the handful of edits that prove the central claim — that your vim fingers already work. Nothing here is deep. The depth starts in Chapter 3. This is the part where you confirm the thing runs and behaves the way the pitch said it would.

`zmax` is a Cargo workspace, so building it is the ordinary Rust ritual: clone the repository, and from its root run `cargo install --path zmax-term --locked`. The `zmax-term` crate is the terminal binary — the workspace’s `default-members` points there, so a bare `cargo build` in the root builds the editor — and the `--locked` flag pins the exact dependency versions recorded in `Cargo.lock`, which on a young project is the difference between a build that works and a build that surprises you. When it finishes you have a single executable. That word *single* is not decoration: everything this book describes — the rope, the tree-sitter integration, the LSP and DAP clients, eventually all five embedded languages — is inside that one file. There is no runtime to install alongside it, no plugin host to keep running, no separate language binary to put on the path.

On first launch `zmax` looks for its runtime files — the tree-sitter grammars, the highlight queries, the themes — and its configuration, in a small set of locations resolved by the `zmax-loader` crate. The search order is deliberate and worth knowing once: a `runtime/` directory next to the executable, a `runtime/` under your config directory (`~/.config/zmax/` on Linux and macOS), the path in the `ZMAX_RUNTIME` environment variable, and a build-time default baked in by `ZMAX_DEFAULT_RUNTIME`. The first one found wins. Most people never touch this; it matters the day you want to override a shipped query or run a development build against a checkout’s `runtime/` instead of the installed one.

The tutor

Before you configure anything, run the tutor. From a shell, `zmax --tutor` opens an interactive lesson file — it lives at `runtime/tutor` — that walks you through the motions and edits hands-on, the way `vimtutor` does. It is the single best twenty minutes you can spend, and it is honest about `zmax` specifically rather than a generic modal-editing primer: the keys it teaches are the keys `zmax` actually binds, so the muscle memory you build in the tutor is the muscle memory you keep. If you already speak vim, the tutor will feel less like learning and more like confirming, which is exactly the feeling the pitch promised.

Three modes, and your first edits

`zmax` has exactly three modes, and the whole editor is organized around them. They are defined, in the source, as a three-variant enum in `zmax-view/src/document.rs`:

```
pub enum Mode {
    Normal = 0,
    Select = 1,
    Insert = 2,
}
```

You start in **Normal** mode, where the letter keys are commands rather than text: `h j k l` move, `w` and `b` hop by word, `0` and `$` go to the ends of the line, `gg` and `G` go to the ends of the file. Press `i` and you enter **Insert** mode, where keys are text again and you type the way you would in any editor; `Esc` (or `C-c`, or `C-[`) returns

you to Normal. Press **v** and you enter **Select** mode, vim's visual mode, where motions extend a highlighted selection instead of just moving. Three modes, three jobs: Normal to command, Insert to type, Select to mark a span before acting on it. That is the entire modal model; everything else is vocabulary inside it.

Now make the edits that prove the claim. Put the cursor on a line and type **dd** — the line is gone, exactly as in vim. Type **dw** — the word ahead of the cursor is deleted. Type **cw**, and the word is deleted *and* you land in Insert mode ready to retype it — the change operator, verb before noun, behaving the way your hands expect. Type **yy** to yank a line and **p** to paste it below. Put the cursor inside a line and type **A** to append at its end, or **o** to open a new line below and start typing. Type **u** to undo and **C-r** to redo. None of this should feel novel; that is the point. If you have typed these keys in vim, you have just typed them in zmax and gotten the same result, on top of an engine that — underneath the surface you just drove — was thinking in selections the entire time. Chapter 5 opens that surface up. For now it is enough that it closed seamlessly over the engine, and that your first hour asked you to learn nothing you did not already know.

Chapter 3 – The Selection Engine

To understand `zmax` you have to understand the object you are actually manipulating, and it is not the cursor. It is the *selection*, and the editor's deepest behaviors — the ones that feel strange if you arrived from `vim` and feel inevitable once they click — all follow from that one fact. This chapter is about the engine `zmax` inherited from `Helix`, which `Helix` in turn took from `Kakoune`: the model in which the fundamental unit of editing is a set of selected ranges, and every motion and every edit operates on the whole set at once. The `vim` keymap of Chapter 2 is a costume worn over this engine. This chapter is the body under the costume.

A range is a span, not a point

In a conventional editor the cursor is a point: a single position between two characters, and that is all the state there is. In `zmax` the irreducible unit is a `Range`, and a range is a *span*. Its definition lives in `zmax-core/src/selection.rs`:

```
pub struct Range {
    pub anchor: usize,
    pub head: usize,
    pub old_visual_position: Option<(u32, u32)>,
}
```

Two numbers carry the meaning. The `anchor` is the fixed end — the end that stays put when you extend a selection — and the `head` is the moving end, the one a motion drags. Both are offsets measured in *characters*, not bytes: the whole engine indexes the text by `char` position, so a multi-byte character counts as one, and the cursor can never land in the middle of one. When `head` is greater than `anchor` the range points forward; when it is less, the range points backward; direction is information the engine keeps and uses. A bare cursor — what looks like a single blinking point on screen — is in this model just a range whose anchor and head sit a single character apart, with the block you see drawn over the character at the head. There is no separate “cursor” type. There are only ranges, some of them one character wide.

The indices are *gap* indices: they count the gaps between characters, not the characters themselves, so position 0 is before the first character and a range from 0 to 3 covers the first three characters — half-open, inclusive of the left edge and exclusive of the right, the same convention as a Rust slice. This sounds pedantic until you are reasoning about an edit at the very end of a line or the boundary between two selections, at which point the half-open gap convention is the thing that makes the arithmetic come out right every time instead of off-by-one half the time. The `old_visual_position` field is a smaller comfort: it remembers the column you were in so that moving down through a short line and back out to a long one returns you to where you visually were, the “sticky column” every good editor has and few explain.

A selection is a set of ranges

One range is a span. The editor's real state is a `Selection`, which is a set of them:

```
pub struct Selection {
    ranges: SmallVec<[Range; 1]>,
    primary_index: usize,
}
```

Two details in that declaration carry the design. The ranges live in a `SmallVec<[Range; 1]>` — a vector that stores a single range inline, with no heap allocation, and only spills to the heap when a second range appears. The common case, one cursor, costs nothing; the multi-cursor case grows on demand. And there is always at least one range: the type maintains the invariant that a selection is never empty, so there is no such thing as “no cursor.” The `primary_index` marks which range is the *primary* one — the one whose position the status line reports and whose viewport the screen follows — while every other range is just as real and just as editable.

The consequence is the whole point of the engine: when you run a motion or an edit, it does not act on “the cursor.” It acts on every range in the selection, independently and simultaneously. Select ten occurrences of a variable name and they become ten ranges in one selection; type `cw` and all ten are changed at once; the ten edits are composed into a single transaction and applied together. What looks like multiple-cursors-as-a-feature in other editors is, here, not a feature bolted on but the default and only shape of editing. There is one cursor or there are five hundred; the code path is identical, because both are just a `Selection` with some number of `Ranges` in it.

Why the vim keymap had to be emulated

Now the tension named in Chapter 1 has a precise mechanical cause, and you can see it. This engine is built to *extend and then operate*: you grow a selection to cover what you mean, and then a verb consumes the selection. That is noun-then-verb, selection-first, and it is wired into every command — `delete` deletes whatever is selected, `change` changes whatever is selected, with no notion of “delete a word” as a single indivisible verb. Vim is verb-then-noun: `d` announces an intent to delete and *then* a motion says how far. The two grammars run in opposite directions over the same machine.

So the vim keymap cannot simply rename engine commands; it has to *translate* between the grammars, and the only way to translate verb-first into a select-first engine is to expand each vim operator into a little script of engine commands that builds the selection the operator implies and then runs the engine verb. `dw` becomes, literally, “collapse to a cursor, extend forward over a word, delete the selection.” The engine never learns what a vim operator is; it only ever sees collapses, extends, and deletes. That expansion — how it is spelled in the keymap, which sequences cover which operators, and exactly which vim edits resist the trick and remain unimplemented — is the subject of Chapter 5. What matters here, at the level of the engine, is *why* the translation is necessary at all: because the object under your keys is a set of spans, and vim’s grammar was written for a single point. `zmax` makes the set wear the costume of the point, and does it well enough that your hands never notice — but the set is what is really there.

Chapter 4 – Three Modes

Chapter 2 named the three modes in passing. This chapter takes each one seriously, because the character of an editor is mostly the character of its modes, and zmax's three are not the three you would predict from vim. The surprises are instructive: where zmax follows vim exactly, where it quietly differs, and where its Helix lineage shows through the vim costume. The enum ordered them `Normal = 0`, `Select = 1`, `Insert = 2`; we take them in the order you meet them.

Normal mode is a language

Normal mode is where you spend your life, and it is best understood not as “the mode where you are not typing” but as a small language whose words are single keys. A key in Normal mode is a verb (`d`, `c`, `y`), a motion (`w`, `$`, `G`), an operator that expects a motion after it (`d` then `w`), or a prefix that opens a whole submenu of further keys (`g`, `z`, `[`, `]`, `SPC`). Counts compose with all of it: `3w` moves three words, `2dd` deletes two lines, and the count is consumed by the keymap machinery before the command ever runs, so commands never have to know they were repeated. This is the vim mental model exactly, and that is deliberate — Normal mode is the part of zmax that most needs to feel like home, and it does.

What is *not* vim is what sits underneath, and Chapter 3 already showed it: every Normal-mode motion is, in the engine, a selection operation that happens to collapse to a one-character range when you are “just moving.” The fluency you feel in Normal mode is the fluency of vim; the machine answering your keys is the selection engine. The keymap that defines all of this lives in one large file, `zmax-term/sic/keymap/vim.rs`, and reading it is the single most direct way to know exactly what every key does, because it is not documentation that can drift; it is the binding itself.

Insert mode is deliberately small

Insert mode is where keys become text, and zmax makes a specific philosophical choice here that is worth naming: Insert mode is kept *minimal*. The modal-first view is that you should be in Insert mode briefly, to enter text, and back in Normal mode to do anything structural — so Insert mode resists growing into a second editor full of its own commands. What it does provide is the set of editing keys a touch-typist's hands reach for without leaving the home row: the readline/emacs bindings. `C-w` deletes the word behind the cursor, `C-u` kills to the start of the line, `C-h` is backspace, `A-d` deletes the word ahead, `A-f` and `A-b` walk forward and backward by word, `C-a`-style line motions are present, and `M-<` / `M->` jump to the ends of the buffer. Completion is here too: `C-n` and `C-p` cycle candidates, and `C-x` opens a vim-style completion submenu (`C-x C-o` for omni/LSP completion, `C-x C-f` for filenames, `C-x C-l` for whole lines), all of which route into zmax's LSP and word completion underneath.

Two details reward attention. First, `C-g` in Insert mode opens a small submap that includes `C-g u`, which commits an undo checkpoint mid-insertion — vim's `i_CTRL-G_u` — so a long Insert session can be carved into sensible undo steps rather than collapsing into one giant undo. Second, the change made during an Insert session is committed as a unit when you press `Esc`: the operator-pending machinery and the undo history both treat “the text you typed between `i` and `Esc`” as one transaction, which is why `.`-style repeat (when it lands) and undo behave the way vim users expect. Insert mode is small on purpose, and the smallness is the feature.

Select mode is vim visual, with a Helix accent

Select mode is the one where the lineage shows most. To a vim user it is visual mode: press `v`, and now motions extend a highlight instead of moving a point; press `V` for linewise; the highlighted span is the thing your next operator acts on. `d` or `x` deletes it, `c` changes it, `y` yanks it, `>` and `<` indent and outdent it, `~` toggles its case, `o` and `O` jump the cursor to the other end of the selection so you can extend the side you did not mean to fix. All of that is vim, and all of it works.

The Helix accent is in two places. The first is that, in the engine, Select mode is closer to the *natural* state of the editor than Normal mode is — remember that the engine is selection-first, so “extend a selection and then operate” is its mother tongue. In Select mode the vim costume is thinnest and the engine is most itself: the operators act on the selection *directly*, with none of the `[collapse, extend, operate]` indirection that Normal-mode operators need (Chapter 5). That is why the Select-mode bindings in `vim.rs` are so much simpler than the Normal-mode ones — they are not translating between grammars; they are speaking the engine’s own.

The second accent is block selection. Vim’s `C-v` enters a true rectangular “visual block” mode, a distinct kind of selection. The Helix engine has no rectangular-selection primitive — it has the set-of-ranges, and nothing else — so `zmax` *emulates* block editing with multiple cursors: you enter Select, size the block with `l/e/$`, and grow it down the lines with `C-v`, which copies the selection onto the next line as an additional range. The result behaves like block editing — type with `I` or `A` and you are inserting on every line of the block at once — but it is multiple cursors underneath, not a rectangle. This is the same move the whole editor makes, one level down: where vim has a special case, `zmax` expresses it as a configuration of the one general mechanism it has. The set of ranges is enough, if you are willing to build the rest out of it.

Chapter 5 – Operator-Pending, Emulated

This is the keystone chapter. Everything before it has pointed at a seam — vim's verb-first grammar worn over a select-first engine — and asserted that zmax hides it well. This chapter opens the seam and shows the mechanism, because the mechanism is genuinely clever and worth understanding. By the end you will know not just that `dd` works but *how* — and how the same trick was stretched to reach the interactive edits (`ciw`, `df`, the `.` repeat) that an earlier edition of this chapter had to defer.

The trick, stated plainly

A comment at the top of `zmax-term/src/keymap/vim.rs` states the whole design in two sentences: where vim is verb-noun — operator-pending, `d{motion}`, `c{motion}`, `y{motion}` — zmax emulates it with nested submaps whose motions run `[collapse_selection, extend-motion, operate]` command sequences. Unpack that. The keymap is a *trie*: pressing `d` does not run a command, it descends into a submap, and the next key you press — `d`, `w`, `e`, `$` — selects an entry in that submap. Each of those entries is not a single command but a *list* of commands run in order. The list for `dw` is, in full:

```
d → { "delete"
  w → [ collapse_selection, subword_extend_w, delete_selection ]
}
```

When you type `d` then `w`, zmax runs three engine commands back to back: `collapse_selection` shrinks whatever is selected down to a bare cursor; `subword_extend_w` grows a selection forward over the word; `delete_selection` deletes it. Three native selection operations, composed by the keymap, produce the vim behavior of `dw`. The engine was never told what `d` is.

Every operator is built this way, though the sequences get longer as the fidelity gets finer. `dd` is four commands rather than three — `[collapse_selection, extend_to_line_bounds, delete_selection_linewise, goto_first_nonwhitespace]` — because vim's `dd` is linewise *and* leaves the cursor on the first non-blank of the following line, and both of those are separate steps here. `cw` is `[collapse_selection, subword_extend_e, change_selection]`, and the `_e` is deliberate: vim's `cw` behaves like `ce`, stopping at the word's end rather than the next word's start, so the change operator binds a different motion than the delete operator does for the same key. `yy` is `[collapse_selection, extend_to_line_bounds, yank, collapse_selection]` — copy the line, then tidy the selection back to a cursor. The case operators wear the same shape with a different verb: `gUw` is `[collapse_selection, subword_extend_w, switch_to_uppercase, collapse_selection]`. Indent (`>j`), reformat (`=G`), and the shell-filter operator (`!!`) are all more of the same: a collapse, an extend that encodes the motion, and a verb.

That `dw` and `cw` bind different extend-motions is worth pausing on, because it is where the emulation stops being a mechanical transformation and starts being a port. Nobody would derive `cw` → `subword_extend_e` from first principles; it is there because that is what vim does, and someone checked. The file is full of that kind of entry, and it is why it is as long as it is.

Counts ride along for free. The count prefix is consumed by the keymap engine before the command list runs, and it multiplies the motion inside the list, so `3dw` extends across three words before deleting and `2dd` across two lines — without any entry in the submap having to mention counts at all. The engine's existing repeat machinery does the work; the keymap only has to name the motion.

How the trick was extended

The static emulation has a natural ceiling, and the first edition of this chapter stopped at it: a fixed list of commands chosen by keypress cannot pause to read an *interactive* key, so the edits that depend on one — `df`, `ciw`, `di` — were the documented corner a vim user fell into in the first ten minutes. That corner has since been filled in, and how it was filled in is worth seeing, because it did not throw out the static-trie design so much as give a few of its entries a single moment of memory.

The mechanism is the editor's *next-key* callback: a command, instead of finishing, can register a closure that receives the very next keystroke — the same facility the snippet code uses to let a typed character overwrite a placeholder. Operator-plus-find is built on it. `d` descends into the delete submap as before, but its `f` entry is now `delete_find_char_forward`, a command that arms the next-key callback, waits for you to press the target character, then extends to it and deletes. The static list did not learn to pause; one command inside it learned to ask for one more key. `df`, `ct`, `cf`, and the `F/T` backward variants all work this way.

Operator-plus-text-object closed the same way. The `i` and `a` entries in each operator's submap are `delete_textobject_inner` / `change_textobject_around` and their siblings — commands that read the following `w`, `(`, `p`, or `"` as a text-object specifier and select the inner or surrounding object before operating. `ciw` changes the inner word, `di` deletes inside the parentheses, `dip` takes the paragraph, `ya{` yanks around the braces — the whole `operator + i/a + object` matrix a vim user's hands assume. It rests on the same tree-sitter and match-pair text objects of Chapter 13; the keymap simply reaches them through the operator now, not only from Select mode.

The odds and ends followed. The `q/@` macro system records and replays keystrokes to a named register; named marks (`m{a-z}`, then ``{a-z}` / `'{a-z}`) remember positions and are carried along as the buffer is edited; `R` is a true Replace mode that overtypes; and `.` repeats the last change, recorded as a replayable unit — working today for the insert-and-change class of edits, which is where dot-repeat earns most of its keep. Block-visual, which earlier editions of this book listed as the one genuinely emulated piece, is native now: `C-v` enters `visual_block_mode` and the editor carries a real rectangle — `BlockSelect { anchor, to_eol }` in `zmax-view/src/editor.rs`, re-derived after every motion by `block_reproject` — with `to_eol` modelling vim's `CTRL-V $` ragged right edge. Chapter 59 tells the story of where that rectangle came from. The seam this chapter opened is, for the daily grammar, closed — and the honest accounting that once flagged it (Chapter 29) is the same machinery that now records it as covered.

Chapter 6 – The Vim Grammar

Movement I was about the engine and the seam. Movement II is about the vocabulary — the actual keys, mode by mode and prefix by prefix — and it begins with the core vim grammar, the words you will type ten thousand times a day. This chapter is closer to a guided tour than an argument, because the value here is completeness and accuracy: these are the bindings exactly as `vim.rs` defines them, so that when your hand reaches for a key you can confirm `zmax` binds it the way you remember. Where a binding diverges from vim, the chapter says so; where it is identical, it says that too, because “identical” is the feature.

Motions

The home-row motions are vim's: `h j k l` for left, down, up, right, with the arrow keys bound as synonyms. One subtlety: `j` and `k` are bound to `move_visual_line_down` and `move_visual_line_up`, meaning they move by *visual* line, so on a soft-wrapped paragraph they step through screen rows rather than leaping over the whole wrapped logical line — the behavior most people actually want, and the one vim only gives you if you remember to bind `gj/gk`.

Word motion comes in two grades, exactly as in vim. Lowercase `w, b, e` move by *word* — `move_next_word_start`, `move_prev_word_start`, `move_next_word_end` — where a word breaks on punctuation; uppercase `W, B, E` move by *WORD*, the long kind that treats anything non-blank as part of the token. Within a line, `0` goes to the very start, `^` to the first non-whitespace character, `$` to the end; `home` and `end` mirror `0` and `$`. Across the file, `gg` goes to the top and `G` to the bottom — the vim spelling, not Helix's.

Find-character motion is complete and behaves like vim down to the repeat keys. `f` searches forward to a character and lands on it; `F` searches backward; `t` stops one short (“till”); `T` is the backward till. After any of them, `;` repeats the last find in the same direction and `,` repeats it reversed — `repeat_last_motion` and `repeat_find_char_reverse` — so the `f;/;` rhythm of hopping along a line to successive commas is intact.

Search

Regular-expression search is on `/` (forward) and `?` (backward), with `n` and `N` selecting the next and previous match. Two convenience keys round it out the vim way: `*` searches forward for the word under the cursor and `#` searches backward, each first detecting the word boundaries around the cursor so you do not have to select the word yourself. Because search results are *selections* in this engine, `n` does not merely move the cursor to the next match — it selects the match, which means the very next operator acts on exactly the text you searched for, a small but constant convenience that falls out of the selection model for free.

The single-key edits

Beyond the operators of Chapter 5, a set of single keys perform edits directly. `x` deletes the character under the cursor and `X` the character before it; `D` deletes from the cursor to the end of the line and `C` changes to the end of the line; `Y` yanks the whole line. `J` joins the line below onto the current one. `~` toggles the case of the character under the cursor. `r` replaces a single character (it waits for the replacement key); `R` enters Replace mode. `p` pastes after the cursor and `P` before it. `u` undoes and `C-r` redoes. Every one of these is its vim spelling doing its vim job.

Entering Insert mode

Six keys cross from Normal into Insert, and they are the six vim keys: **i** inserts before the cursor, **a** appends after it; **I** inserts at the first non-blank of the line and **A** at the end of the line; **o** opens a new line below and **O** above, each dropping you into Insert ready to type. There is nothing surprising here, and that is the entire intent — the keys that start you typing are the keys your hands already press.

Marks, registers, and macros

Marks use vim's punctuation: **m** followed by a letter sets a mark, a backtick followed by that letter jumps to the exact position, and an apostrophe jumps to the start of the marked line. Registers are reached with the double-quote key: **"** followed by a register name selects which register the *next* yank, delete, or paste will use, so **"ayy** yanks a line into register **a**. Macros use vim's **q** and **@**: **q** followed by a register records a macro into it, **q** again stops recording, **@** followed by the register replays it, and **Q** replays the last one. As Chapter 5 noted, the macro system is partially implemented — record and replay exist, the rougher edges are still being worked — so this is one of the places to keep the “early” badge in mind, but the everyday record-a-fix-and-replay-it loop is present.

Chapter 7 – The g, z, [, and] Prefixes

Vim organizes a great deal of its less-frequent vocabulary under four prefix keys, and zmax follows the convention closely enough that the muscle memory transfers and extends it where the underlying engine offers something vim lacked. A prefix key, in the keymap trie of Chapter 5, is simply a submap: press **g** and you have not run a command, you have descended into a menu, and the next key chooses from it. Because **auto-info** is on by default, holding at a prefix briefly pops a small menu showing what the next keys do — so these prefixes are not something you have to memorize from a manual; the editor shows you the branches as you stand at them.

g – goto and a grab-bag

The **g** prefix is vim’s “goto,” and zmax keeps the navigational core: **gg** to the top of the file, **ge** to the end of the previous word, **gn** and **gN** to the next and previous search match. It also gathers, as vim does, a grab-bag of operations that did not fit elsewhere. The case operators live here — **gU**, **gu**, and **g~** to uppercase, lowercase, and toggle a motion’s worth of text (Chapter 5 showed **gUw**’s expansion). Formatting is here: **gg** and **gw** reflow through the language server. Insertion variants **gi** (“insert at last insert position”) and **gI** (“insert at column one”) are here. **gv** reselects the last visual region; **gp** and **gP** paste and leave the cursor after or before the pasted text; **gx** and **gf** open the URL or file under the cursor.

The most important **g** bindings, though, are the ones vim never had, because they reach the language server: **gd** goes to definition, **gD** to declaration, **gy** to type definition, **gr** to references, **gi** to implementation. These are the keys a modern editor lives on, and zmax puts them exactly where a Neovim user with an LSP config has trained their hand to expect them. Buffer navigation rounds out the prefix: **gt** and **gT** move to the next and previous buffer, the vim “tab” keys repurposed for buffers.

z – the viewport and folds

The **z** prefix controls what the screen shows without moving the logical cursor, again following vim. The view-alignment trio is here: **zz** centers the current line in the window, **zt** scrolls it to the top, **zb** to the bottom, with **z.** as a centering synonym. Horizontal scrolling is **zh** and **zL**. Folding lives under **z** as it does in vim: **za** toggles a fold, **zo** opens and **zc** closes one, and **zf** creates a fold over a selection. Spell-checking, too, takes vim’s **z** bindings: **z=** suggests corrections, **zg** marks a word good, **zw** marks it bad. The organizing idea is vim’s — **z** is the “view and folds and spelling” drawer — and zmax keeps the drawer where you left it.

[and] – unimpaired-style navigation

The bracket pair is the one most associated with Tim Pope’s *vim-unimpaired* plugin, and zmax builds the idiom in: **[** jumps to the previous something and **]** to the next, with the second key naming the something. **[d** and **]d** step through diagnostics — the everyday key for walking the errors in a file. **[f** and **]f** step by function, **[c** and **]c** by change (a git hunk, the version-control integration of Chapter 16 surfacing here as a motion), **[s** and **]s** by spelling error, **[(** and **])** to the previous and next unmatched parenthesis. **[p** and **]p** paste line-wise before and after with indentation adjusted to context. The pattern is uniform and learnable as a whole rather than key by key: once **]d** for “next diagnostic” is in your hand, **]f** and **]c** and **]s** are free, because the bracket

chooses direction and the letter chooses the category — the same compositional economy that makes the whole vim grammar worth learning once and keeping for life.

Chapter 8 – Windows and Buffers

An editor is not one text; it is many, arranged on a screen. `zmax` separates those two ideas the way `vim` does, into *buffers* — the open files themselves — and *windows*, the rectangular views onto them, and it gives each its own set of keys. Understanding the split matters because it is easy to confuse “close this window” with “close this file,” and the two are different operations on different objects.

Windows under `C-w`

Window management hangs off the `C-w` prefix, `vim`’s window key, and the submap under it is thorough. Splitting: `C-w s` splits horizontally and `C-w v` vertically, giving you two windows onto buffers. Moving between windows: `C-w h`, `C-w j`, `C-w k`, `C-w l` move focus left, down, up, right — the home-row motions lifted up a level to operate on windows instead of characters. Rearranging: `C-w H`, `C-w J`, `C-w K`, `C-w L` swap the current window to the far left, bottom, top, or right; `C-w w` and `C-w W` rotate focus forward and backward through the windows; `C-w x` transposes (swaps) two windows; `C-w p` jumps to the previous window. Housekeeping: `C-w n` opens a new split, `C-w q` closes the current window, and `C-w =` equalizes the window sizes so they share the screen evenly. Resizing has its own keys — `C-w +` and `C-w -` change height, `C-w >` and `C-w <` change width. The whole vocabulary is `vim`’s, which means if you have ever run a split `vim` or `tmux`-style layout, your hands already know it.

Underneath, a window is a *View* and the layout is a tree of them, as Movement V will detail: the editor keeps a tree whose leaves are views and whose internal nodes divide space in proportions, and the resize and equalize keys are adjusting those proportions. You do not need the tree to drive the windows, but it is why the windows compose cleanly — splitting a split just adds a node — rather than bottoming out at some fixed two-pane limit.

Buffers

Buffers are the files. The quickest way between them is `gt` and `gT`, next and previous buffer, which Chapter 7 met under the `g` prefix. For anything more than stepping, the picker is faster: the buffer picker (Chapter 12) lists every open buffer and fuzzy-finds among them, which is the right tool once you have more than a handful open. On the command line, the `:buffer-next` / `:buffer-previous` typable commands (aliases `:bn` / `:bp`) and the `:buffer-close` family (`:bc`, and `:buffer-close-all` / `:bca`) manage buffers by name and in bulk; `:open` (aliases `:e`, `:o`, `:edit`) brings a new file in. The split between windows and buffers shows here too: `:quit` (`:q`) closes a *window*, while `:buffer-close` closes a *buffer*, and conflating them is the single most common new-user stumble. Keep the model — buffers are files, windows are views, `C-w` is for windows and `:b.../g t` are for buffers — and the editor’s handling of many files at once stays legible no matter how many you open.

One kind of file gets a buffer of a different shape. Open something binary — an executable, an image, anything that is not text — and `zmax` neither refuses it nor mangles it into garbled UTF-8; it routes the file into a hex editor, the `xxd`-style view also reachable on demand as `:hex`, with byte offsets down the left, the hex columns in the middle, and the ASCII gutter on the right. It is byte-faithful in both directions: `i` or `R` drops you into edit mode, you overwrite bytes in either the hex or the ASCII column (`Tab` toggles which), and `Ctrl-s` writes the raw bytes back exactly, so a round trip through the editor changes nothing you did not change yourself. A binary file is no longer a dead end — it is just a buffer that counts in bytes instead of characters.

Chapter 9 – The Emacs Layer

zmax is named for emacs, and the name is a promise about more than extensibility. A working emacs user has a second set of reflexes — the readline/Meta keys that move and kill and yank without ever leaving the home position — and zmax honors those reflexes by binding the emacs keys *alongside* the vim ones, so that the two traditions coexist on the same keyboard rather than forcing a choice. This chapter is the emacs overlay: what is bound, why it does not collide with vim, and how the two mental models share an engine.

Meta keys in Normal mode

In Normal mode the Meta (Alt) keys carry the emacs vocabulary. **A-x** (that is, **M-x**) opens the command palette — emacs's **M-x**, the “run a command by name” prompt that is the front door to everything the editor can do. **A-<** and **A->** jump to the beginning and end of the buffer, emacs's **M-<** / **M->**. Word motion has its emacs spelling: **A-f** forward a word, **A-b** back a word, **A-d** kill the word ahead. **A-w** copies (emacs's **M-w**, “kill-ring-save”); **A-v** scrolls up a page (emacs's **M-v**). **A-;** toggles comments — emacs's **M-;**, “comment-dwim” — and **A-q** reformats, emacs's **M-q** “fill-paragraph.” **C-g**, the emacs “what am I looking at” reflex, shows file and cursor information. And **C-space** sets the mark — which in this engine means entering Select mode with the anchor pinned, the exact emacs gesture of **C-SPC** to start a region.

That last binding is the clean joint between the two traditions, and it is worth dwelling on. Emacs's “set mark, move, act on region” *is* zmax's “anchor a selection, extend it, operate” — they are the same gesture under two names, because both are selection-first. The vim user reaches this state with **v**; the emacs user reaches it with **C-space**; the engine cannot tell the difference, because there is only one underlying motion: pin an anchor, move the head. The emacs layer is not a compatibility shim bolted over a vim editor; it is a second set of names for an engine that was always, underneath the vim costume, working the way emacs's region model works.

Why the layers do not collide

The natural worry is collision: if both vim and emacs keys are live, do they fight? They do not, and the reason is that they occupy different key *spaces*. Vim's grammar lives in the plain letter keys and their **g/z/[** prefixes; the emacs layer lives in the Meta-modified keys and a few Control keys that vim leaves unused in Normal mode. **d** is vim's delete operator; **A-d** is emacs's kill-word; they share a letter but not a chord, so a hand trained on either never accidentally invokes the other. Insert mode is where the emacs keys earn the most use, because that is where a typist most wants to edit without reaching for the arrow keys — Chapter 4 covered that set (**C-w**, **C-u**, **A-f**, **A-b**, **A-d**, the **C-x** completion submodule, the **C-g u** undo checkpoint). The result is that a person who thinks in emacs can drive zmax in long stretches without consciously switching to vim thinking, and a person who thinks in vim never trips over the emacs keys they are not using. Two traditions, one engine, no referee needed — because the engine underneath was a shape both traditions were always describing.

Chapter 10 – The Spacemacs Leader

The third tradition zmax absorbs is Spacemacs, and what it contributes is not a set of motions but a *philosophy of discovery*: the idea that the editor’s whole command surface should hang off one easy key — the space bar — organized into a labeled, self-revealing tree, so that you can find a command by walking the menus rather than by remembering a chord. Where the vim grammar of Chapters 6 and 7 is the fast surface, optimized for hands that already know it, the leader is the *discoverable* surface, optimized for the command you use once a week and cannot quite recall. Both are present at once, and you move between them without thinking.

The leader tree

In Normal mode, `SPC` — the space bar — is the leader, and pressing it opens the top of a tree. `SPC SPC` runs the command palette, the same name-a-command prompt that `M-x` opens, so the leader’s first branch is “everything, searchable.” Below that, each letter opens a themed submenu, and the set is broad: `SPC f` for file operations (open, save, the file picker, the file explorer), `SPC b` for buffers, `SPC w` for windows (the same operations as the `C-w` prefix, reachable the leader way), `SPC s` for search across the project, `SPC j` for jumps, `SPC g` for git, `SPC e` for errors and diagnostics, `SPC c` for comments, `SPC l` for the language server (rename, code action, hover, symbols), `SPC h` for help, `SPC T` for themes, `SPC R` for run-and-debug configurations. There are more than thirty such submenus, and they nest: `SPC g` opens a git menu whose own keys open further choices. The leader is, in effect, the entire editor laid out as a navigable outline.

Why it is discoverable

The leader would be just another prefix if you had to memorize it, but you do not, because of one default: `auto-info` is on, and for the leader specifically `auto-info-leader-only` focuses that help where it matters most. When you press `SPC` and pause, a panel appears listing the branches — `f` → `file`, `b` → `buffer`, `g` → `git` — and as you descend, the panel refreshes to show the next level’s choices. You are never guessing. This is the Spacemacs insight in full: a menu that shows itself turns “I know there is a command for this somewhere” into a ten-second walk down a labeled tree, with no manual and no memorization. The config can exclude certain keys from the auto-info popup (`auto-info-exclude` defaults to the busy prefixes `g`, `y`, `z`, `d`, where a popup on every press would be noise), so the help appears where it aids and stays quiet where it would nag.

The deeper point is about how the three traditions divide the labor. The vim grammar is for the operations you do constantly, where a popup would only slow you down and muscle memory is everything. The leader is for the operations you do occasionally, where discovery beats memorization and a half-second menu is a bargain. The emacs keys are for the typist who wants to edit text without leaving the home row. None of the three is “the” interface; each is the right interface for a different frequency of use, and zmax’s wager is that an expert wants all three live at once rather than being made to pick one. The space bar is where that wager is most visible: the most beginner-friendly surface in the editor sits one key away from the most expert-only one, and they never get in each other’s way.

Chapter 11 — The Command Line

Pressing `:` drops you onto the command line, and this is the editor's third input surface after the keymap and the leader — the precise, typed, scriptable one. Where a keybinding is fast and a leader menu is discoverable, a typed command is *exact*: it takes arguments, it accepts ranges, it can be spelled out in full or abbreviated to a letter, and — crucially for a book whose fourth movement is about embedded languages — it is the surface through which you will eventually *eval* a tongue. This chapter covers the command line as it stands: the typable-command system, and the deep vein of vim ex-command compatibility zmax mines into it.

The typable-command system

Every `:` command is a `TypableCommand`, a record with a name, a set of aliases, a documentation string, a completer, and a signature describing how many positional arguments it takes. There are more than 340 of them, registered in a single list in `zmax-term/src/commands/typed.rs` and looked up by name or alias when you press Return — and that count has grown fast, because a great many are the selection transforms of Chapter 57. The fundamentals are the vim spellings: `:write (:w)` saves, `:quit (:q)` closes a window, `:write-quit (:wq)` does both, `:write-quit-all (:wqa, :xa)` saves and quits everything, `:exit (:x)` writes-if-modified and quits. File and buffer management — `:open/:e`, `:new/:n`, the `:buffer-*` family of Chapter 8 — is here, as are the configuration commands: `:set-option` and `:toggle-option` change editor settings live, `:theme` loads a color scheme, `:config-open` opens your `config.toml` for editing. Utilities round it out: `:sort` and `:unique` operate on lines, `:format (:fmt)` reformats through the language server, `:character-info` reports the character under the cursor.

Because each command carries a completer and a signature, the command line is not a bare text box: it completes filenames where a path is expected, theme names where a theme is expected, and so on, and it knows how many arguments a command will accept. The aliases mean you can type the full self-documenting name when you are learning and the one-letter vim form when you are fast, from the same prompt, with no mode to switch.

Vim ex-commands, parsed specially

The part that makes the command line feel like home to a vim user is that zmax special-cases vim's ex-command syntax rather than forcing everything through the tokenizer. A bare number jumps: `:42` goes to line 42, and `:42:10` to line 42, column 10. The substitute command is the real one — `:s/pattern/replacement/flags` — parsed as vim parses it, not split on slashes by a generic argument splitter, so the regex and the flags survive intact. The global command `:g/pattern/command` runs a command on every line matching a pattern, vim's most powerful ex-command, and it is here. Line-moving commands are here in vim spelling: `:m5` moves the current line to after line 5, `:t.` (and `:co`) copy a line to a destination, `:>` and `:<` indent and outdent a range. And the shell escape is vim's: `!:cmd` runs a shell command — no space required after the bang — with the existing shell-out machinery behind it, the same machinery that, the book will argue in Movement IV, is one short step from running not an external `sed` but an *internal* AWK or stryke over your buffer without leaving the process at all.

That forward reference is deliberate, because the command line is where the five embedded tongues will most naturally surface. An editor that already accepts `:s/.../.../` and `:g/.../...` and `!:...` as first-class typed commands has the exact shape a language *eval* wants: a prompt that takes a line of code and runs it against the live editor. When Movement IV makes the embedding concrete, the command line is the door it walks through —

which is why this chapter, the last of the keymap movement, ends by pointing at the next one. The three input surfaces — keymap, leader, command line — are complete; what remains is to make the command line able to speak five languages.

Chapter 12 – Pickers and Fuzzy Finding

Movement III turns from the keys to the working editor — the daily tools that make zmax a place to get real work done — and it starts with the one you reach for most: the picker. A picker is a fuzzy-finding list overlay: a prompt at the top, a filtered list below, a live preview beside it, and a matcher fast enough that the list narrows as fast as you can type. It is how you open files, jump between buffers, find a symbol, and walk diagnostics, and once it is in your hands the file tree and the tab bar of other editors start to feel like the slow way.

The matcher

Under every picker is `nucleo`, a fuzzy matcher built by the Helix author and shared with the wider Rust ecosystem. It scores each candidate against your query the way a good fuzzy finder does — subsequence matching, with bonuses for matches at word boundaries and path separators — and it does so across every item on a background thread, so that typing into a picker over a repository with tens of thousands of files stays responsive. The matcher is the reason the picker can be the primary navigation tool rather than a convenience: it is fast enough that “open the file called roughly *config something*” is a three-keystroke thought, not a trip through a directory tree.

The pickers you live in

The file picker is the front door, bound under the leader at `SPC f` and to a direct key, and it lists the files in your workspace for fuzzy selection. Its behavior is configured under `[editor.file-picker]`, and the defaults are chosen for a developer in a git repository: `hidden = true` skips dotfiles, `git-ignore`, `git-global`, and `git-exclude` all default on so that `.gitignore` build output never clutters the list, `follow-symlinks` and `deduplicate-links` keep symlinked trees sane, and `max-depth` is unbounded by default but available when you want to cap a deep tree. The file *explorer* — a tree view, configured separately under `[editor.file-explorer]` — is there for the times you genuinely want to see the shape of a directory rather than fuzzy-find within it.

Beyond files, the same picker mechanism powers navigation everywhere. The buffer picker lists open buffers, the answer to “I have twenty files open and want the one from ten minutes ago.” The symbol picker lists the symbols in the current document (and, in its workspace variant, across the project), powered by the language server, so jumping to a function by name is a fuzzy-find rather than a search. The diagnostics picker lists every error and warning for selection. There are pickers for commands (the palette of Chapters 9 and 10 is itself a picker), for themes, for the git commit log. The interaction is uniform across all of them — type to filter, move with the same keys, press Return to choose — so learning the picker once teaches you to drive all of them.

How a picker sits on the screen

Mechanically, a picker is a layer in the compositor — the stack of UI components Movement V describes — pushed on top of the editor view while it is open and popped when you choose or cancel. That is why it floats over your work without disturbing it, why a live preview can render the file under the cursor in a pane beside the list, and why pressing `Esc` returns you exactly where you were: the picker never touched the editor state underneath; it was a transient layer on top, asking one question. This is the same compositor that draws the auto-info menus of Chapter 10 and the completion popups of Chapter 4, and seeing that they are all the same

mechanism — transient layers over a stable editor — is the first hint of the architecture Movement V makes explicit.

Chapter 13 – Tree-sitter in Daily Use

zmax parses your code, continuously, with tree-sitter, and keeps a real syntax tree of the buffer up to date as you type. Movement V explains how that machinery works; this chapter is about what it *buys you* at the keyboard, because the syntax tree is not just for coloring text — it is the substrate for a whole class of edits that know about the structure of code rather than the position of characters. Once an editor genuinely understands that a function is a function and an argument is an argument, motions and selections can speak in those terms, and the edits get dramatically more precise.

Text objects

The clearest payoff is text objects: selections defined by syntax rather than by distance. Each grammar ships a `textobjects.scm` query that labels regions of the tree — function, class, parameter, comment, and more — and zmax exposes them as things you can select and operate on. “Select this whole function,” “select the argument the cursor is in,” “select the body of this loop” become single operations, and because the selection is the universal currency of the engine, any operator applies to them: select the function and delete it, change it, indent it, yank it. This is the structural editing that makes a syntax-aware editor feel qualitatively different from a line-and-character one — you stop counting lines to delete a function and just select the function.

It is worth being precise about the seam here, because Chapter 5 was: the *vim* spelling of these — `ciw`, `di{`, `va{`, operator-plus-text-object — now works, wired through the next-key mechanism of Chapter 5 that lets `ci` read the following `w` or `{` as a text-object specifier. The objects are reachable both ways: from Select mode (select the object, then operate) and through the operator directly (`ci{`, `daw`, `yip`, `dit`), the chord a vim user’s hands assume. What decides how far `iw` or `i{` or `it` reaches is the same tree-sitter and matched-pair logic described here — the keymap only chooses when to invoke it.

There is a faster way to reach them than naming each one. Press `<ret>` — Enter, in normal mode — and zmax selects the smallest text object around the cursor; press it again and the selection grows to the next object out, the word becoming the expression, the expression the argument, the argument the call, the call the function, each Enter climbing one rung of the syntax tree. `<backspace>` walks back down, and a number before Enter jumps straight to the Nth level. This is the *wildfire* idiom, ported from vim: the structural counterpart to extending a selection by characters. You need not decide in advance whether you want the parentheses or the braces or the whole block — you press Enter until the highlight is the shape you meant, then operate. It is text objects without the vocabulary, the tree made navigable by a single key.

Surround, brackets, and injections

Surround operations — adding, changing, and deleting the delimiters around a selection — are built in: wrap a selection in quotes or brackets, change the parentheses around an expression to braces, delete the pair surrounding the cursor. Bracket matching is syntax-aware rather than character-counting, so it pairs the right delimiters even through nesting and strings. Rainbow brackets — colored by nesting depth so you can see which paren closes which — are available, gated behind `rainbow-brackets` in the config and driven by each grammar’s `rainbows.scm` query, off by default and a single setting to enable.

The most quietly impressive use of the syntax tree is *injection*: a grammar’s `injections.scm` query marks regions that are written in another language — the SQL inside a string literal, the Rust inside a Markdown code fence, the CSS inside an HTML `<style>` — and zmax parses and highlights those regions with the *other*

language's grammar, nested inside the first. A Markdown file's code blocks light up in their real languages; a shell heredoc of SQL is colored as SQL. This is the kind of thing that is invisible when it works and conspicuous by its absence elsewhere, and it falls directly out of having a real tree rather than a line-oriented highlighter. Indentation, too, is syntax-driven: each grammar's `indents.scm` tells the editor how a new line should be indented given where it sits in the tree, so pressing Return inside a nested block lands the cursor at the right depth without a language-specific indent hack. All of it — objects, surround, brackets, injection, indent — is the same syntax tree, queried different ways, and Movement V is where the tree itself is finally taken apart.

Chapter 14 — Language Servers

The Language Server Protocol is how a modern editor knows what your code *means* rather than merely how it looks, and `zmax` is an LSP client built in, not added. Where the syntax tree of Chapter 13 understands structure, the language server understands semantics — types, definitions, references, errors — by running the same analysis engine your compiler does, as a separate program the editor talks to. This chapter is the LSP from the user’s chair: the keys that drive it, the capabilities behind them, and how servers are configured.

The capabilities and their keys

The protocol’s surface is wide, and `zmax` wires the breadth of it; the source enumerates the features as a `LanguageServerFeature` enum with around twenty variants — `GotoDefinition`, `GotoDeclaration`, `GotoTypeDefinition`, `GotoReference`, `GotoImplementation`, `Hover`, `SignatureHelp`, `DocumentHighlight`, `Completion`, `CodeAction`, `Format`, `Diagnostics`, `PullDiagnostics`, `RenameSymbol`, `InlayHints`, `DocumentColors`, `DocumentSymbols`, `WorkspaceSymbols`, `CallHierarchy`, and more. Each maps to keys you have already met. The gotos are on the `g` prefix: `gd` to definition, `gD` to declaration, `gy` to type definition, `gr` to references, `gi` to implementation — the Neovim-trained hand finds them unchanged. `K` shows hover documentation for the symbol under the cursor. Diagnostics navigate on the unpaired brackets, `[d` and `]d`, and gather in the diagnostics picker. The leader’s `SPC l` opens the language-server menu — rename, code action, hover, symbols — and the common ones have direct keys too: code actions, `SPC a`; rename, `SPC r`; document and workspace symbols, `SPC s` and `SPC S`. Formatting is `:format` or `gg`. Signature help pops automatically as you type a call; inlay hints (type annotations rendered inline) are available, off by default behind `display-inlay-hints`.

The behavior of all this is tuned under `[editor.lsp]`: `enable` turns the whole client on or off, `display-messages` and `display-progress-messages` control how chatty servers are, `auto-signature-help` toggles the automatic parameter hints, `display-inlay-hints` and `auto-document-highlight` toggle the heavier visual features, `snippets` enables snippet expansion in completions. The defaults assume you want the help and let you dial back the noise.

Configuring servers

Servers are declared in `languages.toml`, the same file that configures languages, in `[language-server.NAME]` blocks. Each takes a `command` and `args` to launch the server, an `environment` table, a `config` table merged into the server’s initialization options, a `timeout`, and `required-root-patterns` that say which project markers must be present for the server to apply. A language then lists the servers it uses in its `language-servers` array, and that array is more expressive than a list of names: an entry can be a name with `only-features` or `except-features`, so you can run `rust-analyzer` for everything and a second server for just diagnostics, or split features across cooperating servers. A document can have several language servers at once, each tagged so their diagnostics and results stay attributed. The shipped `languages.toml` names over two hundred distinct servers across roughly three hundred and fifty languages — the generated `lang-support.md` is the current tally — so for most languages the right server is already wired and the editor just needs the server binary on your path.

A deliberate boundary, named

It is worth marking a contrast the book will sharpen in Movement IV. A language server is an *external process*: zmax's `zmax-lsp` crate spawns it as a child, speaks JSON-RPC to it over pipes, and converts editor positions to and from the server's offset encoding across that boundary. That subprocess-and-protocol design is exactly right for language servers, because a language server is somebody else's compiler, written in whatever language they chose, shipped on its own schedule — it *has* to live behind a process boundary. Hold that picture, because it is the precise opposite of how the five embedded tongues work. The language server is the canonical example of the seam that Movement IV's languages do *not* have: zmax talks to `rust-analyzer` across a process and a protocol, and it talks to its embedded elisp across a function call. Same editor, two integration strategies, each chosen for what it integrates — and seeing why an LSP *should* be a subprocess is the cleanest way to appreciate why an extension language should not.

Chapter 15 – The Debugger

If LSP is the protocol for understanding code at rest, the Debug Adapter Protocol is its mirror for code in motion: a standard way for an editor to drive a debugger — set breakpoints, step, inspect variables — without knowing the debugger’s private interface. zmax ships a DAP client, `zmax-dap`, built on the same pattern as its LSP client, and this chapter is debugging from the editor: how it is configured, how you drive it, and an honest note on its maturity.

Adapters and templates

Debuggers, like language servers, are external programs, and they are configured in `languages.toml` — but per language, in a `[language.debugger]` block. The block names the adapter (`name`), how to reach it (`transport`, `command`, `args`, and an optional `port-arg`), and then a list of `templates`, each describing one way to start a debug session. A template has a `name`, a `request` of either `launch` (start a fresh process under the debugger) or `attach` (connect to a running one), a `completion` spec describing what the editor should prompt you for — a filename, a process id — and an `args` table of the DAP launch arguments, with `{0}`, `{1}` placeholders filled from your answers to the prompts. Rust and C and C++ are wired to `lldb-dap` with templates for running a binary, running it in a terminal, and attaching; Go is wired to Delve. The template system means “debug this” is a menu of pre-built session shapes rather than a JSON launch configuration you hand-edit.

Driving a session

Breakpoints toggle on a line and show in the gutter; the `dap_*` commands — `dap_launch`, `dap_toggle_breakpoint`, `dap_continue`, and the stepping commands — drive the session, and the leader gathers them under `SPC R`, the run-and-debug menu, alongside the configured templates. When execution stops at a breakpoint, the adapter reports the stop and the editor builds its view of threads, stack frames, and variables from the DAP event stream. The protocol is request-and-response rather than the streaming-notification style of LSP, which shapes the client: zmax asks for a stack trace when execution suspends rather than receiving a continuous feed.

Honesty, again, because the book promised it: debugging is among the younger parts of zmax, inherited from Helix’s own DAP support and carrying its rough edges. The core loop — configure an adapter, set breakpoints, launch, step, inspect — is present and real, and for the languages with shipped templates it works without ceremony; but it is less polished than the editing core, and a reader who lives in a heavyweight IDE debugger should calibrate expectations to “capable and improving” rather than “feature-complete.” For the common case of dropping a breakpoint in a Rust or Go program and stepping through it from the same editor you wrote it in, without launching a separate tool, it earns its place in the working-editor movement.

Inside the client: sequence numbers and back-channels

The maturity note above is about the user-facing loop. The transport underneath it is a different matter, and reading `zmax-dap/src/transport.rs` is worth doing because DAP’s wire model differs from LSP’s in one way that shapes everything built on it.

The wire carries three things, and the port models them as one enum tagged by the protocol’s own `type` field:

```
pub enum Payload {
    Event(Event),
    Response(Response),
    Request(Request),
}
```

The presence of `Request` in that list — inbound, not just outbound — is the difference. A debug adapter may send *the editor* a request and wait for an answer. Reverse requests such as “run this program in a terminal for me” exist because the adapter cannot open a terminal and the editor can. A client that modelled the connection as request-out/response-in would have no place to put that message.

`Request` carries a field that does not go on the wire at all:

```
pub struct Request {
    #[serde(skip)]
    pub back_ch: Option<Sender<Result<Response>>>,
    pub seq: u64,
    pub command: String,
    pub arguments: Option<Value>,
}
```

`back_ch` is the reply channel for *this* request, attached to the request structure and skipped by `serde` so it never reaches the adapter. That is the whole correlation mechanism: the sender includes the channel it wants the answer on, the transport records it against the sequence number, and when the response arrives it is routed to that channel rather than to a general dispatch that would have to figure out whose it was.

`Response` carries `request_seq` rather than a `seq` of its own, and the comment in the struct explains why: “seq is omitted as unused and is not sent by some implementations.” That is a real-world tolerance note. The specification has the field; adapters in the wild sometimes omit it; and since nothing in the client consumed it, the port dropped it rather than making deserialization fail on adapters that are technically out of spec but otherwise fine. Parsing what tools actually send is a recurring theme in this book — the compilation-mode parser of Chapter 87 makes the same kind of concession — and it is the difference between an implementation of the standard and an implementation that works.

The registry: many adapters, one stream

`zmax-dap/src/registry.rs` owns the clients, and the type says how:

```
pub struct Registry {
    inner: SlotMap<DebugAdapterId, Client>,
    current_client_id: Option<DebugAdapterId>,
    pub incoming: SelectAll<UnboundedReceiverStream<(DebugAdapterId, Payload)>>,
}
```

A `SlotMap` rather than a `Vec` or a `HashMap`, because clients are created and destroyed over a session and an index into a `Vec` would dangle after a removal while a slot-map key is generational and cannot. The registry “holds the responsibility of managing the lifecycle of each client.”

`incoming` is the interesting field. Each client has its own message stream, and `SelectAll` merges them into one, with every message tagged by the adapter that sent it. The main loop therefore polls a single stream regardless of how many debug sessions are live, and the tag tells it whose message it is. Adding a second adapter adds a stream to the merge and changes no consumer.

`current_client_id` is the concession, and the source marks it as one:

```
/// TODO: You can have multiple active debuggers, so the concept of a single
/// active debugger may need to be changed
```

The transport and the registry are already plural — `SlotMap`, `SelectAll`, tagged messages — and the

single-active-client field is the layer above them not yet having caught up. That is a much better place for the limitation to sit than the reverse would be. Growing to multiple simultaneous sessions is a change to which client the commands address, not a rewrite of the message plumbing.

`start_client` handles both connection styles from one entry point: a socket address, if given, produces a TCP client; otherwise the adapter is spawned as a process with its transport, command, arguments, and optional port argument taken from the `DebugAdapterConfig` that Chapter 15's `languages.toml` block describes. Both paths are driven through `block_on` inside `try_insert_with_key`, so the client is constructed with its own identity already known — the id it will be filed under is passed *into* its constructor, which is what lets the client tag its own outbound messages without a second registration step.

Reading these two files alongside the chapter's honesty note is the right calibration. What is young about zmax's debugging is the interface — the views, the polish, the number of pre-built templates. What is underneath is a protocol client with proper correlation, adapter-tagged multiplexing, and a lifecycle owner, which is the part that would be expensive to retrofit and is already there.

Chapter 16 — Version Control

Version control is the quiet integration — the one you notice only when it is missing — and zmax builds it in through the `zmax-vcs` crate. Today that means git, behind a feature flag, with the crate structured as a provider registry so that other systems could join it later; git is the only provider shipped, and it is the one nearly everyone needs. This is a short chapter because the integration is deliberately unobtrusive: it surfaces version control where it helps and stays out of the way otherwise.

The diff gutter and change motions

The most-used piece is the diff gutter. zmax asks git for the *base* version of the file you are editing — the committed content — diffs it against your working buffer, and renders the result as marks in the gutter: a line added, a line changed, a line removed. The gutter is one of the configurable gutter components (`[editor.gutters]`, where `Diff` sits alongside line numbers, diagnostics, and marks), so you can order or omit it. The diff is computed on a background task, the same fire-and-forget pattern the crate uses throughout, so a large repository never stalls the editor to compute a gutter.

Those changed regions are navigable as motions, which is where version control rejoins the keymap: `[c` and `]c`, from the unimpaired family of Chapter 7, jump to the previous and next change — the “review my own diff hunk by hunk” motion, without leaving the buffer. The version-control state can also surface in the status line: `VersionControl` is one of the status-line elements, so the current branch can ride in the corner of your screen.

Beyond the gutter

For history rather than diff, there are commit-log pickers — `:Commits` over the repository, `:BCommits` for the current file — and it is worth being precise about how they work, because they are the one place in this chapter that leaves the process. Both are fzf.vim ports and both run the external `fzf` binary rather than the in-process picker of Chapter 12: `queue_fzf_cmd` hands the TTY to `fzf` with `git log --oneline` piped to its stdin, and the preview flag is passed as `false`, so there is no diff shown while you scroll the list. What you pick is fed back to `fzf-git-show`, which runs `git show --stat -p <sha>` and drops the result into a scratch buffer. So: an external picker, no preview during selection, and a real diff after it — which is a reasonable design and not the one an earlier edition of this chapter described.

The status buffer

For staging and committing without leaving the editor, `:magit` (also `:git`, or `<space>g s` from the leader menu) opens a Magit-style status porcelain: a single buffer that lists the repository’s state in sections — staged, unstaged, untracked, stashes — and lets you act on it in place. You stage and unstage at the granularity that matters, *per hunk* rather than per file, expanding a changed file to send only the lines you mean into the index; you discard, you commit, you switch and create branches from a branch menu, you push and pop from a stash menu, and you can drive an interactive rebase — the full ceremony that an earlier draft of this chapter sent you to the terminal for, now a porcelain inside the editor. It is the one place zmax steps past the unobtrusive-gutter philosophy on purpose, because staging is the git operation you do most and the one most worth keeping under the same keys as everything else. For the commonest case — stage everything, commit, and push in

one move — `[-x v c (git_acp)` prompts for a message and runs the whole chain off-thread, confirming the pushed branch and commit in the status line, so a routine “save my work upstream” never needs the porcelain or the terminal at all.

Resolving conflicts

The other place git stops being quiet is a merge that does not apply cleanly, and zmax meets it with a dedicated resolver rather than leaving you to edit `<<<<<<` markers by hand. `:merge` (or `:resolve`, and `<space>g m / <space>g c r`) opens a three-pane view modeled on the JetBrains merge tool: *ours* on the left, the *result* you are building in the middle, *theirs* on the right, with the common *base* available as a fourth pane so a three-way conflict reads as two divergences from a shared ancestor rather than an opaque clash. Differences are highlighted down to the character, the panes scroll together horizontally for long lines, and you resolve each conflict by choosing a side — keep ours, keep theirs, keep both — either from the resolver or straight from the buffer with the `:conflict-ours` / `:conflict-theirs` / `:conflict-both` commands. The conflicts are navigable like any other change: `]n` and `[n` jump to the next and previous conflict marker, and `<space>g c 0 / <space>g c T` take *all* of one side at once when a merge is lopsided. The IDE layout surfaces the same state passively — its Git panel grows a Conflicts section listing every file still in conflict — so the editor tells you there is a merge to finish before you go looking for it.

None of this turns zmax into a full git client, and it does not try to be: the design bet has simply moved one notch. The needs that benefit most from being inline — see my changes, navigate them, stage them by hand, finish a merge — now live in the editor under the keys you already use, while the rarer ceremony still belongs to git proper.

Chapter 17 – Registers, Marks, and the Jumplist

This chapter collects three small systems that share a theme: remembering places and pieces of text so you can return to them or reuse them. None is large, but each is the kind of thing that, once in the hand, you use constantly without thinking — and each is a place where zmax’s vim spelling sits over the selection engine’s own notion of state.

Registers

A register is a named slot that holds text. In vim and in zmax, the double-quote key selects which register the next operation uses: `"a` before a yank puts the yanked text in register `a`, `"a` before a paste pastes from it. Without an explicit register, yanks and deletes flow through the default — configurable as `default-yank-register`, the unnamed `"` register by convention — and the system register for the OS clipboard is reachable too, so copying between zmax and other applications is a register choice rather than a special command. Because every yank and delete in this engine acts on a *selection* that may hold many ranges, a register can hold many pieces at once — yank a multi-cursor selection and the register carries each range’s text — which is multiple-clipboard behavior falling out of the selection model again, for free.

Marks

A mark is a named position. `m` followed by a letter drops a mark at the cursor; the backtick key followed by that letter returns to the exact spot, and the apostrophe returns to the start of the marked line — vim’s spelling exactly. Marks are for the deliberate “I will want to come back here” gesture: pin a spot, wander off to consult or edit elsewhere, jump back with two keystrokes. They are explicit and persistent within the session, the bookmark you place by hand.

The jumplist

Where marks are deliberate, the jumplist is automatic: zmax records the places you jump *from* as you move through a file and across files — a search, a goto-definition, a leap to the bottom of the buffer — so that you can retrace the path without having marked anything. `C-o` jumps backward through that history and `C-i` (which the terminal sees as Tab) jumps forward, vim’s jumplist keys unchanged. The distinction between the two systems is worth keeping: the jumplist answers “take me back where I just was,” automatically, in order; marks answer “take me to the specific place I named,” on demand, by name. Together with registers they form the editor’s short-term memory — text you set aside, places you named, places you have been — and like most of zmax, the keys that drive them are the keys you already press, sitting over an engine that thinks in selections and positions rather than cursors and points. With that, the working editor is complete; Movement IV turns to the capability no other editor has.

Chapter 18 – The Five Tongues

Here is the claim the rest of this movement defends: `zmax` is the first editor in the history of the craft to embed five complete programming languages in its core. Not one extension language. Not a host language with a foreign-function escape hatch. Five full language runtimes — **elisp**, **vimscript**, **stryke**, **AWK**, and **Zsh** — compiled into the editor as native code, sharing its address space, reachable by a function call. This chapter is the inventory and the rationale: which five, why those, and what having them changes. The two chapters after it open the machine they share and the seam by which they reach the editor; the chapter after that walks each tongue. Start with the fact itself, because it is the kind of claim that should be checked rather than waved past.

The state of the art it surpasses

Every editor that matters has wrestled with the same problem: a text editor must be programmable, but in *what*. Emacs answered with `elisp` and built a civilization on it — but `elisp` is the *only* first-class language emacs has, and it runs in a bespoke C interpreter. Vim answered with `vimscript`, slow and idiosyncratic, and later let you reach Python or Ruby through a C ABI into a dynamic library. Neovim made Lua first-class — singular — and reaches it through a C FFI, with everything else living across an RPC channel in a separate process. VS Code put extensions in a separate Node process and talks to them over JSON-RPC. In every one of these the count of *embedded* languages — languages running in the editor’s own process, reached without a protocol or an ABI — is one, or zero. The plural has never been shipped.

`zmax` ships the plural. The reason it can, where forty years of editors could not, is the four words Chapter 1 kept returning to: it is all Rust. The five interpreters are Rust crates — `elisp`, `vim`, `stryke`, `awk`, `zsh` — and the editor is a Rust crate, and a Rust crate calls another Rust crate by linking it in and calling its functions. There is no boundary to bridge because there is no boundary. The editors above had to choose a protocol or an ABI precisely because their cores and their languages were written in different languages or run in different processes; `zmax` sidesteps the whole problem by making the editor and all five languages the same kind of thing, compiled together into one binary.

Why these five

The five are not arbitrary; they are the two languages an editor of this lineage owes and the three that an editor most *uses*. The owed pair is heritage. `zmax` is named for emacs, so it must speak `elisp` — the tongue in which emacs is programmable all the way down — and it wears vim’s keymap, so it should speak `vimscript`, the language of every `.vimrc` its users have ever written. Embedding those two is `zmax` keeping faith with both of its parents at once: configure it in the language of the editor whose keys you use, *or* the language of the editor whose name it bears.

The used three are the text-and-shell powerhouses: AWK, Zsh, and stryke. AWK and Zsh are the languages people already reach for when they have to *transform text* and drive a shell — the exact things an editor is for — and stryke is MenkeTechnologies’ own flagship, a Perl-5-compatible language (not Perl itself) that bills itself as the fastest dynamic language in the world for parallel work and carries the largest standard library of any of the five. An editor that embeds them is not just scriptable in the abstract; it can run a stryke substitution across your buffer, an AWK program over a selection’s worth of columns, a Zsh pipeline whose output replaces a region — with the language running *inside* the editor over the editor’s own text, no shell-out, no temp file, no process. The `!cmd` of Chapter 11 pipes a selection through an external program; the embedded tongues let the same gesture run an *internal* one, on the buffer, in the editor’s memory. The owed pair makes `zmax`

configurable in its heritage; the used trio makes it a text-processing instrument in the languages that own that job.

What the rest of the movement proves

A claim this large has to be cashed out in mechanism, and the next chapters do it. Chapter 19 opens `fusevm`, the single bytecode machine all five compile down to — the reason “five interpreters” is really “five front ends onto one engine.” Chapter 20 takes `elisp`, the tongue furthest along, and shows the embedding working end to end: how `eval` runs, how a script reaches the editor. Chapter 21 walks the other four and is honest about how far each has come, because — as the “early” badge has warned since Chapter 1 — this capability is being built out crate by crate as the book is written, and a reader deserves to know which tongues are speaking today and which are still clearing their throat. Chapter 22 is the architecture: the precise, slightly dangerous mechanism by which an in-process script gets its hands on the editor’s own state. The claim is the headline; the mechanism is the proof; and the mechanism, it turns out, is only possible because of a single shared machine.

Chapter 19 — fusevm: One VM to Run Them All

Five embedded languages could have been five unrelated interpreters that happen to be linked into the same binary. They are not, and the difference is the most important engineering fact in this movement: all five compile down to a single, shared, language-agnostic bytecode virtual machine called **fusevm**, whose own motto is “one VM to run them all.” Understanding fusevm is understanding why the five tongues are cheap rather than five times as expensive as one — and why an embedded elisp here can outrun the elisp in emacs.

A machine, not a language

fusevm is not tied to any source language. It is a stack-based bytecode machine with — at the time of writing — 224 opcodes organized across 21 sections, and its premise is that *any* language front end can lower its source to those opcodes and inherit, for free, everything the machine has been engineered to do well. What it has been engineered to do well is run fast: it ships eleven *fused superinstructions*, single opcodes that do the work of common opcode pairs so the hot path dispatches once where a naive bytecode would dispatch twice; it uses slot-indexed fast paths for local variables; and behind a feature flag it carries a *three-stage Cranelift JIT* — linear, then block, then tracing — that promotes hot code to native machine instructions in escalating tiers. A front-end author writes a lexer, a parser, and a lowering to fusevm opcodes, and in exchange the language gets fused dispatch, fast locals, and a native-code JIT that someone else built and tunes. The economy is the whole point: the expensive part of a fast language — the VM and the JIT — is written once and shared five ways.

Two of fusevm’s 21 opcode sections tell you exactly which languages it was built to host. It has, by count, twenty-nine first-class *shell* operations and 87 first-class *AWK* operations — pipelines, redirections, field splitting, the `$1/$2` field model — baked into the machine as opcodes rather than synthesized awkwardly on top of a general-purpose core. That is not an accident; it is fusevm being designed, from the metal up, to host a shell (`zshrs`) and an AWK (`awkrs`) as naturally as it hosts arithmetic. When `zshrs` lowers a pipeline or `awkrs` lowers a field reference, it is emitting a single opcode the VM understands natively, which is why those two notoriously special languages embed as cleanly as they do.

AOT machine, JIT’d eval

Chapter 1 drew the distinction; here it has a concrete home. The fusevm machine itself — its dispatch loop, its opcode handlers, its JIT — is Rust, compiled ahead of time by `cargo` into the `zmax` binary along with the five front ends. Nothing about the *engine* is built at runtime; there is no toolchain the user installs, no warm-up that compiles the interpreter. When you launch `zmax`, the machine that will run your elisp is already native code. The just-in-time compiler exists for the one thing that genuinely cannot be compiled in advance: code you type at runtime and `eval`. That code is lowered to fusevm bytecode on the spot and, as its loops get hot, Cranelift lifts them to native instructions through the three JIT stages. So the steady state is an AOT-native machine running JIT-native eval — interpreted in the slow tree-walking sense at no point — which is the technical reason an embedded tongue on fusevm can be *faster than its own reference implementation*, whose canonical interpreter (elisp’s C evaluator, vimscript’s tree-walker) never had a JIT at all.

The 21 sections, surveyed

The claim that fusevm is a *machine* and not a language is best felt by looking at how its opcodes are grouped, because the grouping is where a general-purpose core stops and a deliberately-hosted design begins. The 224 opcodes fall into 21 sections, and those sections divide cleanly into two kinds. The first kind is the furniture every bytecode machine has: an arithmetic and comparison section, so a front end never lowers `+` or `<` to anything but a single opcode; a control-flow section of jumps and conditional branches, the raw material a lowering builds `if`, `while`, and `for` out of; a call section for invoking functions and returning from them; a locals section that reads and writes the slot-indexed variables the fast path is built around; and a heap section for the objects a garbage-collected language allocates — the cons cells and strings and vectors that live behind handles rather than raw pointers, so that a moving or compacting collector can relocate them without a front end noticing. None of that is surprising; it is the cost of being a machine at all.

The second kind is the part that tells you what fusevm is *for*, and it is the reason the five-tongue claim is tractable rather than heroic. Two of the 21 sections are not general at all: one is a *shell* section of twenty-nine first-class operations — pipelines, redirections, job control, the machinery a shell language spends its whole existence expressing — and one is an *AWK* section of 87 first-class operations, field splitting, the `$1/$2/$NF` field model, the record loop, the associative arrays AWK leans on. A general-purpose VM would force `zshrs` and `awkrs` to *synthesize* those behaviors out of primitives, wrapping a foreign shape around a core that never anticipated it; fusevm instead answers a pipeline or a field reference with one native opcode, because the machine was designed from the metal up knowing a shell and an AWK would be lowered onto it. That is why the two most idiosyncratic of the five tongues — the two whose reference implementations are famously peculiar C — embed as cleanly as the arithmetic does. The lesson generalizes past those two: a language is cheap to add to fusevm exactly to the degree that fusevm already carries opcodes shaped like its primitives, and the sections are the ledger of which shapes it carries.

The three JIT stages, named

The three-stage Cranelift JIT deserves its stages named, because “three-stage” is not marketing granularity — it is three genuinely different compilers escalating on the same code as the machine learns how hot that code is. The first stage is **linear**: a straight-line translation of a bytecode run into native instructions, one opcode at a time, taking almost nothing on itself beyond skipping the dispatch loop. It is the cheapest to produce and the first to fire, and for code that runs a handful of times it is all the acceleration the path will ever need or deserve. The second stage is **block**: when a region proves warmer, fusevm recompiles it a basic block at a time, letting Cranelift see across the boundaries the linear stage respected — folding constants, keeping values in registers across a whole block instead of round-tripping them through slots, eliminating the redundant loads a per-opcode translation leaves behind. The third stage is **tracing**: for the genuinely hot loop, fusevm records the *trace* the execution actually takes — the specific path through the branches, the types the values actually held — and compiles that one path as straight-line native code with guards where reality might diverge from the recording, the technique that lets a dynamic language’s inner loop run at a speed a statically-typed compiler would recognize.

The escalation is the point, and it is why the JIT can be an unconditional win rather than a warm-up tax. Cold code is never compiled past linear, so a script that runs once pays nothing for machinery it will not use; only the loop that runs a million times earns the tracing compiler that makes it fly. And because all three stages live in fusevm and not in any front end, a stryke loop, an elisp loop, and an AWK record scan are promoted by the *same* three compilers — a tuning improvement to the tracing stage makes all five tongues’ hot loops faster in one commit, which is the coherence dividend of Chapter 19 paid out in the currency that matters most for a language: speed under load.

Why one machine matters for an editor

Sharing a machine buys more than speed; it buys *coherence*, and coherence is what makes five embedded languages tolerable rather than a maintenance catastrophe. One value representation underlies all five, so a string is the same string whether elisp or stryke made it. One JIT means a performance improvement to fusevm makes all five tongues faster at once. One machine means one place where the embedding hooks into the editor — the host-callback mechanism of Chapter 22 is defined against fusevm, not re-invented per language — so wiring the editor into the languages is wiring it into *the VM they share*. The five front ends are thin; the machine is the substance; and because the machine is one machine, the cost of the fifth language is a lexer and a lowering, not a fifth runtime. That is the architecture that turns “no editor has ever shipped five embedded languages” from a boast into a tractable engineering plan — and the next chapter shows the first of the five actually running.

Chapter 20 — elisp in zmax

Of the five tongues, elisp goes first, and not only because the editor is named for the one that elisp built. elisp goes first because it is the furthest along: its embedding crate, `elisprs`, already exposes exactly the surface an editor needs, so it is the language in which the whole architecture can be shown *working* rather than *planned*. This chapter is the proof of concept — what `elisprs` offers, how an elisp form gets evaluated against the live editor, and how the editor's own operations become callable from elisp.

`elisprs`, an elisp on fusevm

`elisprs` is an elisp implemented as a fusevm front end. That phrasing carries a design decision worth stating: `elisprs` holds no virtual machine and no JIT of its own — it lowers elisp to fusevm opcodes and lets the shared machine of Chapter 19 do the running, with elisp values living as fusevm values (heap objects reached through handles rather than raw pointers). So embedding `elisprs` into zmax is not embedding a second runtime beside fusevm; it is adding a front end to the fusevm that the editor already links. The elisp you run and the AWK you run are the same machine underneath, which is the coherence dividend Chapter 19 promised, collected here.

`eval`, the emacs gesture

The defining thing about emacs is that you can evaluate elisp against the running editor at any moment — `M-:` to type a form, `C-x C-e` to evaluate the one before the cursor — and the form acts on the live editor immediately. zmax offers the same gesture, and `elisprs` is built for it: its embedding API centers on an `eval_str` entry point that takes a string of elisp and runs it, against a host that *persists* between calls so that definitions you make in one evaluation are still there in the next. That persistence is what makes an editor's elisp a programmable environment rather than a calculator: you define a function once and call it for the rest of the session, exactly as in emacs. The command line of Chapter 11 is the natural front door — a typed command that hands its argument to `eval_str` — and a scratch buffer of elisp evaluated form by form is the natural workshop.

`defsubr`: the editor as elisp builtins

An eval that cannot touch the editor is a toy; the point is for elisp to *drive* zmax. `elisprs` provides the hook for exactly this: `defsubr`, which registers a native function as an elisp builtin under a name, with a minimum and maximum argument count and a Rust function to run. This is the mechanism by which the editor's own operations become elisp primitives. What zmax registers through it is worth being exact about, because the obvious guess is wrong. `ensure_builtins` in `zmax-term/src/commands/scripting/elisp.rs` installs twenty-two subrs, and they are all *editor-level*: the echo area (`editor-message`, `editor-error`), command dispatch (`editor-command`), buffer identity (`buffer-name`, `buffer-file-name`), the mark and region (`set-mark`, `push-mark`, `mark`, `region-beginning`, `region-end`, `region-active-p`, `use-region-p`, `deactivate-mark`, `mark-whole-buffer`), and the kill ring (`kill-new`, `kill-region`, `copy-region-as-kill`, `kill-ring-save`, `yank`, `current-kill`, `kill-line`, `kill-whole-line`).

`insert` is not on that list, and its absence is deliberate rather than unfinished. The source says so in a comment above the registrations: buffer-text builtins — `insert`, `point`, `goto-char`, `forward-line`, `search-forward`, `looking-at` — are *elisprs's own* subrs, and if zmax overrode them “the two would fight over point.” So (`insert`

"hello") in zmax runs elisprs's own insert against elisprs's own `EditBuffer`, not a zmax function against the live rope. What makes it nonetheless land in the buffer you are looking at is the subject of the next paragraph, and it is a mirror rather than a shared pointer.

Above the subrs sits a second layer written in elisp itself. The `EDITOR_LISP` string in the same file defines `find-file`, `save-buffer`, `write-file`, `kill-buffer`, `switch-to-buffer`, `goto-line`, `split-window-below`, `message` and a dozen more as ordinary `defuns` that route through `editor-command` to zmax's typable commands. So "open a file from elisp" does work, and works under the emacs name — it is just a defun over a subr rather than a subr of its own.

The subr callbacks themselves are bare `fn` pointers: `h.defsubr("editor-message", 1, Some(1), b_message)` hands over a plain function item, not a closure. They can be, because they do not capture the editor — Chapter 22 is devoted to how they reach it instead, through a thread-local pointer that the evaluation publishes before it begins. The `defsubr` callback is a bare function pointer because it does not need to be anything heavier — it runs in the editor's own process, on the editor's own thread, with the editor's own state a dereference away. `elisprs` is first among the five because this surface — `eval_str`, a persistent host, `defsubr` for host functions — is already clean and present, so the editor can be wired to it today while the other four tongues grow the matching surface. It is the language in which "five embedded tongues" stops being a sentence in a pitch and becomes a REPL you can type into.

Chapter 21 – The Other Four Tongues

elisp proved the shape; the other four fill it out, each bringing something elisp does not and each at a different stage of readiness. This chapter walks vimscript, stryke, AWK, and Zsh in turn — what each is *for* as an embedded tongue, what its crate offers today, and what it still needs — and it is the most forward-looking chapter in the book, because the wiring of these four is in active construction as this is written. The book’s standing promise applies in full here: where something is a stub or a gap, it is named as one.

vimscript – configure it in its own keymap’s language

zmax wears vim’s keys, so vimscript closes a loop: configure the editor in the language of the `.vimrc` its users already have. The crate is `vimlrs`, a path dependency at `vendor/vimlrs` (there is no separate `zmax-viml` crate), and it can evaluate expressions and source scripts — `eval_expr` and `eval_source`, with persistent globals across calls.

Earlier editions of this chapter described the host side as the largest gap of the four, with `:map`, `:command`, `:set` and `:autocmd` present only as stubs. That is no longer the state of the tree. `install_viml_host_hooks` in `zmax-term/src/commands/scripting/mod.rs` installs a full set: an `EditorHost` carrying `line_count`, `get_line`, `set_lines`, `cursor`, `set_cursor` and `buf_name` onto the `api_*` surface, so `getline/setline/line()/col()/cursor()` read and write the live buffer; a `set_hook` that reruns each `:set` through zmax’s own `:set`; a `map_hook` that records `:map/:nmap/:nnoremap` lines into the runtime keymap overlay and asks the application to merge them onto `config.keys`; an `excmd_hook` that claims any line whose first word is a known zmax typable command and dispatches it — which is how `:command` arrives, backed by a real `USER_COMMANDS` registry with `:delcommand` and `:comclear` beside it in `zmax-term/src/commands/typed.rs`; and `highlight/colorscheme` hooks that rebuild the zmax theme from vimlrs’s highlight registry.

One exception survives, and the source names it: `:autocmd` has no vimlrs host hook. Autocommands work in zmax — there is a native registry, `vim_autocmd`, with `:augroup` support — but they are a parallel implementation rather than something a sourced vimscript reaches through. That is the remaining seam, and it is a much narrower one than the chapter used to describe.

stryke – the flagship power language

stryke arrives as `strykelang`, MenkeTechnologies’ own language and a major product in its own right (with its own companion book). It is *not* Perl — it is a Perl-5-compatible interpreter in Rust that bills itself as the fastest dynamic language in the world for parallel work, second only to LuaJIT single-threaded and beating it on several benchmarks, with NaN-boxed values, a bytecode VM and Cranelift JIT, rayon parallelism across all cores, and the largest standard library of any of the five tongues. As an embedded tongue its draw is exactly that: raw text-processing power and an enormous builtin shelf, a stryke transformation run over your buffer, in process. The crate exposes a `VMHelper` and a `parse_and_run_string`, and the eval half is wired: `zmax-term/src/commands/scripting/stryke.rs` keeps a thread-local `VMHelper` so globals and subs survive across `:stryke` calls, runs the source, and renders the program’s last expression.

stryke is the one tongue of the five whose host side is still genuinely open, and its own module doc is the place that says so. It has no native host-function registration of the elisp `defsubst` kind — the path from stryke out to native code remains its `rust{}` blocks, compiled by `rustc`, rather than a table of editor primitives — so

the `api_*` surface every other tongue drives is not exposed to it. `print` is a second, smaller casualty: stryke routes it through its embedded-zsh fd layer, which the editor's own fd redirect cannot intercept, so `print` is suppressed to keep the TUI clean and capturing it needs an upstream output-sink hook. Both are upstream VM work rather than editor work. Where the other four now *drive* the editor, stryke computes in it.

AWK — a column processor over a selection

AWK arrives as `awkrs`, and its embedded purpose is the one AWK was born for: run a program over text organized in columns — a selection's worth of tabular data — and get the transformed text back. fusevm's first-class AWK opcodes (Chapter 19) mean `awkrs` lowers especially cleanly. This chapter used to describe the library-shaped door as missing; it exists. `awkrs::run_program(program, input)` is the entry point, and `zmax-term/src/commands/scripting/awk.rs` is eleven lines of wrapper around it: a fresh awk runtime per call, no process stdio touched, `print/printf` output captured and returned. The orchestration that reads the region and writes the result back is `run_awk_filter` in the parent module. AWK's embedded role is the stateless one — a text filter over a region — which is why it needs the least host surface of the five.

Zsh — a pipeline whose output replaces a region

Zsh arrives as `zshrs`, and its embedded purpose is to run a shell pipeline *inside* the editor — the `:!cmd` of Chapter 11, but with no external process, the shell itself embedded. The crate has a `ShellExecutor` and an `execute_script`, and the output capture this chapter once listed as the gap is closed. `zsh.rs` keeps the executor in a thread-local so variables, functions and the working directory survive across `:zsh` calls, and wraps each run in `capture::with_captured_fds`, which redirects the real file descriptors for the duration and hands back `(exit status, captured output)` — the fd-level route, since an embedded shell's children write to descriptors and not to a Rust `Write`. “Replace this region with the output of this Zsh pipeline” works without leaving the process.

One machine, one version

A cross-cutting note ties the four back to Chapter 19. Because all five tongues ride the one fusevm, they must ride the *same* fusevm — and they do. Every one of the five `vendor/*/Cargo.toml` files pins the identical version, differing only in which features they turn on (`awkrs` and `vimlrs` ask for `jit`; all five ask for `jit-disk-cache` and `aot`). Earlier editions of this chapter described the pins as scattered and named the unification as pending work; it landed, and what is worth carrying forward is not the version number — which will move — but the constraint that produced it. A shared machine is only a shared machine if everyone links the same one, so a single crate drifting to a different fusevm major does not merely duplicate code, it means two bytecode machines in one binary and the end of the claim this movement is built on. The pin is checkable in one command, and any future divergence is a build break rather than a silent regression.

Chapter 22 – The Embedding Architecture

Twice now the book has deferred the central mechanism to this chapter: how an in-process script gets its hands on the editor's own state. The answer is precise, a little dangerous, and forced — not chosen for elegance but dictated by the shape of the crates being embedded. This chapter states it plainly, defends why it is sound despite looking unsafe, and connects it to the no-boundary thesis that opened the movement. It is the engine room of the whole five-tongue claim.

What the crates force

Begin with the constraint, because the design is downstream of it. The interpreters were written to be embedded, and the way they let a host reach back into native code has a shared property: their host state is *thread-local*, and the callback itself carries no context. Sometimes that is literal — elisp's `defsubst` takes a plain `fn` item, as in `h.defsubst("editor-message", 1, Some(1), b_message)`, an address to jump to and nothing more. Sometimes the callback is heavier in type but no richer in content: vimlrs's `EditorHost` fields are `Box<dyn Fn>`, and `zmax` fills them with closures that capture nothing (`Box::new(|| api_line_count().unwrap_or(1))`). Either way the consequence for the editor is the same and it is sharp: the callback cannot carry the editor with it. When `fusevm` calls the registered `editor-message` function, that function receives the *language's* arguments — the string to show — and nothing else. So how does it reach the editor it is supposed to mutate?

The thread-local context pointer

It reaches the editor through a thread-local raw pointer that the evaluation installs before it begins and takes down when it ends. What is published is not an `&mut Editor` directly but the active command's `&mut compositor::Context` — the wider handle a typable command runs under, from which `cx.editor` is one field away — and it is stored type-erased, as a `Cell<*mut ()>` cast back to `*mut compositor::Context` on the way out. The real declaration, in `zmax-term/src/commands/scripting/mod.rs`, is four lines:

```
thread_local! {
    /// Type-erased pointer to the `compositor::Context` of the in-flight eval.
    static CX_PTR: Cell<*mut ()> = const { Cell::new(ptr::null_mut()) };
}
```

The store is wrapped in an RAI guard, and here the detail that matters is one the book got wrong in earlier editions. The guard does not *clear* the slot on the way out; it *restores* whatever was there before:

```

impl CxGuard {
    fn new(cx: &mut compositor::Context) -> Self {
        let prev = CX_PTR.with(|c| c.get());
        CX_PTR.with(|c| c.set(cx as *mut compositor::Context as *mut ()));
        CxGuard { prev }
    }
}

impl Drop for CxGuard {
    fn drop(&mut self) {
        CX_PTR.with(|c| c.set(self.prev));
    }
}

```

Save-and-restore rather than set-and-clear is what makes nested evaluation sound — a script that triggers a second eval leaves the outer eval's context intact underneath it, and unwinding one level restores one level. Because the slot starts null, restoring past the outermost guard puts it back to null, so the “outside an eval there is no pointer to misuse” property still holds; it is a consequence of the stack discipline rather than a special case.

Reading it back is `with_cx`, which is where the null check lives: a callback that fires with no eval in flight gets `Err("editor API called with no active context")` rather than a dereference of null. Above `with_cx` sit nine `api_*` helpers — `api_message`, `api_error`, `api_command`, `api_line_count`, `api_get_line`, `api_cursor`, `api_set_cursor`, `api_set_lines`, `api_buf_name` — each returning `Result<_, String>` so that any language can surface a failure as its own error type. That set is the entire language-agnostic editor API, and every binding in the movement is a marshalling layer over those nine functions.

The lifecycle of one eval, traced

It is worth walking a single evaluation from keystroke to editor change, slowly, because the whole soundness argument lives in the *order* of the steps and the order is easy to take on faith without seeing. Suppose you type `:elisp (message "hi")` and press Enter. The editor is, at that instant, inside a command handler, holding the one `&mut compositor::Context` the main loop lent it — the same borrow every other command runs under. `eval_elisp` fires, and its first act is to construct the RAI guard: it coerces that `&mut` to a raw pointer and writes it into `CX_PTR`. That write is the only moment the borrow leaves the type system's sight; from here until the guard drops, the editor is reachable two ways at once — through the `&mut` the command still nominally holds, and through the raw pointer in the thread-local — which is exactly the aliasing Rust forbids and exactly why the discipline that follows is load-bearing rather than decorative.

With the pointer installed, `eval_elisp` calls `elisprs's eval_str`. The interpreter lowers the form to fusevm opcodes and runs them on the shared machine. `message` is one of the elisp `defuns` in `EDITOR_LISP`, so it formats its arguments with `elisprs's` own `format` and then calls `editor-message`, which *is* a registered subr: fusevm invokes `b_message`, a bare `fn` that receives one argument — the string `"hi"` — and no editor. So it does the reciprocal move. It calls `api_message`, which calls `with_cx`, which reads `CX_PTR`, reconstitutes the `&mut compositor::Context`, and calls `cx.editor.set_status(...)` — the identical method any other command would call. The echo area changes and the subr returns a value up into fusevm. Crucially, while that subr held its reconstituted `&mut`, the command's original `&mut` was *not being touched* — the command is blocked in the `eval_str` call, doing nothing but waiting for it to return — so at no instant are two live mutable borrows actually dereferenced at once, even though two exist on paper. The aliasing is nominal, not dynamic, and that gap is the seam's whole safety.

`eval_str` returns; control unwinds back through the command; the guard goes out of scope and its destructor restores the previous pointer, which at the outermost level is null. From this point there is no pointer to misuse — a stray later read of the slot finds null and `with_cx` returns an error rather than dereferencing it — and the editor loop turns again holding the single `&mut` it always held. Trace the same path for a panic instead of a clean return and the guarantee holds better, not worse: if the elisp form errors and unwinds, the guard's

destructor still runs on the way out, still restores the slot, so an interpreter that blows up mid-eval cannot leave a dangling pointer behind for the next command to trip over. The RAI guard is doing in three lines what a `try/finally` does in a language with exceptions and what nothing does in a language with neither — it ties the pointer's validity to a stack frame, and a stack frame is unwound whether it returns or throws.

Buffer text takes a different road

`(message "hi")` is the clean case, and it would be dishonest to leave the impression that every elisp form reaches the editor that way. Buffer text does not. Consider `:elisp (insert "x")`. `insert` is not among the twenty-two subrs zmax registers, and Chapter 20 explained why: `insert`, `point`, `goto-char`, `forward-line`, `search-forward` and the rest are elisprs's own builtins, operating on elisprs's own `EditBuffer`, and overriding them would leave two implementations fighting over point.

So `eval_elisp` mirrors instead. Its body is five lines and they are the honest shape of the boundary:

```
let _guard = CxGuard::new(cx);
elisp::ensure_builtins();
elisp::ensure_editor_lisp();
elisprs::with_host(load_buffer_into_host);
let value = elisprs::eval_str(src)?;
elisprs::with_host(flush_host_into_buffer);
```

`load_buffer_into_host` reads the live rope, calls `to_string()` on it, collects that into the host buffer's `Vec<char>`, and sets point from the primary selection's cursor (mirroring an anchor-distinct-from-head selection into elisp's mark). `eval_str` then runs against that copy. `flush_host_into_buffer` collects the `Vec<char>` back into a `String`, compares it to the document, and if it differs replaces the *whole* buffer in one `Transaction` — deliberately not a minimal diff, which costs an eval's edits their individual undo steps and buys simplicity.

That is a marshallng boundary, and a whole-buffer one, crossed twice per eval. Earlier editions of this book claimed the opposite — that `(insert "x")` ran a registered Rust function against the live rope with no copy in either direction — and the claim was wrong. It is worth being precise about what survives the correction, because most of the thesis does. There is still no IPC, no serialization format, no FFI boundary, no second process, no plugin host; the copy is a `Rope` → `String` → `Vec<char>` conversion inside one address space on one thread, and the editor-level operations of the previous section genuinely do reach the live editor through a pointer with no copy at all. What is not true is the strongest form of the claim. zmax has *two* paths from a script to the editor: a direct one for editor operations, and a mirrored one for buffer text. The second is a real cost — it is linear in buffer size per eval, and it collapses an eval's edits into one undo step — and it was chosen to avoid a worse problem, two engines disagreeing about where point is. A design that trades a copy for coherence is a defensible design. It is not a design with no boundary, and the book should not have said it was.

Why this is sound, and why it is unsafe

Call the danger by its name: this involves a raw pointer to the editor and bare function pointers reaching it through a global-ish thread-local, and Rust's borrow checker is not watching that seam. The soundness argument is a discipline the editor holds to, and it has three parts. First, the editor is single-threaded for this purpose: eval runs *synchronously, on the editor's own thread, inside a command handler*, so there is exactly one `fmmt Editor` in existence and the thread-local points at it — there is no second thread to race and no second mutable borrow to alias. Second, the RAI guard bounds the pointer's validity to the eval's extent, so the thread-local is never read while it is stale; outside an eval there is no pointer to misuse. Third, the editor is not re-entered mid-eval — the eval runs to completion before the editor loop turns again — so the `fmmt` the callbacks reconstitute is never live at the same time as the one the loop holds. Under those three conditions the unsafe seam is sound: the pointer is always valid when read, never aliased, never stale. It is unsafe in the language's eyes and correct in the program's, and the gap between those two is exactly the price of integration

with no boundary.

That price is worth naming squarely, because it is the trade the whole movement rests on. The editors of Chapter 18 paid for their language integration in latency and marshalling — a process boundary, a protocol, an ABI — and got memory safety across that boundary for free, because a separate process cannot corrupt your heap. `zmax` makes the opposite trade: it pays in a small, carefully-bounded patch of `unsafe` at the host-callback seam, and gets in return an integration with no process boundary, no protocol and no ABI — a script that moves the cursor by calling the cursor-moving function. Not, as the previous section was careful to establish, an integration with *nothing* copied anywhere: buffer text crosses by mirror. The claim that survives is narrower and still unusual, namely that the copy is a conversion between two in-process representations rather than a message to another program. The language server of Chapter 14 sits on one side of that trade and the five tongues on the other, in the same editor, each placed where its trade makes sense: foreign programs behind a safe boundary, native tongues through a fast pointer.

Where it plugs into the editor

The seam needs anchors on the editor side, and they are the clean hook points the editor already has. The `TypableCommand` registry of Chapter 11 is where an `eval` command lives — a `:elisp`, a `:stryke`, a typed front door that funnels its argument through the thread-local guard into the right interpreter. The config-load moment in `main.rs`, where `zmax` reads its TOML before the main loop starts, is the natural place to run an *init script* — the `.zmax` answer to emacs's `init.el` — so a tongue can configure the editor at startup. The runtime-modifiable `KeyTrie` keymaps mean a script can *bind keys*, reaching the same structure the vim keymap of Movement II is built from. And the existing shell-out machinery is the precedent that the synchronous “run something, get a result, update the buffer” shape already exists and works. None of these hook points had to be invented for the languages; they are the editor's own command, config, keymap, and shell surfaces, and the embedding is the languages reaching through them by function call. That is the architecture in one sentence: five front ends onto one VM, reaching one editor through a thread-local pointer held for the length of a synchronous eval — no process, no protocol, no ABI, because it is all Rust, and Rust calls Rust by calling it.

Chapter 23 – The Rope

Movement V goes under the floorboards. The four movements before it described what `zmax` is to use; this one describes what it is made of, for the reader who wants to know how — or to hack on it. It begins where the editor begins, with the representation of the text itself, because every other mechanism in the editor is ultimately a way of reading or changing this one structure. The structure is a *rope*, and the choice of a rope over the obvious alternatives shapes everything above it.

Why not an array

The naive representation of a text buffer is a single contiguous array of bytes, and it is a fine choice right up until you edit the front of a large file, at which point inserting one character means shifting every byte after it — megabytes of memory copied to type one letter. Editors have escaped this in various ways (gap buffers, piece tables); `zmax` inherits Helix's choice of a *rope*, by way of the `ropey` library, re-exported at the top of `zmax-core`. A rope stores the text as a balanced tree whose leaves are small chunks of string, so a position is found by walking the tree in logarithmic time and an edit touches only the handful of nodes along one root-to-leaf path. Inserting at the front of a ten-megabyte file rebuilds a few tree nodes, not ten megabytes of array. The rope is why `zmax` stays responsive on files that would make a naive editor stutter.

Characters, not bytes

One property of the rope deserves emphasis because it pervades the whole codebase: positions are measured in *characters*, not bytes. The engine indexes the text by `char` offset, and `ropey` makes char-indexed access as cheap as byte-indexed access would be, which is what lets the selection model of Chapter 3 define its anchors and heads in chars without paying for the conversion. The dividend is that a multi-byte character — an accented letter, a CJK ideograph, an emoji — counts as one position, the cursor can never split it, and a selection boundary always falls between whole characters. Unicode correctness, which is a constant source of off-by-one pain in byte-oriented editors, is mostly a non-issue here because the foundational index is the character.

An edit is a transaction

The rope is, in the functional spirit, treated as nearly immutable: you do not poke new characters into it in place, you *apply a transaction* to it and get a new rope that shares most of its structure with the old one. The vocabulary lives in `zmax-core`: a `Change` is a single edit (replace this range with this text), a `ChangeSet` is a composed sequence of them, and a `Transaction` wraps a change set together with an optional new selection. Applying a transaction produces the next version of the document, and — this is the elegant part — the same change set knows how to *remap positions* through itself: every selection in the editor, and every mark and diagnostic anchored to a position, is moved through the edit by `ChangeSet`'s position-update logic, so that after you insert five characters at the top of the file, the cursor that was on line 200 is still on line 200, automatically. The transaction is the unit of change, and position-remapping is built into it rather than bolted on after.

Undo is free

Because applying a transaction yields a new rope that structurally shares the old one, keeping the old one around costs almost nothing — the shared nodes are not copied. That is what makes the undo history cheap, and zmax keeps it as a *tree*, not a line: `History` records transactions so that undo and redo walk a branching structure, and because each state is a cheap structural variant of its neighbor, the tree can be deep without the memory cost a naive “snapshot the whole buffer per undo” scheme would incur. Short strings throughout the editor use a `Tendril`, which in zmax is a type alias rather than the crate of that name — `pub type Tendril = SmartString<smartstring::LazyCompact>` in `zmax-core/src/lib.rs` — a small-string type that stores brief text inline without a heap allocation, another small efficiency in the same spirit. The through-line of the chapter is that one good choice — a rope with structural sharing — pays out in four places at once: large files stay fast, Unicode stays correct, position-remapping comes for free with the transaction, and a branching undo history is cheap. The rest of the machine is built on top of this, and the next chapter follows a single keystroke all the way down to it.

Chapter 24 — How a Keystroke Becomes an Edit

This chapter traces one keystroke from the keyboard to the screen, because following the whole path once makes the architecture click in a way that describing the parts separately cannot. Press `x` to delete a character, and a remarkable amount happens in a specific order; by the end of this chapter you will be able to name every stage, and — more usefully — you will see that the vim operator of Movement II, the LSP edit of Chapter 14, and the embedded-language edit of Movement IV all merge into the *same* path before they touch the buffer.

From key to command

A key arrives from the terminal as an event, read by the main loop (Chapter 27) and handed to the *compositor*, the stack of UI layers. It travels down the stack until a layer consumes it; in the ordinary case that layer is the editor view, which looks the key up in the current mode's *keymap* — the *KeyTrie* of Movement II. The lookup has three possible outcomes: the key is a complete binding and yields a command; the key is a prefix and the trie descends to await the next key (this is how `g` and `d` and `SPC` work); or the key is unbound and nothing happens. When it yields a command, recall from Chapter 5 that a “command” may be a single function or a *list* of them — the `[collapse_selection, extend_next_word_start, delete_selection]` sequence that makes `dw` work — but either way the result is one or more commands to run.

From command to transaction

Each command runs against a *Context* — a bundle holding `Emut Editor`, the job queue, and the surrounding state — which is how a command reaches everything it might need to touch. A motion command edits the selection and leaves the text alone; an *edit* command builds a *Transaction* against the current document. This is the convergence point the chapter promised: the command does not mutate the rope directly, it constructs a transaction describing the change, and that is true whether the command came from a vim key, from a code action the language server proposed, or from an `(insert "x")` evaluated in embedded elisp. All three produce a transaction; none of them is special downstream of that.

From transaction to screen

The transaction is applied to the document's rope, yielding the next version of the text (Chapter 23). The same change set remaps every selection, mark, and diagnostic through the edit, so all the anchored state stays anchored. The *History* records the transaction so undo can find it. Events fire — `DocumentDidChange` and its kin (Chapter 26) — which is how the language server learns to send a `didChange` notification, how the diff gutter knows to recompute, how the syntax tree (Chapter 25) knows to reparse the touched range. A redraw is requested, and on the next turn of the main loop the compositor renders the changed buffer: the editor view queries the syntax tree for highlights over the visible lines, draws them into a back buffer, and the TUI layer (Chapter 27) diffs that buffer against what is already on screen and emits only the escape sequences needed to update the changed cells. The character is gone from the screen, and the entire trip — key, compositor, keymap, command, transaction, rope, remap, history, events, redraw, diff, flush — happened between two

frames. Every edit zmax makes, from the simplest [x](#) to a five-hundred-cursor change to a script-driven insertion, runs this same path; learn it once and you have the spine of the whole editor.

Chapter 25 – The Syntax Layer

Chapter 13 showed what the syntax tree buys at the keyboard; this chapter shows how the tree is built and kept current, because the difference between a syntax-aware editor and a *syntax-highlighting* one is entirely in this machinery. `zmax` does not run regular expressions over your lines to guess at keywords; it maintains a real parse tree of the buffer, updated incrementally as you type, and queries it. The engine is *tree-sitter*, reached through a wrapper.

tree-house over tree-sitter

`zmax`'s syntax code, in `zmax-core/src/syntax.rs`, is built on `tree-house`, a *tree-sitter* wrapper (a workspace dependency at version 0.4) that handles the parts of using *tree-sitter* that are fiddly to do by hand: incremental updates, iterating query matches, and resolving language *injections* (Chapter 13's nested languages). Raw *tree-sitter* gives you a parser and a tree; `tree-house` gives you the editor-shaped operations on top of it. Each language the editor knows is described by a `LanguageData` record, and its most telling feature is laziness: the compiled query sets — `SyntaxConfig` for highlights, plus the indent query, the text-object query, the tag query for bracket matching, the rainbow query — each live behind a `OnceCell` and are compiled on *first use*, not at load. A language you open pays to compile its highlight queries the first time it is highlighted, and a query family you never trigger (rainbows, if you never enable them) is never compiled at all. Multiply by the hundreds of languages that ship and the lazy compile is the difference between a fast start and a slow one.

Incremental reparsing

The reason *tree-sitter* belongs in an editor rather than a batch tool is that it reparses *incrementally*. When the transaction of Chapter 24 changes the buffer, `zmax` tells the parser exactly what changed — the edited byte range and how it shifted — as a *tree-sitter* `InputEdit`, and the parser reuses the entire existing tree except the subtree the edit actually disturbed. Type a character inside a function and the parser re-derives that statement, not the file. `tree-house` caches the tree between queries, so the next highlight query reuses the freshly-patched tree. This is what makes “a real parse tree, always current” affordable on every keystroke: the work per edit is proportional to the size of the edit, not the size of the file.

The query families

A grammar is more than a parser; each ships a set of *tree-sitter queries*, the `.scm` files in `runtime/queries/<language>/`, and each query file is a different question asked of the same tree. `highlights.scm` maps tree nodes to highlight scopes (`@keyword`, `@string`, `@function`) that themes color. `injections.scm` marks regions to reparse with another grammar. `locals.scm` tracks variable scopes so a local can be distinguished from a global. `indents.scm` drives auto-indentation. `textobjects.scm` labels functions, parameters, and classes for the text objects of Chapter 13. `rainbows.scm` colors brackets by depth. `tags.scm` is symbol tagging only — its captures are `@definition.function`, `@definition.struct`, `@definition.constant`, `@name` and their siblings, the ctags-style index of what a file *defines*; brackets are `rainbows.scm`'s job, not its. The runtime ships query sets across essentially the whole `languages.toml` inventory of Chapter 14, and adding a language is, in large part, supplying its grammar and these queries, which is why the editor's language support grows without touching the editor's own code. The tree is one structure; the query families are the many lenses; and the thing that picks a grammar for a buffer is `zmax_`

`core::syntax::Loader`, whose `language_for_shebang` and extension lookup resolve a document to a language (`zmax-core/src/syntax.rs`). The separately-named `zmax-loader` crate is not that — it is the configuration and runtime-directory layer (`config_file`, `runtime_file`, `cache_dir`), and only loads things by name. The syntax layer is, in the end, a single well-kept tree and a drawer of questions to ask it.

Chapter 26 — The Event System

A large editor is a knot of subsystems that need to react to each other: when the document changes, the language server must be told, the diff gutter must recompute, the syntax tree must reparse, the completion popup must perhaps close. The naive way to wire this is direct calls — the edit code calls the LSP code calls the gutter code — and that way lies a tangle of mutual dependencies where every subsystem must know about every other. `zmax` avoids the tangle with a dedicated event system, the `zmax-event` crate, and this chapter is how that decoupling works.

Hooks and dispatch

The model is publish-and-subscribe with a twist. A subsystem declares an event type through an `events!` macro, other subsystems register *hooks* on that type with `register_hook!`, and when the event occurs someone calls `dispatch` with it; every registered hook runs. The edit code does not know the LSP code exists; it dispatches a `DocumentDidChange`, and whoever cared — the LSP client, the gutter, the syntax layer — had registered a hook and gets called. Add a new reactor to document changes and you register a hook; you do not edit the edit code. The dependency graph collapses from every-subsystem-to-every-subsystem down to every-subsystem-to-the-event-bus.

The twist is that hooks are *synchronous* by default: a registered hook is a function run immediately, in dispatch order, and it may mutate editor state directly. This is deliberate and occasionally essential — some reactions must happen *now*, before anything else, such as hiding the completion popup the instant the document changes out from under it — and a synchronous hook that holds the editor can do that without a deferral. The cost is that a synchronous hook must be quick, because it runs on the path of the thing that triggered it.

The async escape

Work that is *not* quick — talking to a language server, computing a diff, anything that touches I/O or takes real time — cannot run in a synchronous hook without stalling the editor, so `zmax-event` provides the `AsyncHook`, built around debouncing. An async hook receives events on a tokio channel and runs background work, and it *debounces*: a flurry of events — the keystrokes of fast typing — collapses so the expensive reaction fires once after you pause, not once per character. This is exactly right for the idle-triggered features: completion and signature help and inlay hints want to fire when you stop typing, not on every key, and the debounced async hook is the mechanism. The synchronous hook mutates the editor immediately; the async hook defers and coalesces background work; between them, every subsystem reacts to every other without any of them depending on any other directly. The event system also owns *redraw coordination* — the `lock_frame`, `request_redraw`, and `start_frame` primitives that decide when the screen is allowed to repaint — which is the natural segue to the chapter on the loop that does the painting.

The vtable, hand-rolled

The description above is the model. The implementation has one part that is worth opening, because it explains a constraint you will hit if you ever add an event type: `zmax-event/src/hook.rs` builds its own dynamic dispatch rather than using `dyn Trait`. The header states why:

rust dynamic dispatch is extremely limited so we have to build our own vtable implementation. Otherwise implementing the event system would not be possible.

The limitation is lifetimes. An event type is generic over the lifetimes of the references it carries — a document-change event borrows the document — and a hook must be callable for *any* of those lifetimes. Rust's trait objects cannot express “generic over all lifetime parameters of `E`” in a way that survives erasure into a `Box<dyn Fn(&E)>`, so the crate erases it manually.

The result is a two-word structure:

```
pub(crate) struct ErasedHook {
    data: NonNull<Opaque>,
    call: unsafe fn(NonNull<Opaque>, NonNull<Opaque>, NonNull<Opaque>),
}
```

A pointer to the closure and a function pointer that knows how to call it, with `Opaque` standing in for the erased type parameter — the comment notes that “if extern types were stable, this could be implemented as `extern { pub type Opaque; }.`”

The header also points out the incidental win. A normal trait object is two pointers, the second of which points at a static vtable containing several more — the methods, drop, size, align — so a call is a double indirection. Here there is exactly one function in the trait and, in the header's words, they “don't need a drop implementation (event system is global anyway and never dropped),” so the entire vtable is a single inline function pointer. One indirection instead of two, on a path that runs on every dispatched event.

That “never dropped” clause is the trade being made. The hook registry lives for the process and hooks are never unregistered, which is what makes it sound to skip the drop glue. It is a design constraint, not an oversight, and it is the reason hooks are registered at startup rather than dynamically attached and detached.

Identity by name, and the panic that enforces it

`registry.rs` keys events by their type *name*, and it is unusually blunt about what that costs:

The registry identifies event types using their type name so multiple event with the same type name may not be registered (will cause a panic to ensure soundness).

`register_event` asserts that the event's `TypeId` matches its `Static` associated type's, then checks the name table. A second registration of the same name panics — one message if the `TypeId` matches (the same event registered twice) and a different one if it does not (two distinct types with a colliding name). Neither is recoverable, and neither should be: the second case would let a hook written against one type be invoked with a pointer to another, which is the unsoundness the manual erasure is otherwise careful to avoid.

Panicking at registration time means the failure happens at startup, deterministically, on the developer's machine, rather than as a corrupted call at runtime on a user's. The one exception is under the `integration_test` feature, where duplicate registration does not panic — because the test harness runs several editor instances in one process and would otherwise trip on its own second startup.

Statics that are not quite static

That same test harness forces the strangest thing in the crate. `runtime.rs` explains it:

The event system makes use of global to decouple different systems. However, this can cause problems for the integration test system because it runs multiple zmax applications in parallel. Making the globals thread-local does not work because a applications can/does have multiple runtime threads.

(Quoted exactly, typos and all — “makes use of global”, “a applications”. This book tidied both in an earlier edition, which is a small thing to get wrong and a bad habit to keep: a block quote that has been silently

improved is no longer evidence, and the reader loses the ability to check it against the file.)

Two editor instances in one process would share one hook registry, one redraw notifier, one status queue. Thread-locals do not fix it because one instance spans several threads. So `runtime_local!` declares statics that are local to a *tokio runtime*: in a normal build the macro expands to a plain static with a `Deref` that costs nothing, and under `integration_test` it expands to a map keyed by runtime id behind a lock. The comment is candid that “the implementation requires locking so it’s not exactly efficient” — which is acceptable precisely because it only exists in test builds.

Every global in the crate goes through this macro: the registry, `REDRAW_NOTIFY`, `RENDER_LOCK`, the status `MESSAGES` sender. The pattern is worth noting as a general technique. Isolating globals for tests usually means either threading context through every call site or accepting that tests must run in separate processes. This is a third option, and the cost is confined to the configuration that needs it.

Redraw: a notifier and a lock

The chapter mentioned that the event crate owns redraw coordination. There are two primitives and they do opposite things.

`REDRAW_NOTIFY` is a `Notify`. `request_redraw()` wakes it, `redraw_requested()` returns a future that resolves when it is woken, and the documentation states the policy: “The redraws are debounced (currently to 30FPS) so this can be called many times without causing a ton of frames to be rendered.” Any code anywhere can ask for a repaint without knowing whether one is already pending.

`RENDER_LOCK` is a `RwLock<()>` used inside out. Background tasks take *read* locks via `lock_frame()`, and the renderer takes a *write* lock in `start_frame()` — which therefore blocks until every outstanding read lock is released. The comment describes the intent: it “allows asynchronous tasks to acquire non-exclusive locks to prevent the next frame from being drawn until a certain computation has finished.”

The inversion is the point. Many concurrent holders can each delay the frame, they do not contend with each other, and the renderer’s single write acquisition is the join. A syntax highlight that will land in a few milliseconds can hold the frame rather than let it paint stale and repaint immediately after. `start_frame` then drains any leftover notification so a request that arrived during the wait does not immediately schedule a second redundant frame.

Cancellation

`cancel.rs` supplies the other half of the async story: work that should be abandoned rather than awaited. `cancellable_future(future, handle)` selects between the future and the handle’s cancellation signal, returning `Option<T>` — `None` when cancelled — with `biased`; on the select so cancellation is checked first and an already-cancelled task cannot slip one more poll through.

The shared state is a `Notify` plus an `AtomicU64` generation counter, and the comment explains why the counter is there rather than just the notifier:

if we only cared about async code then we could also only use a notify (without the generation count) ... but it would be extremely slow to frequently check for cancellation from sync code.

Synchronous code — a long highlighting pass, a big search — wants to check “should I stop?” in its inner loop. Against a `Notify` that is an expensive question. Against a relaxed atomic load it is nearly free. The counter also makes the handle reusable: bumping the generation cancels the previous round without allocating a new controller for the next one.

There is a second comment on that struct that is worth quoting for what it says about the project’s engineering standard:

`Notify` has some features that we don't really need here ... we could reimplement the waiter linked list with modest effort and reduce memory consumption by one word/8 bytes and reduce code complexity/number of atomic operations. I don't think that's worth the complexity (unsafe code).

An optimization identified, quantified at eight bytes, and declined in writing. That is the correct disposition of a micro-optimization, and having the reasoning in the source means nobody has to rediscover it in order to reject it again.

Status messages

`status.rs` is the smallest module and closes the loop for background work that has something to say. A `StatusMessage` is a severity — `Hint`, `Info`, `Warning`, `Error`, ordered — and a `Cow<'static, str>`, so a constant message costs no allocation and a formatted one is still accepted. `From` implementations for `anyhow::Error` and `&'static str` mean the common cases are one `.into()`.

The queue is a `runtime_local!` sender, drained by the main loop. That is the same shape as the job queue of Chapter 92: background code produces, the main loop consumes, and no background thread ever touches the editor's status line directly. Between the job queue, the redraw notifier, the render lock and this, the whole async-to-main-thread surface of the editor is four channels and no shared mutable editor state.

Chapter 27 – The Main Loop and Rendering

At the center of any interactive program is a loop that waits for things to happen and responds, and the character of an editor is partly the character of that loop: what it waits on, in what order, and how often it redraws. zmax's loop lives in the `Application` struct in `zmax-term`, and it is an asynchronous, priority-ordered, draw-on-demand loop built on tokio. This chapter is that loop and the rendering path it drives.

What the loop waits on

The `Application` holds the pieces that must be coordinated: the `Compositor` (the UI layer stack), the `Terminal` (the drawing surface), the `Editor` (all document and view state), the `Config`, and `Jobs` (the queue of background callbacks and futures). The heart of it is a `tokio::select!` — in the function `event_loop_until_idle` — that waits on several sources at once and acts on whichever fires first: operating-system signals, terminal input events, completed job callbacks, status messages, finished async futures (an LSP or DAP response landing), and the editor's own idle events. The `select!` is marked `biased`, which means it checks its branches in a fixed priority order rather than at random. The order in `zmax-term/src/application.rs` is `accept_server` first, then signals, then terminal input, then job callbacks — the server-socket accept leads because a connection left unaccepted blocks its client, and after that a suspend-signal or a keystroke is never starved by a flood of background callbacks. The loop is responsive by construction because the things a human is waiting on are checked before the things a machine is waiting on.

Draw on demand

The single most important efficiency in the loop is that it does *not* redraw every iteration. Rendering happens only when something changed — after handling input, after a job callback updates state, after an editor event — not on a timer and not on a spin. An idle editor displaying a static buffer does no rendering work at all; it sits in the `select!` waiting, costing nothing. This is why zmax is quiet on battery and cheap on a remote connection: the cost of displaying text you are not changing is zero. The config is held in an `arc_swap::ArcSwap`, so a configuration reload swaps in a new `Config` atomically without locking the loop — you can change settings live and the loop picks them up on its next turn.

This loop is also, to close the circle from Chapter 22, the thread on which an embedded language `eval` runs. Eval is invoked from a command handler, which runs inside this loop's input-handling branch, synchronously, holding the `fm::Editor` — which is exactly the single-threaded, one-mutable-borrow condition that made the thread-local editor pointer sound. The languages are not given their own thread; they borrow the editor's, briefly, between two turns of this loop.

From state to screen

When a render does happen, it flows through the compositor and the TUI. The `Compositor` is a stack of `Component` layers — the editor view at the bottom, then any pickers, popups, menus, and prompts above it — and rendering walks the stack bottom to top, each layer drawing onto a surface, so the editor view paints the buffer and a picker paints itself over the top. The drawing surface is the `Terminal` in `zmax-tui`, and it is

double-buffered: it keeps the cells currently on screen and the cells the new frame wants, and on flush it *diffs* them and emits only the terminal escape sequences needed to turn the old screen into the new one. Change one character and only that cell's worth of output is written, not the whole screen. Underneath, a **Backend** trait abstracts the actual terminal — a crossterm backend, a termina backend on Unix, and a test backend for the integration tests — so the same rendering code drives a real terminal or a simulated one. The path is uniform and lazy: render only on change, draw through layers, diff before flush, write the minimum. It is the output mirror of the draw-on-demand loop, and together they are why a full-featured editor feels light.

Chapter 28 — The Crate Map

zmax is not one program but fourteen crates in a Cargo workspace, and the way the work is divided among them is itself a piece of the design — it is what keeps a large editor comprehensible and what makes the five-language embedding land in a sensible place. This chapter is the map: what each crate owns, and why the boundaries fall where they do.

The spine

Three crates form the spine, layered from pure to concrete. `zmax-core` is the foundation: the rope, the `Selection` and `Range` model, the `Transaction/ChangeSet` machinery, and the syntax layer — text and edits and parsing, with no notion of a screen or a terminal in it. `zmax-view` sits above core and owns *editor state*: the `Editor` itself, the `Document` (text plus its history, selections, diagnostics, and language-server connections), and the `View` (a window onto a document, with viewport and gutters), along with the `Theme`. It knows about documents and views and layout, but still not about the terminal. `zmax-term` is the top: the `Application`, the main loop, the command implementations, the UI components, and the command-line — the crate that turns the editor state into a running terminal program. Below `term` sits `zmax-tui`, the drawing layer of Chapter 27, generic over its backend.

The discipline in that layering is that dependencies point one way — `term` depends on `view` depends on `core`, never the reverse — and the payoff is concrete: because `core` has no UI and `view` has no terminal, the editor logic can be tested without a screen, which is exactly what the integration tests do, driving a `TestBackend` through the real `view` and `core`. A boundary that points one way is a boundary you can test against.

The services and the utilities

Around the spine sit the service crates, each owning one external-facing concern. `zmax-lsp` and its type crate `zmax-lsp-types` are the language-server client of Chapter 14; `zmax-dap` and `zmax-dap-types` are the debugger client of Chapter 15; `zmax-vcs` is the git integration of Chapter 16; `zmax-loader` is the runtime, grammar, theme, and config discovery of Chapter 2; `zmax-event` is the hook system of Chapter 26. Three smaller crates round it out: `zmax-parserc`, a parser-combinator library used to parse snippet syntax; `zmax-stdx`, the standard-library extensions — path normalization, rope helpers, URL and range types — shared everywhere; and `zmax-plugin`, the native plugin SDK, which is the one crate this book otherwise barely mentions and deserves a sentence. It defines a versioned, hand-rolled C ABI — `#[repr(C)]` structs and `extern "C"` function pointers — that both the editor and a third-party plugin depend on, so a plugin can ship as an ordinary `cdylib`, be `dlopened` at runtime by `:plugin load <path>`, and register typable commands that resolve exactly like built-in ones. It is a genuinely different extension road from the five embedded tongues of Movement IV: compiled rather than interpreted, across an ABI rather than inside the address space, versioned so a stale plugin is rejected rather than trusted. Each service crate is replaceable and testable on its own because the seam between it and the editor is explicit.

The map also answers a question Movement IV raised: where do the five embedded languages plug in? Into `zmax-term`, at its command, config, and keymap surfaces — because those are the surfaces a language drives, and `term` is the crate that owns them. The interpreters depend on the editor crates to reach the editor, and the editor binary links the interpreters; the embedding is a `term`-level concern precisely because driving the editor is what `term` is for. The crate map is not bureaucracy; it is the reason a fourteen-crate editor with

five embedded languages stays legible enough to reason about — and the last chapter of the movement is about how the project keeps itself honest about what all that machinery actually covers.

Chapter 29 – Measuring the Port

This book has made a habit of admitting what does not work yet — the unimplemented vim edits, the half-wired language embeddings, the young debugger — and that habit is not just authorial manners. It mirrors something the project itself does: zmax measures its own coverage against the editors it borrows from, mechanically, with infrastructure built for the purpose, and refuses to claim a feature it cannot point at in source. This final chapter of the machine movement is about that infrastructure, the `port/` directory, because a project that counts its own gaps honestly is a different kind of thing from one that markets its hits — and the counting is itself a piece of engineering worth seeing.

A numerator and a denominator, both from source

The question “how much of vim does zmax cover?” is easy to answer dishonestly and hard to answer well, and the `port/` machinery answers it well by computing a fraction whose two halves both come from real artifacts rather than opinion. The *denominator* is a set of cited inventories — the commands and keybindings of the editors being measured against — parsed from upstream sources rather than hand-listed: Vim and Neovim normal, insert, visual, command-line, and ex commands; the GNU Emacs command set; the Spacemacs bindings; the fzf.vim command list; the Cursor command set; and the JetBrains IntelliJ IDEA default keymap, against which zmax’s IDE-side work — the magit status buffer, the three-pane merge tool, the run configurations, the workbench tool windows, the debugger — is measured the same honest way. These are not zmax’s own idea of what vim or IntelliJ contains; they are extracted from those tools’ own documentation, so the target cannot be quietly shrunk to flatter the score.

The *numerator* is re-derived from zmax’s source at report time: the static commands read out of `commands.rs`, the typable commands out of `typed.rs`, the keybindings out of `keymap/vim.rs`. And every claimed mapping must carry *evidence* — a resolvable reference to the real thing, `static:<command>`, `typable:<command>`, or `key:<mode>:<chord>` — so that “zmax covers vim’s `dw`” is not an assertion but a pointer to the binding that implements it. Each mapping is marked `ported`, `partial`, or `absent`, and `partial` is used honestly: a mapping is `partial` only while its binding covers some of a feature and not the rest. The status moves as the code does — the operator-plus-find and operator-plus-text-object edits this report once carried as `partial` flipped to `ported` when the next-key machinery of Chapter 5 landed, and the generated numbers moved with them rather than being talked up.

Reports that resist drift

From those two halves the machinery generates reports — `port_report` and a focused `keybinding_report`, in both HTML and Markdown — by running `gen_port_report.py`. Because the numerator is re-derived from source every time, the reports track the code far more closely than hand-maintained feature lists do. Delete a binding and the next report shows the coverage drop; the document is downstream of the source, not a parallel claim that rots.

It is worth being exact about what the script does with a mapping whose evidence does not resolve — a `broken` entry — because earlier editions of this book overstated it. They said the generator *fails* on a broken mapping and called that failure condition the load-bearing part of the argument. It does not fail. `resolve()` marks the entry `broken`, the totals count it as `absent`, the report lists it in a dedicated section, and `main()` prints `WARNING: N broken mapping(s) – see report.` before writing the files normally. The script’s

only `sys.exit` fires when it cannot find the `static_commands!` invocation in `commands.rs` at all — a parse failure, not an evidence failure.

The distinction matters for how much weight the mechanism can carry. Counting a broken mapping as absent is the conservative choice — an unresolvable claim reduces the score rather than inflating it — so the number cannot be gamed by breaking evidence, and the report at the time of writing carries zero broken mappings. But a warning printed to stdout in a build that succeeds is a *reporting* discipline, not a *gate*. Nothing stops a run with broken mappings from publishing. The honest description is that the script makes drift visible and expensive to ignore, and that a human still has to read the warning. The book should not have claimed a gate that does not exist, and the argument the rest of this chapter makes stands without one.

The counted totals themselves live in `docs/port_report.md` and `docs/keybinding_report.md` and move on every regeneration, so treat any figure quoted here as a snapshot with the artifact as the authority. At the time of writing the source surface is 2,103 static commands, 2,483 typable command spellings (aliases included), and 4,109 default keybindings. Measured against the cited inventories — 6,847 distinct feature items drawn from Vim/Neovim, the GNU Emacs manual, Spacemacs, fzf.vim, the JetBrains keymap, and the Cursor command set — the report counts 5,957 ported (87.0%) and 373 partial. The keybinding-only slice, which excludes ex-commands, options, functions and `M-x` commands to count just the key-press surface, is close behind: 3,113 cited keys, 2,692 ported (86.5%), 179 partial.

Capabilities versus muscle memory

A single per-source percentage can mislead in either direction, and the report knows it, so it carries a second, deliberately different measure alongside. The per-source fractions answer “how much of *vim*’s muscle memory transfers,” “how much of *emacs*’s,” and so on — and they overlap heavily, because go-to-line is counted once for Vim, once for Emacs, once for Spacemacs, and once for JetBrains, four rows for one capability. They also flatter or punish by scope: the Emacs denominator is the *entire* GNU Emacs manual — Dired, Gnus, Calc, TeX-mode, the games — so whatever that row reads reflects how much of the encyclopedia a modal editor declines to be as much as it reflects anything missing. Read alone, those numbers are honest but easy to misread.

So the report leads instead with **functionality coverage**: distinct editor *capabilities*, deduplicated, one row each regardless of how many ancestors expose them. That table answers the question a user actually has — “what can zmax *do*” — and the answer is the headline the per-source rows bury: of 186 tracked capabilities the report records 185 implemented, one partial, and none absent. Nearly every area reads 100%; **editing**, at 20 of 21, is the single row that does not. The two measures are not in tension; they answer different questions. Per-source compatibility tells a vim refugee how much of their reflexes survive the move (high — Neovim normal mode and visual mode both above 90%); the capability table tells anyone how complete the editor is as an editor. The project publishes both rather than picking the flattering one, which is the whole ethic of the `port/` directory restated as a table layout.

Honesty as infrastructure

The deeper point, and the right note to end the machine movement on, is that zmax has turned honesty into infrastructure. It would be easy to write “vim-compatible” on the box and let users discover the gaps one frustration at a time; instead the project maintains a mechanism whose entire job is to enumerate, with evidence, exactly how far the compatibility goes and exactly where it stops. This is the same value that runs through this whole book — ground every claim in a file, mark every provisional thing as provisional — with a script doing the counting so the number is not a matter of opinion.

That is a weaker claim than “enforced by a script rather than a conscience,” and the weaker claim is the true one. The script re-derives, tallies, and warns; it does not refuse. The discipline of reading its warnings, of regenerating before quoting a figure, and of writing down what a mechanism does rather than what it ought to do, is human and stays human. What the machinery buys is that the human’s job is *checkable*: anyone

can run `gen_port_report.py` and compare. A book whose numbers can be re-derived by its reader is in a different position from one whose numbers must be trusted, and that — rather than an imagined build gate — is what the `port/` directory actually provides.

It is also, not incidentally, the foundation the reference companion to this book is built on: where this volume is the argument you read in a chair, the reference is the enumerated, generated, evidence-backed catalog of every command and key, and it is generated from the very machinery this chapter describes. The book you are holding tells you why `zmax` is shaped the way it is; the port reports prove, line by line, how much of the three great traditions that shape `actually` delivers — and refuse to let the answer be rounded up.

Chapter 30 – Multiple Cursors in Anger

The five movements behind you were explanation; this one is practice. Each chapter takes a real task and works it end to end with the keys and tools the book has described, because the surest way to know an editor — is to watch someone use it on something that matters. We start with the capability that most distinguishes the engine — multiple selections — and a task every programmer faces: making the same change in many places at once, correctly, without a regex you have to debug.

The task

You have a function that takes a configuration struct, and a dozen places in the file read fields off a local named `cfg`. You have decided `cfg` is a poor name and want it to be `settings` everywhere in this scope — but *only* this scope, so a blind search-and-replace across the file is wrong, because `cfg` appears in three other functions you must not touch. This is precisely the case where multiple cursors beat both manual editing and a global substitution: you want to see every site, confirm each is the right one, and change them together.

The selection-first solution

Drop into Select mode and select the body of the function — the region within which the rename is safe. Now search within that thought: with the function's text selected, the search-and-select operations of the engine narrow a selection to the matches of a pattern inside it, turning “the function body” into “every `cfg` in the function body” — a single `Selection` holding a dozen `Ranges`, one per occurrence, each a real cursor. This is the move that has no equivalent in a point-cursor editor: you did not loop, you did not replace; you *selected all twelve at once*.

With twelve cursors live, the edit is ordinary. Type `c` to change the selection and all twelve `cfgs` vanish together, dropping you into Insert mode; type `settings`, and the word appears at all twelve sites simultaneously as you type each character; press `Esc`, and you are back in Normal mode with the rename done and twelve cursors still standing on the twelve new words, in case you want to keep editing them. Every keystroke you typed went to every cursor, because in this engine a keystroke always goes to every range — the multi-cursor case and the single-cursor case are the same code path (Chapter 3), so there was nothing special to turn on.

Column editing with `C-v`

A different shape of the same power is column editing, and it has its own idiom. Suppose you have ten lines and want to prefix each with a comment marker, or align an assignment operator down a block. Enter Select mode, then press `C-v` repeatedly — each press runs `copy_selection_on_next_line`, dropping a copy of the current selection onto the next line down, so after ten presses you have ten cursors stacked in a column. Now `I` inserts at the left of every line in the block at once and `A` appends at the right of every line; type your prefix and it lands on all ten. The book noted in Chapter 4 that `zmax` has no true rectangular block mode because the engine has no rectangle — only the set of ranges — and here is that fact as a workflow: block editing is a column of cursors you built with `C-v`, and `o/O` flip you to the other end of the block if you sized it from the wrong side. It behaves like vim's visual-block insert, and it is multiple cursors underneath, which is why it composes with everything else the selection can do.

The lesson of the chapter is a mental shift more than a key sequence: stop thinking “find and replace” and start thinking “select all the places, then edit them as one.” The engine was built for that thought, the vim keys expose it, and once it is in your hands the daily small refactors — rename in a scope, prefix a block, fix the same typo in six strings — stop being a regex you write and become a selection you make.

Chapter 31 – A Day with the Language Server

The language server (Chapter 14) is the difference between editing text that happens to be code and editing code that the editor understands. This chapter is a day's worth of that understanding put to work: navigating an unfamiliar codebase, making a safe rename, applying a fix the server suggests, and clearing the diagnostics — all without leaving the editor or typing a symbol's name into a search box.

Navigating by meaning

You open a Rust project you did not write and land in a function that calls `resolve_path`. Where is it defined? Put the cursor on the name and press `gd` — goto definition — and the editor jumps to it, in this file or another, opening the buffer if need be. You read it, and want to know who else calls it: `gr` lists every reference through the symbol picker, each previewed, so you can survey the call sites without a text search that would also match the string `"resolve_path"` in a comment. The semantic gotos — `gd`, `gD` for declaration, `gy` for the type's definition, `gi` for implementations — turn “understand this code” from reading-and-grepping into jumping, and because each landing is recorded in the jumplist, `C-o` walks you back out the way you came (Chapter 17). Hover with `K` for the type and doc of whatever is under the cursor; the picture of the code assembles without your hands leaving the home position.

A safe rename

You have decided `resolve_path` should be `canonicalize_path`. You could do the multi-cursor rename of Chapter 30, but it is scoped to a buffer and this symbol is public, used across the crate — so this is the language server's job, because only it knows every use across every file, including the ones not open. Put the cursor on the name and run rename, `SPC r`; type the new name once; the server computes a workspace edit touching every file that references the symbol, and the editor applies it as a set of transactions. The rename is *semantic* — it changes the symbol, not the text, so it does not touch a same-named field on an unrelated type, and it reaches files you never opened. This is the rename multi-cursor cannot do, and the editor offers both because each is right for a different scope: a buffer-local change is a selection, a project-wide one is a server edit.

Fixing what the server flags

The server has underlined a line: an unused import. Navigate diagnostics with `jd` to land on it, then ask the server what it can do about it — code actions, `SPC a` — and a menu offers “remove unused import.” Choose it and the edit applies. Code actions are the server's repair suggestions, and they range from the trivial (drop an import) to the structural (fill in a match's missing arms, implement a trait's required methods), each one a workspace edit the editor applies as a transaction so it lands in the undo history like any other change. Walk the rest of the diagnostics with `jd`, applying actions or making manual fixes, and watch the count in the status line fall to zero. Format the file on the way out with `:format`, and you have navigated, renamed, repaired, and tidied an unfamiliar codebase using the server's knowledge throughout — the editor as a front end to the same analysis your compiler runs, which is exactly what an LSP client is for.

Chapter 32 – Text Surgery with AWK

Here the book cashes a promise five movements in the making. `zmax` embeds AWK (Chapter 21), which means the column-processing language Unix programmers reach for can run *inside* the editor, over your buffer, with no shell-out and no temp file. This chapter is that capability as a workflow — and an honest note, per the book’s habit, that this is the embedding being built out as we write, with `awkis` needing the small library-shaped entry point Chapter 21 described before the recipe below runs unchanged. What follows is the intended shape, and it is close.

The task

You have selected a block of lines that look like a `/etc/passwd` dump or a CSV export — colon- or comma-separated columns — and you want to keep two of the columns, swap their order, and reformat. In a terminal you would pipe the text through `awk`. In `zmax` you do the same thing without the terminal: select the lines, and run the selection through an embedded AWK program whose output replaces it. The gesture is the `:!` of Chapter 11 turned inward — instead of piping the selection to an external `awk` process, you run it through the AWK that is compiled into the editor.

The program

Say the columns are colon-separated and you want column 5 (a description) followed by column 1 (a name), tab-separated:

```
BEGIN { FS = ":"; OFS = "\t" }
{ print $5, $1 }
```

Select the block, invoke the embedded AWK over the selection with that program, and the twelve lines are replaced by twelve reformatted ones, in place, in the buffer, in the editor’s own memory. Recall from Chapter 19 *why* this is fast and clean: `fusevm` has 87 first-class AWK opcodes, so `FS`, the field references `$5` and `$1`, and the `print` lower to opcodes the shared VM runs natively — this is not AWK bolted on, it is AWK as a front end to the machine the editor already carries.

Why in-editor beats the pipe

The external pipe works too — `:!awk -F: '{print $5 "\t" $1}'` shells out and is a fine tool. The embedded version wins on three counts that matter once it is wired. It needs no `awk` on the system, because the AWK is in the binary (Chapter 2’s “single executable” paying off again). It crosses no process boundary, so there is no fork, no pipe buffering, no second program to fail to find. And it shares the editor’s process, which means the same mechanism that runs a standalone AWK program could, in time, expose the editor’s own state to the program — the host-callback architecture of Chapter 22 applied to AWK. The column transform is the first taste; the deeper point is that a text-processing language living *in* the editor is a different tool from one you pipe to, and the next two chapters push the same idea through `stryke` and `Zsh`.

Chapter 33 – Text Surgery with stryke

AWK is the right tool when your text is columns. When it is not — when the transform needs real regular expressions, data structures, and logic — you reach for stryke, and zmax embeds it via `strykelang` (Chapter 21). stryke is MenkeTechnologies' own language — *not* Perl, but a Perl-5-compatible interpreter in Rust that bills itself as the fastest dynamic language for parallel work, with a bytecode VM, a Cranelift JIT, and the largest standard library of the five tongues. The examples here use its Perl-compatible core (so they read like Perl), but stryke is its own language with a terser native idiom and a vast builtin shelf. This chapter is stryke as in-editor text surgery, with the same honesty as the last: the run path (`VMHelper`, `parse_and_run_string`) exists, and the native host-function surface that lets stryke *drive* the editor is the gap Chapter 21 named; the recipe below is the intended shape, and it runs over a selection the way the AWK one does.

The task

You have a block of Markdown and want to rewrite every inline link — `[text](url)` — into a reference-style link, collecting the URLs into a numbered list at the bottom. This is beyond AWK: it needs a real regex with capture groups, a counter, and an accumulator. It is exactly stryke's home ground.

The program

Select the block and run it through embedded stryke:

```
my @refs;
s{\[([^\]]+)\]\([^\)]+\)}{
    push @refs, $2;
    "[${1}][" . scalar(@refs) . "]"
}ge;
print;
print "\n";
print "[${_+1}]: $refs[${_}]\n" for 0 .. $#refs;
```

The `s{...}{...}ge` substitution walks every link in the selection, the `e` flag runs the replacement as *code* so it can push the URL onto `@refs` and compute the reference number, and the trailing loop prints the collected URL list. The selection is replaced by the rewritten Markdown plus its reference list — a transform with state and logic that no column processor could express, running in the editor's own process.

When stryke, when AWK, when a cursor

The three text-surgery chapters are really one lesson about picking the right tool for the shape of the change. If the edit is *the same everywhere* and you can see all the sites, it is a multi-cursor selection (Chapter 30) — no language needed. If the text is *columnar* and the transform is per-record, it is AWK (Chapter 32). If the transform needs *regex with logic, state, or data structures*, it is stryke. zmax is unusual in offering all three as first-class, in-editor options rather than making you leave for a terminal, and more unusual still in that the second and third are not external programs it pipes to but languages compiled into it. The same `:!-` shaped gesture reaches a cursor refactor, an AWK program, or a stryke script, and you choose by the shape of the problem, not by what happens to be installed.

Chapter 34 — Text Surgery with Zsh

The last of the three text-surgery chapters embeds not a text-processing language but a *shell*: `zmax` carries `zshrs` (Chapter 21), so a Zsh pipeline can run with its output replacing a region — the `!:cmd` of Chapter 11, except the shell itself is inside the editor. The honest note recurs: `zshrs` runs scripts today through its `ShellExecutor`, and the gap is output capture — a command-line shell writes to real file descriptors, while an embedded one whose output should land in the buffer needs that output captured — so the recipe is the intended shape once that route exists.

The task

You have selected a list of lines — log entries, import statements, anything with repetition — and you want the unique entries sorted by how often each appears, most frequent first. This is the classic shell incantation, and it is muscle memory for anyone who lives in a terminal:

```
sort | uniq -c | sort -rn
```

Running it inward

Select the block and run it through embedded Zsh with that pipeline, and the selection is replaced by the counted, sorted result — `42 import os, 17 import sys`, and so on, in order. The point is not that the pipeline is clever; it is that you ran *the shell you already know*, with the exact syntax you would type in a terminal, without leaving the editor and without spawning a shell process — because the shell is embedded. Chapter 19's detail pays off here too: `fusevm` has twenty-nine first-class shell opcodes, so pipelines, redirections, and the rest lower to opcodes the shared VM runs natively, which is why a shell embeds onto this VM as cleanly as a calculator would.

Two shells, one gesture

It is worth being clear about what coexists. `zmax` already had `!:cmd` (Chapter 11), which pipes a selection to an *external* program over a real process and pipe — and that still works and is the right tool when you want the system's actual `sort` or a program zsh does not implement. The embedded Zsh is the other option: no external process, the shell in the binary, the same syntax. You will reach for the external pipe when you need a specific system tool and the embedded shell when you want the shell's own language — loops, globbing, parameter expansion — over your buffer with nothing else installed. As with `stryke` and `AWK`, the distinctive thing is that the in-editor option exists at all, and that it is a real Zsh rather than a reimplementaion: the tongue you script your dotfiles in is the tongue you can now run over your text.

Chapter 35 – Configuring zmax from Scratch

Enough surgery; this chapter builds the editor itself to taste, walking a real `config.toml` from an empty file to a complete personal setup. Everything here draws on Appendix A's schema, but where the appendix is a reference, this is a narrative: the choices a returning vim user actually makes on day one, and why.

A first config

Start with the essentials — a theme, relative line numbers for jump counts, and mode-shaped cursors so you can see at a glance which mode you are in:

```
theme = "onedark"

[editor]
line-number = "relative"
scrolloff = 8
cursorline = true
true-color = true

[editor.cursor-shape]
normal = "block"
insert = "bar"
select = "underline"
```

`line-number = "relative"` makes `5j` and `12k` a glance-and-count rather than mental arithmetic; `scrolloff = 8` keeps a margin of context around the cursor; `cursorline` draws a subtle highlight on the current line; the cursor shapes give Insert mode a bar and Select an underline so the mode is legible without reading the status word.

Turning on the help you want

Next, the language-server niceties — inlay hints and document highlights are off by default because they are visually heavy, but many people want them — and a soft-wrap setting for prose files:

```
[editor.lsp]
display-inlay-hints = true
auto-document-highlight = true

[editor.soft-wrap]
enable = true
wrap-indicator = "↪"
```

Reshaping the chrome and the keys

Tune the status line to show what you care about and trim what you do not, and remap a key or two. Suppose you want `;` to enter the command line (a popular vim remap) and a leader binding to save:

```
[editor.statusline]
left  = ["mode", "file-name", "file-modification-indicator"]
right = ["diagnostics", "selections", "position", "file-type"]

[keys.normal]
";" = "command_mode"

[keys.normal.space]
"w" = ":write"
```

The `[keys.normal]` table maps a key to a command by its name — the same `command_mode`, `:write`, and the hundreds of others the keymap of Movement II is built from — and a nested table like `[keys.normal.space]` hangs a binding off the leader. Because the config is hot-swapped through the `ArcSwap` of Chapter 27, `:config-open` lets you edit this file and reload without restarting, so building your config is a live loop: change a line, save, watch the editor change, repeat. From an empty file to this took a dozen lines, and none of them required learning a configuration *language* — it is TOML, declarative, with every key documented in Appendix A — which is the deliberate counterpoint to the next chapter, where configuration becomes a *program* in one of the five embedded tongues.

Chapter 36 – Bringing Your Config With You

Chapter 35 configured `zmax` in TOML — declarative, bounded, exactly as much configuration language as a settings file needs. This chapter does the thing only `zmax` can do: configure the editor as a *program*, in one of the languages you already use to configure your editor, because the five embedded tongues include the two an editor of this lineage owes — vimscript and elisp. This is the heritage promise of Chapter 18 made concrete, and it is, again, the capability being built out as the book is written; the recipes are the intended shape, honest about the wiring still in progress (Chapter 21).

In vimscript

You arrive from vim with a `.vimrc` full of mappings and settings, and the dream is to run it. `zmax`'s embedded vimscript (`vimlrs`) is the path: an init written in vimscript, sourced at startup, doing what a `.vimrc` does.

```
set scrolloff=8
set relativenumber
nnoremap ; :
nnoremap <Space>w :write<CR>
command! Reload :config-open
```

The `set` lines change options; the `nnoremaps` bind keys in normal mode; the `command!` defines a new `:Reload`. This is where Chapter 21's honesty matters most: vimscript *evaluates* in `zmax` today, but the ex-commands that actually reach into the editor — `:map`, `:set`, `:command`, `:autocmd` — are the host stubs still being wired to the editor's real keymap, config, command registry, and event system. When that wiring lands, a meaningful fraction of a real `.vimrc` runs unchanged, which is a thing no other editor that wears vim's keys can claim — because in the others, vimscript is the *native* config language, whereas here it is one embedded tongue among five, reaching the editor through the same host bridge they all share.

In elisp

Or you think in emacs, and configuration means a program that *builds* your editor. The embedded elisp (`elisprrs`, the tongue furthest along, Chapter 20) is sourced as an init script at the config-load moment of Chapter 22 — the `.zmax` answer to `init.el` — and because `elisprrs` already exposes `eval_str` and `defsubr`, this is the recipe closest to running today:

```
;; a small command, defined in the editor's own elisp
(defun save-and-format ()
  (format-buffer)
  (write-buffer))

;; bind it under the leader
(define-key 'normal "SPC w" 'save-and-format)
```

The `defun` defines a function; the editor primitives it calls — `format-buffer`, `write-buffer`, `define-key` — are Rust functions registered through `defsubr` (Chapter 20), so elisp here is not a sandbox computing values, it is a program *driving the editor*, exactly as in emacs. The difference from emacs is underneath: this

elisp is AOT-compiled to fusevm and its hot loops are JIT'd (Chapter 19), and it reaches the editor by a function call through a thread-local pointer (Chapter 22), not through a bespoke C interpreter.

Two heritages, one editor

The chapter's point is the editor's whole thesis in miniature. A vim refugee configures zmax in vimscript; an emacs native configures it in elisp; a pragmatist configures it in TOML (Chapter 35); and a power user mixes all three, because they coexist in one binary with no boundary between them. No other editor lets you bring *either* heritage's configuration language, let alone both, as first-class embedded code — and that is the five-tongue capability stopping being a benchmark boast and becoming the most personal thing an editor can offer: the ability to shape it in the language you already think in.

Chapter 37 — A Debugging Session

The practical movement ends where a coding day often does: not writing code but finding out why it is wrong. zmax’s debugger (Chapter 15) brings the debug loop into the editor through the Debug Adapter Protocol, and this chapter walks one session end to end — set a breakpoint, launch, stop, inspect, step — with the honest reminder that debugging is among the younger parts of the editor, capable and improving rather than IDE-complete.

Setup is a template, not a config file

You have a Rust program that panics somewhere it should not, and you want to stop just before the panic and look. The adapter is already wired: `languages.toml` configures Rust to `lldb-dap` with templates for running a binary (Chapter 15), so there is no JSON launch configuration to hand-write. You open the file, move to the suspect line, and toggle a breakpoint with `dap_toggle_breakpoint`; a marker appears in the gutter where the diff and diagnostic marks also live (Chapter 16). The breakpoint is set without leaving the buffer.

Launch, stop, inspect

Open the run-and-debug menu under the leader, `SPC R`, choose the “binary” template, and the editor launches your program under `lldb-dap`. It runs until it hits your breakpoint, and the adapter reports the stop; zmax suspends there and builds its view of the moment from the DAP event stream — the stack of frames that led here, the threads, the local variables in scope. You read the values that should not be what they are. Because the protocol is request-and-response rather than streaming (Chapter 15), the editor asks for the stack trace at the moment it suspends rather than tailing a feed, but from your chair it is the ordinary debugger experience: stopped at a line, state laid out, the call path visible.

Step and continue

From the stop you drive with the `dap_*` commands gathered under `SPC R`: step over the next line, step into a call to follow it down, continue to the next breakpoint with `dap_continue`. You narrow the fault by stepping until the variable goes wrong, find the line that corrupts it, and you have your bug — located by stepping through the running program in the same editor you wrote it in, with no separate debugger UI to launch and no context to lose. That is the close of the practical movement, and of the case the whole book has made: an editor that is conservative about your fingers and radical about its internals can take you from writing a line, through understanding an unfamiliar codebase, through transforming text in five embedded languages, through configuring itself in your heritage’s tongue, all the way to watching the program run and stop and yield its bug — without ever leaving the one binary. The appendices that follow are the reference you keep open while you do it.

Chapter 38 — First Programs in Five Tongues

The text-surgery chapters used each embedded language for the job it is best at; this chapter does the opposite, and sets all five the *same* small task so you can see their characters side by side. The task is the “hello, world” of an extensible editor: insert the current date at the cursor. Five languages, one job, one editor.

A caveat belongs at the top rather than in a footnote, because it is the most useful thing the comparison teaches. The five do *not* reach the editor equally far, and a chapter that wrote the same program five times with five cosmetic syntaxes would be hiding that. Each example below is written against its tongue’s real entry point — the typable commands `:elisp`, `:vim`, `:awk`, `:zsh`, `:stryke` — and where a tongue cannot do part of the task, the text says so instead of inventing a primitive that would make it look symmetric.

elisp

The emacs heritage shows: `defun` to define, the standard buffer builtins to act.

```
(defun insert-date ()
  "Insert today's date at the cursor."
  (insert (format-time-string "%Y-%m-%d")))

(insert-date)
```

Run it with `:elisp`. `insert` and `format-time-string` are elisp’s own builtins acting on the mirrored buffer (Chapter 22), and the definition persists across calls because the host does. What elisp cannot do is the second half of the classic task: there is no bridge from elisp’s keymaps to zmax’s, so the key comes from `config.toml` or from vimscript (Chapter 46).

vimscript

The same in the language of your `.vimrc`, and this is the tongue that can do the whole classic task — insert *and* bind — because vimlrs is the one with a keymap bridge.

```
function! InsertDate()
  put =strftime('%Y-%m-%d')
endfunction

command! InsertDate call InsertDate()
nnoremap <Space>id :InsertDate<CR>
```

Every line of that lands somewhere real. `strftime` is a vimlrs builtin. `put` is a zmax typable command, reached because the `excmd_hook` claims any ex-line whose first word names one. `command!` registers into zmax’s `USER_COMMANDS` table (with `:delcommand` and `:comclear` beside it). And `nnoremap` goes through `install_map_hook` into the runtime keymap overlay, which the application merges onto `config.keys`. Source this from an `init.vim` and `<Space>id` is bound.

AWK

AWK's embedded role is not “define a command” — it is a *filter*: `:awk` runs a program over the selection (or the whole buffer) and replaces that region with the program's output. So the task has to be re-posed to fit the grain, which is itself the point of including it. Appending the date to every selected line:

```
{ print $0 " " # " strftime("%Y-%m-%d") }
```

There is no editor primitive in that program and there is not meant to be. `awk.rs` is eleven lines around `awkr::run_program`; the region I/O is `run_awk_filter` on the Rust side. AWK computes, the editor moves the text.

Zsh

The shell, where the date is a command rather than a library call. `:zsh` runs a line in the embedded shell and shows the captured output:

```
date +%Y-%m-%d
```

State persists between calls — the `ShellExecutor` lives in a thread-local, so variables, functions and the working directory survive — which makes `:zsh` a real shell session rather than a series of one-shot subprocesses. Like AWK, it computes and hands back text; it does not register editor commands.

stryke

stryke brings its text-processing weight and its sigils. `:stryke` evaluates source with persistent globals and subs and renders the last expression:

```
sub insert_date {  
  my @t = localtime;  
  sprintf("%04d-%02d-%02d", $t[5] + 1900, $t[4] + 1, $t[3]);  
}  
insert_date();
```

stryke is the tongue with the least editor surface today, and its own module documentation says why: it has no native host-function registration, so the `api_*` primitives are not exposed to it, and its `print` routes through a file-descriptor layer the editor's capture cannot intercept. Both are upstream VM work. Today stryke computes a value and shows it; it does not yet drive the editor.

What the five show

Set side by side, the five make a better case for plurality than a symmetric comparison would have. They are not five spellings of one capability. `elisp` has the deepest buffer surface — the full emacs vocabulary against a mirrored buffer — and no way to bind a key. `vimscript` has the widest *editor* surface — options, keymaps, user commands, ex-command passthrough — and is the only one that can do the classic task end to end. AWK and Zsh are filters and pipelines, deliberately stateless about the editor and excellent at the text. stryke is a compute engine waiting on a host-function surface.

That unevenness is the honest shape of the thing, and it argues for having all five rather than against it: the tongue you reach for depends on whether your task is a buffer edit, a configuration change, a text transformation, a shell pipeline, or a computation. What they share is not a uniform API but a substrate — one binary, one address space, one bytecode machine, no subprocess — and the `api_*` surface of Chapter 22 that each binding marshals onto as far as its interpreter can currently reach.

Chapter 39 – Snippets and Completion

Two features make typing code feel less like typing: completion, which proposes the rest of what you are writing, and snippets, which expand a short trigger into a structured template you tab through. zmax has both, and they are wired together — completion candidates can *be* snippets — so this chapter takes them as one workflow.

Completion as you type

With `auto-completion` on (the default), candidates appear as you type once you pass the trigger length, drawn from the language server and from words already in the buffer. The keys are vim's insert-mode completion keys (Chapter 4): `[-n` and `[-p` cycle the menu, and the `[-x` submenu picks a *source* explicitly — `[-x [-o` for the server's omni-completion, `[-x [-f` for filenames, `[-x [-l` for a whole-line completion, `[-x [-n` for buffer words. The menu is a compositor layer (Chapter 12), so it floats over your text and vanishes on `Esc` without disturbing the buffer. Tuning lives under `[editor]:` `completion-trigger-len` sets how many characters precede the popup, `completion-timeout` bounds the wait, and `auto-completion = false` turns the automatic popup off entirely for those who prefer to summon it.

Snippets and the parsec engine

A snippet is a template with holes. The language server offers them as completion candidates — accept one and it expands, dropping you at the first hole with the others queued — and the syntax is the LSP/TextMate standard that zmax parses with its own parser-combinator crate, `zmax-parsec` (Chapter 28). A snippet body looks like this:

```
for ${1:item} in ${2:iterable}:
    ${3:pass}$0
```

The `${1:item}` is the first tab stop with a default of `item`; `${2:iterable}` is the second; `$0` is where the cursor finally rests. Accept the snippet and `item` is selected so you can type over it; press Tab and you jump to `iterable`; Tab again to the body; and the editor's multiple-selection model means a tab stop that appears twice — a variable used in two places in the template — is edited at both sites at once, the same one-edit-to-many- cursors mechanism of Chapter 30 surfacing inside snippet expansion. `zmax-parsec` is a small functional parser — combinators like `or`, `one_or_more`, and `map` composed into a grammar for `${...}` — which is why the snippet syntax is parsed robustly rather than with a fragile regex.

Your own snippets

The language server is not the only source. zmax keeps a personal snippet library — a `snippets.toml` beside your config — that you edit inside the editor with `:snippets`: a small panel where each entry is a *trigger* word, a *scope* (a language name, or `*` for every language), a description, and a body in the same `${1:...}/$0` syntax, validated live as you type it so a malformed template never reaches the disk. These expand without a language server at all. Type a trigger and press Tab: if the word just before the cursor matches a stored trigger whose scope fits the current language, the body expands in place with its tab stops armed, exactly as an LSP snippet would — the same `zmax-parsec` parser, the same transaction path, the same mirrored-hole behavior. Your triggers are tried before emmet's HTML abbreviations, so an explicit snippet always wins over

a guess, and the whole library is cached in memory so the Tab that expands it never waits on the disk. It is the mechanism of this chapter made yours: the boilerplate *you* type most, under a word you choose, on the key you already press to walk a template.

Why they are one feature

Completion and snippets feel like one thing because mechanically they nearly are: both are the editor proposing structured text and you accepting it, both render through the same popup layer, both insert through the same transaction path (Chapter 24), and both lean on the selection engine — completion to place the cursor, snippets to drive the tab stops and mirror repeated holes. Enabling snippets is a single `[editor.lsp]` key, `snippets = true` (the default). The result is that the most mechanical part of writing code — typing the boilerplate you have typed a thousand times — is the part zmax most tries to type for you, using the same machinery it uses for everything else.

Chapter 40 – Macros and Repetition

Some edits are not the same everywhere — so multiple cursors (Chapter 30) do not fit — but they are *the same shape* repeated down a list, with small variations a cursor cannot capture. For those, the tool is the macro: record yourself doing the edit once, then replay the recording. This chapter is the macro workflow, with the standing caveat from Chapter 6 that the macro system is among the features still maturing — record and replay work; the sharpest edges are still being filed.

Record, then replay

The keys are vim's. `q` followed by a register letter starts recording into that register; every key you press is captured; `q` again stops. `@` followed by the register replays it, and `Q` replays the last one. The discipline that makes macros reliable is the same in `zmax` as in vim: record an edit that is *position-independent*, so it works wherever the cursor starts. Suppose you have a list of names and want each turned into a quoted, comma-terminated entry — `Alice` becomes `"Alice",`. Position the cursor at the start of a line and record:

```
qa          start recording into register a
I"          insert a quote at the line start
<Esc>
A",         append a quote and comma at the line end
<Esc>
j           move down to the next line
q           stop recording
```

Now register `a` holds the whole edit, ending with the move to the next line so it is ready to run again. Replay it with `@a`, and the next name is transformed and the cursor steps down; `@a` again for the next. Because the recording ended by advancing a line, each replay leaves you positioned for the following one, and a count multiplies it — `10@a` runs the macro down ten lines.

Macro, cursor, or language

Chapter 30 promised that the daily refactors divide among a few tools, and the macro completes the set. A multi-cursor edit is right when the change is *identical* and the sites are *visible at once*. A macro is right when the change is a *repeated shape* but each application touches different text — quoting a line, renumbering an entry, reformatting a record — and especially when it includes motions that must re-run per item. And an embedded language (Chapters 32 through 34) is right when the transform needs real logic or runs over a whole region as data. Three tools, three shapes of repetition: the same-everywhere selection, the same-shape macro, and the programmable language. Knowing which to reach for is most of what makes an expert fast, and `zmax` is unusual in putting all three within the same few keystrokes — the selection on `v`, the macro on `q`, the language on the command line — over the one engine, in the one editor.

Chapter 41 — Working Across Many Files

Real work is rarely one file. It is a change that starts in a handler, follows a call into a helper two modules away, checks a type definition in a third file, and lands back where it began — across buffers, across windows, without losing the thread. This chapter is that session: the tools of Movement III (pickers, windows, the jumplist, the language server) used together the way a working day uses them.

Spreading out

You open the project and the file picker (`SPC f`) drops you into the entry point. You want the configuration module beside it: `C-w v` splits the window vertically, and in the new window you `SPC f` to the config file. Two files, side by side, two windows you move between with `C-w h` and `C-w l` (Chapter 8). A third file is worth a glance but not a permanent pane, so you `gd` into it from a symbol — goto-definition opens the buffer — read what you need, and `C-o` jumps you back out along the jumplist (Chapter 17), no window juggling required. The discipline that keeps this legible is the buffer/window split of Chapter 8: the files you are keeping are *buffers*, present whether or not a window shows them, and the panes are *windows*, cheap to open and close over those buffers.

Keeping the thread

The session's continuity comes from three things working together. The jumplist records every semantic leap, so `C-o` and `C-i` walk the path you actually took through the code, across files, in order. The buffer picker answers “back to the file I had ten minutes ago” without remembering its name. And workspace symbols (`SPC S`) let you jump to any symbol in the *project*, not just the open files, by fuzzy name — so “go to wherever `parse_config` is defined” is a few keystrokes whether or not that file is open. The result is that a multi-file change never becomes a tab-management chore: you follow the code's own structure with the language server, and the jumplist remembers the trail so you can always retrace it. When a project ships a ctags `tags` file rather than a language server, the vim tag stack gives the same jump-and-retrace by other means: `:tag name` jumps to a definition and pushes the stack, `:tnext` and `:tprevious` cycle multiple matches, `:pop` returns, and `:tags` shows the stack — LSP-independent navigation for the languages a server does not cover.

Closing down

When the change is done you `:wqa` — write all and quit — or close windows one at a time with `C-w q` while keeping buffers, and `:buffer-close` the files you are truly finished with. `zmax` persists enough of the session that reopening lands you near where you left. The whole loop — fan out into windows, follow the code across buffers, retrace with the jumplist, write and close — is the unglamorous daily reality an editor lives or dies by, and it is built from the same handful of mechanisms the book has already named, used in concert rather than in isolation.

Chapter 42 – Search and Replace at Scale

Chapter 30 made a change in many places with cursors and Chapter 40 with a macro; this chapter makes changes at the scale where you reach for patterns — a regex across a file, a command run on every matching line, a search across the whole project. These are the ex commands of Chapter 11 and the search of Chapter 6, put to work on real volume.

Within a file: substitute and global

The substitute command is vim's, parsed faithfully (Chapter 11). To rename every `foo_bar` to `fooBar` in the file — a mechanical case change a semantic rename would be overkill for — you write a substitution with a backreference:

```
:%s/foo_\(\w\)/foo\U\1/g
```

The `%` ranges over the whole file, the `\(\w\)` captures the letter after the underscore, and `\U\1` uppercases it in the replacement. When the change is not a substitution but a *command run on matching lines*, the global command is the tool. To delete every line containing a `DEBUG` log call:

```
:g/DEBUG/d
```

or to indent every line that opens a block, run the indent command on the matches. `:g` is vim's most powerful ex command — “for each line matching this pattern, run this command” — and `zmax` parses it specially (Chapter 11) so the pattern and the command survive intact rather than being mangled by a generic argument splitter.

Across the project

When the search leaves the current file, the workspace search and the pickers take over. A project-wide text search lists every match across the tree with a live preview, and because matches are selections in this engine (Chapter 6), choosing one drops you onto it ready to act. For *semantic* breadth — every reference to a symbol rather than every textual match — the language server's `gI` (Chapter 14) is more precise than any regex, because it knows a use of the symbol from a same-named string in a comment. The rule of thumb the chapter leaves you with: within a file, `:s` and `:g` for textual patterns; across the project, the search picker for text and `gI` for meaning; and whenever the change is semantic rather than textual, prefer the server's rename and references over a pattern, because the server cannot be fooled by a coincidence of spelling.

Chapter 43 – Making a Theme

zmax ships over two hundred themes, and one of them is probably close to what you want — but “close” is not “yours,” and a theme is a single TOML file, so making your own is an afternoon, not a project. This chapter writes one from scratch, which is also the most concrete possible tour of the scope model of Appendix C.

The palette and the UI

Start by naming your colors once in a `[palette]`, so the rest of the file refers to names rather than repeating hex:

```
[palette]
bg      = "#11121a"
fg      = "#c0caf5"
dim     = "#565f89"
accent  = "#7aa2f7"
string  = "#9ece6a"
warn    = "#e0af68"
err     = "#f7768e"
```

Then paint the editor’s own chrome — the UI scopes — so the frame of the editor is themed before a single token is:

```
"ui.background"      = { bg = "bg" }
"ui.text"            = { fg = "fg" }
"ui.linenr"          = { fg = "dim" }
"ui.linenr.selected" = { fg = "accent" }
"ui.selection"        = { bg = "#283457" }
"ui.cursor.primary"   = { fg = "bg", bg = "fg" }
"ui.statusline"       = { fg = "fg", bg = "#1a1b26" }
```

The syntax scopes

Now the code itself. The syntax scopes map tree-sitter captures to styles, from general to specific, most-specific winning (Appendix C):

```
"comment"           = { fg = "dim", modifiers = ["italic"] }
"keyword"            = { fg = "accent" }
"function"           = { fg = "#7dcfff" }
"type"               = { fg = "#2ac3de" }
"string"             = { fg = "string" }
"constant.numeric"   = { fg = "warn" }
"variable.builtin"    = { fg = "err" }
"diagnostic.error"    = { underline = { color = "err", style = "curl" } }
```

Save it as `mytheme.toml` under `runtime/themes/` (or your config’s themes directory), set `theme = "mytheme"` in `config.toml`, and `:theme mytheme` to load it live — or use `:theme` with the picker to preview as you iterate. Because the loader (Chapter 2) searches your config directory before the shipped runtime, your theme overrides a shipped one of the same name, so you can fork `tokyonight.toml`, tweak three scopes, and keep the name. A theme is data, not code, which is why the editor can carry hundreds and why adding

one more is seven lines of palette and a handful of scopes — the same data-not-code principle that lets the language and grammar support grow without touching the editor, applied to its colors.

Chapter 44 — The Single Binary in Practice

Chapter 2 called `zmax` “a single executable” and promised the word was not decoration. This chapter cashes that promise in the situations where it matters most — a remote server, a fresh machine, a slow connection — because the architectural choices the book has described (one binary, no external runtime, draw-on-demand rendering, minimal-diff output) add up to a specific practical character: `zmax` is light where editors are usually heavy.

Deploy is a copy

You have a server with nothing on it but an SSH daemon and the program you are debugging. Putting a full editor on it is normally a chore — a package manager, a language runtime, a plugin tree, a config sync. With `zmax` it is a file copy. The binary carries everything the book has described inside it: the rope and the selection engine, the tree-sitter integration and the shipped grammars, the LSP and DAP clients, and — when the embedding lands — the five language interpreters and their shared VM. There is no Node to install for the editor’s sake, no separate language binary to put on the path, no plugin host to keep running. `scp` the binary, run it, edit. The single-binary design that Chapter 1 framed as a deployment property is, from the chair, the difference between “I can fix this on the server right now” and “first let me set up my editor.”

Light on the wire and the battery

Over a slow or remote connection, the rendering path of Chapter 27 is what you feel. `zmax` draws on demand — only when something changes — and emits only the *diff* between the old screen and the new one, so an idle editor sends nothing and a keystroke sends the handful of cells that changed, not a repaint. On a laggy SSH link this is the difference between an editor that feels local and one that feels like it is underwater; on a laptop it is the difference between an editor that wakes the fans and one that sits quiet while you read. The same property serves a `tmux` or screen session: suspend `zmax` with `C-z` to drop back to the shell (it handles the suspend signal cleanly, Chapter 27), run a build, and `fg` to resume exactly where you were. None of this is a feature you turn on; it is the shape of the loop and the renderer, felt as lightness.

The cost, named

The single binary has one honest cost, the mirror of its convenience: size. Everything being present means the binary is larger than a minimal editor’s, and — once five interpreters and their VM are inside — larger still. The book’s habit is to name costs, so: you trade disk and download size for the elimination of an install step, a runtime dependency, and a plugin tree. On the server-fix and fresh-machine cases that trade is lopsided in your favor, which is exactly the workflow a terminal editor is for. You are paying megabytes you have to save minutes you do not.

Chapter 45 – Trust and Safety

An editor that can run code — five languages of it, in its own process — has to take a position on *which* code it runs and when, because the same power that lets you script the editor in elisp lets a repository you just cloned ship a config that runs the moment you open it. `zmax` has a position, configured under `[editor.workspace-trust]`, and this chapter is that model and the discipline around the embedded-language seam, from the user's side rather than the implementer's.

Workspace trust

The risk is concrete. `zmax` reads a workspace-local `.zmax/` config, and language servers launch external programs named by a project's `languages.toml`; a malicious repository could, in principle, point those at something you did not intend. The `workspace-trust` setting is the gate:

```
[editor.workspace-trust]
level = "servers"          # none | servers | insecure
prompt = true
trusted = ["~/work/**"]
```

The `level` decides how much an untrusted workspace may do — `servers` (the default) gates the launching of language servers behind trust, `none` is the most restrictive, `insecure` turns the gate off. `prompt` asks you the first time you open an unfamiliar workspace, and `trusted` is a list of path globs you have pre-approved, so the directories where your own code lives never prompt while a freshly-cloned stranger does. The model is the sensible one: your own trees are trusted standing, anything new asks first, and what a trusted workspace may do is your policy to set.

The languages and the seam

The embedded languages sharpen the same question, and Chapter 22 was honest about the mechanism: a tongue's host callback reaches the editor through a thread-local pointer with a small, bounded patch of `unsafe` at the seam, sound under the single-threaded synchronous discipline the editor holds to. From the user's chair the thing to understand is *when* code runs. An init script (Chapter 36) runs at startup, from a location you control; an `eval` you type runs because you typed it; a script bound to a key runs when you press it. The editor does not evaluate arbitrary embedded code from a file just because you opened it — running a tongue is a deliberate act, the same way `!cmd` is. That deliberateness is the user-facing safety story for the five languages: they are powerful precisely because they run in the editor's own process with no boundary, and the discipline that keeps that power safe is that *you* decide when a tongue speaks. Trust the workspaces that are yours, let the rest prompt, and treat the embedded languages the way you treat a shell — enormous leverage, invoked on purpose.

What trust actually stores

The user-facing model above is the policy. `zmax-loader/src/workspace_trust.rs` is the mechanism, and it is worth reading because the storage format encodes several decisions that the configuration keys do not reveal.

A grant is one small file per workspace:

```
path = /home/user/proj1
hash = sha256:abc123...
excluded = false
```

filed under `data_dir()/workspace_trust/<sha256(workspace_path)>` — the filename is the hash of the workspace’s absolute path, and the path is stored again inside as a plain field. Hashing the path for the filename avoids every question about path characters, length limits, and case folding across filesystems; keeping the readable path in the contents means the directory is still auditable by a human with `grep`.

The “one file per workspace” shape is a concurrency decision, and the source says so: it “is safe under multiple concurrent zmax instances writing trust for *different* workspaces (different filenames). Two instances racing to trust the same workspace converge to identical content.” No lock file, no database, no merge. A single shared trust file would have needed all three, because the user’s sixteen synchronized editor panes are exactly the scenario that finds the race.

Four capabilities, not one bit

`TrustQuery` is what callers ask about, and there are four of them:

```
pub enum TrustQuery {
    Lsp,           // language server permissions
    Dap,           // debug adapter permissions
    LocalConfig,   // whether .zmax/ config can be loaded
    Git,           // whether git integration can trust .git/config
}
```

Four distinct questions rather than one boolean, because they have different risk profiles and different answers. `Git` is on the list for a reason that is easy to miss: a repository’s `.git/config` can name commands — the classic hook and external-diff vectors — so trusting a repository’s *contents* and trusting its *git configuration* are separate grants.

Stale, and why it is not untrusted

`TrustStatus` has four values, and the third is the one that carries the design:

```
pub enum TrustStatus {
    Trusted,
    Untrusted,
    Stale,
    Excluded,
}
```

A grant snapshots a hash of every file under `.zmax/`. If those files change later, the workspace becomes `Stale`, and the documented consequence is precise: local config is no longer loaded until the user re-runs the trust command, but “language server launches may still proceed under stale trust because they use the (unchanged) globally-configured binaries.”

That distinction is the entire value of having a third state. The thing that changed is the workspace-local configuration; the thing that did not change is the set of binaries the user configured globally. Revoking everything on any change would train the user to re-trust reflexively, which destroys the meaning of the prompt. Revoking exactly the surface that changed keeps the prompt informative.

`Excluded` is the fourth state and is a permanent no — “never prompts again” — which is the answer for a directory full of cloned repositories you read but never run.

The escape hatch, and its warning label

The `trusted` glob list from the configuration above has a documentation block that is unusually blunt for a feature the project ships:

This is deliberately weaker than an explicit `:workspace-trust` grant and is discouraged: it bypasses the `.zmax/` hash pin (changes to local config are never re-checked) and it trusts any repository that happens to land under a matching directory, including ones cloned there later.

Both failure modes are named. A glob-trusted workspace can never go `Stale`, because there is no snapshot to compare against. And a glob is a statement about a *location*, not about a repository, so anything cloned into that location afterwards inherits the trust. The section heading in the source is literally “Trusted globs (discouraged).”

The feature exists anyway, because the alternative for a user with hundreds of repositories in a predictable layout is granting trust one repository at a time forever, and a security control that is too tedious to use gets switched off entirely. Shipping it with its weaknesses documented in the source is the honest middle: the user who reaches for it knows what they are giving up.

One ordering rule closes the model: “An explicit exclude still wins over a matching glob.” The most specific negative decision beats the broad positive one, which is the same precedence principle Chapter 89 applies to configuration — whoever was most specific about this exact thing wins.

Chapter 46 – A Complete elisp Extension

The recipes so far have been fragments. This chapter writes a *whole* small extension — the kind you would actually keep in your init — end to end in embedded elisp, then walks it line by line, because seeing a complete program is worth more than a dozen snippets. The extension adds three conveniences a working programmer reaches for: duplicate the current line, leave a commented copy of the region, and a “scratch note” that drops a timestamped line. It is written against the elisp embedding of Chapter 20, the tongue furthest along, and it is the shape your init grows into.

Everything it calls is real, and it is worth saying which half is which before reading it, because Chapter 22 drew the distinction and this chapter is where it bites. The buffer-text calls — `point`, `insert`, `forward-line`, `beginning-of-line`, `end-of-line`, `buffer-substring`, `format-time-string` — are elisp’s own builtins, running against the mirrored `EditBuffer` that `eval_elisp` syncs in before the `eval` and flushes out after. The editor-level calls — `message`, `editor-command` — are the subrs `zmax` registers, reaching the live editor through the thread-local context pointer. Both work; they take different roads.

```
;;; zmax init - small editing conveniences

(defun duplicate-line ()
  "Copy the current line and paste it just below, cursor on the copy."
  (let ((start (progn (beginning-of-line) (point)))
        (end   (progn (end-of-line) (point))))
    (let ((line (buffer-substring start end)))
      (goto-char end)
      (insert "\n")
      (insert line))))

(defun comment-and-copy (token)
  "Leave a commented copy of the region above it, original intact."
  (let ((text (buffer-substring (region-beginning) (region-end))))
    (goto-char (region-beginning))
    (beginning-of-line)
    (insert (concat token " " text "\n"))))

(defun scratch-note (token msg)
  "Insert a timestamped note - handy for TODOs you will grep later."
  (insert (format "%s NOTE(%s): %s\n"
                  token
                  (format-time-string "%Y-%m-%d")
                  msg)))

(defun save-and-format ()
  "Reformat the buffer and write it - two editor commands, one call."
  (editor-command "format")
  (editor-command "write"))

(message "init loaded")
```

The walkthrough

`duplicate-line` is the shape of every buffer-editing command in the language, and it is ordinary emacs elisp because elisprs implements ordinary emacs elisp. `beginning-of-line` and `end-of-line` move point, `point` reports where it is, `buffer-substring` lifts the text between two positions, and `insert` puts text back. Nothing here is a zmax invention; if you have written this function in GNU Emacs it is the same function. What zmax supplies is the mirror underneath: the buffer these calls see is a copy of your live document, and when the eval finishes `flush_host_into_buffer` writes it back as a single transaction. The practical consequence to know is that the whole eval lands as *one* undo step, which is usually what you want for a scripted edit and occasionally surprising.

`comment-and-copy` reaches the selection through `region-beginning` and `region-end` — these are among the twenty-two subrs zmax registers (Chapter 20), and they report zmax's live selection because `load_buffer_into_host` mirrors a selection whose anchor differs from its head into elisp's mark. The comment token is a parameter rather than a (`comment-token`) primitive; there is no such primitive, and passing it in keeps the function honest about what the embedding actually offers.

`save-and-format` is the other road. `editor-command` is the general escape hatch: it takes the name of any zmax typable command and runs it, which means the entire `:` surface of Chapter 11 is callable from elisp without zmax having to register a subr per command. Most of the emacs-named commands you would reach for are already wrapped this way — `find-file`, `save-buffer`, `switch-to-buffer`, `goto-line` and a dozen more are `defuns` in the `EDITOR_LISP` layer that route through `editor-command` — so (`save-buffer`) works, and what it does underneath is run `:write`.

Binding it

One thing this extension does *not* do is bind itself to a key, and the reason is a real limit rather than an omission. elisprs has a `define-key`, but it operates on elisprs's own keymap objects; there is no bridge from those to zmax's `KeyTrie`, so an elisp `define-key` will not put a command on `SPC m d`. Binding is done from the editor side, in `config.toml`, where a key may be mapped to any typable command — and `:elisp` is a typable command:

```
[keys.normal.space.m]
d = ":elisp (duplicate-line)"
c = ":elisp (comment-and-copy \"/\"")"
n = ":elisp (scratch-note \"/\" \"\")"
```

Or from vimscript, which *does* have a keymap bridge — `install_map_hook` in `zmax-term/src/commands/scripting/mod.rs` feeds `:nmap` lines into zmax's runtime keymap overlay — so a sourced `.vimrc` can bind an elisp call:

```
nnoremap <Space>md :elisp (duplicate-line)<CR>
```

That asymmetry is worth carrying away, because it is the clearest small illustration of the book's general point about the five tongues: they do not all reach the editor equally far. elisp has the deepest buffer surface and no key bridge; vimscript has the key bridge. Used together they cover the task, which is an argument for plurality that a chapter claiming each tongue does everything could not have made.

Chapter 47 – A Complete Configuration, Annotated

Chapter 35 built a config from scratch a few lines at a time. This chapter shows a *whole* one — the config a returning vim user might actually settle into after a week — as a single annotated listing, so you can see how the pieces of Appendix A fit together into a coherent setup rather than a pile of settings. Read it top to bottom; the comments are the tour.

```
# ~/.config/zmax/config.toml - a worked, opinionated setup

theme = "tokyonight"

[editor]
line-number = "relative"      # 5j / 12k become a glance, not arithmetic
scrolloff = 8                 # keep context around the cursor
cursorline = true             # subtle highlight on the current line
true-color = true             # 24-bit color on a modern terminal
rulers = [100]                # a guide at column 100
auto-format = true            # format on save where a formatter exists
text-width = 100
bufferline = "multiple"      # show the bufferline only with >1 buffer

[editor.cursor-shape]         # see the mode without reading the word
normal = "block"
insert = "bar"
select = "underline"

[editor.lsp]
display-inlay-hints = true    # inline type annotations
auto-document-highlight = true
display-signature-help-docs = true

[editor.soft-wrap]
enable = true                  # wrap prose instead of scrolling sideways
wrap-indicator = "↵"

[editor.statusline]
left = ["mode", "spinner", "file-name", "file-modification-indicator"]
center = ["diagnostics"]
right = ["selections", "position", "file-encoding", "file-type"]

[editor.indent-guides]
render = true
character = "│"

[editor.search]
smart-case = true             # case-insensitive unless you type a capital

[editor.file-picker]
hidden = false                 # show dotfiles in the picker

[keys.normal]
";" = "command_mode"         # ; for : - a classic vim remap
"X" = "extend_line_above"

[keys.normal.space]
"w" = ":write"
"q" = ":quit"
```

Why these choices hang together

The settings are not a random grab; they cohere around a returning vim user who wants the editor legible at a glance and quiet underfoot. The `line-number = "relative"` and the mode-shaped cursors make the *state* of editing visible — which line is which distance, which mode you are in — without reading text. The `[editor.lsp]` block turns on the semantic help (inlay hints, signature docs, document highlight) that is off by default because it is heavy, the deliberate choice of someone who has decided the richness is worth the visual

weight. The `soft-wrap` and `rulers` and `text-width` together say “I write to a hundred columns and want to see that boundary and wrap past it rather than scroll.” The statusline is tuned to show diagnostics centrally and position and file facts on the right, trimming the defaults to what this user watches. And the `[keys]` tables make the two or three remaps every vim user carries — `;` for the command line, leader bindings for save and quit. Because the config hot-reloads (Chapter 27), this file was not written in one sitting; it accreted, a line at a time, each change saved and felt live. That is the intended relationship with the config: not a thing you set once and forget, but a thing you tune continuously, in a declarative language documented leaf by leaf in Appendix A — with the option, when declarative is not enough, of dropping into one of the five tongues (Chapter 46) and configuring the editor as a program instead.

Chapter 48 – Porting a Real .vimrc

Chapter 36 sketched vimscript configuration; this chapter does it at the scale of a real `.vimrc` — a substantial init a long-time vim user might bring across — so you can see how much of a working vim configuration maps onto zmax and where the seams are. As Chapter 21 was careful to say, the vimscript host wiring (the ex-commands that reach the editor) was the largest of the embedding gaps, and it has moved fastest of the four tongues since (Chapter 71): the `set` and `map` blocks below now reach the editor's real options and keymap, so what follows is close to a `.vimrc` you can source today, with the `command!` definition the last piece still landing.

```
" ~/.zmax/init.vim – a working vim user's config, ported

" --- options -----
set number relativenumber      " hybrid line numbers
set scrolloff=8
set ignorecase smartcase      " smart-case search
set textwidth=100
set expandtab shiftwidth=4

" --- leader mappings -----
let mapleader = " "
nnoremap <leader>w :write<CR>
nnoremap <leader>q :quit<CR>
nnoremap <leader>e :file-explorer<CR>
nnoremap <leader>f :file-picker<CR>

" --- quality-of-life remaps -----
nnoremap ; :
nnoremap Y y$                  " yank to end of line, like D and C
nnoremap <C-l> :nohlsearch<CR> " clear search highlight

" --- a small function: strip trailing whitespace -----
function! StripTrailingWS()
  let l:save = winsaveview()
  keeppatterns %s/\s\+$/e
  call winrestview(l:save)
endfunction
command! StripWS call StripTrailingWS()
nnoremap <leader>sw :StripWS<CR>
```

What ports, and what to watch

Read top to bottom, most of this is the muscle memory of a vim user's `.vimrc` arriving intact. The `set` block maps onto editor options — `number/relativenumber` onto `line-number`, `scrolloff` directly, `ignorecase/smartcase` onto the smart-case search of Chapter 42, `textwidth` and the indent settings onto their `[editor]` equivalents. The `nnoremap` lines bind normal-mode keys to commands, and the commands they target are zmax's own — `:write`, `:quit`, `:file-picker`, `command_mode` via `:` — so a vim user's leader scheme transfers by changing the *targets* of the maps, not their *shape*. The `StripTrailingWS` function is the interesting case: it is real vimscript with a local variable, a `keeppatterns` substitution, and view save/restore, exactly as it would sit in a `.vimrc`, exposed as a `:StripWS` command and bound to the leader.

The seam, stated plainly: the `set` and `nnooremap` and `command!` lines are the ex-commands whose host wiring Chapter 21 flagged as in progress — and, as Chapter 71 records, most of it has since landed. The `set` block now reaches real options (a mapped subset changes live editor state, the rest are stored the way vim keeps inert options), and the `nnooremap` lines bind into the actual keymap; the one still awaiting its wiring is `command!`, the user-defined `:StripWS` command, which is why that line is the last thing between this init and sourcing cleanly. The point of showing the whole file anyway is the size of the prize — a vim user porting a config that is *this close* to their real one, in the actual language they wrote it in, into an editor whose keys they already know — which is a migration story no other editor offers, because in every other editor vimscript is either the native tongue (so there is nothing to embed) or absent entirely.

Chapter 49 – A Data-Wrangling Session

The book's last worked example is the one that puts the most pieces together: a messy data file turned clean, using selections, the embedded tongues, and the command line in one sitting. It is the kind of task that, outside `zmax`, scatters across a terminal, a couple of one-off scripts, and the editor; here it happens in one buffer without leaving the process. As with the other embedding recipes, treat the tongue invocations as the intended shape (Chapter 21).

The mess

You have pasted a few hundred lines of server log into a buffer. Each line looks like:

```
2026-06-28T11:42:07Z WARN auth user=alice latency=412ms
2026-06-28T11:42:09Z ERROR payments user=bob latency=1903ms
2026-06-28T11:42:10Z INFO auth user=alice latency=88ms
```

You want, from this: only the `WARN` and `ERROR` lines, reduced to subsystem and latency, sorted by latency descending, with a count of how many came from each subsystem at the top. Four transforms — filter, project, sort, summarize — of four different shapes.

The pipeline, step by step

First, *filter* to the lines that matter. The global command deletes the rest:

```
:g/\sINFO\s/d
```

Now every remaining line is a `WARN` or `ERROR`. Second, *project* down to subsystem and latency with embedded AWK, which is built for exactly this columnar pull (Chapter 32):

```
{ gsub(/latency=/, "", $5); print $3, $5 }
```

Select the block, run it through AWK, and the lines become `auth 412ms`, `payments 1903ms`. Third, *summarize* the subsystem counts with embedded Zsh, the shell incantation of Chapter 34, run over a copy of the column:

```
awk '{print $1}' | sort | uniq -c | sort -rn
```

yank that result to the top of the buffer as your header. Fourth, *sort* the detail lines by latency descending — strip the `ms`, sort numerically — which is one more small pass, an embedded stryke `sort` when the key needs computing:

```
print for sort { ($b =~ /(\d+)ms/)[0] <=> ($a =~ /(\d+)ms/)[0] }
split /\n/, selection_text();
```

The point of the whole movement

Step back from the keystrokes and look at what just happened: a filter in the command line, a projection in AWK, a summary in Zsh, a sort in stryke — four languages and the editor's own command surface, applied to one buffer, in one sitting, without a single context switch to a terminal or a temp file. Each transform was expressed in the tool whose grain it ran with, and every tool was *in the editor*. That is the thesis of the whole book made operational: an editor conservative enough to take your vim keys and your shell reflexes unchanged, and radical enough that the shell, and AWK, and stryke, and elisp, and vimscrip are not programs it shells out to but languages it *is* — five tongues on one VM in one address space, reachable by a function call, driving the one buffer in front of you. The reference that follows catalogs the parts; this chapter was the parts working together, which is the only test that finally matters.

Chapter 50 – A Complete AWK Report

Chapter 32 used AWK for a one-line projection; here is AWK doing what it is *most* famous for, a grouped report with running totals, as a complete program over a buffer. The task: a buffer holds a CSV of sales — `region,rep,amount` — and you want a report of total and average sale per region, plus a grand total, written back into the buffer. This is the AWK associative-array idiom in full, and it runs over the selection through the embedded interpreter (Chapter 21).

```
BEGIN { FS = ","; OFS = "\t" }

NR == 1 { next }           # skip the header row

{
    total[$1] += $3        # sum by region
    count[$1] += 1         # tally by region
    grand += $3            # grand total
}

END {
    print "REGION", "TOTAL", "AVG", "N"
    for (r in total)
        printf "%s\t%d\t%.2f\t%d\n", r, total[r], total[r] / count[r], count[r]
    print "----"
    printf "GRAND\t%d\n", grand
}
```

The walkthrough

The `BEGIN` block sets the input field separator to a comma and the output separator to a tab, run once before any line. The `NR == 1 { next }` rule skips the header — `NR` is the record number, so the first row is dropped. The bare `{ ... }` block runs for every remaining line and is the heart of the idiom: `total[$1] += $3` uses the region (field one) as a key into an associative array and accumulates the amount (field three), `count[$1]` tallies the rows per region, and `grand` sums everything. AWK creates the array entries on first reference, so there is no declaration — the regions appear in the arrays as the data introduces them. The `END` block runs once after the last line, printing a header, then looping `for (r in total)` over the regions to print each one's total, computed average, and count with `printf` formatting, and finally the grand total. Select the CSV, run it through embedded AWK, and the selection is replaced by the report — grouping, summing, and averaging that would be tedious by hand and is four lines of body in AWK, running in the editor's process over the editor's buffer. This is why AWK earns a place among the five despite being the odd one for editor *scripting*: for tabular *reporting*, nothing is terser, and now it lives where your data already is.

Chapter 51 — A Refactor, Start to Finish

This chapter is a single refactor worked completely, to show the editor's tools composing on real code rather than in isolation. The starting code is a Rust function that has grown a repeated pattern and a poorly-named local:

```
fn summarize(items: &[Item]) -> Report {
    let mut r = Report::new();
    for it in items {
        r.total += it.amount;
        r.count += 1;
        if it.amount > r.max { r.max = it.amount; }
    }
    r
}
```

You want three changes: rename `r` to `report` (it is returned and deserves a real name), rename the loop variable `it` to `item`, and you have noticed `summarize` is called from a dozen places under an old name `tally` you have been meaning to retire.

The sequence

The two local renames are *buffer-scoped* and mechanical, so they are multi-cursor work (Chapter 30), not a language-server rename — `r` and `it` are local, and a semantic rename would be overkill. Select the function body, narrow the selection to the occurrences of `r` as a standalone word, and you have five cursors on five `rs`; type `creport` — change to `report` — and all five update together. Repeat for `it` to `item`. The body now reads:

```
fn summarize(items: &[Item]) -> Report {
    let mut report = Report::new();
    for item in items {
        report.total += item.amount;
        report.count += 1;
        if item.amount > report.max { report.max = item.amount; }
    }
    report
}
```

The third change is different in kind, and that difference is the lesson. `summarize` is a *public* name used across the project, so renaming it is the language server's job, not a buffer edit: put the cursor on `summarize`, run rename (`SPC r`, Chapter 14), type the new name once, and the server rewrites every call site in every file — including the ones you never opened — as a set of transactions. Finish with `:format` to normalize the spacing, and the refactor is done. The point is the *division of labor*: two of the three renames were selections because they were local and textual; the third was a server rename because it was semantic and project-wide. An expert does not reach for one tool; they read the *scope* of each change — this buffer, or the whole project; textual, or semantic — and pick accordingly, and `zmax` puts both tools a keystroke apart so the choice is the only effort.

Chapter 52 – A New Command in AWK

The practical movement closes the way it has run: with a complete, usable extension, this time in an embedded tongue other than elisp, to balance Chapter 46 and show a second road into the editor. The command aligns the current selection on a delimiter — turning a ragged block of `key = value` lines into a column-aligned one — which is the canonical “I wish my editor did this” task.

The tongue is AWK, and the choice is not arbitrary. This task is a *filter*: read a region, transform it, write the region back. That is exactly the shape `:awk` has — `run_awk_filter` in `zmax-term/src/commands/scripting/mod.rs` reads the primary selection (or the whole buffer if the selection is empty), hands the text to `awkrs::run_program`, and replaces the range with the output as one transaction. The editor supplies the region plumbing; the AWK program is pure text-to-text, which is the only thing AWK ever wanted to be. A two-pass alignment is also the classic AWK idiom, since AWK has a `BEGIN`/body/`END` structure built for exactly “measure everything, then emit.”

```
{
    lines[NR] = $0
    if (index($0, "=") > 0) {
        lhs = substr($0, 1, index($0, "=") - 1)
        sub(/[ \t]+$/, "", lhs)
        left[NR] = lhs
        rhs = substr($0, index($0, "=") + 1)
        sub(/^[\t]+/, "", rhs)
        right[NR] = rhs
        if (length(lhs) > width) width = length(lhs)
    }
}

END {
    for (i = 1; i <= NR; i++) {
        if (i in left)
            printf "%-*s = %s\n", width, left[i], right[i]
        else
            print lines[i]
    }
}
```

The walkthrough

The body block is the measuring pass, and it runs once per line because that is what an AWK body block does — there is no loop to write. For each line it finds the first `=` with `index`, splits the line there with `substr`, trims trailing whitespace from the left-hand side and leading whitespace from the right, stashes both in arrays keyed by `NR`, and tracks the widest left-hand side in `width`. Lines with no `=` are kept verbatim in `lines` and deliberately left out of `left`, which is how they survive untouched.

The `END` block is the rewrite. It walks the line numbers in order and uses `printf` with `%-*s` — left-justify in a field whose width is supplied as an argument — to pad every left-hand side to the same column. `if (i in left)` is the test for “did this line have a separator”; if not, the original text is printed unchanged. The output of the whole program is what replaces the selection.

Note what is *not* in the program: there is no `editor::` anything, no binding call, no host primitive. Earlier edi-

tions of this book showed this chapter with an invented `editor::selection_text()` / `editor::replace_selection()` / `editor::bind()` API and no such API exists — not in `stryke`, not in any of the five. The region I/O is on the Rust side, in the `typable` command, which is a cleaner division than the fictional one and has the advantage of being true.

Binding it

`:awk` takes the program itself as its argument — `awk_filter` joins the whole argument list into the program string — so there is no `-f script.awk` form to bind to. A bound key carries the program, which means squeezing it onto one line:

```
[keys.select.g]
a = ":awk { l[NR]=$0; if (index($0,\"=\")==0) { a=substr($0,1,index($0,\"=\")==0-1); sub(/[
  ↪  \\t]+$/,\"\",a); b=substr($0,index($0,\"=\")+1); sub(/^[ \\t]+$/,\"\",b); l[NR]=a;
  ↪  R[NR]=b; if (length(a)>w) w=length(a) } } END { for (i=1;i<=NR;i++) if (i in l) printf
  ↪  \"%s-%s = %s\\n\", w, l[i], R[i]; else print l[i] }"
```

That is unpleasant to read and it is the honest cost of a filter whose program is its argument. The multi-line version above is what you would actually type at the `:awk` prompt while working; the one-liner is what you would commit to a config once you had settled on it. The same program bound from a sourced `.vimrc` goes through the keymap bridge instead:

```
vnoremap ga :awk { l[NR]=$0; ... }<CR>
```

Across two chapters you now have two real extensions in two of the five tongues, reaching the editor by two different roads — `elisp` through a mirrored buffer and the `editor-command` `subr`, `AWK` through a region filter — and bound by a third, `vimsript`'s keymap hook. That is the practical shape of the five-tongue claim: not one uniform API wearing five syntaxes, but several genuine roads into one editor, and the judgment to pick the one whose grain matches the task.

Chapter 53 – A Zsh Build-and-Test Loop

The tightest loop in programming is edit, build, test, read the error, edit again — and the friction in it is usually the trip out to a terminal and back. `zmax`'s embedded Zsh (Chapter 34) collapses that trip: run the build from inside the editor and capture its output into a buffer you can navigate like any other text. This chapter is that loop, with the standing caveat from Chapter 21 that the output-capture route `zshrs` needs is the gap being closed; the shape below is the target.

The command

You want a single key that saves everything, runs the project's build and tests, and drops the output into a scratch buffer where the errors become navigable. In embedded Zsh:

```
run_tests() {
  local out
  out="$(cargo build 2>&1 && cargo test 2>&1)"
  editor scratch "build-output" "$out"
  if print -r -- "$out" | grep -q '^error'; then
    editor status "build failed - see *build-output*"
  else
    editor status "ok: tests passed"
  fi
}
editor bind normal 'SPC t t' run_tests
```

The function runs `cargo build` and, if it succeeds, `cargo test`, capturing the combined output into `$out`; `editor scratch` opens (or reuses) a buffer named `build-output` and fills it with that text; a `grep` decides the status-line message. The binding hangs it off `SPC t t`. Press the key, and without leaving the editor the project builds, the tests run, and the result is a buffer in front of you.

Why a buffer, not a pane

The move that makes this more than a convenience is that the output lands in a *buffer*, not a throwaway terminal pane, and a buffer is the thing `zmax` is best at. The compiler's `file:line:col` error locations are now text in a buffer you can search with `/`, walk with the diagnostic-style motions, and — because they are real paths — jump from. The edit-build-test loop becomes edit, `SPC t t`, read the errors in a buffer, jump to the first one, fix, `SPC t t` again, all inside the editor, with the build driven by the shell you already know running in the editor's own process. The external `:!cargo test` of Chapter 11 does the build too, over a real subprocess; the embedded version's distinction is that the shell, the capture, and the navigation are all one process and one set of buffers, which is the whole point of embedding a shell rather than shelling out to one.

Chapter 54 – A Snippet Library

The book's last practical chapter is a small, usable artifact: a library of snippets for the patterns you type most, presented as the definitions you would actually keep. Chapter 39 explained the mechanism — tab stops, mirrored holes, the `zmax-parser` parser — and this chapter is the payoff, a set of bodies worth stealing, with notes on what makes each one earn its trigger.

A handful worth keeping

Snippet bodies use `${N:default}` for the Nth tab stop with a placeholder, `$N` for a bare or mirrored stop, and `$0` for the final cursor. A practical starter set:

```
# trigger: fn - a Rust function
fn ${1:name}(${2:args}) -> ${3:()} {
    ${0:todo!()}
}

# trigger: test - a Rust test, name mirrored into nothing but the body waiting
#[test]
fn ${1:it_works}() {
    ${0}
}

# trigger: for - a loop whose variable is mirrored into the body
for ${1:item} in ${2:iter} {
    ${3:process}(${1});
}

# trigger: main - the entry point
fn main() {
    ${0}
}
```

The `fn` snippet walks you name → args → return type → body with three Tabs and rests in the body. The `for` snippet is the one that shows the engine off: `${1:item}` appears in the header *and* is mirrored as `${1}` in the body, so editing the loop variable at the first stop updates both sites at once — the multiple-selection model of Chapter 30 surfacing inside the template, because a repeated tab stop is just two ranges in one selection. The `test` snippet selects a default name you usually type over and drops you in the body.

Where they live and why it matters

Snippets reach you two ways. Many come from the language server as completion candidates — accept one and it expands (Chapter 39) — so a great deal of this library you already have for free wherever a good server is configured. The rest are your own: the four bodies above go straight into your personal library through `:snippets`, each under the trigger named in its comment, and from then on `fn` or `for` expands it in place with the tab stops armed — no server required, scoped to Rust so the triggers stay out of your way in other languages. The reason a snippet library is worth assembling once is the reason the whole In Practice movement exists: the editor's leverage is highest on the things you do most, and nobody types more boilerplate than the patterns above. Between snippets for the boilerplate, completion for the names, multiple cursors for

the repeated edits, macros for the repeated shapes, and five embedded languages for the transforms that need real logic, zmax's wager is that the mechanical parts of editing should mostly be done *for* you — by the same few mechanisms, over the same one engine, reached by the keys you already know. That is the book's case, and the reference that follows is the catalog you keep open while you put it to work.

Chapter 55 — The Workbench

The working-editor movement (Chapters 11–17) covered what `zmax` can *do* — the command line, pickers, language servers, version control. This chapter covers the surfaces that wrap those capabilities into something that looks and behaves like an IDE, for the times you want one. They are all optional, and the modal editor underneath is unchanged whether they are showing or not.

IDE mode

`:ide` (alias `:workbench`, the static command `toggle_ide`, or `F2`) turns the plain editor into a JetBrains-style workbench rendered inside the same view: a left column holding a **PROJECT** file-tree above a **STRUCTURE** outline, a bottom panel for **PROBLEMS** and a run console, and a right-hand error-stripe minimap. The leader reaches the same toggle at `SPC z`, and you can boot straight into it with `--ide`. Nothing about the buffer changes — the panels are chrome drawn around the same `View`, so every key you know still works; you have only surrounded the text with the context an IDE keeps on screen.

What makes it livable across sessions is that the layout *persists*. The drawer's width, which panels are collapsed, whether the minimap is folded, the height of the bottom panel and its splits — all of it is written to an `[ide]` table in `appdata.toml` when you quit and restored when you next open `zmax` without a file argument. You arrange the workbench once; it stays arranged.

The panels

The workbench is the full JetBrains tool-window set, and each panel is focused by name the way IntelliJ's `Alt+number` does it — under the `SPC W` (“workbench”) prefix, which reveals the workbench, opens the target panel, and routes your keys into it until you return to the editor. The **PROJECT** file-tree is `SPC W t` (or `SPC W p`); `reveal_in_tree` finds the current file inside it, and an auto-reveal toggle keeps it following the buffer you edit. (This is distinct from `SPC a f`'s dired-style file *explorer* picker, which is a transient fuzzy list rather than a standing sidebar.) The **STRUCTURE** outline — `document_outline` walking the tree-sitter tag query for the file's functions, types, and other named regions — is `SPC W s` (or `SPC W o`), and the same symbols are a fuzzy jump away through the symbol picker (`SPC s j`). Along the bottom sit the tool windows you act *from*: **PROBLEMS** (`SPC W e`) lists the diagnostics, the **RUN** console (`SPC W r`) holds program output and the jumpable hits from project search (Chapter 58 — scroll it with `j/k`), the **GIT** changes panel (`SPC W g`) is the Commit tool window, the staging surface of Chapter 16, and the **CI** panel (`SPC W c`) lists GitHub Actions runs and opens one in the browser on `Enter`. `SPC W m` maximizes the bottom panel to full height for reading a long log or diff, and restores it again. The **minimap** is the thin error-stripe down the right edge; it has no key of its own — you fold it by dragging it toward the edge and unfold it with a click, and its folded state rides along in the persisted layout.

Two smaller pieces complete the IDE feel. A **breadcrumb** runs along the toolbar — the path to the symbol the cursor sits in — and clicking it reveals the file in the project tree, the JetBrains “where am I” strip made navigable. And the toolbar carries a **run/debug** control: the Run/Debug Configurations you manage below (`SPC R c`) launch from here, and a live session drives the **DEBUG** tab — the same DAP debugger of Chapter 15, surfaced as a tool window with a toolbar rather than reached as a bare command.

Preferences, run configs, and the keymap switch

`SPC ,` (or `:preferences`) opens a single tabbed Preferences window — Settings, Keymap, Theme, Run/Debug Configs, Help, and a Dashboard — so the editor’s configuration surface is a place you visit rather than a file whose keys you must remember; `SPC 5` opens it straight to Settings. The **run-configuration manager** is the Run/Debug Configs tab, reached directly with `SPC R c` / `SPC R e`: a JetBrains-style list of named launch configurations rather than a single hard-wired run command. And because zmax wears three keymaps at once, `:keymap` switches the active preset between `spacemacs`, `vim`, `helix`, and `emacs` at runtime — the whole personality of the editor changed without a restart.

The integrated terminal

`:terminal` (alias `:term`, or `SPC p '`) opens a real PTY shell running your `$SHELL` inside a pane, so the shell is one more split rather than a context switch out of the editor. The one problem a terminal pane must solve is that almost every key belongs to the shell — so window management hangs off a leader of its own: `C-\` arms a one-shot window command, and the next key splits (`v` / `s`) or moves focus (`h` / `j` / `k` / `l`) instead of being typed at the prompt. A click in another pane focuses it and a click back returns to the terminal, so the mouse crosses the boundary the keyboard is careful about.

The start screen and the mouse

Launch zmax with no file and it opens on a **start screen** — a vim-startify port, a fortune-and-cowsay header over two lists of recent files: a frequency-ranked set (most-used, weighted by recency) and a plain most-recently-used set scoped to the current directory. It is the “what was I working on” page, one keystroke from any of them.

zmax is keyboard-first but not keyboard-only. A click focuses the pane it lands in; the scroll wheel scrolls; the tab strip and gutter answer clicks; and the divider between splits is draggable — press on it and drag to resize the windows, the same proportions the `C-w +/->/<` keys adjust. The mouse is there for the moments it is genuinely faster, without ever being required.

The help browser

Finally, the editor documents itself. `SPC h h` (or `:help`, `:h`) opens a searchable Help browser — the Help tab of the Preferences window — over every command, keybinding, and topic, so “what runs on this key” and “what does this command do” are a fuzzy search rather than a trip to a manual. The `SPC h` family fans out from there: the describe submenu (`SPC h d`) explains a binding, a key, or an ex-command through the browser, while the describe-symbol keys (`SPC h d f` and kin, and `SPC m h h` at point) route through the language server’s hover instead — because the authoritative description of a symbol lives in the server, not the editor. The Spacemacs leader’s own which-key popups (Chapter 10) are the same instinct in miniature: the editor would rather show you the next choice than expect you to have memorized it. And for the most direct question of all — “what is this language actually doing” — there is the REPL (`SPC a r`, `:repl`), a live prompt fronting all five embedded tongues (Chapters 18–22), defaulting to elisp and switching languages with Tab. The workbench, in the end, is the editor’s headless power with its discoverability turned up: the same commands, the same engine, wrapped in surfaces that tell you what they can do.

Chapter 56 — Org Mode

Org mode is emacs's outliner, planner, and notebook in one, and zmax ports the part of it that earns its keep inside a text editor: a `.org` file as a foldable tree of headings, each able to carry a TODO state, a priority, and a date, with an agenda that gathers those across files into one dated to-do list. This is not the whole of org — no spreadsheet tables, no literate-programming babel blocks — it is the planning core, reached through the leader's major-mode menu (`SPC m`) on any org buffer.

The outline

A heading is a line of leading stars — `* Project`, `** Task`, `*** Subtask` — and its depth is the star count, exactly as in emacs. `SPC m <tab>` (or `SPC m z`, or `:org-cycle`) folds and unfolds the subtree under the heading at the cursor — the TAB-cycling that turns a long org file into a collapsed table of contents you open one branch at a time — while `SPC m a` / `SPC m A` (`:org-fold-all` / `:org-unfold-all`) collapse or open the whole buffer at once. `SPC m j` and `SPC m k` step to the next and previous heading, and `SPC m H` / `SPC m L` promote and demote the heading, removing or adding a star to move it up or down the outline.

TODO states and priorities

A heading becomes a task by carrying a keyword after its stars. `SPC m t` (`:org-todo`) cycles the current heading through `none` → `TODO` → `DONE` → `none`, the three-beat rhythm of checking work off; `SPC m p` (`:org-priority`) cycles a priority cookie `none` → `[#A]` → `[#B]` → `[#C]` → `none`. Those two marks — a state and a priority — are all the agenda needs to treat a heading as something to be done.

The agenda and capture

`SPC a o a` (`:org-agenda`, or `:agenda`) opens the agenda: it scans the TODO/DONE items across every open `.org` buffer and every `*.org` file under the working directory, reads each one's scheduled and deadline dates, and presents them grouped by date — overdue first, then today, then upcoming — so “what is due” is answered from all your org files at once rather than file by file. Its companion is capture: `SPC a o c` (`:org-capture`, or `:capture`) prompts for a line of text and appends it as a `* TODO <text>` entry to an inbox file (`inbox.org` in the working directory by default, or a path you pass), so a thought becomes a tracked task without leaving the buffer you are in or hunting for where to put it. Together they are the smallest loop that makes org useful: capture catches the task, the agenda surfaces it when it comes due.

Chapter 57 — The Transform Library

Movement six showed how to wrangle text with the embedded languages — an AWK one-liner, a stryke pipeline, a zsh loop (Chapters 49–50). That path is for the transform that needs *logic*: a condition, a computed field, a shape you could not have anticipated. But most of the transforms you actually reach for are not bespoke — they are the same hundred chores everyone does to text, and zmax ships them pre-built as a library of typable commands so you do not have to rewrite “convert this JSON to CSV” or “align these on the equals sign” from scratch each time. There are well over two hundred of them. This chapter is the map of that shelf, not the full catalog (Appendix F points at the rest); the point is to know the shape of what is there so you reach for `:json-to-csv` instead of opening AWK.

The model: selection in, text out

Every transform follows one rule: it runs on the **selection** — or, when nothing useful is selected, the whole buffer — and replaces it with the result. That is the entire interface, and it is why the library composes with everything else in the book: select with a motion, a text object, or a multi-cursor set, run a transform, and the engine’s transaction machinery (Chapter 24) applies it as one undoable change across every selected range. A handful are *non-destructive* — `:stats`, `:json-validate`, `:count-matches`, `:bases` report into the status line rather than rewriting — but the default is “the text you selected, transformed.”

Structured data: JSON, CSV, tables

The deepest family is JSON. `:json-query users.0.name` pulls a value at a dot-path; `:json-flatten` explodes an object into greppable `path = value` lines and `:json-unflatten` rebuilds it; `:json-to-csv`, `:json-to-toml`, `:json-to-lines`, `:json-to-jsonl` and their inverses convert between shapes; `:json-pick/:json-omit` keep or drop keys, `:json-sort` and `:json-unique` and `:json-group-by` reshape arrays, `:json-table` renders an array of objects as an aligned plain-text grid, and `:json-validate` reports the first syntax error with its location. The CSV/TSV side mirrors it: `:csv-column` keeps the Nth column, `:csv-to-tsv/:tsv-to-csv` convert, `:csv-to-html-table` and `:md-table` (align a Markdown pipe table) and `:transpose-grid` (rows become columns) reshape, `:csv-validate` checks that every row has the same field count. This is the path the data-wrangling chapter took through AWK, available as single commands for the cases that do not need a program.

Numbers and statistics

A column of numbers is its own little spreadsheet. `:stats` (and `:sum`) report count/sum/mean/min/max without touching the buffer; `:running-total`, `:running-max`, `:running-min`, and `:percent-of-total` replace each line with a cumulative or relative value; `:scale`, `:offset`, `:clamp`, `:to-fixed`, and `:abs` do per-line arithmetic; the `:sum-fields / :avg-fields / :max-fields` family reduces each row of whitespace fields to one number; `:bases` shows an integer in decimal, hex, octal, and binary at once, and `:calc` evaluates an arithmetic expression (or each selection in place). Numbers in text get their own helpers too: `:increment-numbers`, `:pad-numbers`, `:human-bytes`, `:ordinal`, `:dec-to-hex/:hex-to-dec`.

Text, identifiers, and case

The identifier-case set — `:to-snake`, `:to-kebab`, `:to-camel`, `:to-pascal`, `:to-constant`, plus `:cycle-case` and `:change-case` to rotate through them — is the refactoring chore of renaming `userId` to `user_id` made one keystroke, and because it runs on a multi-cursor selection it renames every occurrence at once. Around it sit the small line chores that fill a day: `:align/:tabularize` on a delimiter, `:reflow` to a width, `:head/:tail`, `:dedup` and `:dedup-adjacent`, `:shuffle` and `:sample`, the sort family (`:sort-by-field`, `:sort-by-length`, `:natural-sort`, `:sort-words`), `:number-lines`, `:trim-lines`, `:squeeze-blank-lines`, `:rotate-lines`, `:pad-left/:pad-right`, `:quote-lines/:unquote-lines`, `:join-with/:split-on`, `:unwrap-paragraphs`.

Extraction, encoding, markup, and generators

When the text contains what you want buried in what you don't, the extraction family pulls it out one match per line: `:extract-urls`, `:extract-emails`, `:extract-ips`, `:extract-numbers`, `:extract-quoted`, `:extract-between <a> <b>`, and the general `:extract <regex>`; `:count-matches` and `:uniq-count` count rather than extract. The encoder family covers the everyday cryptography-adjacent chores — `:base32-encode`, `:caesar/:rot47`, `:morse-encode`, `:crc32`, `:to-binary`, `:unicode-escape`, `:nato`, `:hexdump` — joined by the static `SPC x` shelf (URL-encode, Base64 and Base64URL, HTML entities, JWT decode, roman numerals, hex↔RGB colour). The markup family writes Markdown and HTML for you: `:code-fence`, `:markdown-link`, `:linkify`, `:checkbox-list`, `:ordered-list`, `:to-html-list`, plus the typographic polish of `:smart-quotes`, `:typographic-dashes`, `:to-ascii`, and `:de-typography`. And when you need text that is not in the buffer yet, the generators insert it at every cursor: `:uuid`, `:lorem`, `:date/:datetime/:timestamp`, `:seq <start> <end>`, and the static `SPC i` set (UUIDs, lorem, strong/phonetic passwords, a digraph and a Unicode picker).

When to drop to a language

The library is wide but not infinite, and its edge is exactly where Chapter 49 began: the moment a transform needs a *condition* the pre-built command does not have — keep rows where column three exceeds a threshold, join two files on a key, compute a field from three others — you drop to AWK, stryke, or zsh (`:awk`, `:stryke`, `:zsh`) and write the three lines. The two paths are the same idea at different grain: the transform library is the common case made instant; the embedded languages are the uncommon case made possible. Between them, the rule holds that the rest of the book has argued — the mechanical part of editing should be done *for* you — extended now from the keys to the content itself.

Chapter 58 — The Long Tail

A book has to stop somewhere, but an editor with over a thousand typable and two thousand static commands has more capabilities than it has chapters. This one gathers the rest — the features that do not each need their own chapter but should not go unmentioned, because a tool you do not know is there is a tool you do not have.

Clipboard and the primary selection

zmax distinguishes three places text can go. The unnamed register is vim's default yank/paste (Chapter 17). The **system clipboard** is the OS-wide one your other applications share: `:clipboard-yank` and `:clipboard-paste-after/-before/-replace` (and the static `SPC y/SPC p` bindings) reach it explicitly. And on X11 there is the **primary selection** — the middle-click buffer that holds whatever you last highlighted — reached through the parallel `:primary-clipboard-*` family. `:show-clipboard-provider` tells you which backend is wired up, and `:yank-join` copies a multi-selection joined by a separator you choose.

Moving around the filesystem

The working directory is mutable and stacked. `:cd` changes it; `:pushd` changes it while remembering where you were and `:popd` returns; `:dirs` (`:show-directory-stack`) shows the stack and `:pwd` the current entry — the shell's directory stack, in the editor. File operations live on the buffer too, vim-eunuch style: `:move` renames the file and the buffer together, `:delete-file` removes it from disk and closes it, `:mkdir` makes a directory (its parents included), `:chmod-x` makes the current file executable, and `:read` pulls another file's contents into the buffer. And when you close the wrong thing, `:reopen` (`SPC b u / SPC f u`) brings back the most recently closed file.

Project search and the Run console

Separate from the language server's semantic tools (Chapter 14), zmax keeps a set of purely *textual* project tools that report into the IDE Run console (Chapter 55). `:grep` (`:rg`) runs ripgrep across the project and lists jumpable hits; `:grep-word` (`:find-references`) does the same for the word under the cursor; `:todos` (`:fixme`) scans the tree for `TODO/FIXME/HACK/XXX/BUG/NOTE` markers; `:rename-word` renames every whole-word occurrence of a symbol *in the current buffer* — the textual cousin of the LSP's semantic rename, for the times there is no server or you want a literal replace. `:run` drives the Run tool window itself (defaulting to `cargo run`).

Navigating by leaps and pins

Beyond motions and search, two faster ways to jump. The **leap** motions label every visible occurrence of a two-character target and jump on the label you type — `goto_word` / `extend_to_word` (`SPC j l / SPC j w`), with `sneak` on `s/S` for the vim-sneak feel — so crossing the screen is two keystrokes regardless of distance. **Harpoon** (under `SPC H`) pins a handful of files you are actively working between and jumps among them by number (`harpoon_1–harpoon_4`, `harpoon_next/harpoon_prev`, `harpoon_menu`) — a manual, order-preserving alternative to the frequency picker (`SPC f r`), which ranks recent files by a z-style frequency-and-recency score.

Structural editing, folds, and narrowing

For Lisp and other paren-dense code, the **paredit** commands under `SPC k` edit the tree rather than the text: `slurp` and `barf` (pull the next form into the current parens, or push the last one out), `splice` (remove a wrapping pair), `raise`, `transpose`, and `wrap-sexp` — the structural moves that keep parentheses balanced as you reshape an expression. Orthogonally, **folds** go beyond Chapter 7's `zf: fold_toggle/open/close` and their `_all` variants collapse and expand regions, and **narrowing** (`narrow_to_region`, `SPC n r`) hides everything outside the selection so you can edit a region as if it were the whole file — the emacs idiom, distinct from narrowing a *selection* down to a match.

Spelling and macro counters

Spell-checking rides the unimpaired family: `]s / [s` jump to the next and previous misspelling, `z=` suggests corrections, and `zg / zw` teach the dictionary a word is good or bad (`zug` undoes the lesson). And the macro system of Chapter 40 carries a **counter**: `SPC K c a` seeds it and `SPC K c c` inserts its current value and increments — so a recorded macro can number the lines it touches, the classic “insert 1, 2, 3...” macro without editing the recording. None of these is large enough for a chapter; together they are the difference between an editor you use and one you have actually finished learning.

Chapter 59 — The JetBrains Layer

The first edition of this book named three editing traditions — vim, emacs, Spacemacs — and treated them as the whole of zmax’s personality. That was true when it was written and is not true now. A fourth tradition has been added, and it is the one that changes the pitch the most, because it is not another way to *move the cursor* but another way to *reason about code*: the JetBrains/IntelliJ IDEA keymap and the IDE workflows that hang off it. The README now lists the reference workflows as “Spacemacs and JetBrains,” in that order, and the port report tracks the IntelliJ default keymap as a first-class source — 218 cited keybindings, of which 177 are ported and 41 partial, an 81.2% muscle-memory match that puts the JetBrains layer second only to Spacemacs among the four traditions. This chapter is what that layer contains and why an editor that already speaks vim, emacs, and Spacemacs needed a fourth voice at all.

Why a fourth tradition

The three traditions of Movement II are all *editing* traditions: they disagree about which keys move and edit text, but they agree about what the editor is — a thing that edits text. JetBrains comes from a different lineage. IntelliJ IDEA and its siblings are not text editors that grew an IDE; they were born as IDEs, and their keymap reflects a different center of gravity. The operations a JetBrains user reaches for by reflex are not “delete a word” but “extract this expression into a variable,” “introduce this literal as a field,” “inline this call,” “rename this symbol across the project,” “run to the cursor,” “show me the structure of this file.” These are *semantic* operations — they require the editor to know what the code *means*, not just where the characters are — and they are exactly the operations a language server can answer. zmax already had the language server (Chapter 14) and the debugger (Chapter 15); what it lacked was the JetBrains vocabulary laid over them, so that a developer whose hands learned refactoring in IntelliJ finds those moves where their fingers expect them. The JetBrains layer is that vocabulary, and it sits on the same engine as everything else.

One structural point keeps the rest of the chapter honest: the JetBrains layer is *not* a keymap preset at all. The switchable presets are `spacemacs`, `vim`, `helix` and `emacs` (Chapter 66) — and IntelliJ is not among them. The JetBrains work is instead a set of *named static commands*, each carrying an IntelliJ doc-string, registered in `zmax-term/src/commands.rs` and implemented in `commands.rs`, `commands/lsp.rs`, and `commands/dap.rs`. You reach most of them through the command palette (`M-x`, `SPC SPC`) and a handful through the Spacemacs `SPC` menus; the IntelliJ chords the doc-strings cite — `Ctrl-Alt-Shift-T`, `Opt-F9`, and the rest — are the *reference* those commands model, not the keys zmax binds. So the JetBrains tradition is real, and its semantics are faithful, but it arrives as vocabulary reached by name rather than as a chord space you switch into.

Refactoring as a first-class verb

The heart of the JetBrains layer is refactoring, because that is the heart of the IntelliJ experience. zmax ports the canonical refactoring set as dedicated commands, each one driving the language server’s rename, code-action, and text-edit machinery underneath. The extract family is the core: **Extract Method**, **Extract Variable**, and **Extract Constant** each take the selected expression or statement range and lift it out — a method out of a block, a local out of a subexpression, a constant out of a literal — replacing the original with a reference to the new name; the commands are `extract_function`, `extract_variable` (which carries IntelliJ’s name *Introduce Variable*), and `extract_constant`, with a catch-all `extract_refactor` that offers every `refactor.extract` action the server reports as one menu. The introduce family is its near twin from

the other direction: `extract_field` (Introduce Field) and `extract_parameter` (Introduce Parameter) promote a local value up a level, turning a computed local into a field on the enclosing type or a parameter on the enclosing function. `inline_refactor` runs the extract operations in reverse, folding a name back into its single use, and `rewrite_refactor` reaches the change-signature class of edits. All of these route through the LSP's code-action and workspace-edit protocol — a *pinned* refactoring whose title matches a single action applies straight away, the way IntelliJ's dedicated Extract does, while an ambiguous one shows a menu — so the refactor is computed by the same server that powers `gd` and `gr`, and applied as one undoable transaction across however many files it touches.

The keystone is **Refactor This** — `refactor_this`, IntelliJ's `Ctrl-Alt-Shift-T` popup. Rather than make you remember which refactoring lives on which chord, it gathers every refactoring applicable at the cursor into a single menu and lets you pick. This is the JetBrains discovery idiom, and it is the same instinct as the Spacemacs leader and the vim auto-info popup seen earlier in the book: when there are more operations than a hand can memorize, show the menu rather than demand the chord. Refactoring was, for most of this layer's life, the thinnest of the JetBrains categories and earlier drafts of this book named it as the frontier. It is not one any more: `docs/port_report.md` scores `jetbrains / refactoring` at 13 of 13 and the deduplicated capability row `functionality / refactoring` at 5 of 5, both complete. The frontier moved, and what it moved to is smaller than what it left — of the 186 tracked capabilities the report now records 185 implemented, one partial and none absent, with `editing` (20 of 21) the only row not at 100%. That is a better position than the book originally claimed, and the reason to state it here rather than quietly is that the number is regenerated from source on every run: a reader who re-runs the generator gets whatever is true then, not whatever was true when this paragraph was written.

Multiple carets, the JetBrains way

`zmax` already had multiple cursors as the engine's native shape (Chapter 3), but JetBrains users reach them through different gestures, and the layer ports those. **Clone Caret** — `copy_selection_on_next_line / copy_selection_on_prev_line`, IntelliJ's `Ctrl-Ctrl`-then-arrow — drops an additional caret on the line above or below, the column-preserving way to grow a vertical stack of cursors one row at a time. **Select All Occurrences** — IntelliJ's `Ctrl-Alt-Shift-J` — turns every occurrence of the symbol under the cursor into a caret at once, the multi-cursor rename without a regex. And **Paste from History** ports IntelliJ's `Ctrl-Shift-V`, the clipboard-ring picker that lets you paste not just the last copy but any of the recent ones — the same idea as the emacs kill-ring of the next chapter, surfaced as a JetBrains gesture. Between clone-caret, select-all-occurrences, and the engine's own select-and-search (Chapter 30), the JetBrains multi-caret coverage is complete — 7 of 7 in `docs/port_report.md`.

Column selection and real block mode

There is one place the JetBrains layer did something the vim layer could not, and it is worth correcting an earlier chapter to mark it. Chapter 4 described vim's block-visual mode (`C-v`) as *emulated* with a stack of cursors — good enough for typing on every line, divergent in the edge cases where a rectangle and a cursor-stack are not the same shape. The JetBrains work changed that. It added a genuine `column_selection` command and, more consequentially, promoted Select mode's `C-v` to a *real* rectangular block — `visual_block_mode` — so the `C-v` you press is now a true column region, not the multi-cursor imitation the earlier chapter described. Inside it, `o` runs `block_swap_corners` and `O` runs `block_swap_columns`, the rectangle-aware versions of flipping the selection's ends. This is a rare case of the JetBrains tradition contributing a primitive the vim tradition then inherits: the block model IntelliJ users expect became the block model `C-v` gives everyone, and the selection category sits at 100% — three of three — on the strength of it. Where the two diverge, the truthful note is now the opposite of Chapter 4's: the rectangle is real, and only the deepest corners of vim's visual-block semantics remain approximate.

The View menu, ported to toggles

A great deal of the JetBrains daily experience is in the **View** menu — the toggles that change how code is *shown* without changing what it is. zmax ports the set as individual toggle commands, each mirroring a View-menu item: `toggle_line_numbers` (Show Line Numbers) flips the line-number gutter, `toggle_indent_guides` the vertical guides that trace nesting, `toggle_whitespace_render` (Show Whitespaces) the rendering of spaces and tabs, `toggle_soft_wrap` (View > Soft-Wrap) the wrapping of long lines, and `toggle_inlay_hints` the LSP-provided type and parameter annotations that float inline in the text. Scrolling the view by a line without moving the cursor — IntelliJ's `EditorScrollUp/EditorScrollDown` — lands on the keys vim already uses for it: zmax binds `C-e` to `scroll_down` and `C-y` to `scroll_up`, one place the two traditions agree on the chord and only differ on the name. The folding work is here too: **recursive fold expand/collapse** — `fold_open_recursive` and `fold_close_recursive`, IntelliJ's Expand/Collapse Recursively — opens or closes a region *and everything nested inside it* in one action, and **doc-comment folding** collapses the documentation blocks that pad a file — the code-folding category lands at a full 100%, eight of eight. None of these change the buffer; they change the lens, and a JetBrains user reaches for them by reflex the moment a file gets dense.

Navigation, carets, and bookmarks

JetBrains navigation has its own texture, and the layer ports it. The **page-caret motions** move the caret by a screenful while keeping it on screen — IntelliJ's `Ctrl-PageUp/Ctrl-PageDown` and the move-caret-to-page-top/bottom actions — a coarser grain than vim's `C-f/C-b` that scroll the *view*. **Bookmarks** — modeled on IntelliJ's `F11` toggle and numbered-bookmark set — pin positions you return to, a named cousin of vim marks; the workbench reaches the Bookmarks tool window at `SPC W b` (`focus_bookmarks`), alongside the source-navigation actions (back, forward, to-declaration) that a JetBrains user drives without thinking. **Goto previous splitter** and **change-splitter-orientation** and **split-resize** port the IntelliJ editor-tab and split-management actions, so the window grammar a JetBrains user knows — not vim's `C-w`, not Spacemacs's `SPC w`, but IntelliJ's own — is present too. The navigation category is complete in the port report — 21 of 21 — with the adjacent `navigate-from-symbol` row the one still carrying a partial, at 9 of 10, because a little of IntelliJ's symbol navigation leans on project-model features zmax does not have. The report says which rather than rounding up.

The workbench surfaces one more piece of JetBrains navigation that has no keystroke of its own: the **breadcrumb**. Chapter 55 described it — the strip along the toolbar showing the path to the symbol the cursor sits in, package to type to method — and it is pure IntelliJ, the “where am I in the structure” readout that a JetBrains user reads without consciously looking. Clicking it reveals the file in the project tree, making the breadcrumb navigable rather than merely informative. Paired with the STRUCTURE outline (`SPC W s`, `document_outline`, the tree-sitter tag query) and the symbol picker (`SPC s j`), it gives the three JetBrains ways of orienting in a file — see where you are (breadcrumb), see the whole shape (outline), jump to a name (picker) — all backed by the same syntax tree, none of them requiring you to scroll and count.

Live templates and the run/debug tools

Two more JetBrains subsystems round the layer out. **Live templates** are IntelliJ's snippet system — abbreviations that expand to parameterized code with tab stops — and zmax maps them onto its own snippet engine (Chapter 39), so the JetBrains gesture reaches the same expansion machinery the vim and emacs users get. And the **run/debug** integration is where the JetBrains layer meets the debugger of Chapter 15. **Run To Cursor** — `dap_run_to_cursor`, bound at `SPC d C`, modeling IntelliJ's `Opt-F9` — is the standout: it sets a breakpoint at the cursor line and continues execution, running the program *to where you are looking* in one gesture. It diverges from IntelliJ in one honest detail the source notes: the breakpoint **persists** rather than being one-shot, so you arrive at the cursor and the breakpoint stays — a small difference from the IDE it models, recorded rather than glossed. **Debug restart** and the **debug-configs** actions, plus the quick-switch-scheme and TODO

tool-window actions, fill out a run-debug category that lands at 95.2% (20 of 21). And the **tool windows** — the workbench panels of Chapter 55 — are the JetBrains contribution made most visible: `hide_active_tool_window` (`SPC W q`, IntelliJ's Shift-Esc), `jump_to_last_tool_window` (`SPC W TAB`, the F12 “toggle last tool window” reflex), and focusing a panel by name, the IntelliJ `Alt`+number muscle memory ported to the `SPC W` prefix. That category is at 100%, twelve of twelve. The JetBrains layer, in sum, is not a costume over vim the way the emacs and Spacemacs layers partly are; it is a different *kind* of vocabulary — semantic, IDE-shaped, server-backed — laid over the same engine, and it is the part of zmax growing fastest as this edition is written.

The tool windows in practice

The piece of the JetBrains layer you live in most is the tool-window set — the panels of the workbench (Chapter 55) — and the IntelliJ instinct of focusing one by name is ported to the `SPC W` prefix. A JetBrains user's daily loop is “open a panel, act in it, dismiss it,” and that loop reads naturally here:

```
SPC W t      focus the PROJECT tree      SPC W s      the STRUCTURE outline
SPC W e      the PROBLEMS panel          SPC W r      the RUN console
SPC W g      the GIT changes panel       SPC W c      the CI runs
SPC W b      the Bookmarks tool window
SPC W q      hide the active tool window (IntelliJ Shift-Esc)
SPC W TAB    jump back to the last tool window (IntelliJ F12)
SPC W m      maximize the bottom panel for a long log or diff
```

The two that make it feel like IntelliJ are `SPC W q` and `SPC W TAB`: hide the panel you just used to reclaim the space, and snap back to the last one when you need it again, without hunting. This is the JetBrains tool-window muscle memory — `Alt`+number to focus, `Shift-Esc` to hide, `F12` to toggle the last — rehomed onto the leader, and it is why the tool-windows category scores a clean 100% in the port report: the panels and the gestures that drive them both ported whole. The breadcrumb, the structure outline, and the project tree together give the “where am I” orientation a JetBrains user expects, and the problems/run/git panels give the “what is the state of things” readout, all reachable without the mouse the IDE usually assumes.

The JetBrains commands at a glance

Because most of the layer is palette-reached rather than bound, the reference that matters is the command names. These are the ones you type after `SPC SPC` (or `M-x`), plus the handful that carry a default chord:

```

REFACTORING (palette; LSP code-action backed)
  extract_function      Extract Method / Function
  extract_variable      Introduce Variable
  extract_constant      Extract Constant
  extract_field         Introduce Field
  extract_parameter     Introduce Parameter
  extract_refactor      menu of every refactor.extract action
  inline_refactor       Inline (variable / method)
  rewrite_refactor      Change Signature class
  refactor_this         show all applicable refactorings (Refactor This)

MULTIPLE CARETS
  copy_selection_on_next_line / _prev_line  clone caret up / down
  (select-all-occurrences)               every occurrence -> a caret

SELECTION / BLOCK
  column_selection      rectangular column block
  C-v (Select) visual_block_mode      true block mode
  o / O (block) block_swap_corners / _columns

VIEW TOGGLES
  toggle_line_numbers  toggle_indent_guides  toggle_whitespace_render
  toggle_soft_wrap     toggle_inlay_hints
  C-e scroll_down      C-y scroll_up
  fold_open_recursive  fold_close_recursive

RUN / DEBUG and TOOL WINDOWS
  SPC d C  dap_run_to_cursor      Run To Cursor (breakpoint + continue)
  SPC W q  hide_active_tool_window (IntelliJ Shift-Esc)
  SPC W TAB jump_to_last_tool_window (IntelliJ F12)
  SPC W b  focus_bookmarks        Bookmarks tool window

```

Chapter 60 — Emacs, In Earnest

Chapter 9 introduced the emacs layer as a set of readline reflexes — the Meta keys that move and kill and yank without leaving the home row — and called it “a second set of names for an engine that was always working the way emacs’s region model works.” That was an accurate description of a thin layer. It has since stopped being thin. The emacs personality has grown the machinery that makes emacs *emacs* rather than just a set of keybindings: a kill-ring with yank-pop, a mark-ring with pop-to-mark, point and number registers, true rectangle editing, named persistent bookmarks, and dynamic abbrevs. This chapter is that deeper emacs, the one a long-time emacs user reaches for past the arrow keys.

The kill-ring and yank-pop

In emacs, cut and copy do not overwrite a single clipboard slot; they *push onto a ring*. Every kill — `C-w`, `C-k`, `M-d`, `M-w` — prepends its text to the kill-ring, and a yank (`C-y`) inserts the most recent entry. The move that makes the ring a ring is **yank-pop**, `M-y`: pressed immediately after a yank, it *replaces* the just-yanked text with the next-older entry, and pressing it again steps further back, so you yank, then cycle backward through your kill history until the text you wanted lands. zmax ports this faithfully: the kill-ring accumulates kills, `C-y` yanks the head, and `M-y` walks the ring, the exact gesture an emacs user’s hand performs without conscious thought. It is the same idea as the JetBrains paste-from-history of the previous chapter and the vim numbered registers of Chapter 17 — three traditions’ spellings of “the clipboard remembers more than the last thing” — and zmax now speaks all three.

The mark-ring and pop-to-mark

Emacs’s mark is two things at once: the anchor of the current region, and a *position you can return to*. Every time you set the mark — `C-SPC` — and every time a command sets it for you (a search, a paragraph motion), the old mark is pushed onto the **mark-ring**, a per-buffer stack of recent positions. **Pop-to-mark** — `pop_to_mark`, bound at `C-x C-SPC` in the emacs preset — jumps the cursor back to the previous mark and rotates the ring, so repeated `C-x C-SPC` walks backward through where you have been; emacs’s canonical `C-u C-SPC` spelling does not apply here, because in this preset `C-u` is bound to kill-to-line-start, so the pop binding moved to `C-x C-SPC`, the honest difference a ported keymap inherits. Its companion `C-x C-x` exchanges point and mark — `flip_selections`, emacs’s “swap the two ends of the region” — so you can jump to the far end of what you marked and back. zmax ports the mark-ring, pop-to-mark, and the exchange together, which means an emacs user has their familiar “go back to where I was marking” motion, distinct from and parallel to the vim `C-o/C-i` jumplist that the same engine also exposes. Two traditions, two histories of position, one set of cursors underneath.

Registers: point, number, and text

Emacs registers are a small lettered filing system, and the `C-x r` prefix is their door. zmax ports the prefix and its three most-used kinds. **Point registers** — `C-x r SPC` to store the cursor position into a register, `C-x r j` to jump to it — are named bookmarks scoped to the session, the emacs analogue of vim’s `m/` marks. **Number registers** hold an integer you can store (`C-x r n`), increment (`C-x r +`), and insert (`C-x r i`), the building block of emacs’s insert-a-counter-into-a-macro idiom. And **text registers** store a copied region into a letter and paste it back later, a clipboard with two dozen named slots. The whole `C-x r` family is one of the

densest corners of the emacs keymap, and porting it is what lets an emacs user keep the muscle memory of “stash this here, grab it there” that registers exist to serve.

A worked example makes the three register kinds concrete. Suppose you are reorganizing a file and want to mark a spot to return to, stash a snippet to drop in three places, and number a series as you go:

```
C-x r SPC a      store the cursor position into point register a
... edit elsewhere ...
C-x r j a        jump back to register a's position
C-x r n 1 c      store the integer 1 into number register c
C-x r i c        insert register c's value; C-x r + c to bump it
```

Point register **a** holds where you were; number register **c** holds a counter you insert and increment; and a text register (a copied region stored into a letter) holds a snippet you paste back by name. Three lettered filing systems, one prefix, and the emacs habit of “put it in a register and forget about it until I need it” works exactly as it does in emacs proper.

Rectangles

The **C-x r** prefix also opens emacs’s **rectangle** commands, and these are the genuine article — not the cursor-stack emulation the vim block layer uses, but true rectangular region operations. With a region marked, **kill-rectangle** (**C-x r k**) cuts the rectangle of text between point and mark, **delete-rectangle** (**C-x r d**) removes it without saving, **copy-rectangle-to-register** and the **M-w** rectangle copy save it, **yank-rectangle** (**C-x r y**) inserts a saved rectangle at the cursor, and **clear-rectangle** (**C-x r c**) blanks it to spaces. A rectangle is the column of text a region’s two corners define, and operating on it — deleting a vertical strip, pasting one back in elsewhere — is the emacs gesture for column work. Combined with the JetBrains column-selection mode of Chapter 59, zmax now has *two* real rectangle models, reached by two traditions’ keys, where the first edition had only the vim block emulation.

Bookmarks and abbrevs

Two more emacs subsystems made the move. **Named persistent bookmarks** — **C-x r m** to set a bookmark with a name, **C-x r b** to jump to one, **C-x r l** to list them — are registers that *survive across sessions*, written to disk so “jump to the bookmark I called **config**” works tomorrow as well as today. They are the emacs counterpart to the JetBrains numbered bookmarks of Chapter 59 and the Harpoon pins of Chapter 58, three more spellings of “pin the places I keep coming back to.” And **abbrevs** — emacs’s dynamic abbreviation system — port the define-and-expand pair: **C-x a g** defines a global abbrev from the word before the cursor, and **C-x '** expands the abbrev the cursor sits on, the lightweight text-expansion emacs users lean on for boilerplate they have not bothered to make a snippet. None of these is large on its own; together they are the difference between an emacs layer that *looks* like emacs in a cheat sheet and one that *behaves* like emacs under a long-time emacs user’s hands. The port report credits the whole cluster — kill-ring, mark, registers, rectangles, bookmarks, abbrevs — as the emacs features that moved the editor’s emacs-command coverage off its floor.

Three clipboards, reconciled

Adding the kill-ring raises an obvious question: zmax now has *three* places copied text can live — the kill-ring just described, vim’s numbered and named registers (Chapter 17), and the system clipboard (Chapter 58) — so which does a copy land in, and do they fight? They do not, because they are three traditions’ answers to the same need and zmax keeps all three live rather than collapsing them. A vim user’s **yy** lands in the unnamed register and pastes with **p**; an emacs user’s **M-w** pushes onto the kill-ring and yanks with **C-y**; and the system clipboard — the OS-wide buffer your other applications share — is reached explicitly through the **:clipboard-*** commands and the **SPC y/SPC p** bindings. The JetBrains paste-from-history of Chapter 59 is

a fourth door onto the same idea, the clipboard *ring* surfaced as an IntelliJ gesture. The reason the multiplicity is not chaos is the reason the whole book gives for everything: underneath, each is a place text is stashed and retrieved, and the engine does not care which tradition's key did the stashing. You use the clipboard your hands learned, and the other two are there when you want them — the kill-ring's **M-y** cycling, the registers' named slots, the system clipboard's cross-application reach — each strongest at the job it was designed for.

The emacs deep keys at a glance

These live in the `emacs` preset (`:keymap emacs`), riding the Meta keys and the `C-x` prefix that vim leaves free:

```
KILL-RING and MARK-RING
C-y      yank_from_kill_ring    yank the most recent kill
M-y      yank_pop               replace it with the next-older kill
C-SPC    set_mark_command       set the mark / start a region
C-x C-SPC pop_to_mark           jump back through the mark-ring
C-x C-x   flip_selections       exchange point and mark

REGISTERS (C-x r ...)
C-x r SPC point_to_register     store the cursor position
C-x r j   jump_to_register      jump to a stored position
C-x r n   number_to_register    store an integer
C-x r +   increment_register    increment it
C-x r i   emacs_insert_register insert a register's contents

RECTANGLES (C-x r ...)
C-x r k   kill_rectangle        C-x r d   delete_rectangle
C-x r c   clear_rectangle       C-x r y   yank_rectangle
C-x r M-w copy_rectangle_as_kill

BOOKMARKS (persistent) and ABBREVS
C-x r m   bookmark_set          C-x r b   bookmark_jump   C-x r l   list
C-x a g   define_abbrev         C-x '     expand_abbrev
```

Chapter 61 — Narrowing and Indirect Buffers

One emacs idea earned its own chapter because it changes how you *relate* to a file rather than how you edit one: **narrowing**. To narrow is to tell the editor “for now, this region is the whole buffer” — everything outside the narrowed span vanishes from view and from the reach of every command, until you widen and it returns. It is the concentration tool: the way emacs lets you treat one function, one section, one page as if nothing else in the file existed. zmax ports true narrowing and the indirect-buffer machinery that makes it composable, under the `SPC n` leader.

True narrowing

The core is `narrow_to_region` — `SPC n r` — which takes the current selection and makes it the entire editable buffer. The text outside is not deleted; it is *hidden*. Motions stop at the narrowed bounds, `gg` and `G` go to the top and bottom of the *region*, a search finds only matches inside it, and a transform that falls back to “the whole buffer” operates on the region rather than the file. **Widen** reverses it, restoring the full buffer and putting the cursor back where it was. This is the emacs idiom exactly, and it is distinct — Chapter 58 was careful to note — from narrowing a *selection* down to its matches, which is a multi-cursor operation on the engine’s selection; narrowing the *buffer* is a change to what the editor considers the file to be. The two share a word and nothing else.

zmax ports the convenience narrowings emacs ships, the ones that pick the region for you. **Narrow to function** scopes the buffer to the function the cursor sits in, walking the tree-sitter tree (Chapter 13) to find its bounds — the daily “let me work on just this one function” gesture. **Narrow to page** — `SPC n p` — scopes to the page between form-feed boundaries, emacs’s older unit of division. Each is `narrow_to_region` with the bounds chosen by structure rather than by hand, and each widens back the same way.

Indirect buffers

Narrowing has one famous limitation, and emacs’s answer to it is the **indirect buffer**. When you narrow, you can only see the narrowed region — but sometimes you want to narrow *and* keep the whole file visible elsewhere, to edit one function in a focused pane while the rest of the file scrolls in another. An indirect buffer is a second buffer that *shares the same underlying text* as the first: edits to either appear in both, because there is one text and two views onto it, but each view can be narrowed independently. zmax ports this as **per-view narrowing** for indirect buffers — `SPC n F` to clone the current buffer into an indirect view narrowed to the function, `SPC n P` for the page — so you can have the focused, narrowed view and the full view of the same file side by side, both live, both editable, both the same text. It is one of the subtler emacs features, the kind that sounds abstract until the first time you want to edit a function with its call sites visible in the next pane, and it falls out of the engine’s existing buffer-and-view split (Chapter 8) more naturally than it does in emacs, because zmax already separates the document from the views onto it. The org-capture and agenda machinery (Chapter 56) leans on the same indirect-buffer plumbing, which is why the two features arrived together.

What narrowing composes with

The reason narrowing is more than a focus gimmick is that *everything else in the editor respects it*. Because narrowing changes what the editor considers the buffer to be, the features built on “the buffer” inherit the narrowing for free. A search (`/`) finds only matches inside the region. A transform from the library of Chapter 57, which falls back to “the whole buffer” when nothing is selected, operates on the *region* — so `:sort` inside a narrowed function sorts that function’s lines and nothing else, `:to-snake` normalizes identifiers only within it, `:stats` reports on its numbers alone. The language server still sees the whole file (it must, to resolve symbols), but your edits and your text operations are fenced to the region. This composition is the payoff: narrowing is not a special mode with its own command set, it is a single change to the buffer’s effective bounds that every buffer-scoped feature then honors automatically, the same way the selection engine of Chapter 3 lets one motion serve one cursor or five hundred. You narrow once, and the whole editor narrows with you until you widen.

The narrowing keys at a glance

All under the Spacemacs `SPC n` leader:

<code>SPC n r</code>	<code>narrow_to_region</code>	narrow to the selection
<code>SPC n f</code>	<code>narrow_to_function</code>	narrow to the enclosing function
<code>SPC n p</code>	<code>narrow_to_page</code>	narrow to the form-feed page
<code>SPC n w</code>	<code>widen</code>	restore the full buffer
<code>SPC n R</code>	<code>narrow_region_indirect</code>	selection in an indirect split
<code>SPC n F</code>	<code>narrow_to_function_indirect</code>	function in an indirect split
<code>SPC n P</code>	<code>narrow_to_page_indirect</code>	page in an indirect split

Chapter 62 — Paredit and Structural Editing

Chapter 58 mentioned paredit in a paragraph; it deserves a chapter, because for the Lisp the editor now embeds — elisp, and the s-expression spellings of the other tongues — structural editing is not a luxury but the only sane way to work. Paredit is the emacs package that edits the *tree* of an expression rather than its *characters*, keeping parentheses balanced through every transformation so you can never strand a paren. zmax ports the suite under the **SPC k** leader, and with the five embedded languages of Movement IV it now has real Lisp to point it at.

The principle: never unbalance a paren

The premise of paredit is that you do not edit parentheses as text at all. You do not type a close-paren and you do not delete one in isolation; instead you reshape the expression with operations that *move the boundaries* of forms while keeping every pair matched. The payoff is that the structure is always valid — there is never a moment where the buffer holds an unbalanced expression that the reader would choke on — and the cost is a vocabulary of structural verbs to learn in place of the character edits. For paren-dense code the trade is overwhelmingly worth it, which is why paredit has been standard equipment for Lisp programmers for decades.

Slurp, barf, splice, and raise

The four motions every paredit user knows come first. **Slurp** pulls the next form *into* the current parentheses — the close-paren moves rightward to swallow the sibling that was outside it — and **barf** pushes the last form *out* — the close-paren moves leftward to expel it. Together they are how you grow and shrink the reach of an expression without ever touching a paren key: slurp to extend a **let** binding over the form after it, barf to push a form back out of a **when**. **Splice** removes a wrapping pair entirely, promoting its contents up a level — the structural “unwrap” — and **raise** replaces the enclosing form with the single child you point at, deleting everything else in the parent. These four cover the bulk of daily structural editing, and each is bound under **SPC k** so the whole suite shares a roof.

The advanced four: split, transpose, absorb, convolute

zmax ports the rest of the suite, including the operations even seasoned paredit users reach for less often. **Split-sexp** — **SPC j s** / **SPC j S** — breaks one form into two at the cursor, closing the first and opening the second, the structural counterpart of splitting a line. **Transpose s-expression** — **SPC x t e**, alongside the transpose-sentence and transpose-paragraph siblings on the same **SPC x t** branch — swaps the form before the cursor with the one after it, reordering arguments or clauses without selecting either. **Absorb** — **SPC k a** — is the rarer move that pulls the enclosing form *into* the current one, the inverse of a barf across a level. And **convolute** — **SPC k c** — is the famously hard-to-describe operation that reorders nesting, pulling an inner form's wrapping outward around its parent; it is the move that completes the paredit suite, and zmax's commit history marks it as exactly that, “completes the paredit suite.” Few editors outside emacs ship convolute at all; that zmax does is a measure of how seriously the structural-edit port was taken. For the elisp extension author

of Chapter 46, working on the editor’s own configuration language, the paredit suite is the difference between fighting the parentheses and forgetting they are there.

Where it applies, and where wildfire takes over

Paredit is for paren-balanced code, and with the embedded tongues of Movement IV *zmax* finally has it in quantity — the elisp you write to extend the editor (Chapter 46), the s-expression forms the other Lisps accept — but the structural instinct is not confined to Lisp. The wildfire idiom of Chapter 13 (`<ret>` to select the closest text object and grow outward, `<backspace>` to shrink) is the *general* structural selection that works in any tree-sitter language, and paredit is its sharper, Lisp-specific cousin: where wildfire *selects* by structure, paredit *edits* by structure, moving the boundaries of forms rather than the spans of characters. For a Rust or Python file you reach for wildfire to grab the right syntactic unit and then operate with an ordinary verb; for a Lisp you reach for paredit to reshape the tree directly. Both rest on the same syntax tree (Chapter 25), queried two ways — and the existence of both is why a syntax-aware editor feels categorically different from a line-and-character one: structure is something you can both select and edit, not just something the highlighter colors.

The paredit keys at a glance

The structural suite lives under the `SPC k` (“Lisp / sexp”) leader, with split on `SPC j` and transpose also reachable on `SPC x t`:

<code>SPC k s / S</code>	<code>paredit_slurp_forward / _backward</code>	swallow the next/prev form
<code>SPC k b / B</code>	<code>paredit_barf_forward / _backward</code>	expel the last/first form
<code>SPC k w</code>	<code>wrap_sexp</code>	wrap the form in a new pair
<code>SPC k W</code>	<code>paredit_splice</code>	remove a wrapping pair (unwrap)
<code>SPC k r</code>	<code>paredit_raise</code>	replace parent with child at point
<code>SPC k t</code>	<code>paredit_transpose</code>	swap two sibling forms
<code>SPC k a</code>	<code>paredit_absorb</code>	pull the enclosing form in
<code>SPC k c</code>	<code>paredit_convolute</code>	reorder the nesting outward
<code>SPC k e / E</code>	<code>paredit_splice_kill_forward / _back</code>	splice and kill in a direction
<code>SPC k (/)</code>	<code>paredit_insert_sexp_before / _after</code>	insert a sibling form
<code>SPC j s / S</code>	<code>paredit_split</code>	split one form into two
<code>SPC x t e</code>	<code>transpose_sexp</code>	transpose s-expression

Chapter 63 – Layouts, Workspaces, and Winner Mode

Chapter 8 covered windows and buffers — the splits you make and the files you open. This chapter covers the layer above them: *named arrangements* of windows and buffers that you save, switch between, and undo, the Spacemacs window-layout and workspace subsystem that zmax ports under the `SPC l` leader. Where a window is a view and a buffer is a file, a layout is a *workspace* — a whole screen configuration with a name — and the ability to keep several and flip between them is what turns the editor from a place you arrange once into a place you arrange per task.

Layouts and the `SPC l` tree

The `SPC l` prefix is the window-layout and workspace menu, and its shape follows Spacemacs closely. A layout bundles the current window splits and the buffers shown in them under a name; `layout_next` / `layout_prev` (`SPC l n` / `SPC l p`) step between saved layouts, `layout_last` (`SPC l TAB`) flips to the previous one, and the numbered switches — `SPC l 1` through `SPC l 9` — jump to a layout by number the way a tiling window manager switches desktops. **Layout add-buffers** (`SPC l A`, `layout_add_buffers`) folds another layout's buffers into the current set; **layout load** (`SPC l o` / `SPC l L`) and **layout rename** (`SPC l R`) manage the named collection, and `SPC l S` saves it; and the workspace-tier bindings — the `SPC l w` submenu with its own switches and `SPC l w R` rename — add a second level above layouts, a workspace being a named *group* of layouts. The mental model is three deep: buffers live in windows, windows compose a layout, layouts group into a workspace, and the `SPC l` tree gives each level a name and a switch. For a developer juggling several features at once — one layout per feature, each with its own files arranged its own way — this is the difference between re-opening files all day and flipping to the arrangement you left.

Winner mode: undo for window layouts

The single most beloved Spacemacs window feature is **winner mode**, and zmax ports it: undo and redo *for the window configuration itself*. Splits, closes, and rearrangements form their own history, separate from the text undo history, and **winner-undo** — `SPC w u`, or `SPC w . u` — steps backward through it, restoring the window arrangement you had before that accidental `C-w o` closed every split but one. **winner-redo** (`SPC w . U`) steps forward again. It is the safety net that makes aggressive window management painless: close splits freely, knowing one keystroke brings the layout back, exactly as `u` brings back deleted text. The two undo histories — text and layout — are independent, which is the whole point; undoing a botched split should not undo the edit you made in it.

Golden ratio and ace-window

Two smaller Spacemacs window conveniences round the chapter out. **Golden-ratio** mode — toggled at `SPC t g`, or applied once at `SPC w . g` — automatically resizes the focused window to the larger share of the screen and shrinks the others, so the window you are working in is always the big one and the rest stay visible but out of the way; it is the auto-balancing answer to constantly adjusting splits by hand. **Ace-window** — `SPC w . a` — labels every window with a letter and jumps to the one whose label you press, the visual-jump

answer to “focus that window over there” when there are more than two and the `[w h/j/k/l]` directional motions get ambiguous. Both are the same instinct the leader embodies everywhere: let the editor do the mechanical part — the resizing, the targeting — so your attention stays on the work. Together with the layouts and winner mode, they make the window system something you *compose* rather than merely *operate*, which is what the workspace idea was always for.

A worked switch

Pulling the pieces together, a realistic switch between two tasks reads like this:

```
SPC l l          capture the current arrangement as a layout
SPC l R bugfix <ret> name it
... open and arrange the files for the second task ...
SPC l l          capture that one too
SPC l R feature <ret>
SPC l 1 / SPC l 2 flip between the two whole arrangements by number
SPC w u          if a switch left the windows wrong, winner-undo the layout
SPC t g          let golden-ratio keep the focused window large
```

The whole point is in the last lines: once two layouts exist, switching tasks is a single number key, recovering from a botched window change is `SPC w u`, and the focused window keeps itself sized without your fiddling. The arrangement work happened once, per task, up front; the daily cost of moving between tasks dropped to one keystroke. That is the workspace idea delivering — the editor remembering your arrangements so you do not have to rebuild them.

Layouts that outlive the session

A layout you have to rebuild every morning is only half a layout, and the workbench addresses that: the IDE layout persists. Chapter 55 named the mechanism — when you quit, the drawer widths, the collapsed panels, the minimap’s folded state, the bottom panel’s height, and the active colorscheme are written to an `[ide]` table in `appdata.toml`, and restored when you next open `zmax` without a file argument. The layout system of this chapter rides the same instinct: you arrange the workbench once, and it stays arranged across launches, so the “what was I working on” question is answered by the editor opening the way you left it rather than the way it shipped. Combined with the start screen’s frequency-ranked recent files (Chapter 55), the effect is that launching the editor drops you back into your work rather than into a blank slate — the named layouts hold the *arrangement*, the persisted `appdata` holds the *chrome*, and the start screen holds the *files*, three pieces of “resume where I was” that together make a cold start feel warm.

The layout and window keys at a glance

Layouts under `SPC l`, the window conveniences under `SPC w`:

```
SPC l l          layout_create      capture current windows+buffers as a layout
SPC l n / p      layout_next / _prev step between layouts
SPC l 1..9       layout_goto_N      jump to a layout by number
SPC l TAB        layout_last        flip to the previous layout
SPC l A          layout_add_buffers  fold another layout's buffers in
SPC l R          layout_rename      SPC l S layout_save  SPC l o/L layout_load
SPC l w ...      workspace tier (named groups of layouts)
SPC w u          winner_undo        SPC w . U winner_redo
SPC t g          golden_ratio_resize SPC w . g apply once
SPC w . a        ace-window         jump to a window by its label
```

Chapter 64 – The Help System and Discoverability

zmax’s design goal, the README says, is “a maximally powerful CLI IDE with zero user configuration” — and a tool that powerful with no config file to read is only usable if it can *explain itself*. The earlier movements showed the discoverability built into the keys: the vim auto-info popups (Chapter 7), the Spacemacs which-key menus (Chapter 10), the Refactor This popup (Chapter 59). This chapter is the dedicated help *system* that backs them — the `SPC h` family and the searchable Help browser — because the editor that documents itself is the editor you can actually learn without leaving it.

The Help browser

`SPC h h` (or `:help`, `:h`) opens the **Help browser**, the Help tab of the Preferences window (Chapter 55), a searchable index over every command, every keybinding, and every topic the editor knows. “What runs on this key” and “what does this command do” become a fuzzy search rather than a trip to an external manual, which matters most precisely because zmax ships no config you wrote — the browser is how you find the command you did not know existed.

Most of its index is built at runtime from the editor’s own tables rather than written by hand, which is the property that makes it trustworthy. `HelpPanel::new` in `zmax-term/src/ui/help.rs` walks `MappableCommand::STATIC_COMMAND_LIST` and `TYPABLE_COMMAND_LIST` directly, and `key_index` builds the command-to-chord map by calling `keymap::default()` and reversing each mode’s `KeyTrie`. So a command that exists is in the browser and a command in the browser exists, for the same reason the palette is never stale: there is no second list to forget to update.

Two qualifications keep that from being the stronger claim earlier editions made. First, the browser’s `Topics` category is a hand-maintained `TOPICS` const in the same file — prose, not generated — so the narrative help *can* go stale even while the command index cannot. Second, this is a different derivation from the port report’s. The browser reads the runtime tables and the runtime keymap; `gen_port_report.py` scrapes the *source text* of `commands.rs` and `typed.rs` and re-implements the keymap layering in Python. They describe the same editor by two independent routes, which is a useful cross-check but not a shared source of truth — the two can disagree, and if they do, the runtime is right and the report needs fixing.

A worked discovery

The help system earns its keep in the moment you know what you want to *do* but not what it is *called* or *bound to*, which for an editor with hundreds of commands and no config you wrote is a constant moment. Suppose you want to align a block of assignments on the `=` sign and have no idea whether there is a command for it:

```
SPC h h      open the Help browser
align        fuzzy-filter the index to alignment commands
              -> :align / :tabularize (Chapter 57) surfaces
<ret>        read what it does and how it is invoked
```

Two keystrokes and a word turned “is there a way to do this” into the answer, with the command, its aliases, and its description in front of you — and now you know `:align` exists for next time. The same path answers the inverse question, “what does this key I just pressed do,” through the describe family below, and “what

does this *symbol* mean” through the LSP-hover describers. The discovery loop is the whole reason an editor with zero starting configuration is usable at all: every capability is one fuzzy search from being found, so the absence of a config file you wrote is not a wall but an invitation to ask the editor what it can do. You learn `zmax` by using its help, not by reading a manual first — which is exactly the Spacemacs ethic this chapter set out to describe.

The describe-* family

The `SPC h d` submenu is the **describe-*** family, emacs's introspection vocabulary ported. It splits cleanly in two by what is being described. The *editor* describers explain the tool: **describe key** explains what a binding does, **describe current modes** (`SPC h d m`) reports the active modes, **describe text properties** (`SPC h d t`) reports the faces and properties at the cursor, **describe language package** (`SPC h d p`) explains the current buffer's language configuration, and the copy-helpers — **copy system info** (`SPC h d s`) and **copy last pressed keys** (`SPC h d l`) — put diagnostic information on the clipboard for a bug report. These route through the Help browser, because they describe the editor and the editor knows itself.

The *symbol* describers are the interesting half, because they route somewhere else. When you ask `zmax` to describe a *symbol* in your code — a function, a type, a variable — `SPC h d f`, `SPC h d v`, and `SPC h d a` (describe-function, describe-variable, describe-at-point) do not consult an editor manual; they each run `hover`, the **language server's hover**. The authoritative description of `tokio::spawn` does not live in `zmax` and never could; it lives in `rust-analyzer`, which read the docs and the signatures. So the describe-* symbol keys forward to LSP hover, and the emacs gesture of “describe the thing under the cursor” gets answered by the same server that powers `gd`. This is the joint the whole help system is built around: the editor describes the editor, and the language server describes the code, and `SPC h d` routes each question to whichever one can actually answer it.

Searching the world outside

The `SPC h` family reaches past the editor, too, porting the Spacemacs help-search bindings that look things up *outside* `zmax`. **Search man pages** (`SPC h m`), **search info pages** (`SPC h i`), **search packages** (`SPC h p`), **search dotfile variables** (`SPC h .`), and **search capability layers** (`SPC h l`) each open the respective external index, so “what does this man page say” is a binding rather than a shell-out you compose by hand. **Browse the FAQ** (`SPC h f`) and **browse release notes / NEWS** (`SPC h n`) open the project's own documentation in the browser. None of these is essential to editing; collectively they are the difference between a tool that assumes you have its manual open in another window and one that brings the manual inside. That ethic — show, don't assume; describe, don't require memorization — is the Spacemacs contribution distilled, and the help system is where it is most concentrated.

which-key and auto-info, the in-line help

The Help browser is the help you go *to*; the which-key popups are the help that comes *to you*, and they are the more constant of the two. `zmax`'s `auto-info` setting, on by default, means that pausing at any prefix — `SPC`, `g`, `z`, `C-w` — pops a small panel listing what the next keys do, refreshing as you descend a submenu (Chapter 10). For the leader specifically, `auto-info-leader-only` can focus that behavior where it pays most. The point is that the leader and the prefixes are not something you memorize from a manual and then drive blind; they are self-revealing, and the manual is the fallback for when you want to *search* rather than *browse*. The config tunes this: `auto-info-exclude` names the busy prefixes — `g`, `y`, `z`, `d` by default — where a popup on every press would be noise rather than help, and `SPC t k k` toggles the which-key display entirely for the moments you want the screen quiet. The two layers of help compose: the popup shows you the branch you are standing on, and the browser answers the question the popup cannot — “where is the command for a thing I cannot find by walking the tree.” Between them, an editor that ships no configuration you wrote is still an editor you can learn without leaving it, which was the whole design goal the chapter opened with.

The help keys at a glance

All under the `SPC h` leader; the `SPC h d` submenu is the describe-* family:

```
SPC h h      help                the searchable Help browser
SPC h f      browse_faq          SPC h n  browse_news (release NEWS)
SPC h m      man_page_search     SPC h i  info_search
SPC h p      package_search      SPC h l  layer_search (capability areas)
SPC h .      config_variable_search
SPC h d m    describe_current_modes  SPC h d t  describe_text_properties
SPC h d p    describe_language_package
SPC h d s    copy_system_info       SPC h d l  copy_last_keys
SPC h d f/v/a hover (describe-function / -variable / -at-point, via LSP)
```

Chapter 65 — ediff and the Diff Tools

Comparing two versions of a thing is one of the operations an editor is asked to do constantly and rarely does well. Chapter 16 covered version control and the magit-style git porcelain; Chapter 55 covered the three-pane merge resolver. This chapter covers the layer between them: **ediff**, emacs's interactive diff tool, ported under the **SPC D** leader, and the read-only diff views that make comparison safe. The reason it earns a chapter is that ediff is *not* the merge resolver — it is the gentler, read-mostly cousin you reach for to *understand* a difference before you decide to resolve one.

Safe ediff: read-only by default

The defining choice in zmax's ediff port is that it opens **read-only by default**, in a dedicated DiffView mode. A diff you open to *look* at should not be a diff you can accidentally *edit*, and the safe-ediff design enforces that: the comparison panes are read-only until you deliberately choose to act, so walking a difference to understand it carries no risk of fat-fingering a change into one side. This is a deliberate departure from the merge resolver of Chapter 55, which is editable because its whole job is to produce a resolved result; ediff's job is to *show*, and the read-only default matches the job.

The ediff variants

zmax ports the ediff comparisons you actually reach for. **ediff current buffer vs a picked file** — **SPC D f f** — compares what you are editing against another file you fuzzy-pick, the “how does this differ from that” question. The **3-way ediffs** handle the merge-shaped comparison without committing to a resolve: **SPC D f 3** over three files and **SPC D b 3** over three buffers show ours, theirs, and the base side by side, the diff3 view that the merge resolver builds on but presented for reading rather than editing. There are **ediff-windows** and **ediff-versions** variants too — comparing two windows' worth of text, or two git revisions of the same file — so the same gesture covers “diff these two things on screen” and “diff this file against an older commit.” Each opens in the safe, read-only DiffView, and each is one **SPC D** keystroke from the buffer you are in.

A worked comparison

Concretely, suppose you suspect your working copy of a config file has drifted from the version a colleague committed, and you want to understand the difference before deciding whether to keep yours. The read-only ediff path:

```
SPC D f f      ediff this buffer against a file you fuzzy-pick
                (opens read-only, in DiffView mode)
... walk the highlighted differences, understanding each ...
SPC D f 3      escalate to a 3-way ediff if a base version matters
:merge         escalate to the editable resolver only if you reconcile
```

The key is the escalation discipline: you opened a *read-only* view to look, so there was no risk of fat-fingering an edit into either side while you studied the difference; and only if you decided to actually reconcile did you step up to **:merge**, the editable tool. You compared before you committed to changing anything, which is precisely the gap between ediff and the merge resolver, and precisely why both exist.

How it relates to the git and merge tools

It is worth placing ediff among its neighbors, because zmax now has a *spectrum* of comparison tools and the right one depends on the question. At one end is `:diff` (Chapter 55), the quick read-only side-by-side of the buffer against git `HEAD` — one keystroke, no choices, “what have I changed.” In the middle is ediff — interactive, multi-source, still read-only, “let me walk this difference and understand it.” At the far end is `:merge`, the editable three-pane resolver with inline character highlighting and conflict navigation (`]n/[n`), “let me produce the resolved file.” And underneath all three is the magit porcelain (Chapter 16) for staging and committing what you decided. The progression is deliberate: zmax gives you the gentlest tool that answers your question and lets you escalate to an editable one only when you mean to change something. The ediff layer is the contemplative middle of that spectrum, the place you compare before you commit.

The change gutter, the lightest comparison of all

Below even `:diff` sits the most ambient comparison in the editor, the one you never invoke because it is always on: the **change gutter**. As you edit a file under version control, zmax marks each line in the left gutter against its git `HEAD` state — added, modified, or removed — so the shape of your uncommitted change is visible at a glance without opening any diff at all. The unimpaired motions of Chapter 7 turn those marks into navigation: `[c` and `]c` jump to the previous and next changed hunk, walking your own edits the way `[d/]d` walks diagnostics. This is the zeroth rung of the comparison spectrum — beneath ediff, beneath `:diff`, beneath the merge resolver — the comparison that costs nothing and is simply present, telling you what you have touched since the last commit. From there the escalation is the one the chapter has drawn: the gutter shows *that* a line changed, `:diff` shows *what* against `HEAD`, ediff lets you *walk* a difference read-only, and `:merge` lets you *resolve* one. Four degrees of “compare,” each the gentlest that answers its question, and the gentlest of all is the one you never have to ask for.

The diff and merge keys at a glance

The ediff comparisons live under `SPC D`; the editable tools are typable commands:

```
SPC D f f      ediff current buffer vs a picked file (read-only DiffView)
SPC D f 3      3-way ediff over three files      SPC D b 3 over three buffers
:diff :gdiff side-by-side, buffer vs git HEAD (read-only)
:merge :resolve editable 3-pane (ours/result/theirs) conflict resolver
]n / [n      goto_next_conflict / _prev      walk conflict markers
SPC g m / SPC g c r resolve_conflicts
SPC g c O / SPC g c T conflict_take_all_ours / _theirs
```


Chapter 66 — Four Traditions, Four Presets, One Engine

The book began with a promise — conservative about your fingers, radical about its internals — and it has spent the chapters since making good on it. This chapter closes the reference tour of the traditions by stepping back to the question all four raise together: how can one editor be vim *and* emacs *and* Spacemacs *and* JetBrains at the same time without becoming an incoherent pile of bindings? The answer has two parts, because the four traditions are not all the same *kind* of thing. Four of them are switchable *keymap presets* — `PRESETS` in `zmax-term/src/keymap.rs` lists `spacemacs`, `vim`, `helix`, `emacs` — and each owns the plain keys while `:keymap` chooses which. JetBrains is the exception: it is a *layered vocabulary*, a command set reached by name rather than a chord space you switch into. Getting that distinction right is what keeps the whole thing coherent.

The relationship between the first two presets is worth stating precisely, because it is where the “default is vim” shorthand comes from and where it goes wrong. `spacemacs` and `vim` share one base keymap. `spacemacs` is that base with the emacs `C-x / C-c / C-h` prefixes overlaid; `vim` is the same base with the `SPC` leader stripped out instead (`zmax-term/src/keymap/spacemacs.rs`, module docs). So the default preset is the vim grammar in the plain keys — every `dd`, `cw`, `y$` in this book works untouched under it — with two extra prefix trees hanging off keys vim was not using. That is why the vim chapters describe the out-of-the-box editor accurately even though the preset named `vim` is not the one that loads.

Why the traditions do not collide

The worry is obvious and the book has answered it piecemeal; here is the whole answer. The traditions do not collide because they occupy different *key spaces* and, more deeply, because they all reduce to operations on the *same engine*. Vim’s grammar lives in the plain letter keys and the `g/z/[` prefixes. The emacs layer lives in the Meta-modified keys and the `C-x` prefix that vim leaves alone. The Spacemacs leader lives entirely behind `SPC`, a key vim and emacs both leave free in Normal mode. And the JetBrains layer — this is the part Chapter 59 was careful about — does not occupy a live chord space at all: its commands are reached from the palette by name, and the few that are bound ride the same `SPC` leader (`SPC W` for tool windows, `SPC d C` for Run To Cursor). The IntelliJ chords its doc-strings cite are a *reference*, not a binding, so there is nothing for them to collide *with*. `d` is vim’s delete; `M-d` is emacs’s kill-word; `SPC d` opens a Spacemacs menu; `refactor_this` is a palette command — four traditions, three chord vocabularies and a name-reached fourth surface, one letter, no conflict. The separation is not an accident the layers stumbled into; it is a property of the traditions, which evolved in different chord spaces, and zmax’s contribution is to notice they are disjoint and lay them over each other.

But key-space separation is only the surface reason. The deeper reason is the one Chapter 3 established and every chapter since has confirmed: there is only one engine underneath, the selection engine, and every binding in every tradition is ultimately a sequence of operations on a *Selection*. Vim’s `dw` is `[collapse, extend, delete]`. Emacs’s `C-SPC` then motion then `C-w` is anchor-extend-delete — *the same three operations*. Spacemacs’s `SPC` menu items call the same commands the vim keys call. JetBrains’s `extract_function` drives the same LSP the vim `gr` drives. The traditions are not separate editors bolted together; they are *vocabularies* for one machine, and that is why they compose instead of fighting. A vim user and an emacs user editing the same buffer are pressing different keys to run the same engine commands, and the engine cannot tell which tradition asked.

The `:keymap` switch

The switchable part is the *presets*, and `:keymap` is how you choose. It takes one argument — `spacemacs`, `vim`, `helix`, or `emacs`, validated against the `PRESETS` list in `zmax-term/src/keymap.rs` — and swaps the active base keymap live, the whole base personality of the editor changed without a restart. The default is `spacemacs` (`DEFAULT_KEYMAP`, `zmax-term/src/config.rs`): the verb-before-noun grammar this book has documented throughout, plus the `SPC` leader and the emacs prefixes, starting you in Normal mode. `vim` is the same grammar with the leader tree removed, for anyone who wants `SPC` to stay a plain motion. `helix` restores the original selection-first keymap `zmax` forked from — the `zmax-term/src/keymap/default.rs` fallback of Appendix E, for the Helix user who wants their noun-then-verb order back. And `emacs` foregrounds the emacs bindings as the primary layer, and tellingly it starts in *Insert* mode — the emacs preset is effectively modeless, because that is what emacs is, so the `:keymap` handler sets the matching start mode along with the bindings. The preset chooses which tradition's base grammar owns the plain keys; the Spacemacs leader, the Meta keys, and the palette-reached JetBrains commands ride along on top of whichever base you pick, because they live in surfaces the base does not touch. The same choice is available without the command: a `keymap = "..."` line in `config.toml` sets it at startup, and the Preferences window (Chapter 55) exposes it in the Keymap tab, so the editor's personality is a menu pick as much as a command.

The synthesis

Stand back and the shape of the whole editor is visible in one frame. `zmax` is the Helix engine — rope, selections, tree-sitter, LSP, DAP — wearing four editing traditions at once: four switchable presets and one name-reached vocabulary, none of them a reimplement, all of them vocabularies over the one engine, in a single static binary that also contains five embedded programming languages compiled to one shared bytecode machine. The conservatism is in the keys: whatever tradition your hands learned, they keep working, and you can run them together or commit to a base preset with `:keymap`. The radicalism is in the internals: the five tongues need no subprocess, no socket, no second configuration language — the editor is programmable in the languages you already write, in its own address space. (`zmax` does *also* ship a conventional native plugin host, `zmax-plugin` — a versioned C ABI and `:plugin load` over `dlopen` — for people who want to extend it in compiled Rust. That is a separate road from the embedded tongues, and the book would be overclaiming if it said the editor had no FFI at all.) Conservative about your fingers, radical about its internals: the chapters behind this sentence are the long version, and every key, command, and claim in them is traceable to a file in the source tree. That was the promise. The port report, regenerated from source on every run and refusing to round up, is the receipt. What remains is to watch the traditions at work — the final movement does exactly that, taking the new vocabularies of these eight chapters onto real tasks.

Chapter 67 – A Refactoring Session

The traditions movement was reference; this last movement is practice, the way Movement VI was practice for the engine and the keymaps. Each chapter here takes one of the new vocabularies and works a real task with it, because the surest test of a ported tradition is whether it survives contact with an actual editing job. We start where the JetBrains layer is most itself: a refactoring session, the kind of five-minute restructuring an IntelliJ user does without thinking and a plain modal editor makes you do by hand.

The task

You have a Rust function that has grown a tangle in its middle — a block that computes a timeout from three configuration fields, inline, in the body of a larger function. You want to lift that computation out: first into a local variable so it has a name, then, once you see it is needed elsewhere, into a method, and finally to promote one of its inputs to a field on the struct so it stops being threaded through as an argument. This is the canonical refactoring escalation — variable, then method, then field — and it is exactly the path the JetBrains commands of Chapter 59 were ported to walk.

Extract Variable

Select the subexpression — the arithmetic that computes the timeout — and run the extract-variable refactoring. Because the JetBrains commands are palette-reached (Chapter 59), you open the palette and type the command name:

```
SPC SPC          open the command palette
extract_variable  Introduce Variable (IntelliJ)
<ret>            apply
```

The language server computes the edit: it replaces the selected expression with a fresh local — `let timeout = ...;` inserted above, the use site rewritten to reference it — and applies the whole thing as one undoable transaction (Chapter 24). You did not type the `let`, you did not choose the indentation, you did not move the cursor to the line above; the server did all of it, the same server that answers `gd`. If the expression appeared more than once, the refactoring rewrites every occurrence, which is the multi-cursor payoff (Chapter 30) arriving through a refactoring rather than a manual selection.

Extract Method

Now the variable wants to be a method. Select the statement — or the block, if the computation grew to several lines — and run the extract-method refactoring:

```
SPC SPC          open the palette
extract_function  Extract Method/Function (IntelliJ)
<ret>            apply (pinned: applies straight away)
```

`extract_function` is one of the *pinned* refactorings of Chapter 59: when exactly one `refactor.extract` action matches the title that names a function or method, it applies without showing a menu, mirroring IntelliJ's dedicated Extract Method. The server lifts the block into a new `fn`, threads in the locals it reads as parameters,

returns the value it produces, and replaces the original block with a call. What was inline is now a named method with a signature, and the call site reads as the intent rather than the mechanics.

Introduce Field

The method takes three parameters, and two of them are the same struct's fields, read the same way at every call. Promote one to a field with the introduce-field refactoring:

```
SPC SPC          open the palette
extract_field    Introduce Field (IntelliJ)
<ret>           apply
```

`extract_field` promotes the local to a field on the enclosing type, initializes it where the value was first computed, and rewrites the method to read `self.field` instead of taking the parameter. The signature shrinks; the threading disappears. Had you wanted the reverse later — to fold a one-use method back into its caller — `inline_refactor` runs the whole escalation backward, and `rewrite_refactor` reaches the change-signature edits for when the parameter list itself needs reshaping.

Rename: the semantic and the textual

The most-used refactoring of all is rename, and it is worth seeing that zmax offers it two ways, because the two answer different needs. The **semantic** rename is the language server's, reached at the LSP keys of Chapter 14 or through the leader: it understands scope, so renaming `cfg` renames the binding and its references and nothing that merely shares the spelling, across every file the symbol reaches. The **textual** rename is `:rename-word` (Chapter 58) — it renames every whole-word occurrence of a symbol *in the current buffer*, a literal replace with no semantic understanding. You reach for the textual one exactly when the semantic one cannot help: there is no language server for the file, or you genuinely want a literal whole-word substitution rather than a scope-aware one. The pair is the same shape as the rest of the JetBrains layer's relationship to the engine — the powerful, semantic version when the server can answer, and a mechanical fallback for when it cannot — and a power user keeps both in hand, reaching for the semantic rename by default and the textual one when the job is literal.

Why this is the JetBrains layer's reason to exist

Step back and notice what none of this involved: no manual cut-and-paste of the block, no hand-typed `fn` signature, no hunting for every call site, no risk of leaving the buffer in a half-refactored state, because every edit was a single server-computed transaction. This is the operation a JetBrains user reaches for by reflex and a plain modal editor cannot offer, because it requires the editor to *understand the code* — to know that this expression has these free variables, that this block returns this type, that this local is read at these three sites. zmax has that understanding, because it has the language server, and the JetBrains layer is the vocabulary that puts it one palette command away. The refactoring category is, as Chapter 59 admitted, the least complete of the JetBrains areas — but the extract-and-introduce core, the part you use hourly, is exactly the part that works.

All-or-nothing, and undoable

One property of the whole session deserves to be named, because it is what makes refactoring *safe* rather than merely *convenient*: every one of these edits is a single transaction. A refactoring that touches five call sites across three files does not leave the buffer in a state where three are updated and two are not — the workspace edit the server computes is applied as one atomic change, all sites or none, and a single `u` reverses the entire thing. This matters more than it sounds. The danger of refactoring by hand is exactly the half-finished

state: you rename the definition, start updating the references, get interrupted, and leave the code broken in a way that compiles in some files and not others. The server-computed, single-transaction refactoring removes that danger by construction — there is no intermediate broken state to be caught in, because the edit never passes through one. It is the same guarantee `paredit` gives for parentheses (Chapter 69) and transforms give for text (Chapter 74), applied to cross-file restructuring: the operation is atomic, so the buffer is always valid, and undo is always one keystroke. Refactoring you can trust is refactoring you will actually use, and atomicity is what earns the trust.

Chapter 68 — Rectangle and Column Editing

The second new vocabulary worth practicing is the one that finally gave `zmax` a *real* rectangle — two of them, in fact, reached by two traditions' keys: the emacs `C-x r` rectangle commands of Chapter 60 and the JetBrains-derived block mode that Chapter 59 promoted `C-v` into. This chapter works a task that is pure pain in a point-cursor editor and almost free with a rectangle: editing a column of a fixed-width table.

The task

You have a block of text aligned into columns — a license header, a table of constants, a chunk of aligned assignments — and you need to operate on one *vertical strip* of it: delete the third column, or blank it, or copy it out and paste it elsewhere. The strip is a rectangle: the same character range on each of a run of lines. Neither a linewise nor a characterwise selection describes it, which is why both vim's emulated block and emacs's rectangles exist.

The emacs way: `C-x r`

In the emacs preset (`:keymap emacs`, Chapter 66), the rectangle commands hang off the `C-x r` prefix, and the gesture is set-mark-at-one-corner, move-to-the-other, operate:

```
C-SPC          set the mark at the top-left corner of the strip
<motion>       move point to the bottom-right corner
C-x r k        kill-rectangle (cut the vertical strip)
C-x r y        yank-rectangle (paste it back, here or elsewhere)
C-x r d        delete-rectangle (remove it, no save)
C-x r c        clear-rectangle (blank it to spaces)
C-x r M-w      copy-rectangle-as-kill (copy without cutting)
```

The two corners define a column span over a line range, and the operation applies to exactly that geometry — a true rectangle, padding short lines with spaces so the columns stay aligned, applied as one whole-document transaction (Chapter 24). Kill the third column and the fourth slides left to meet the second; yank it back in at another position and it inserts as a block. This is the emacs `emacs_rect` module of Chapter 60 doing precisely what the emacs manual describes, in a modal editor that also speaks vim.

The JetBrains way: `C-v` block mode

The same task has a second spelling through the block mode Chapter 59 made real. From Select mode, `C-v` now enters a genuine rectangular block — `visual_block_mode`, not the old cursor-stack emulation — and the block keys reshape it:

v	enter Select mode
C-v	switch to rectangular block (visual_block_mode)
<motion>	size the block: down the lines, across the columns
o	swap to the opposite corner (block_swap_corners)
O	swap which column edge moves (block_swap_columns)
d	delete the block c change it y yank it
I	insert at the left edge of every line
A	append at the right edge of every line

The **I** and **A** inserts are the payoff that makes block mode worth a separate primitive: type once, and the text appears on every line of the block at the same column — adding a `//` comment prefix to twenty lines, or a trailing comma to a column of values, in one gesture. Where the emacs rectangle is a region you *operate on*, the block is a region you *type into*, and `zmax` now has both, reached by the keys each tradition trained your hands to press.

A worked column: prefixing a block

Make the difference concrete with the most common column task there is — prefixing every line of a block with the same text, the gesture you reach for to comment out a region by hand or to indent a paste that arrived flush-left. With the JetBrains block:

v	enter Select mode at the first line's column 0
C-v	switch to rectangular block (visual_block_mode)
j j j ...	grow the block straight down the lines, zero width
I	insert at the left edge of every line in the block
// <esc>	type the prefix once; it lands on every line

The **I** is the whole trick: a zero-width block down the left margin, and one insert that the block replays on every line at the same column. To do the inverse — strip a column — size the block to cover the prefix and press **d**. The emacs rectangle reaches the same result from the other direction: mark the rectangle that covers the prefix, `C-x r k` to kill it or `C-x r c` to blank it. Two traditions, one task, and the lesson of the chapter: the rectangle is a real region now, however you spell your way into it, and the column edits that were tedious in a point-cursor editor are a handful of keys.

Two rectangles, one set of ranges

It is worth recalling, from Chapter 3, what is underneath both: there is no “rectangle type” in the engine any more than there is a “cursor type.” The emacs rectangle is computed geometry applied as a transaction; the `C-v` block is a real column region the Select-mode machinery now understands. Two traditions reached the same capability the engine was always able to express, and the practice of using them is identical even though the keys are not — which is the whole book in one task.

Chapter 69 — Reshaping a Lisp with Paredit

The third vocabulary to practice is paredit, and it earns a practice chapter precisely because it is the one whose value does not land until you watch it move parentheses that never come unbalanced. The embedded elisp of Movement IV gives us real Lisp to point it at: we will reshape an elisp form, in the editor's own configuration language, using only the structural commands of Chapter 62.

The starting form

Begin with an elisp expression that is not wrong but is awkwardly grouped — a **when** guarding a single **message**, which you have decided should guard two statements, and whose condition you want to tighten:

```
(when (and ready loaded)
  (message "go"))
```

You want to end with the **message** and a new **setq** both inside the **when**, the condition extended to a third clause, and at one point you change your mind about the nesting entirely. In a character editor every one of these moves risks stranding a paren. In paredit, none of them can.

Slurp, barf, and the new sibling

Put the cursor in the **when** and add a statement after the **message**. Type the new form *outside* the closing paren, then **slurp** it in:

```
SPC k s      paredit_slurp_forward - the ) moves right to swallow the next form
SPC k b      paredit_barf_forward  - the ) moves left to expel the last form
```

Slurp pulls the sibling that was outside the **when** into it; if you overshoot, barf pushes it back out. At no point did you type or delete a parenthesis — you moved the *boundary*, and the expression stayed balanced through every keystroke. Tighten the condition the same way: slurp the new clause into the **and**.

Splice, raise, and convolute

Now change your mind about the structure. To remove the **and** wrapper entirely — promoting its clauses up into the **when** — **splice** it:

```
SPC k W      paredit_splice - delete the wrapping ( ), keep the contents
SPC k r      paredit_raise  - replace the parent form with the child at point
SPC k c      paredit_convolute - pull an inner form's wrapping out around its
  ~ parent
```

Splice unwraps; raise replaces a whole form with one of its children, deleting the rest; and **convolute** — the hardest paredit move to describe and the one whose presence Chapter 62 flagged as the mark of a serious port — reorders the nesting, pulling a form's wrapping outward around its parent. Even here, reshaping the

tree at its most aggressive, the parentheses stay matched, because paredit edits the tree and the text is downstream of it.

The guarantee, stated as a test

Here is the cleanest way to see what paredit buys: at no point in this whole session — not mid-slurp, not mid-splice, not mid-convolute — would the buffer have failed to parse. You could have run `:elisp` over it after any keystroke and it would have evaluated, because every paredit command transforms one valid expression into another, never passing through an invalid intermediate the way a character edit does. Contrast the character path: delete a close-paren to start moving it, and for the keystrokes until you type the replacement, the expression is unbalanced — a reader would reject it, and if your fingers slip you have introduced a bug that the editor's bracket matching will flag but not fix. Paredit removes that window of invalidity entirely. The guarantee is not “it is harder to unbalance a paren”; it is “you cannot, through these commands, ever have one unbalanced,” which is a categorical difference and the reason the suite is worth the dozen verbs. For code where the parentheses *are* the structure, editing the structure directly and letting the text follow is not a convenience — it is the correct level to work at.

Transpose and split

Two reordering moves finish the session. **Transpose** swaps the form before the cursor with the one after it, reordering two clauses without selecting either; **split** breaks one form into two:

```
SPC x t e      transpose_sexp - swap the two sibling forms
SPC j s        paredit_split  - close the current form, open a new one at point
SPC k a        paredit_absorb - pull the enclosing form into the current one
```

Run the elisp when you are done — `:elisp` over the buffer, or the REPL at `SPC a r` — and it evaluates, because it was never not a valid expression. That is the paredit guarantee made concrete: there was no moment in the whole reshaping where the form would not have parsed, which is why the structural vocabulary is worth the dozen verbs it asks you to learn. For the elisp extension author of Chapter 46, this is not a party trick; it is how you edit the editor's own configuration without fighting its parentheses.

Chapter 70 – A Day of Layouts and Narrowing

The fourth practice chapter is about *attention* rather than text — the two Spacemacs and emacs vocabularies, layouts (Chapter 63) and narrowing (Chapter 61), that exist to shape what you are looking at. We work a realistic day: two features in flight, each wanting its own arrangement of files, and within one of them a function dense enough to want isolating.

Two features, two layouts

You are working a bug fix in one part of the tree and a feature in another, and switching between them keeps meaning re-opening the same four or five files in the same arrangement. Make each a layout:

```
SPC l l      layout_create    - capture the current windows+buffers as a layout
SPC l R      layout_rename    - name it ("bugfix")
SPC l n / SPC l p  layout_next/prev - step between layouts
SPC l 1 .. SPC l 9 layout_goto_N - jump to a layout by number
SPC l TAB     layout_last     - flip to the previous layout
SPC l A       layout_add_buffers - fold another layout's buffers in
```

Arrange the bug-fix files, capture and name the layout, then build the second layout for the feature. Now `SPC l 1` and `SPC l 2` flip between two whole screen configurations — the right files, in the right splits — the way a tiling window manager flips desktops. You arranged each once; switching costs one keystroke.

Winner mode: the safety net

Mid-session you close splits aggressively to focus, then want the arrangement back. Winner mode is window-layout undo, independent of the text undo history:

```
SPC w u      winner_undo    - restore the previous window arrangement
SPC w . U     winner_redo    - step forward again
SPC t g       golden_ratio_resize - auto-grow the focused window
SPC w . a     ace-window     - jump to a window by its on-screen label
```

Close every split but one to read a long function, then `SPC w u` and the layout you had is back — the edit you made in the surviving split untouched, because the two undo histories are separate. This is the feature that makes aggressive window management painless: you never lose an arrangement you cannot recover.

Narrowing the dense function

Inside the feature you hit a function long enough that the rest of the file is just noise. Narrow to it:

```

SPC n f      narrow_to_function - fold everything outside the function
SPC n r      narrow_to_region   - narrow to the current selection
SPC n p      narrow_to_page     - narrow to the form-feed page
SPC n w      widen              - restore the full buffer
SPC n F      narrow_to_function_indirect - focus it in an indirect split,
                                         full file still visible in the other pane

```

`SPC n f` makes the function the whole buffer — `gg` and `G` go to its ends, search finds only inside it, a transform falls back to *it* rather than the file. When you want the function isolated *and* the call sites visible, `SPC n F` clones it into an indirect view (Chapter 61): the same text, two windows, one narrowed and one whole, edits to either appearing in both. Widen with `SPC n w` and the file returns, cursor where you left it. Between layouts holding the *files* you care about and narrowing holding the *region* you care about, the day's attention is managed by the editor, which is exactly what these two emacs-and-Spacemacs vocabularies were ported to do.

Two scales of attention

It is worth naming what makes layouts and narrowing a *pair* rather than two unrelated features, because they operate at the two scales attention actually has. Layouts manage attention at the scale of the *project*: which files, in which arrangement, are in front of you — the coarse “what am I working on” that changes when you switch tasks. Narrowing manages it at the scale of the *buffer*: which region of one file is in front of you — the fine “what am I working on *right now*” that changes minute to minute. A working day moves between both scales constantly, and `zmax` gives each its own vocabulary so the move is one keystroke at either grain: `SPC l 1` to bring up a whole task's files, `SPC n f` to isolate one function within them. Neither tool edits a single character; both shape what you are looking at, which is the part of editing that the keys of the earlier movements cannot touch. The editor that manages your attention as deliberately as it manages your text is a different kind of tool from one that only does the latter, and these two vocabularies are where that difference lives.

Chapter 71 — Vimscript That Drives the Editor

This practice chapter is also a correction, and the book’s honesty discipline requires it. Chapter 21 described the embedded vimscript as the tongue with “the most editor-facing surface to build” — a crate that could evaluate expressions and source scripts but whose ex-commands were “stubs awaiting wiring.” That was true when it was written and is substantially less true now, because the vimscript embedding is the part of Movement IV that moved most across the recent work. This chapter records where it landed.

What changed

The `vimlrs` crate advanced on two fronts the port report can measure. First, the Vimscript *builtin functions* — the `strlen`, `substitute`, `printf`, `map`, `filter`, `get` library a real `.vimrc` calls constantly — went from unimplemented to complete: the port report’s neovim `function` category now reads 451 of 451, **100%**, a category that was at zero in the first edition. Second, the *ex-statements* that a configuration uses for control flow — `:let`, `:if`, `:for`, `:try`, `:echon` — now actually run through `vimlrs`, rather than parsing into stubs. The tongue that “evaluates but does not yet fully drive” has started to drive.

Sourcing a real vimrc fragment

The concrete payoff is that a meaningful fraction of an existing `.vimrc` now executes. `zmax` sources `~/.zmax/init.vim` at startup (the scripting setup of Chapter 36), and a fragment like this — variables, a conditional, a loop, captured output — runs:

```
let g:greeting = 'editing in '
let g:editor = 'zmax'
if strlen(g:editor) > 0
  for i in range(1, 3)
    echon g:greeting . g:editor . " (" . i . ")\n"
  endfor
endif
```

Evaluate it with `:vim` over the buffer, or interactively in the REPL (`SPC a r`, then Tab to the vimscript language), and the `let` bindings persist across calls, the `if` guards the loop, the `for` iterates `range(1, 3)`, and `echon` accumulates the output — each one a statement that was a stub before. The string functions in the condition and the body (`strlen`, the `.` concatenation) are among the 426 builtins now present.

Iterating in the REPL

The fastest way to feel the change is the REPL (`SPC a r`, Chapter 64 and Appendix H), because it turns “edit `init.vim`, restart, see if it worked” into a live loop. Open the REPL, press Tab until the active language is vimscript, and type expressions against the running editor:

```

SPC a r          open the REPL
<Tab to vimscript>
:let g:n = 5      a binding persists across REPL evaluations
echo g:n * 2      -> 10, captured and shown
echo printf("%03d", g:n) -> 005, a builtin that used to be missing

```

Because globals and functions persist across calls (the `vimlrs` crate keeps them), the REPL is a genuine read-eval-print loop for vimscript, not a one-shot evaluator — you build up state interactively, test a `printf` format or a `substitute` call before committing it to `init.vim`, and watch the captured `echo` output appear. The same panel fronts the other four tongues a Tab away, so comparing how `elisp` and `vimscript` and `stryke` each express the same small transformation is a three-keystroke experiment. For a feature the first edition described as barely wired, an interactive REPL with persistent state is a concrete marker of how far it came.

What still has to land

The honest remainder has shrunk since the first edition, and this is where the recent work shows most. Of the four editor-mutating ex-commands Chapter 21 flagged as stubs, three now reach through. `:set` drives real state: a compiled table of every Vim and Neovim option means each one round-trips as vim does — `:set opt=val` stores it, `:set opt?` reports it, `:set` and `:set all` list them — while a mapped subset changes live editor state, so `set number` and `nonumber` toggle the line-number gutter, `expandtab`, `shiftwidth` and `tabstop` reshape indentation, `hlsearch` and `incsearch` drive search highlighting, and `foldmethod`, `spell`, `undofile`, `splitright`, `splitbelow`, `colorcolumn` and dozens more land on their `zmax` equivalents (the rest are kept inert, exactly as vim keeps options it has no use for). `:map` and `:noremap`, with the mode-variant clears and unmaps, bind into the real keymap. And `:autocmd` is a working event engine: it registers an events, pattern and command triple and fires it on `BufReadPost` and `BufEnter` when a file opens and around a save. What is left is `:command` to register a user-defined typable command, and the tail of options that are stored but not yet behavioral — the surface that turns “runs most of your vimrc” into “runs all of it.” The trajectory is the point: the tongue the first edition flagged as least wired is now the one with measured, climbing coverage, and the book updates the claim rather than leaving the old caution to rot. The one-machine-one-version note of Chapter 21 still governs — all five tongues must ride the same fusevm — and `vimscript`’s progress is the first large proof that the shared machine pays off as the embedding matures.

Where the other four stand

For the reader tracking the whole embedding, a brief status of the other tongues against Chapter 21’s account, so the picture stays current. `elisp` (`elisp.rs`) remains the proven one — the `defsubr` host-function surface is clean, the editor is wired to it, and `init.el` is sourced at startup; it is the language in which “five embedded tongues” is already a REPL you type into. `stryke` (`strykelang`) runs over the buffer via `:stryke` with state persisting across calls, drawing on the largest standard library of the five; its remaining gap is native host-function registration of the `elisp` kind, a smaller gap than `vimscript`’s because the language was built to be embedded. `AWK` (`awk.rs`) filters the selection through a program and replaces it as one undo step, lowering especially cleanly onto fusevm’s first-class AWK opcodes (Chapter 19). `Zsh` (`zsh.rs`) runs a command in the embedded shell with captured output and persistent shell state, the `:!` of Chapter 11 with no external process. All five are reached the same way — a typable command per language, or the REPL at `SPC a r` that fronts all of them and cycles with Tab — and all five ride the one fusevm, which is the architectural bet the whole movement rests on. The trajectory across the recent work is uniform: each tongue moved from “runs in the editor” toward “drives the editor,” `vimscript` furthest, and the book tracks the progress rather than freezing the first edition’s snapshot.

Chapter 72 – Resolving a Merge Conflict

Version control was Chapter 16's subject and the merge resolver was named in passing in Chapter 55; this practice chapter works a real conflict end to end, because resolving a merge is one of the moments an editor most earns or loses your trust, and zmax ports a JetBrains-style three-pane resolver precisely so the moment is survivable.

The setup

You pull, and a file comes back conflicted — the familiar <<<<<<, =====, >>>>>> markers wrapping two versions of the same lines. In a plain editor you now hand-edit those markers, hoping you delete exactly the right ones. zmax gives you two better paths: walk the markers in place, or open the structured resolver.

Walking the markers in place

The lightest path stays in the buffer. The conflict markers are navigable as motions, and each conflict can be resolved with one command without touching the markers by hand:

```
]n / [n      goto_next_conflict / goto_prev_conflict  jump between conflicts
SPC g c O      conflict_take_all_ours           keep our side everywhere
SPC g c T      conflict_take_all_theirs         keep their side everywhere
SPC g m        resolve_conflicts                open the resolver on this file
```

For a file where one side wins wholesale, `SPC g c O` or `SPC g c T` finishes the job in a keystroke. For a file where it is conflict-by-conflict, `]n` lands you on the next marker and the per-conflict take-ours / take-theirs / take-both commands resolve it, then `]n` again — the markers consumed one at a time without a hand-typed deletion that could clip a line.

The three-pane resolver

For a genuinely tangled conflict, `:merge` (alias `:resolve`) opens the structured tool: three panes, ours on the left, the merged result in the middle, theirs on the right, with a diff3 base pane showing the common ancestor so you can see *what each side changed from*. Inline character highlighting marks the exact spans that differ, not just the lines, and the result pane is editable — because, unlike the read-only ediff of Chapter 65, the merge resolver's whole job is to produce a resolved file. You accept a hunk from either side into the result, edit the result directly where neither side is quite right, and move on. The IDE Git tab (Chapter 55) lists the conflicted files, so a multi-file conflict is a worklist rather than a memory test.

Why three panes, and the diff3 base

It is worth understanding why the resolver shows *three* sources and not two, because the third is what makes a hard merge tractable. A naive conflict view shows you ours and theirs and asks you to pick — but with only

two sides you cannot always tell *why* they differ, whether a line is something one side added or something the other side deleted. The diff3 **base** pane shows the common ancestor: the version both sides started from. With it, the two changes become legible as changes *against a shared origin* — this side added a parameter, that side renamed the function, and the base proves they touched different things and can both be kept, or the same thing and must be reconciled. The inline character highlighting sharpens it further, marking the exact spans that differ rather than flagging whole lines, so a one-word divergence inside an otherwise-identical line is visible as one word. The three-pane-plus-base layout is the JetBrains merge tool's design, ported because it is the design that turns “guess which side is right” into “see what each side did and decide” — the difference between resolving a merge and gambling on one.

Finishing in magit

With the file resolved, the magit-style porcelain of Chapter 16 closes the loop: stage the resolved file per hunk, write the merge commit, and the conflict is done — all without leaving the editor for a separate merge tool. The spectrum of Chapter 65 is visible one more time: **:diff** to see what changed, ediff to understand a difference read-only, **:merge** to produce the resolution, magit to commit it. Four tools, one process, each the gentlest one that answers the question in front of you.

The whole resolution, in one stream

To see the loop without the prose, here is a two-file conflict resolved end to end without leaving the editor:

```
git pull          (in the integrated terminal, SPC p ')  
SPC g s          magit status – the conflicted files are listed  
]n              jump to the first conflict in the first file  
SPC g c O        take ours here ( or SPC g c T for theirs )  
]n ]n           walk to the remaining conflicts, resolving each  
:merge          for the tangled one, open the 3-pane resolver  
... accept hunks into the result, edit where neither side is right ...  
SPC g s          back to magit; stage the resolved files  
                write the merge commit
```

Every step happened in one process: the pull in the integrated terminal, the worklist in magit, the per-conflict resolution by marker command, the hard case in the three-pane tool, and the commit back in magit. No separate merge program, no leaving for the shell, no losing your place — the conflict went from “git stopped the merge” to “committed” inside the editor you were already in, which is the entire reason zmax ports a resolver rather than shelling out to one.

Chapter 73 — The Macro Ring and Counters

Macros were Chapter 40's subject — record a fix, replay it. This chapter is the part the recent Spacemacs work added on top: a *ring* of recorded macros and a *counter* that a macro can insert and increment, the two pieces that turn the record-and-replay loop into something closer to a tiny programmable transform.

The macro ring

vim's `q/@` records into one named register at a time; the Spacemacs keyboard-macro *ring* keeps several recent macros and lets you cycle among them, so you are not limited to the last thing you recorded. The ring lives under `SPC K`:

```
SPC K r n    kmacro_ring_next    rotate to the next macro in the ring
SPC K r p    kmacro_ring_prev    rotate to the previous
SPC K r d    kmacro_ring_delete  drop the current macro
SPC K r s    kmacro_ring_swap    swap the top two
SPC K r L    kmacro_ring_view    list the ring
SPC K e r    kmacro_to_register  save a macro into a named register
```

The ring is the answer to the common frustration of recording a second macro and losing the first: now both are there, one rotation apart, and either can be promoted into a lettered register for permanent reuse.

The counter

The counter is the feature that makes a macro able to *number* the lines it touches — emacs's `kmacro-insert-counter`, ported. You seed it, then a recorded macro inserts its current value and increments it on each replay:

```
SPC K c a    kmacro_add_counter    seed / add to the counter
SPC K c c    kmacro_insert_counter insert the value, then increment
```

The classic use is the “insert 1, 2, 3, ...” macro: record a macro that runs `SPC K c c` to drop the counter's value, makes its edit, and moves to the next line; each replay inserts the next integer. Numbering a list, generating sequential identifiers, labeling rows — the operations that would otherwise need a script — fall to a recorded macro plus a counter, no editing of the recording required. It is the smallest possible step from “replay an edit” to “replay an edit that counts,” and it is exactly the step that makes macros feel programmable.

A worked numbering macro

Concretely, suppose you have a column of list items and want each prefixed with its ordinal — `1.`, `2.`, `3.` — down the block. Seed the counter, record a macro that inserts the counter and moves on, then replay it across the lines:


```

SPC K c a      seed the counter at 1
q a            start recording into register a
I              insert at the line start
SPC K c c <space> insert the counter's value, then '.', and increment
<esc> j        leave insert, drop to the next line
q              stop recording
9 @ a          replay the macro nine times (or @@ to repeat)

```

Each replay drops the next integer because `SPC K c c` increments on every insertion, so the lines come out numbered without your editing the recording or computing the values. The same shape numbers anything sequential — generated identifiers, labeled rows, ordered steps — and it is the canonical demonstration that a macro plus a counter is a tiny program you wrote with your hands instead of a keyboard.

Saving a macro for keeps

A macro recorded into the ring is transient — useful for the task in front of you, gone a few rotations later. When you record one worth keeping, promote it out of the ring into a named register, where it persists like any register and can be replayed by name:

```

q a ... q      record a macro into register a (vim's record)
SPC K e r      kmacro_to_register – save the current ring macro to a register
@a            replay register a's macro      Q replay the last

```

This is the bridge between the two macro systems the editor carries: vim's `q/@` register macros (Chapter 6) and the Spacemacs ring of this chapter. The ring is the working set — several recent macros, cycled with `SPC K r n/r p` — and a register is the keepsake, a macro you reach for by letter days later. Recording into the ring keeps your last several attempts at hand; promoting the good one to a register makes it permanent. Between them you never lose a macro you wanted, which is the frustration the ring was added to solve.

When to reach for a language instead

The honest boundary, the same one Chapter 57 drew for transforms: a macro plus a counter handles the mechanical, regular case — the same edit, line after line, optionally numbered. The moment the per-line edit needs a *condition* or a *computation* the macro cannot express, you drop to an embedded tongue (`:awk`, `:stryke`, `:zsh`) and write the three lines instead. The macro ring is the common case made instant; the languages are the uncommon case made possible — and a power user keeps both in hand and reaches for the lighter one first.

Chapter 74 — A Transform-Library Session

Chapter 57 mapped the transform library; this chapter walks a session through it, because the library's whole argument — that the mechanical reshaping of text should be a command, not a program — only lands when you watch a real chore dissolve into three keystrokes. We take a small but realistic data task and never leave the editor or write a line of code.

The task

You have a buffer of JSON: an array of objects, each a user record with an `id`, a `name`, and a `signups` count. You need it as a CSV for a spreadsheet, you want a quick statistical read on the counts, and you want the `name` fields normalized to snake_case identifiers. Three transforms, three commands.

JSON to CSV

Select the JSON array — or rely on the whole-buffer fallback (Chapter 57) — and convert:

```
:json-to-csv      array of objects -> aligned CSV, header row from the keys
:json-table       the same data as an aligned plain-text grid (to read it)
:json-query users.0.name  pull a single value at a dot-path (to spot-check)
```

`:json-to-csv` reads the keys off the first object, emits a header row, and writes one CSV line per object, applied as one undoable change across the selection. If you only wanted to see the shape first, `:json-table` renders the same array as an aligned grid without committing to CSV, and `:json-query` pulls one value to confirm you are looking at what you think you are.

Destructive, non-destructive, and undo

One habit makes the transform library safe to use fast: knowing which commands *rewrite* the buffer and which only *report*. Most transforms are destructive by design — they replace the selection with the result, which is the point — but a deliberate handful are non-destructive, reporting into the status line and leaving the text untouched: `:stats`, `:json-validate`, `:count-matches`, `:bases`. You reach for those to *look* without changing, and for the rest you rely on the safety net underneath all of them. Every destructive transform applies as a single transaction (Chapter 24), which means one `u` undoes the whole thing — the entire `:json-to-csv`, not line by line — and the engine's position-remapping keeps your cursor sane afterward. So the working rhythm is fearless: run a transform, glance at the result, and `u` if it was not what you meant, with no partial-application mess to clean up. The non-destructive reporters and the single-transaction undo together are what let you treat the library as a place to experiment on real text rather than a thing to use only when you are sure.

Statistics without a spreadsheet

Now the counts. Select the `signups` column — a multi-cursor or block selection over the field (Chapters 30, 68) — and report on it without rewriting anything:

```
:stats          count / sum / mean / min / max, to the status line
:running-total  replace each line with the cumulative sum
:percent-of-total  replace each with its share of the total
:bases          show an integer in dec / hex / oct / bin at once
```

`:stats` is non-destructive — it reports into the status line and leaves the buffer alone — so it is the column-of-numbers equivalent of glancing at a spreadsheet's status bar. When you *do* want the buffer rewritten, `:running-total` and `:percent-of-total` replace each line with a cumulative or relative value, the per-line arithmetic that would otherwise be an AWK one-liner.

Renaming identifiers in bulk

Finally the names. Select every `name` value — the multi-cursor select-all of Chapter 30 turns “every name” into one selection of many ranges — and normalize the case:

```
:to-snake      userName -> user_name, on every selected range at once
:to-camel :to-kebab :to-pascal :to-constant  the other identifier cases
:align /      align the rows on a delimiter (tidy the result)
```

Because the transform runs on a multi-cursor selection, `:to-snake` rewrites every name in one undoable change — the rename chore of Chapter 57 made a keystroke, applied across the whole column simultaneously. Three transforms, and a JSON dump became a normalized CSV with a statistical read taken along the way, none of it a program. The session is the library's thesis in miniature: select, transform, repeat — and drop to a language only when a transform needs a condition the pre-built command does not have.

Cleaning up: whitespace and tidying

A data session usually ends in cleanup, and the library has a tidying shelf for it that is worth a separate mention because these are the transforms you run last, on the finished result. The whitespace and line-tidy commands square the edges:

```
:trim-lines      strip trailing whitespace from every line
:squeeze-blank-lines collapse runs of blank lines to one
:dedup :dedup-adjacent drop duplicate lines (anywhere / only adjacent)
:align :tabularize line the columns up on a delimiter
:pad-left :pad-right pad each line to an even width
:number-lines    prefix each line with its number
```

There is a static counterpart too — `normalize_whitespace_selection` collapses internal whitespace runs inside the selection, and `trim_selections` strips the edges of each range — so the cleanup is reachable both as a `:` command over the buffer and as a command over a multi-cursor selection. The pattern across the whole library holds here: the chore that is mechanical and regular — “make every line trailing-whitespace-free,” “line these up,” “drop the duplicates” — is a command, not a program, and the program is only for the chore that needs a thought the command cannot encode.

Generators and encoders, in passing

Two adjacent families round out a real session, because data work is not only reshaping what is there — sometimes you need to *produce* text, or *encode* it. The generators insert content at every cursor: `:uuid` drops

a fresh UUID, `:lorem` placeholder prose, `:date/:datetime/:timestamp` the current moment in various forms, and `:seq <start> <end>` a numeric range — so seeding a test fixture with ten UUIDs is a multi-cursor and a `:uuid`, not a copy-paste loop. The encoders cover the cryptography-adjacent chores a data task keeps hitting: `:base32-encode`, `:crc32`, `:caesar/:rot47`, `:morse-encode`, `:to-binary`, `:unicode-escape`, joined by the static `SPC x` shelf for URL-encoding, Base64, HTML entities, JWT decoding, and hex↔RGB colour. None of these needed a language either; they are the same select-and-run gesture pointed at “make this” and “encode this” rather than “reshape this,” and together with the structured-data and statistics families they are why the transform library replaced an AWK script for the overwhelming majority of the chores it covers.

Chapter 75 – Binary Files and the Hex Editor

The last practice chapter is the shortest, for the narrowest tool, because it closes a gap most editors simply leave open: what happens when you open a file that is not text. `zmax` routes it into a hex editor rather than refusing it or mangling it, and the round trip is byte-faithful.

Opening a binary

Open an executable, an image, a compiled artifact — anything that is not valid text — and `zmax` does not garble it into broken UTF-8 and it does not refuse the buffer. It routes the file into an `xxd`-style hex view, the same one reachable on demand:

```
:hex :hexview :hexedit    open the byte-faithful hex editor
```

The layout is the classic three columns: byte offsets down the left, the hex bytes in the middle, and the ASCII gutter on the right, with the printable characters shown and the rest dotted. A binary file is no longer a dead end in the editor; it is a buffer that counts in bytes instead of characters.

Why a hex editor belongs in a text editor

It is fair to ask why a modal *text* editor ships a hex editor at all, and the answer is the design goal the README states: “a maximally powerful CLI IDE with zero user configuration,” the editor you do not have to leave. The moment opening a non-text file either errors or garbles the buffer, you have been ejected from the editor and sent to a separate tool — `xxd`, a hex viewer, `vim -b` with a pipeline — for a task the editor could have handled. `zmax`’s wager is that the editor should absorb that task the way it absorbs the shell (the integrated terminal, Chapter 55), the diff tool (Chapter 65), and the five scripting languages (Movement IV): not because a hex editor is glamorous, but because every tool you do not have to leave the editor for is a context switch you do not have to pay. The hex editor is the smallest instance of the whole `zmax` thesis — fold the capability in, in-process, rather than shell out to a separate program — applied to the narrowest possible case, and the fact that even this corner is byte-faithful and round-trips exactly is the evidence that the thesis was taken seriously all the way down.

Editing bytes faithfully

The hex editor is editable, and the discipline is that nothing changes unless you change it. Drop into edit mode, and you overwrite bytes in either the hex column or the ASCII column:

```
i or R    begin editing (insert / overwrite bytes)
Tab       toggle between the hex column and the ASCII column
Ctrl-s    write the raw bytes back, exactly
```

Type a hex pair to set a byte, or `Tab` to the ASCII side and type a character to set it there; the two columns are two views of the same byte. `Ctrl-s` writes the raw bytes back with no re-encoding, so a round trip through the editor changes exactly the bytes you changed and nothing else — the byte-faithfulness that makes a hex

editor trustworthy for patching a binary or inspecting a file format. It is a small tool with one job, and it does that job exactly, which is the right note on which to leave the narrow tools and turn to the broad ones — fast navigation, the regex workbench, and a full workday — that close the movement: even the narrowest corner of the editor is finished enough to rely on.

Chapter 76 – Navigation at Speed

The practice movement has worked editing, refactoring, comparison, and reshaping; this chapter works the thing you do *between* all of them — moving. Beyond the vim motions of Chapter 6 and the pickers of Chapter 12, zmax ports three faster ways to get the cursor where you want it, each suited to a different distance: leap for anywhere on screen, harpoon for a small set of files you bounce between, and frecency for “the file I was just in.” Together they make the cursor’s travel time stop mattering.

Leap: anywhere on screen in two keys

The leap motions label every visible occurrence of a short target and jump to the one whose label you type, so crossing the screen costs the same two-or-three keystrokes regardless of distance. They live under `SPC j` and on the sneak keys:

```
SPC j l      goto_word      label every word on screen; type the label to jump
SPC j w      extend_to_word  the same, extending the selection to the target
s / S        sneak          vim-sneak: jump to a two-character target forward / back
SPC j j      a line-leap variant to a labelled line
```

Press `SPC j l` and every word on screen sprouts a one- or two-character label drawn from the `jump-label-alphabet` config (Appendix M, default `abcdef...`); type the label and the cursor is there. The point is that the gesture is *flat* in distance — a word at the top of the screen and a word at the bottom both cost the label, so you stop using `w/b` to walk across a function and just leap. `extend_to_word` is the same motion in Select mode, so you can leap-to-extend a selection over to a distant token before operating on it.

Harpoon: a handful of pinned files

Pickers are fast, but when you are actively working *between* four or five files — a header and its implementation, a test and the code it tests, a config and the code it configures — even a fuzzy-find is more friction than you want for a switch you make fifty times an hour. Harpoon (under `SPC H`) is the answer: pin a small, ordered set and jump among them by number.

```
SPC H a (or the add key)  pin the current file into the harpoon list
harpoon_1 .. harpoon_4    jump to the Nth pinned file by number
harpoon_next / _prev      cycle through the pinned set
harpoon_menu              open the list to reorder or remove
```

The distinction from the frecency picker is *intent*: harpoon is a manual, order-preserving set you curate for the task in front of you, where the pins stay put until you change them, so `harpoon_1` is always the same file through the whole session. It is the navigation counterpart to the layouts of Chapter 63 — you decide what matters and pin it, rather than letting an algorithm rank it.

Frecency: the file you were just in

For the unplanned jump — “take me back to that file from ten minutes ago” — the frecency picker (`SPC f r`) ranks recent files by a combined *frequency-and-recency* score, the z-style weighting that surfaces the file you are most likely to want at the top. It is the same ranking the start screen (Chapter 55) uses for its recent-files

list, exposed as a picker you can open any time. Where harpoon is the set you chose, frecency is the set the editor inferred from how you have actually been moving, and the two cover the planned and unplanned halves of “go back to where I was.”

Three distances, three tools

The three are not redundant; they partition by distance and intent. Leap is for *on screen* — anywhere you can see, in two keys. Harpoon is for *the task’s files* — a curated handful, by number. Frecency is for *the recent past* — ranked by behavior, when you did not plan the jump. A motion (`w`, `f`, `}`) is for the near and precise; a leap for the far and visible; a pin for the deliberate; a frecency pick for the half-remembered. Once all four are in your hands, the cursor’s location stops being something you navigate *to* and becomes something you simply *are*, which is the quiet luxury of an editor that took navigation as seriously as editing.

Moving the text instead of the cursor

There is a dual to navigation worth naming here, because it uses the same instinct in reverse: sometimes you do not want to move the cursor to the text, you want to move the text under the cursor. The **drag-stuff** commands do exactly that, sliding the current line or selection up and down through its neighbors:

```
SPC x . j    drag the line / selection down one row
SPC x . k    drag the line / selection up one row
```

Each press swaps the current line (or the whole selection) with the one in that direction, carrying it through the surrounding lines — the “move this line up/down” gesture that would otherwise be a delete-and-paste dance. Because it operates on the selection, dragging a multi-line block moves the whole block as a unit. It is a small command, but it pairs with the leap-and-pin navigation above to complete the picture: you can move the cursor to any text in two keys, and you can move any text past its neighbors in one — the editor handling both halves of “get this and that into the right relationship” so you never reach for cut-and-paste to reorder a couple of lines.

Chapter 77 – The Regex Workbench

Regular expressions are the connective tissue of text work — search, substitution, extraction, the `:g` command — and `zmax` ports a small workbench of conversions for them, because the same pattern is spelled three different ways across the traditions it serves. This chapter is that workbench: the `SPC x r` family that converts among PCRE, Emacs regex syntax, and the `rx` s-expression form, so a pattern you know in one dialect is usable in the others.

Why three dialects

The problem the workbench solves is that “regex” is not one language. The PCRE-style syntax `zmax`’s search uses `(\d+, (?:...), \b)` is what most modern tools speak. Emacs regex is gratuitously different — `\(...\)` for groups, `|` for alternation — and a user coming from emacs has those in their fingers. And `rx` is the *structured* form: an s-expression that spells a pattern as nested forms — `(one-or-more digit)` instead of `\d+` — which is verbose but readable and, crucially, composable like code. A user who thinks in any one of the three should be able to write it and get the others, and that is what the `SPC x r` menu does.

Converting among the forms

```
SPC x r x      convert the selected regex to the rx s-expression form
SPC x r /      the reverse: rx form back to a regex
SPC x r e t    replace an Emacs-style regex with the rx form
SPC x r p x    convert an explicit-PCRE pattern to rx
SPC x r p /    the reverse: rx to explicit PCRE
SPC x r t      a transform within the family
```

Select a pattern, pick a conversion, and the selection is rewritten in the target dialect as one undoable change. The use is concrete: you have an Emacs regex from an old config, you `SPC x r e t` it into the readable `rx` form to understand what it matches, and you `SPC x r /` it back out into the PCRE the search bar wants. The `rx` middle step is the pivot — a pattern is easiest to *read* and *edit* as nested forms, even if you ultimately run it as a flat regex.

How it joins the rest of the editor

The workbench is not an island; it feeds the regex-consuming features the book has already covered. The pattern you converted is the pattern you paste into `/` search (Chapter 6), into the `:s/.../.../` substitution and `:g/.../...` global of Chapter 11, and into the `:extract <regex>` and `:count-matches` transforms of Chapter 57. The conversion family exists precisely because those consumers speak PCRE, while the patterns you inherit or think in may not — so the workbench is the adapter between the dialect in your head and the dialect the search engine runs. It is a small subsystem, but it is the kind of small subsystem that turns a recurring papercut — “this regex is in the wrong dialect” — into a three-key fix.

Chapter 78 – A Data-Wrangling Workday

Chapter 72's workday interleaved the four *editing* traditions; this one interleaves the *text-processing* tools — the transform library, the embedded languages, multiple cursors, and the structured-data commands — on a single realistic data task, because the strongest proof that the pieces compose is watching them solve a problem none of them solves alone.

The task

You are handed a log export: a few thousand lines, each a whitespace-separated record of a timestamp, a user id, an HTTP status, and a byte count. You need three things from it: a clean CSV of just the user ids and byte counts, a per-user total of bytes, and the top handful of users by traffic. No single command does all three; the session is the point.

Reshape with transforms, compute with a language

Start where the transform library reaches (Chapter 57). To pull two columns into a clean CSV, the field-keeping transforms do it without a program:

```
SPC SPC :sort-by-field 2      group the records by user id
:extract \S+\s+(\S+).*(\S+)$ pull the id and the trailing byte count per line
:csv-column 1                keep a single column when you need just one
```

But the *per-user total* needs logic a pre-built transform does not have — sum a column, grouped by a key — so this is exactly the moment Chapter 57 said to drop to a language. Select the block and run an AWK program over it in place (Chapter 32):

```
:awk { tot[$2] += $4 } END { for (u in tot) print u, tot[u] }
```

The selection is replaced by the AWK output — one line per user, with their byte total — as a single undo step (Chapter 24). What the transform library could not express because it needed a grouped accumulation, three lines of embedded AWK expressed exactly, in process, with no external `awk` and no leaving the buffer.

Rank with stryke, polish with the library

Now the top users by traffic. A sort plus a head is the library's job again, but if you want the ranking *computed* — say, each user's share of the total — drop to stryke for the arithmetic (Chapter 33), then return to the library to format:

```
:stryke -> @lines |> map { split _ } |> sort_by { _[1] } |> take 5 |> p
:percent-of-total      each line's share of the column total
:align                 line up the columns for reading
:md-table               render the result as a Markdown table to paste into a report
```

The rhythm is the thesis of the whole text-processing story: the transform library for the common reshape, an embedded language the moment a *condition* or a *computation* is needed, and back to the library to format the result — three tools, one buffer, one undo history, no context switch. A multi-cursor selection (Chapter 30) threads through all of it, so any of these steps that should hit *every* matching line hits them together.

Why this is the editor, not a pipeline

The instructive contrast is the shell pipeline you would otherwise write — `awk ... | sort | head | column -t` — which is powerful but *blind*: you cannot see the intermediate results, cannot undo a stage, cannot eyeball the data between steps. The zmax session is the same power made *visible and reversible*: every transform and every language run leaves its result in the buffer where you can read it, `u` reverses any step, and the next operation sees what the last one produced. The pipeline is faster to type once; the buffer is faster to *think in*, because the data is in front of you the whole time. That is why the text-processing tools live in the editor rather than being shelled out to — not because the shell cannot do it, but because doing it where you can see it is a different and better experience, which is the same argument the hex editor (Chapter 75) and the embedded languages (Movement IV) make in their own corners.

Chapter 79 – Snippets and Live Templates in Practice

Snippets appeared in Chapter 39 as a feature and Chapter 54 as a library; this chapter works them, because snippet expansion is one of those capabilities that sounds minor and turns out to be load-bearing once it is in your hands. `zmax` ships a full snippet subsystem — a TUI to manage reusable snippets, trigger-word expansion with live tab stops, and `emmet` for markup — and the practice is in how the three layer.

The snippet library

`:snippets` (alias `:snip`) opens the snippet library editor, a TUI for creating, editing, and deleting reusable snippets persisted to `snippets.toml`. Each snippet carries four things: a **trigger** word, a **scope** (the language it applies in), a **description**, and a **body** written in LSP snippet syntax — the `${1:placeholder} / $0` form that marks tab stops. You define a snippet once in the TUI, and it is available in every buffer of its scope:

```
:snippets      open the library editor
                fields: trigger | scope | description | body
                body uses ${1:name} tab stops and $0 for the final cursor
```

Expanding a trigger

In a buffer, type a snippet's trigger word and press `Tab`, and the body expands with its tab stops live:

```
<type the trigger>  Tab      expand the snippet body
Tab / 5-Tab          goto_next_tabstop / goto_prev_tabstop
<type at a stop>    fills it; mirrored stops update together
```

Pressing `Tab` after expansion jumps to the next `${N:...}` placeholder; type to fill it, and if the snippet repeats a placeholder number the mirrored copies update as you type — the behavior the worked snippet library of Chapter 54 demonstrated. The cursor lands at `$0` when you are done. This is the same `snippet_expand` machinery the LSP completion uses, so a server-provided completion that *is* a snippet (a function call with parameter holes) expands the same way an LSP omni-completion lands with its arguments as tab stops.

Triggers, emmet, and priority

Two expansion systems share the `Tab` key, and their priority is defined: **user snippet triggers take precedence over emmet**, and triggers are scoped per language, so a trigger you define for Rust does not fire in a Python buffer. Emmet — `emmet_expand`, also reachable as `:emmet` / `:zencode` — is the markup abbreviation expander, turning `ul>li*3` into a nested list in an HTML buffer. The resolution order on `Tab` is therefore: a user trigger if one matches in this language, then emmet if the buffer is markup, then the smart-tab behavior of Appendix M, then a literal tab. That layering is why `Tab` can do so much without ambiguity — each consumer is gated by whether its precondition (a matching trigger, a markup buffer, non-whitespace to the left) holds, and the first that applies wins.

Why snippets and live templates converge

The JetBrains layer's "live templates" (Chapter 59) map onto this same engine, which is the point worth ending on: zmax did not build three snippet systems for three traditions. It built one — the trigger-plus-tab-stops expander — and let the vim, emacs, JetBrains, and LSP paths all reach it. A JetBrains user's live template, a vim user's snippet trigger, and an LSP server's snippet completion are three front doors onto one expansion mechanism, the same way the four keymaps are four front doors onto one selection engine. The convergence is the recurring shape of the whole editor: many vocabularies, one machine underneath.

Chapter 80 – The Start Screen, the Mouse, and Coming Back

The last practice chapter before the synthesis is about the edges of the session — opening the editor and the moments you reach for the mouse — because an editor is not only what happens while you are typing in it. zmax ports a startify-style start screen, full mouse support, and session persistence, and together they make launching and resuming the editor feel like picking up where you left off rather than starting cold.

The start screen

Launch zmax with no file argument and it opens on a **start screen** — a vim-startify port: a fortune-and-cowsay header over two lists of recent files. One list is frequency-ranked (most-used, weighted by recency, the same ranking as the `SPC f r` picker of Chapter 76); the other is a plain most-recently-used list scoped to the current directory. Each file is one keystroke away, so the first thing you see is “what was I working on,” not an empty buffer. It is a small touch, but it is the difference between an editor that opens *ready* and one that opens *blank*.

The mouse, where it is faster

zmax is keyboard-first but not keyboard-only, and the mouse is wired for the moments it is genuinely faster:

```
click a pane      focus it
scroll wheel      scroll the view
click a tab        switch buffers
click the gutter   ( breakpoints, fold markers )
drag the split divider  resize the windows ( the C-w +/->/< proportions )
```

None of these is required — every one has a keyboard equivalent the book has covered — but each is there for when reaching for the mouse beats reaching for the keys: dragging a split to exactly the width you want is faster by hand than nudging it with `C-w >`, and clicking the pane you can see is faster than counting `C-w l` presses. The design rule is that the mouse is *available* everywhere it helps and *required* nowhere, so a keyboard purist never needs it and a hand already on the trackpad is not made to leave.

Coming back

The third edge is resumption, and it is where the workbench’s persistence (Chapter 55) and the layout system (Chapter 63) pay off together. The IDE layout — drawer widths, which panels are collapsed, the minimap’s state, the bottom panel’s height — and the active colorscheme are written to `appdata.toml` on quit and restored when you next open zmax without a file. The named layouts hold your task arrangements; the start screen holds your recent files; the appdata holds your chrome. So “coming back” is not a cold start: the editor opens the way you left it, offers the files you were in, and one `SPC l` brings up a whole task’s window arrangement. Between the three, the gap between closing the editor and being productive again shrinks to a keystroke — which is the quiet, unglamorous form of the zero-configuration promise, applied not to the first launch but to every launch after.

Files that change underneath you

One more edge belongs here, because it is about the world changing while the editor is open: **auto-reload**, zmax's port of vim's **autoread**. When a file you have open is changed on disk by something else — a **git checkout**, a code formatter run from the shell, a build step that rewrites a generated file — zmax notices and reloads the buffer to match, so the version you are looking at is the version on disk. It is on by default and configurable. The care is in the conflict case: if you have *unsaved local edits* and the file also changed on disk, the reload keeps your local edits rather than silently discarding them, so the feature never costs you work. Paired with that is a guard the changelog notes explicitly — a fix against an autosave path that could truncate a file after an undo — which is the same instinct showing up as a bug fix: the editor's job is to never lose your text, whether the threat is an external change or its own save logic. Auto-reload is the convenience; not-losing-your-edits is the discipline underneath it, and the two together are why an editor that follows the filesystem is safe rather than alarming.

Chapter 81 – A Multi-Tradition Workday

The last chapter is a single session that uses all four traditions at once, not to show off but because a real workday does exactly this: a vim motion here, a Spacemacs leader menu there, an emacs kill-ring when it is faster, a JetBrains refactoring when the job is semantic. The point the whole book has argued — that the traditions are vocabularies over one engine, not rival editors — is easiest to believe when you watch them interleave in a single half hour.

The session

You open the project. The start screen (Chapter 55) offers a frequency-ranked recent file; you take it, and you are in a Rust buffer with your vim fingers.

```
# vim: navigate and edit by reflex
} } w ciw    jump two paragraphs, change the inner word
/handler<ret> search — the match is selected (Chapter 6)
n n          walk the matches

# JetBrains: the change is semantic, so reach for a refactoring
SPC SPC extract_function <ret>  lift the selected block into a method

# Spacemacs: discover the git menu rather than recall a chord
SPC g s      magit status (Chapter 16)
SPC g b      blame the line you just changed

# emacs: the kill-ring is faster for this reorder
C-SPC <motion> C-w  set mark, kill the region onto the ring
C-y M-y        yank it back, M-y to cycle to an earlier kill

# back to vim to finish
SPC n f      narrow to the function while you polish it
SPC n w      widen when done
:w           write
```

No line of that session is contrived; it is the natural path of a small task, and the striking thing is how little friction the tradition-switching carries. You did not “switch modes” between vim and emacs and Spacemacs and JetBrains; you reached for whichever vocabulary spelled the next operation most directly, and the engine ran all of them, because — Chapter 66 — it cannot tell which tradition asked. The vim `ciw` and the emacs `C-w` and the JetBrains `extract_function` and the Spacemacs `SPC g s` are four spellings dispatched to one selection engine and one language server.

The afternoon: data, then prose

The afternoon is a different shape of work, and it pulls in the other halves of the editor — the text-processing tools and the reference surfaces — without ever feeling like a different program:


```

# a data file lands in your lap
:json-to-csv          reshape it ( Chapter 57 )
:awk { s[$2]+=$4 } END {...}  compute what no transform expresses ( Chapter 78 )
:md-table             format the result for a report

# you forget a command's name
SPC h h align <ret>      find it in the Help browser ( Chapter 64 )

# you need to bend one key for this project
# ... edit .zmax/config.toml: [keys.normal] "+t" = ":run-shell-command cargo test"

# write the report in an org buffer
SPC m t                cycle a heading to TODO ( Chapter 56 )
SPC n f                narrow to the section you are drafting ( Chapter 61 )
SPC a r                test a stryke expression in the REPL ( Appendix H )

```

Nothing in that afternoon is a context switch, and that is the whole point restated from the other direction: the morning showed four *editing* traditions over one engine; the afternoon shows the transform library, an embedded language, the help system, a config remap, org mode, narrowing, and the REPL — seven different subsystems — all reached from the same buffer, with the same undo history, in the same process. The editor that absorbed the four keymaps also absorbed the shell, the data tools, the planner, and five languages, and a real day moves through all of them the way it moves through the keymaps: by reaching for whichever one spells the next thought most directly.

The whole book in one keystroke stream

That interleaving is the thesis of zmax, and it is the right place to end. The editor is conservative about your fingers: every tradition's muscle memory survives, and you were never made to choose. It is radical about its internals: under those four vocabularies sits one rope, one selection model, one tree-sitter layer, one language server, one debugger, one shared bytecode machine running five embedded languages, in a single static binary with no external runtime to install and no configuration language it invented for the purpose. The first edition of this book documented three traditions and five tongues as a promise. This edition documents four traditions, a maturing embedding, and an IDE workbench grown out of all of them — and every key, command, and number in it is traceable to a file in the source tree, with a port report that regenerates from that source on every run and refuses to round the answer up. That is the editor. The appendices that follow are the quick reference; the argument is done.

The measure of the whole

If a single number had to stand for the editor this book describes, it would be the one from Chapter 29's capability table: of 186 distinct editor capabilities, drawn from four of the most influential editing traditions ever built, zmax implements 185 — one row partial, none absent — in a single static binary, on first launch, with no configuration you wrote. That is the concrete form of “zero user configuration”: not that the editor cannot be configured, but that it does not *need* to be, because the capabilities a developer expects are present and bound the way one of four traditions already trained their hands to expect. Where the table is not at 100% it says so by name rather than hiding it, and the figure is regenerated from source rather than asserted — which is the only reason a number like that is worth printing. Everything else is there: the rope and the selections from Helix, the four traditions' keys, the LSP and the debugger, the five embedded tongues on one fusevm, and the IDE workbench grown out of all of it. The book has tried to earn each of those claims a file at a time; the port report stands behind them a number at a time; and the editor stands behind both, the day you install it, with nothing to set up first. That is the promise the name made — z plus emacs, programmable all the way down, conservative about your fingers — kept, and measured, and refusing to round up.

Chapter 82 – The Calendar, and the Calendars Inside It

There is a particular kind of feature that separates an editor from a text editor, and GNU Emacs has always had it: the calendar. Not a date picker, not a widget bolted onto an agenda plugin — a full-screen mode that knows what day it is in eight different reckonings of time, that can tell you the Julian Day Number of a date in 1583, that can find the next occurrence of a Mayan haab date and the March equinox in a given year. It has no business being in a text editor. It is in Emacs anyway, and a port that claims to be the Emacs port has to carry it.

zmax carries it in `zmax-core/src/calendar.rs`, and the shape of that file is the shape the rest of this book has been describing: a pure, dependency-free, unit-tested core, with the interactive component in the term crate calling into it. The module header states the arrangement plainly — the term-crate component “holds a *point date* and calls these to move it and lay out the month grid.” Every function in the file takes dates and returns dates. Nothing draws, nothing reads a file, nothing consults a clock. That is what makes it testable against known dates, which is the only way a calendar ever gets to be correct.

The one primitive everything else stands on

The base of the module is a two-function pair:

```
pub fn to_serial(d: Date) -> i64
pub fn from_serial(z: i64) -> Date
```

`to_serial` maps a proleptic-Gregorian date to days since the Unix epoch, and `from_serial` inverts it. The comment names the algorithm: Howard Hinnant’s `days_from_civil / civil_from_days`, which the header describes as “exact for any year.” The implementation is the recognizable era arithmetic — shift the year so March begins it, divide into 400-year eras, compute the day-of-era, and adjust by the constant 719468 that puts the epoch where the epoch belongs.

The reason to care which algorithm is used here is that everything above it inherits its correctness. `add_days` is `from_serial(to_serial(d) + n)`. `weekday` is a modular reduction of the serial. `day_of_year` is a subtraction of two serials. `count_days`, which the file identifies as `calendar-count-days-region`, is the absolute difference plus one — inclusive, because Emacs counts both endpoints. A calendar built this way cannot have an off-by-one at a month boundary, because it has no month boundaries; it has one integer line and a bijection onto civil dates.

There is exactly one place where the integer line is not enough, and the file handles it explicitly. `add_months` cannot be done in serial space, because “one month later” is not a fixed number of days. The port does the arithmetic in month-space and then clamps:

```
let total = (d.year as i64) * 12 + (d.month as i64 - 1) + n;
let year = total.div_euclid(12) as i32;
let month = (total.rem_euclid(12) + 1) as u32;
let day = d.day.min(days_in_month(year, month));
```

The comment names the behaviour it is matching: “Emacs `calendar-forward-month` behaviour: Jan 31 + 1mo = Feb 28/29.” This is the sort of decision that is invisible when right and infuriating when wrong. A naive implementation rolls over into March 3rd. Emacs clamps. `div_euclid` and `rem_euclid` rather than `/` and `%`

matter for the same reason: negative month offsets must floor, not truncate toward zero, or moving backward across a January produces month 0 of the wrong year.

Weeks, ISO weeks, and nth-weekday

Above the serial layer sits the week vocabulary. `beginning_of_week` subtracts the weekday index, `end_of_week` adds its complement — with the comment noting these are `calendar-beginning-of-week` “with the default Sunday start,” which is the correct default to port and the correct thing to say out loud, because half the world starts weeks on Monday and a reader needs to know which convention the code assumes.

`iso_week` is separate, and returns a triple — ISO year, ISO week, ISO weekday — because the ISO year is not always the civil year. Late December can belong to week 1 of the following ISO year, and early January to week 52 or 53 of the previous one. Returning a triple instead of just a week number is the port declining to hide that.

`nth_weekday(year, month, target, n)` and `last_weekday(year, month, target)` are the two functions that make floating holidays expressible: the third Monday of January, the last Monday of May. They exist because `holidays` needs them:

```
pub fn holidays(year: i32, month: u32) -> Vec<(u32, &'static str)>
```

The doc comment on that function is a model of how to document a partial port. It says the function covers “the fixed-date observances plus the `nth`-weekday floating holidays” and “deliberately omits astronomically-computed ones (Easter, equinoxes).” That is a scope statement, not an apology, and it is checkable: the fixed table and the floating table are both in the body, and a reader can see exactly which observances are in and which are not. `holiday_on(d)` is the single-date query — `calendar-cursor-holidays` — implemented as a filter over `holidays` for the date’s month.

Eight calendars, one fixed day

The second half of the file is headed, in a comment, as “the zmax port of GNU Emacs cal-julian / cal-hebrew / cal-islam / cal-persia / cal-coptic / cal-french / cal-bahai / cal-mayan.” Each of those Emacs libraries converts between the Gregorian calendar and some other reckoning, and the port takes the standard approach that the calendrical-calculations literature established and that Emacs itself uses: convert everything through a single fixed-day number.

```
pub fn rd(d: Date) -> i64
pub fn from_rd(f: i64) -> Date
```

R.D. — Rata Die — is day 1 = January 1st of year 1 in the proleptic Gregorian calendar. Every other calendar in the module gets a `fixed_from_X` and an `X_from_fixed`, and conversion between any two calendars is composition through R.D. That is why the file can support eight systems without eight-squared conversion functions, and it is why adding a ninth is a self-contained exercise.

The Julian (Roman) calendar shows the pattern in miniature. `julian_leap` is the plain every-fourth-year rule with no century exceptions. `fixed_from_julian` handles the absence of a year zero — the year before 1 CE is -1 , not 0 — which is the classic source of one-day errors in hand-rolled Julian conversion. `julian_from_fixed` inverts it with the standard prior-days-plus-correction scheme, where the correction is 0 before March, 1 in a leap year, and 2 otherwise.

Coptic and Ethiopic share an implementation because they share a structure: both have thirteen months (twelve of thirty days and one short one), and the port gives each its own thirteen-element month-name table and routes both through a common conversion. The Hebrew calendar is the hardest of the group and the file does not shy from it: `hebrew_leap` implements the nineteen-year Metonic cycle, `hebrew_last_month_of_year` returns thirteen in a leap year, and `hebrew_elapsed_days` — annotated as `calendar-hebrew-elapsed-days` — carries the molad computation with its postponement rules, including the condition on

`day % 7 == 2 && parts >= 9924 && !hebrew_leap(year)`. Nobody writes that from intuition. It is a port, from a spec, checked against known dates, which is exactly what the standing instruction about porting demands.

Mayan gets three interlocking cycles — the long count, the 365-day haab, and the 260-day tzolkin — plus the search functions `mayan_next_haab`, `mayan_next_tzolkin`, and `mayan_next_round`. The tests pin them against the canonical correlation: the fixed day for long count `13.0.0.0.0` yields tzolkin `4 Ahau` and haab `3 Kankin`, and the next-occurrence searches land exactly 365, 260, and 18980 days away, the last being the calendar round — the least common multiple of the two cycles. Those constants falling *out* of the search functions rather than being written *into* them is the test that the implementation is real.

Astronomy

The last layer is the one that cannot be done in integers. `astro_from_rd` and `rd_from_astro` convert between R.D. and astronomical day numbers, which begin at noon rather than midnight — the tests assert `astro_from_rd(730120.0)` is `2451544.5`, the half-day offset made explicit. On top of that sits `solar_date_next_longitude`, which finds the next moment the sun reaches a given ecliptic longitude at a given time zone; a longitude of 30 degrees run from March 1st is how the module locates an equinox, and the test does exactly that.

Note what this buys and what it costs. The equinox is *not* in the `holidays` table — the doc comment said so — but the primitive that would compute it is in the module, tested, at floating-point precision. The port distinguishes between “we have the machinery” and “we have wired it into the holiday list,” and says which is which. That distinction, kept honestly, is worth more to a reader than a holiday table with a plausible-looking Easter in it.

Why this is in the editor at all

The instinct is to ask why a modal editor written in Rust should contain the Hebrew molad. The answer is the one that governs the whole port: the claim is *the Emacs port*, and Emacs users do not experience Emacs as a set of features they chose. They experience it as a place where the thing they need is already there. `M-x calendar` being there — and being there with the other calendars and the diary of the next chapter attached to it — is the difference between a keybinding compatibility layer and a port. And the cost, in this architecture, is one pure module with no dependencies that the linker will happily leave cold if nobody calls it.

Chapter 83 — The Diary

The calendar of the previous chapter is a coordinate system. The diary is what you attach to it. In Emacs, `diary-file` — by default `~/diary` — is a plain text file where each entry begins with a date specification and continues with text, and the calendar reads it to mark days and to display what falls on the day under the cursor. `zmax` ports it in `zmax-core/src/diary.rs`, one of the largest files in the core crate — second only to `movement.rs` — and it is large for an instructive reason: the diary's date language is much bigger than it looks.

Three date styles that cannot be told apart

Before any parsing happens, the module has to settle an ambiguity that has no local solution. The `DateStyle` enum ports Emacs's `calendar-date-style`, and its documentation states the problem exactly:

It is a global setting rather than a per-entry one: `10/11/2026` is October 11th under American and November 10th under European, and the two cannot be told apart from the text.

There is no heuristic that fixes this and the port does not pretend there is. The style is a mode — American, European, ISO — with a `next()` that cycles them in the order `calendar-set-date-style` does, and every parse and every format takes it as a parameter. Hence the paired API throughout the file: `parse_line` and `parse_line_with_style`, `parse_file` and `parse_file_with_style`, `format_yearly` and `format_yearly_styled`. The unsuffixed forms use the default (American, matching Emacs); the suffixed forms are the honest ones.

The style's reach goes further than reading. `date_string` formats a full date in the style; `yearly_string` formats the month/day of a year-recurring entry; and `sexp_args` orders the arguments of a `%(diary-anniversary ...)` form, because those arguments follow `calendar-date-style` too. A user who switches from American to ISO changes not just what the file means but what the insert commands write into it. One detail in the implementation is easy to miss and correct: `order2`, which handles the two-number no-year form, treats ISO like American, because only ISO's *three*-number form leads with the year.

The specification language

`DateSpec` is the parsed form of everything that can introduce a diary line, and its variant list is the actual scope of the feature:

- `Specific` — one date, `October 12, 2024` or `10/12/2024`.
- `Yearly` — a month and day, every year.
- `Weekly` — a weekday name, every week.
- `Anniversary` — `%(diary-anniversary MONTH DAY [YEAR])`.
- `Block` — `%(diary-block M1 D1 Y1 M2 D2 Y2)`, a date range.
- `Cyclic` — `%(diary-cyclic N MONTH DAY YEAR)`, every `N` days from a base.
- `Float` — `%(diary-float MONTH DAYNAME N [DAY])`, the `n`th weekday of a month, with `month = None` meaning any month.
- `DateWild` — `%(diary-date MONTH DAY YEAR)` with `t/*` wildcards.

Everything after the first three is a *sexp entry*: a line whose date spec is an elisp form the diary evaluates. This is where the size of the file comes from, and it is the point at which most reimplementations of “Emacs-style

diary” quietly stop. `zmax` does not, and the reason it can is the architectural fact that Chapters 19 through 22 establish: `elisp` is not a foreign language to this editor. But note that `parse_sexp` and `parse_sexp_with_style` handle these forms *textually*, in the pure core, rather than by evaluation — a diary file must parse identically whether or not an interpreter is up, and the `sexp` forms Emacs ships are a closed set with known argument shapes.

Each recurring form gets its own predicate, and each returns more than a boolean where Emacs’s display needs more:

```
pub fn anniversary(month: u32, day: u32, year: Option<i32>,
                  on: Date) -> Option<i64>
pub fn cyclic(n: i64, base: Date, on: Date) -> Option<i64>
pub fn block(start: Date, end: Date, on: Date) -> bool
```

`anniversary` and `cyclic` return `Option<i64>` rather than `bool` because the entry text can interpolate the count — “our 12th anniversary”, “day 400” — and the count is what the predicate already computed. Returning it costs nothing and saves the caller from recomputing. `block` has nothing to count, so it returns a `bool`. That asymmetry is the API telling you which forms carry an ordinal.

`ordinal_suffix` sits nearby, mapping 1 to “st”, 2 to “nd”, 3 to “rd”, and the rest to “th” — with the teens handled, because 11th, 12th and 13th are the trap every naïve implementation falls into.

Other calendars, in the diary

The diary can hold entries dated in a non-Gregorian calendar, and `OtherCal` carries the four Emacs supports there: Hebrew, Islamic, Baha’i, and Chinese. Each has a `prefix()` character (the letter that marks such an entry), a `from_prefix()` inverse, a display `name()`, and a `sexp_kind()` — the string that appears in the `sexp` form: `hebrew`, `islamic`, `bahai`, `chinese`.

The interesting methods are the ones that make a foreign calendar behave like a recurrence rule. `ymd(date)` converts a Gregorian date into the other calendar’s year/month/day. `month_name(year, month)` names a month in that system — and returns `Option`, because in the Hebrew calendar whether month 12 is Adar or Adar I depends on whether the year is a leap year, so not every (year, month) pair has a name. `is_month_name` recognizes one. `years_between(from, to)` counts years in that calendar’s own reckoning, which is what an anniversary in a Hebrew or Islamic date needs. `other_anniversary(cal, month, day, year, on)` puts it together and returns the elapsed count, mirroring the Gregorian `anniversary`.

This is the payoff of Chapter 82’s fixed-day design: none of these functions implement calendar arithmetic. They convert through R.D. into a calendar the previous module already knows, ask the question there, and convert back.

Times, sorting, and includes

Emacs sorts a day’s entries by any time appearing at the start of the entry text, so a day’s agenda reads in chronological order. `diary_entry_time(text)` extracts that time as a comparable integer, `sort_entries` orders a day’s slice of references by it, and `sorted_entries_for(entries, date)` is the composition — filter to the date, then sort — which is what a display actually wants.

`include_path(line)` and `include_paths(contents)` handle the include directive that lets a diary file pull in others. They return paths and nothing else; the module does no I/O, so the command layer reads the files and re-enters `parse_file_with_style` for each. Recursion, cycle-breaking, and error handling are the caller’s, which is the right split: a pure parser that cannot loop forever is easier to trust than one that tries to be clever about the filesystem.

The insertion side mirrors the parsing side. `format_daily`, `format_weekly`, `format_monthly`, `format_yearly`, and `format_anniversary_sexp` produce the headers the `insert-*-diary-entry` commands

write, each with a `_styled` variant where the style matters. Round-tripping is the invariant that makes the family coherent: what `format_yearly_styled` writes, `parse_line_with_style` reads back as the same `DateSpec`.

What this looks like from the keyboard

A user does not see any of the above. They see a diary file:

```
October 12, 2026  Ship the release
Monday           Weekly sync, 10:00
10/31            Halloween
%%(diary-anniversary 3 14 2019)  years since the first commit
%%(diary-cyclic 14 1 5 2026)     Fortnightly review
%%(diary-float 11 4 4)           US Thanksgiving
%%(diary-block 12 20 2026 12 31 2026)  Holiday shutdown
```

and a calendar that marks those days and shows their text. The point of the port is that this file — an existing one, written years ago against GNU Emacs — behaves the same way here. That is the standard the whole book applies: not “we have a diary feature,” but “your diary file works.” The difference between those two claims is measured in exactly the sort of detail this chapter has been walking through — the teens in `ordinal_suffix`, the `Option` on `month_name`, the ISO two-number form falling back to American order. Each one is a place where a plausible implementation and a faithful one diverge.

Chapter 84 — Dired

Dired is the oldest good idea in Emacs: the directory is a buffer. Not a tree widget, not a panel — a buffer of text, which you navigate with the motion keys you already use, mark with `m`, flag with `d`, and act on in bulk. Everything that edits text edits the listing; everything that moves through text moves through the directory. `zmax` ports it, and the split follows the pattern the rest of the port uses: `zmax-core/src/dired.rs` holds “pure, editor-type-free algorithms” and is “filesystem-free and unit-tested in isolation,” while the term layer reads the directory into `DiredEntry` values, calls into the core, and renders.

That split is worth pausing on, because Dired is the feature where it is hardest to maintain and most valuable to have. A directory editor is *about* the filesystem; making the interesting half of it filesystem-free requires deciding that the interesting half is the sorting, the naming, the marking and the predicates — and that the directory read is plumbing. The `DiredEntry` struct enforces that decision:

```
pub struct DiredEntry {
    pub name: String,
    pub is_dir: bool,
    pub is_symlink: bool,
    pub size: u64,
    pub mtime: i64,
}
```

Five fields, all owned, none of them a handle. The comment on `mtime` notes it is “only used as a sort key, so its absolute value is irrelevant” — which is what makes the sort tests writable with arbitrary integers.

Sorting

`SortKey` has four variants — Name, Size, Time, Ext — and a `next()` that cycles them in the order Emacs’s `s` key does, extended: Emacs cycles name and time, `zmax` adds size and extension to the same cycle. The `label()` method returns the string the status line shows, so the UI never spells the names itself.

`sort_entries` has one rule above the key:

```
b.is_dir.cmp(&a.is_dir).then_with(|| { /* key comparison */ })
```

Directories always sort before files, and the doc comment says why: this is the `--group-directories-first` behaviour, which “`zmax` Dired uses unconditionally.” And `reverse` “flips the within-group order but keeps dirs-first.” That is a deliberate divergence from a naive reversal of the whole slice, and it is the right one — reversing a listing to see the largest files should not bury the directories at the bottom.

`extension` backs the `Ext` key and contains one rule worth stating: a leading dot is not an extension separator, so a dotfile has no extension rather than an extension equal to its name. It is implemented as a reverse find of `.` guarded by an index greater than zero, which is the whole fix.

Sizes

`human_size` is nine lines and gets three things right that hand-rolled versions routinely get wrong. It returns bare digits below 1024 rather than a fractional kilobyte. It caps at the last unit in the table rather than running off the end. And it switches precision at ten — one decimal while the number is small enough for it to mean

something, none once it is not. The unit table starts with an empty string so the sub-kilobyte case needs no special formatting path.

Marks, flags, and the predicates behind bulk operations

Dired's power is that marking is separate from acting. `mark_char(marked, flagged)` returns the single character in the left column — `D` for flagged, which takes precedence, `*` for marked, space otherwise — and that precedence order is the port matching Emacs rather than choosing.

The predicates are the interesting part, because each one is the whole content of an Emacs command:

```
pub fn is_backup_file(name: &str) -> bool
pub fn is_auto_save_file(name: &str) -> bool
pub fn is_garbage_file(name: &str) -> bool
pub fn is_executable_mode(mode: u32) -> bool
```

`is_backup_file` tests for a trailing tilde, covering both the plain and the versioned GNU backup forms. `is_auto_save_file` recognizes a name wrapped in `#` with a length guard so the two-character degenerate case does not qualify. `is_garbage_file` is the dired-x `dired-garbage-files-regexp` default, and the port implements it as an extension set — `aux`, `bak`, `dvi`, `log`, `orig`, `rej`, `toc`, `out` — unioned with the backup and auto-save predicates. A reader can audit that list in one glance, which a regexp would not permit. `is_executable_mode` is a mask against the three execute bits, backing `dired-mark-executables`.

`marked_summary` takes a closure rather than a mark set:

```
pub fn marked_summary(entries: &[DiredEntry],
                      is_marked: impl Fn(&str) -> bool) -> (usize, u64)
```

The count-and-total that Dired shows for the marked set, with the mark storage left entirely to the caller. This is the same technique Chapter 87 will show `occur` using with its matcher closure, and for the same reason: it keeps a dependency — here the mark set's data structure, there the regex engine — out of a module that has no other need of one.

Renaming in bulk

Emacs's `%` prefix in Dired is a small language for transforming the marked names. `zmax`'s `NameTransform` covers upcase, downcase, and replace:

```
pub enum NameTransform<'a> {
    Upcase,
    Downcase,
    Replace { from: &'a str, to: &'a str },
}
```

`transform_name` applies one. The doc comment is careful about scope: `% R/% C` regexp rename is implemented here as “a literal find/replace of the first match, keeping the transform dependency-free.” That is a stated limitation with a stated reason. The real regexp path exists separately as `regexp_replace_name`, which takes a compiled regex — so the module gets the dependency only on the function that needs it, and the enum stays usable in a build that never touches the regex crate.

`destination_path(dest, dest_is_dir, src_name)` resolves where a copy or rename lands, handling the “target is a directory, so append the source's basename” case. `relative_path(from_dir, target)` computes the relative form Dired shows for symlink targets. `parse_octal_mode` and `is_valid_filename` guard the `chmod` and `rename` prompts.

Cleaning up

Two functions handle the maintenance operations, and both are more subtle than their one-line descriptions.

`parse_numbered_backup(name)` splits a versioned backup into base and version, and explicitly rejects the unnumbered form. `backups_to_clean(names, keep)` then implements `dired-clean-directory`: group by base, sort each group by version descending, keep the `keep` highest, and return the rest. The default `keep` is Emacs's `dired-kept-versions`, 2. The doc comment notes that Emacs does not specify the result order and states what `zmax`'s is — “base-grouped, oldest-first within a base” — which is the kind of thing that makes a function testable instead of merely correct.

`dirs_differ(here, there)` backs `dired-compare-directories`, returning the names in `here` that are missing from `there` or differ in size or file-vs-directory kind. The comment names the divergence from Emacs and justifies it: Emacs's default predicate “also weighs mtime; size+kind+presence is the portable, copy-stable subset.” Anyone who has compared two directories after a sync and watched every file report as different because the timestamps did not survive knows why that is the better default.

Finding a name under the cursor

`filename_at_point(text, pos)` extracts the file-name token around a byte offset, defined as a maximal run of non-whitespace excluding the shell and markup delimiters. This is the same problem `ffap` solves, and the two implementations are deliberately different: `ffap` scans a whole buffer for every plausible path and filters by plausibility, while this one extracts *the* token at a point and lets the caller decide. Two questions, two functions, neither pretending to be the other.

Why a buffer

The argument for Dired over a tree pane is the argument this whole book has been making about `zmax`'s shape. A tree pane needs its own navigation keys, its own search, its own selection model, its own multi-select. A buffer needs none of them: it has yours. Search works on the listing. Match-stepping steps through the results. Visual line selection selects a run of entries. The multiple-selection engine of Chapter 3 — the one the vim keys sit on — applies to a directory listing exactly as it applies to source code, because to the editor there is no difference. That is not a feature of Dired. It is what you get for free by refusing to invent a second interaction model, and it is why the oldest good idea in Emacs is still a good idea.

Chapter 85 — Comint and the Inferior Process

Every long-lived editor grows a way to talk to a subprocess, and most of them grow it three times: once for the shell, once for the REPL, once for the debugger. Emacs grew it once. `comint-mode` is the base mode for any buffer attached to an interactive subprocess, and the shell, inferior-lisp, and `gud` are all specializations of it. `zmax` ports the same shape, and the port's header states the split: `zmax-core/src/comint.rs` holds “only the pure, unit-tested logic,” while “all process I/O, rendering and key handling lives in `zmax-term::ui::comint`.”

What survives that split turns out to be most of what makes `comint` feel good.

The input ring

`InputRing` is the port of `comint-input-ring` and its navigation index. Newest at index 0, bounded by `DEFAULT_RING_SIZE` — 500, which the comment identifies as “Emacs `comint-input-ring-size` default.” The bound is enforced at construction by clamping the maximum to at least one, so a zero-size ring is impossible and no method has to handle an empty-capacity case.

The state that makes navigation work is one field:

```
index: Option<usize>,
```

with the comment: “Current navigation position into `items`, or `None` when not navigating (the caret is on a fresh, unsubmitted input line).” That `Option` is the whole difference between a history that feels right and one that does not. When you press up-arrow you are navigating; when you type a character you are not; and when you press down-arrow past the newest entry, you must land back on what you were typing rather than on the newest history item. `None` is that state, `navigating()` exposes it, and `reset()` returns to it.

`with_ignoredups(bool)` ports `comint-input-ignoredups` — skip adding an input identical to the most recent — as a builder method defaulting to on. `previous_matching(needle)` is `comint-previous-matching-input`: step backward from the current index looking for an entry containing the needle, which is the prefix-search variant of the history key that makes a 500-entry ring usable.

History expansion

`expand_history(input, ring)` is the shell event language, and its documentation enumerates exactly what it covers:

```
!!    the previous command
!$    the last argument of the previous command
!*    all arguments of the previous command
!-N   the N-th previous command
!STR  the most recent command starting with STR
```

The implementation is a single left-to-right pass over the characters, and its design decisions are all about what to do when there is nothing to expand to. Look at the `!!` arm: if the ring has a newest command, push it and mark the input expanded; otherwise push the two literal characters back. Same for the last-argument and all-arguments forms. The function returns `Option<String>` — `Some` when at least one designator was

replaced, `None` when nothing was. The header states the rule: “A lone `!` or an event with no matching history entry is left verbatim.”

That is the correct behaviour and it is not the obvious one. A naive implementation errors on an unmatched event; a shell does too. But this is an *editor’s* input line, and erroring on an exclamation mark inside ordinary text would be a daily annoyance for no benefit. Returning `None` for “nothing happened” also lets the caller distinguish “expand, then show the user what will run” from “submit as typed” without a string comparison.

`last_argument(cmd)` and `all_arguments(cmd)` back the two argument designators and are also directly bound as the insert-last-argument key, which is the other half of why they are public.

Where the input line begins

The one piece of comint state that is genuinely about the process is the *process mark* — the buffer position after which text belongs to the pending input rather than to the scrollback. `bol_or_process_mark_target(caret, process_mark)` implements `comint-bol`: pressing the beginning-of-line key in a comint buffer should go to the start of your *input*, not to the start of the *line*, because the line starts with the prompt. The pure form of that is a two-argument function over positions, and that is what the module exposes.

`last_output_range(is_prompt)` and `last_prompt_line(is_prompt)` take the same approach from the other direction. Both take a per-scrollback-line boolean slice marking which lines are echoed prompt/input lines, and answer “where is the last command’s output” and “where is the last prompt” as index ranges. `comint-delete-output` and `comint-show-output` are then two lines of caller code each. The slice-of-bools argument is the trick that makes it pure: the term layer knows which lines are prompts, and the core does not need to.

Completing at the right place

Two functions decide what tab-completion should offer. `is_command_position(input, caret)` answers whether the caret sits where a command name goes — the first word, or the first word after a `;`, `|` or `&` — so completion can offer executables there and filenames elsewhere. `filename_fragment(input, caret)` extracts the partial path under the caret, and `split_filename_fragment(frag)` splits it into directory and basename so the caller can list one and filter by the other. Three small functions, and between them the entire decision layer of shell completion, with the actual path scan and directory read left outside.

Moving by command

`forward_command` and `backward_command` port `shell-forward-command` and `shell-backward-command` — the word-motion keys rebound for a shell line, where the unit of motion is a command rather than a word. The separator set is `;`, `|`, `&`, and newline.

`backward_command` is the more careful of the two and its three-phase structure is worth reading:

```
// Step back over separators/space immediately before the caret.
while i > 0 && (is_sep(chars[i - 1]) || chars[i - 1].is_whitespace()) {
    i -= 1;
}
// Step back to the start of this command.
while i > 0 && !is_sep(chars[i - 1]) {
    i -= 1;
}
// Skip forward to the command's first non-blank character.
while i < n && (is_sep(chars[i]) || chars[i].is_whitespace()) {
    i += 1;
}
```

The third loop is the one a first draft omits. Without it, moving backward from inside the second command of a pipeline lands on the space after the pipe rather than on the command's first letter. Emacs lands on the letter. So does this.

Stripping carriage returns

One function in the file exists purely because processes are not tidy: `strip_ctrl_m` removes carriage returns from process output, porting `comint-strip-ctrl-m`. Any subprocess that thinks it is talking to a DOS terminal, or any pipeline that ran a Windows tool, will produce CRLF line endings that render as stray glyphs. It is two lines of code and it is in the pure module with a test, because a two-line function that runs on every line of every process's output is exactly the kind of thing that should have a test.

What this buys

The reason to port `comint` rather than write a terminal-emulator panel is composability. A `comint` buffer is a buffer: the vim keys work in it, search works in it, the multiple selections work in it, `occur` works over its scrollback, and yanking a line of output into a source file is the same yank you use everywhere. A terminal panel is a black rectangle with its own key handling, its own scrollback and its own copy mode. Both have a place — `zmax` has a real terminal too — but the one the editor can *edit* is the one that composes with the rest of the editor, and `comint` is that one.

Chapter 86 — Calc

Emacs Calc is the feature people cite when they want to argue that Emacs is not an editor. It is a full RPN calculator with a stack, a trail of past results, angular modes, an algebraic entry mode, and undo. zmax ports the core of it in `zmax-core/src/calc.rs`, in two layers that the module header names: an infix evaluator that also backs the `:calc <expr>` typed command, and the RPN stack machine that the full-screen Calc component drives.

The infix layer

The grammar is documented at the top of the file, lowest precedence first:

```
expr  = term (('+'|'-') term)*
term  = power (('*'| '/'| '%') power)*
power = factor ('^' power)?
factor = number | '(' expr ')' | fn '(' expr ')' | ('+'|'-') factor
```

It is a textbook recursive-descent evaluator over `f64`, with `power` recursing on its right operand so exponentiation is right-associative.

One line of the header deserves attention: “Unary minus binds tighter than `^` (so `-2^2 == 4`).” That is visible in the grammar — unary sign is part of `factor`, and `power` is built from `factor` — and it is a genuine choice, not an accident. Most programming languages go the other way. Calc’s convention is the one a calculator user expects when they punch in a negative number and square it, and the port matches Calc, states that it does, and gives the disambiguating example. That is the correct handling of a divergence: name it, show it, cite the authority.

`eval_infix` returns `Result<f64, String>` — errors are strings because they go to the status line and nowhere else. `format_value` is the display side, and having it as a public function next to the evaluator means the typed command and the full-screen component render numbers identically.

The stack machine

```
pub struct Calc {
    stack: Vec<f64>,
    trail: Vec<String>,
    undo: Vec<Vec<f64>>,
    redo: Vec<Vec<f64>>,
    pub angle: Angle,
}
```

Level 1 is the *last* element of `stack`, matching Calc’s numbering where the newest value is level one. The undo and redo stacks hold whole snapshots of the value stack. That is a deliberately unclever choice and the right one here: a vector of floats at realistic depth is a handful of words, the operation count is bounded by human typing speed, and the alternative — inverse operations — is wrong for anything lossy. Undoing a square root by squaring does not restore the original bits. Snapshots do.

Compare this with the buffer history of Chapter 23, which does *not* snapshot, because a rope of a large file is a different size class entirely and the transactions are exactly invertible. Two undo systems in one editor with

opposite implementations, each matching its own cost structure. Uniformity for its own sake would have made one of them wrong.

The stack manipulation ports Calc's key bindings directly: `enter` duplicates the top, `pop` drops it, and `roll_down` / `roll_up` rotate the stack.

Errors before mutation

`binop` and `unop` share a structure that is the most instructive thing in the file:

```
pub fn binop(&mut self, op: BinOp) -> Result<(), String> {
    if self.stack.len() < 2 {
        return Err("stack underflow".into());
    }
    let b = self.stack[self.stack.len() - 1];
    let a = self.stack[self.stack.len() - 2];
    let r = match op { /* ... may return Err ... */ };
    if !r.is_finite() {
        return Err("result is not finite".into());
    }
    self.checkpoint();
    self.stack.truncate(self.stack.len() - 2);
    self.stack.push(r);
    self.trail.push(format_value(r));
    Ok(())
}
```

Every early return happens *before* `checkpoint()`. Arity is checked first, operands are read without being removed, the operation is computed into a local, the result is checked for finiteness — and only then is the checkpoint taken and the stack modified. A failed operation leaves the stack byte-for-byte unchanged and adds nothing to the undo history.

The consequence is that error recovery costs the user nothing. Divide by zero, get “division by zero” in the status line, and your operands are still on the stack in the same order; pressing undo does not first have to undo the failure. Anyone who has used a calculator that clears on error, or a REPL that leaves a half-applied operation behind, knows what this is worth. `unop` has the identical shape with a peek at the last element and an in-place write.

The error set is specific rather than generic: “division by zero”, “modulo by zero”, “sqrt of a negative”, “ln of a non-positive”, “log of a non-positive”, “stack underflow”, “result is not finite”. Each names the operation that failed and why. The finiteness check is the catch-all that keeps infinities and NaNs off the stack entirely, which means no later operation has to defend against them — a single guard at the one place values are produced, rather than a guard at every place they are consumed.

Angular mode

`Angle` is `Radians` or `Degrees`, ported from Calc's angular-mode keys, and `unop` handles it with two closures:

```
let to_rad = |v: f64| match self.angle {
    Angle::Radians => v,
    Angle::Degrees => v.to_radians(),
};
let from_rad = |v: f64| match self.angle {
    Angle::Radians => v,
    Angle::Degrees => v.to_degrees(),
};
```

Sine, cosine and tangent take their *argument* through `to_rad`. The inverse functions take their *result* through `from_rad`. Getting the direction right on the inverses is the entire content of angular-mode support, and writing it as two named closures applied at the right ends makes it checkable by reading rather than by testing every combination.

The trail

`trail` is a vector of formatted result strings, appended by every successful operation. It is Calc's scrolling record of what you did, and keeping it as formatted strings rather than values is correct: it is display history, not computational state, and undo does not roll it back — snapshots restore `stack`, not `trail`. That mirrors Calc, where the trail is a log you can scroll and copy from, not a second stack.

Algebraic entry

`algebraic(expr)` evaluates an infix expression and pushes the result — Calc's escape hatch from RPN into ordinary notation. It is three lines, because both halves already exist. This is the payoff of putting the infix evaluator and the stack machine in the same module: the typed command and the algebraic-entry key inside the full-screen calculator run the same parser, so they cannot disagree about precedence, about unary minus, or about what `%` means.

Scope, stated

Calc in Emacs also does exact rationals, bignums, complex numbers, units, matrices, symbolic algebra, and embedded mode. This port is `f64` and a stack. The module header does not claim otherwise — it describes exactly the two layers it contains — and that is the pattern worth extracting from this chapter. A partial port that names its boundary is a usable component. A partial port that implies completeness is a bug report waiting to be filed. Every module this book has walked through takes the first option, and the reason the port report of Chapter 29 can be trusted at all is that the modules underneath it are honest about their own edges.

Chapter 87 – The Error List: Compilation, Tags, and Occur

Three of the oldest Emacs features answer the same question in three different ways: *here is a list of places in the code; take me to each one in turn*. Compilation mode builds the list from a tool's output. Tags builds it from an index. Occur builds it from a search. zmax ports all three, and they are worth reading together, because the differences between them are entirely about where the list comes from and not at all about what is done with it.

Compilation

`zmax-core/src/compilation.rs` is “the pure, dependency-free state machine” behind `next-error` and its relatives. The header explains what Emacs does — run a command, collect output into a compilation buffer, scan it with `compilation-error-regexp-alist`, and walk the hits — and then enumerates precisely which formats the parser recognizes:

```
file:line:col: error: message      GCC/Clang/rustc with a column
file:line:col: warning: message
file:line: message                 GNU-style without a column
file:line:col:message              colon-joined (some linters)
file:line:message                  grep -n matches (classified as info)
```

Five formats, listed, with the tools that emit each. That is a scope statement a reader can act on: if your linter's output is not in that list, you now know why `next-error` does not stop on it, and you know what shape it would have to have.

`ErrorKind` is `Error`, `Warning`, `Info`. The comment is careful about what the three-way split is for: Emacs “only stops on the first two for the plain `next-error`, but we keep all three so the command layer / a compilation buffer can colour them.” Grep matches and note lines are classified as `Info`. The severity is inferred from the message text, not from the location format, which is why the warning and error forms share a parse.

`CompileEntry` records the file “verbatim, as the tool printed it — the command layer expands `~` and resolves it against the working directory.” The pure module does not resolve paths, because resolving requires knowing the compile directory and the user's home, and neither belongs in a parser. It records the line and an `Option<usize>` column, which is `None` for the GNU-style two-field format.

`CompilationList` is entries plus `current: Option<usize>`, and the `Option` is again the state that makes navigation right: `None` before the first move, so that the first `next-error` visits the first entry rather than the second. `set_output` replaces the whole list from a fresh run, which is the right granularity — a rebuild invalidates everything.

The parser's most important property is what it *ignores*. The header lists it: “banners, progress spinners, blank lines, a bare summary such as `error: aborting due to 2 errors`.” That last one is the subtle case. It begins with an error keyword, so a severity-first parser will happily classify it — but it names no location, so stopping on it would mean `next-error` visiting a place that does not exist. The rule is location-first: no file-and-line, no entry, regardless of what the line says about itself.

The header also notes that this list is “kept separate from the Vim quickfix list which lives in `quickfix.rs`.” Two traditions, two lists, deliberately not merged — because the vim and emacs navigation commands have

different populating commands and different lifetimes, and a user driving zmax with vim keys should not find their quickfix list clobbered by a compile started from the Spacemacs menu.

Tags

`zmax-core/src/etags.rs` reads the format `etags` writes, and the header shows a sample rather than describing it: sections separated by a form feed, a `FILE,SIZE` header line, then one line per tag with the shape `PATTERN <DEL> [NAME <SOH>] LINE,BYTE`. The name part is optional, “and when it is absent the tag name is the last identifier in the pattern (which is how `etags` writes C).” Two control characters doing structural work, one of them optional, with a fallback rule that requires tokenizing the pattern. This is a format from the 1980s and it shows.

`parse` splits on the form feed, skips leading empties, takes the header, and splits it from the *right* on a comma — right, not left, because a path may contain commas but the size field is always last. The size is read and discarded; the comment says “advisory and ignored,” which is a claim about the format worth making explicitly, since a reader might otherwise assume it is used for staleness detection.

`Tag` carries the pattern alongside the line number, and the field comment says what for: “the source text `etags` recorded, used to confirm the line has not moved.” That is the mechanism that makes a stale tags file degrade rather than break. Jump to the recorded line, check whether it still contains the pattern, and if not, search nearby. The pure module stores the pattern; the command layer does the search.

Two queries sit on top. `find(table, name)` returns every file-and-tag pair matching a name — a vector, because C headers and implementations define the same symbol and the jump command must be able to offer both. `complete(table, prefix)` returns the names sharing a prefix, backing tag completion at the prompt.

The honest thing to say about etags in current tooling is that the language server of Chapter 14 supersedes it for every language that has one. What it does not supersede is the case where there is no server: a forty-year-old C codebase, a language with no LSP implementation, a machine where you will not be starting a daemon. `etags` runs on all of them, produces one file, and needs no process. The port is a couple of hundred lines. That ratio is why it is there.

Occur

`zmax-core/src/occur.rs` is the smallest of the three and has the cleanest design decision in it. `occur` collects every line containing a match into a list — and it takes the matcher as a closure:

```
pub fn occur(text: &str,
             first_match: impl Fn(&str) -> Option<usize>) -> Vec<Match>
```

The comment states the motivation: “The matcher is a closure so the engine stays free of any regex dependency; the command layer supplies one backed by the `regex` crate, and the tests here supply plain substring/anchored matchers.” The tests do not need a regex engine, and the module does not link one.

But the closure does more than decouple. Because it is applied to each line in isolation, “`^/$` anchors bind to the line’s own start/end,” which is what makes this a faithful port of Emacs’s line-oriented `occur` rather than a multiline regex search that happens to report lines. The semantics fall out of the shape.

`Match` carries the line number, the full line text, and `match_col` — “0-based character column of the first match on the line — where `occur-mode-goto-occurrence` places point.” A line matching five times still yields one `Match`, pointing at the first hit, which is Emacs’s behaviour.

The header distinguishes this from a neighbour that could be confused with it: `region_ops::occur` is “a boolean line filter that returns just `(line_number, text)`” for global-command-style transforms, while this engine also records the column, because the interactive mode needs to place a cursor and the text transform

does not. Two functions with the same name in different modules, each documented as to why the other exists. That is the alternative to one over-general function with a flag argument, and it is the better one.

`multi-occur` extends the same matcher across several named buffers, yielding label-and-match pairs in buffer-then-line order — Emacs's `multi-occur` and `multi-occur-in-matching-buffers`, implemented as a fold over `occur`.

The shared shape

Set the three side by side and the common structure is exact. Each produces an ordered list of locations. Each keeps the *source* of the list — a tool's output, an index file, a search — entirely outside the navigation logic. Each leaves path resolution and file opening to the command layer. And each has a pure core small enough to test exhaustively against the format or the semantics it claims to implement.

That is why the editor can have three of them without three navigation implementations, and why adding a fourth source of locations — a test runner, a grep tool, a code-search backend — is a parser and nothing else.

Chapter 88 – Spelling

zmax spell-checks two different ways, and the split is not an accident of history — it is the same four-traditions problem the rest of the editor has, appearing in a place where the traditions genuinely disagree about how the feature should work.

Emacs does not spell-check. It drives an external program that does — `ispell`, `aspell`, or `hunspell`, named by `ispell-program-name` — over a terse pipe protocol, and parses what comes back. Vim spell-checks itself, from a word list, with a suggestion key and add-to-dictionary keys. zmax ports both. `zmax-core/src/ispell.rs` is the protocol parser; `zmax-term/src/spell.rs` is the self-contained checker.

The pipe protocol

The ispell module's header contains the entire protocol, which is short enough to quote in full:

```
*           the word was found (correct)
-           the word is a run-together / compound, accepted
+ ROOT     found as a derivative of ROOT (correct)
& ORIG N OFFSET: a, b  ORIG is wrong; N near-misses follow after the colon
? ORIG N OFFSET: a, b  ORIG is wrong; N guesses (weaker) follow
# ORIG OFFSET          ORIG is wrong; no suggestions
```

Six line types, three meaning “correct” and three meaning “wrong.” `WordCheck` collapses the first three into a single `Correct` variant and keeps the distinctions that matter from the last three: word, offset, suggestions. The near-miss and guess forms differ in confidence, and the port merges them, because both produce a suggestion list and the editor presents them identically.

`parse_line` dispatches on the first character and returns `Option<WordCheck>`, with `None` for “the version banner, a blank end-of-line marker, or any unrecognised line.” That last clause is the important one. A spell checker's output is not a protocol anyone versions carefully, and different implementations of pipe mode emit slightly different chatter. Returning `None` for anything unrecognized means an unexpected line is skipped rather than crashing the check.

`escape_line` is one line of code and prevents an entire class of bug:

```
format!("{}", line.replace(['\r', '\n'], " "))
```

The leading caret is the protocol's own escape — it forces the whole line to be treated as words to check, bypassing the command characters that would otherwise be interpreted. Without it, a line of source code beginning with a hash becomes a command to the checker instead of text. The newline replacement is because each line is fed separately and an embedded newline would desynchronize the request/response pairing.

`pipe_args(dictionary)` builds the argument vector: pipe mode, plus a dictionary flag when one is named. The comment records the resolution order the command layer follows — “aspell, then hunspell, then ispell,” mirroring Emacs's `ispell-program-name` search.

Knowing what not to check

`message_checkable_lines(lines) -> Vec<usize>` is the most opinionated function in the module, porting `ispell-message`. Given the lines of an e-mail message, it returns the indices worth checking, and the

rules are:

- The header block — everything up to and including the first blank line — is skipped. If there is no blank line, “the whole message is treated as body (no recognizable headers).”
- Cited lines, matched as a leading `>` after optional whitespace, are skipped. You did not write them.
- Everything from a lone signature separator onward is skipped — and the trailing space in that separator is part of it, per the convention.
- Blank lines are skipped as having nothing to check.

Each of those exists because spell-checking without them is useless: the header block is full of addresses and message identifiers, the quoted text is someone else’s, and the signature is a name and a URL. This is one small function that turns a feature from technically-working into actually-usable, and the fact that it is here means the port carried the judgment and not just the mechanism.

The vim side

`zmax-term/src/spell.rs` is a different animal: it *is* the checker. The header states the sourcing chain — vim’s `spelllang` chooses the base list, the default resolving to the system dictionary at `/usr/share/dict/words`, “present on macOS and most Linux installs,” with other languages resolving to a hunspell dictionary. User additions persist to `spell-good` and `spell-bad` under the config directory, with vim’s `spellfile` able to relocate the good list.

The last sentence of that header is the design decision:

If no dictionary is found, nothing is ever flagged misspelled (so the feature degrades to a no-op rather than firing on every word).

Consider the alternative. An empty word list plus a naive membership test flags every word in the buffer. The user opens a file on a machine without a dictionary and every word is underlined. Failing closed here — no dictionary, no misspellings — is the only behaviour that does not make a missing optional dependency look like a broken editor.

Flyspell, as a mode

The term module also carries Emacs’s `flyspell-mode`, and the type shows why it had to be a three-state enum rather than a boolean:

```
pub enum Flyspell {
    #[default]
    Off,
    All,    // flyspell-mode: check every word
    Prog,   // flyspell-prog-mode: check comments and strings only
}
```

`flyspell-prog-mode` is the one that makes spell-checking tolerable in source code, because it uses the syntax layer of Chapter 25 to check only prose — comments and string literals — and leave identifiers alone. Without it, every abbreviation and every camel-cased name is a misspelling.

The state is keyed by `DocumentId`, because both are buffer-local minor modes in Emacs, and `toggle_flyspell` implements the mutual exclusion explicitly: enabling a mode that is already on turns it off, and switching between the two replaces rather than stacks. The comment states the rule — “the last one enabled owns the buffer” — which is what Emacs does and what a user pressing the two toggles in sequence expects.

The renderer reads `flyspell(doc)` to decide “whether (and where) to underline misspellings,” which is the whole integration surface: one function, called from the render path, returning an enum.

Two checkers, on purpose

It would be possible to unify these — drive an external checker always, or check internally always. Neither is right. The internal checker needs no external program, works from a word list, and matches what a vim user's add-word and suggest keys mean. The external checker has real morphology, real dictionaries for languages the system word list does not cover, and matches what an Emacs user's check-word key means.

A user pressing the vim suggest key and a user pressing the Emacs check key are asking different questions with different expectations about what will happen, and answering both correctly requires both implementations. That is the four-traditions problem in its purest form: not “which one is better,” but “these are two different features that share a noun.”

Chapter 89 – The File Configures Itself

There are three conventions by which a file, or the directory containing it, tells the editor how it wants to be edited. Two of them are Emacs's and one is everyone's. `zmax` implements all three, in three modules worth reading together because they solve the same problem at three different scopes.

Scope one: the file's own first line

`zmax-core/src/file_locals.rs` ports the Emacs commands that manage file-local variables. Emacs puts them in two places, and the header names both:

```
 -*- key: val; key2: val2 -*-      the prop line (first line, or second
                                   when the first is a shebang)

Local Variables:                  the block near end of file, each line
key: val                         wrapped in the buffer's comment syntax
End:
```

The shebang rule is implemented as its own function, `prop_line_index`, returning 1 when the first line starts with a shebang and 0 otherwise. That is three lines of code that make every executable script in a `bin/` directory work, and it is exactly the sort of thing that gets forgotten in a reimplementation because it never comes up until it does.

`parse_prop_line` returns a triple — prefix, pairs, suffix — rather than just the pairs, and that shape is what makes the *writing* commands work. The prop line lives inside a comment, so there is text before the opening marker and text after the closing one, and `add-file-local-variable-prop-line` must insert a key into the middle without disturbing either. Keeping the outside text in the parse result means the writer reassembles rather than reconstructs.

There is one special case handled and worth noting: a bare mode declaration, with no colon, is not a malformed pair list. The parser turns it into a single `mode` pair. A parser that required colons would either error or silently drop it, and either way a very common form of prop line would stop working.

The Local Variables block is handled with an explicit comment prefix and suffix supplied by the caller, because the block is wrapped in whatever the buffer's language uses for comments — semicolons in Lisp, slashes in C, an open/close pair in HTML. The pure module takes them as parameters; the command layer gets them from the buffer's language configuration. No language table in the parser.

Scope two: the directory tree

`zmax-core/src/dir_locals.rs` ports `.dir-locals.el`, which is one elisp alist keyed by mode:

```
((nil . ((fill-column . 80)))
 (c-mode . ((indent-tabs-mode . t)
            (tab-width . 8))))
```

`nil` applies to every file in the tree; a mode symbol applies only to files in that mode. Emacs merges the `.dir-locals.el` of every directory from the root down to the file's own, “the deepest winning,” and also merges *directory classes* — named variable sets bound to a directory at runtime via `dir-locals-set-class-variables` and `dir-locals-set-directory-class`.

The port's most consequential decision is in the `parse` doc comment:

Tolerant by design — an unreadable or malformed file yields the entries it could make sense of, never an error, because Emacs's own reader ignores what it cannot use and a half-written file must not wedge the editor.

That second clause is the operational argument, and it is about a real situation: you are editing `.dir-locals.el`, you save mid-edit or your editor autosaves, and the file is momentarily unbalanced. A strict parser makes the editor refuse to open files in that tree until you fix it. A tolerant one degrades to whatever prefix parsed. Given that the file's purpose is to configure the editor, the tolerant reading is the only one that cannot lock you out.

The parser is a small Lisp reader — a tokenizer, a form reader, and a `Form` enum with atom, list, and dot variants — and it strips comments. It accepts both the dotted pair and the bare two-element list, filtering out the dot forms rather than requiring them, which is why the two shapes read identically.

The module's API is the full command set — parse, render, set, delete, merge — because the Emacs commands go both directions: `add-dir-local-variable` and `delete-dir-local-variable` edit the file, and `copy-file-locals-to-dir-locals` and its inverse move a setting between the two scopes this chapter is describing. `render` being the exact inverse of `parse` is what makes those safe: a round trip through both must not reorder or reformat the entries the user did not touch.

Scope three: everyone's convention

`zmax-core/src/editor_config.rs` implements `EditorConfig`, and its header pins the version: “follows the spec at version 0.17.2.” That is the right way to cite a moving external standard — not “implements `EditorConfig`,” but “implements this revision of it.”

The type is deliberately all-`Option`:

```
pub struct EditorConfig {
    pub indent_style: Option<IndentStyle>,
    pub tab_width: Option<NonZeroU8>,
    pub line_ending: Option<LineEnding>,
    pub encoding: Option<static Encoding>,
    pub trim_trailing_whitespace: Option<bool>,
    pub insert_final_newline: Option<bool>,
    pub max_line_length: Option<NonZeroU16>,
}
```

Every field distinguishes “the file said nothing” from “the file said a value,” because `EditorConfig` layers over the user's own configuration and a `None` must fall through to it rather than overwrite it with a default. The non-zero integer types on the width and length fields push the “zero is meaningless here” check into the type, so no consumer has to guard against a division by a zero tab width.

`find(path)` walks the path's ancestors, reads each `.editorconfig`, and — per the spec's file-processing rules — accumulates and applies them root-ward-first. The error handling matches the `dir-locals` decision: an unreadable file is skipped, and a file with a glob that fails to compile is logged at warning level and ignored. The message says the file is being ignored “because a glob failed to compile,” which tells you both what was ignored and why. A malformed `.editorconfig` in a vendored dependency does not stop you from editing.

Globs come from `globset`, and the indent-style and line-ending types are the editor's own — so an `EditorConfig` tab-indent setting lands in exactly the same field an explicit option-setting command would set. There is no parallel `EditorConfig`-only configuration path.

One field is commented out in the struct — the spelling language — and left in place rather than deleted. That is a small honesty: the spec has the key, this implementation does not read it, and a reader diffing against the spec can see which one is missing without checking every field.

Precedence, and why three

Three mechanisms sound like two too many until you notice they have different authors. EditorConfig is written by whoever set up the repository and is meant to be portable across editors — indentation, encoding, final newlines, the things every editor has. `.dir-locals.el` is written by an Emacs user for an Emacs project and can set anything Emacs has a variable for, per mode, per subtree. The prop line is written by whoever wrote *this file*, and travels with it — which is why generated files and one-off scripts use it, and why it wins.

The precedence — file over directory, deeper directory over shallower, and the editor's own configuration underneath all of it — is the ordering that respects authorship. The person closest to the file had the most specific intent. That is the rule, and all three modules exist to implement it at their own scope without knowing about each other.

Chapter 90 – Bytes: Coding Systems and Line Endings

A rope holds text. A file holds bytes. Everything in this chapter is about the two places where that gap has to be crossed, and about what happens when it is crossed wrong.

Mojibake, and how to undo it

`zmax-core/src/coding.rs` ports `recode-region` and `recode-file-name`, and the header explains the problem better than a description could:

When a file is read with the wrong coding system its bytes are decoded into the wrong characters (the classic mojibake). Emacs fixes that without re-reading the file: it encodes the text back with the coding system it was *mistakenly* decoded as — reconstructing the original bytes — and decodes those bytes again with the right one.

The whole implementation is that sentence:

```
pub fn recode(text: &str, interpreted_as: &str,
              really_in: &str) -> Result<String, String> {
    let wrong = lookup(interpreted_as)?;
    let right = lookup(really_in)?;
    let (bytes, _, had_errors) = wrong.encode(text);
    if had_errors {
        return Err(/* cannot be represented */);
    }
    let (decoded, _, malformed) = right.decode(&bytes);
    if malformed {
        return Err(/* not valid in the target */);
    }
    Ok(decoded.into_owned())
}
```

Two error checks, and they are the reason this is a real port rather than a plausible one. An encode error means the text cannot be represented in the coding system it was supposedly decoded as — so the original bytes cannot be reconstructed, and there is nothing to re-decode. A malformed decode means the reconstructed bytes are not valid in the target. Either way the function returns an error and the buffer is untouched. The header states the standard being met: “Emacs signals in those cases rather than silently mangling the buffer.”

Silently mangling is the failure mode that matters here, because the input is *already* mangled text and the user cannot tell good output from bad by looking. An operation that sometimes produces a different flavour of garbage, without saying so, is worse than one that refuses.

Name resolution

`lookup(name)` is where the port meets a naming problem: Emacs’s coding-system names are not IANA labels. The function handles it in two stages.

First, the end-of-line suffix. Emacs writes the line-ending convention as part of the coding-system name — a `-unix`, `-dos` or `-mac` suffix on the encoding. The function strips those before looking anything up, with the comment explaining why: “The EOL half is the document’s line ending, not its encoding.” Two orthogonal properties in one string, separated before either is resolved.

Second, the Emacs spellings that have no IANA equivalent, mapped explicitly:

```
"latin-1" | "latin1" => "iso-8859-1",
"mule-utf-8" | "prefer-utf-8" | "undecided" => "utf-8",
"chinese-gbk" => "gbk",
"japanese-shift-jis" | "sjis" => "shift_jis",
```

The `undecided` mapping is the interesting one. In Emacs, `undecided` means “detect it,” which a pure function cannot do; mapping it to UTF-8 makes the common case work and is a defensible reading of the modern default. Everything else goes to `encoding_rs`’s label lookup, so every label that crate knows is accepted for free, and an unknown name produces an explicit “unknown coding system” error rather than a silent fallback.

The dependency note in the header — “`encoding_rs` (already a core dependency)” — is the sort of parenthetical that matters for a project with a thirty-year horizon. The feature added no new dependency; it used one the rope layer already required.

Line endings

`zmax-core/src/line_ending.rs` is the other half of the byte gap, and its `LineEnding` enum has an unusual shape:

```
pub enum LineEnding {
    Crlf,
    LF,
    #[cfg(feature = "unicode-lines")]
    VT, // U+000B
    #[cfg(feature = "unicode-lines")]
    FF, // U+000C
    #[cfg(feature = "unicode-lines")]
    CR, // U+000D
    #[cfg(feature = "unicode-lines")]
    Nel, // U+0085
    #[cfg(feature = "unicode-lines")]
    LS, // U+2028
    #[cfg(feature = "unicode-lines")]
    PS, // U+2029
}
```

Two variants unconditionally; six more behind a feature flag. Unicode defines eight characters as line terminators and a strictly correct implementation honours all of them — but honouring all of them means a file containing a form feed becomes a file with extra lines, and vertical tabs and form feeds appear in real source files as ordinary characters. The feature flag is the port declining to pick one answer: strict Unicode line-breaking is available, LF and CRLF is the default, and the choice is made at build time where it costs nothing at runtime.

`NATIVE_LINE_ENDING` is CRLF on Windows and LF everywhere else, selected by target OS. It is what a *new* buffer gets. An existing file keeps whatever it had — which is the behaviour that stops an editor from producing a diff where every line changed because someone opened a CRLF file on Linux and saved it.

`len_chars()` returns 2 for CRLF and 1 for everything else, and it is `const`. That function is the single point where the two-character line ending is accounted for, and every position calculation that has to skip a terminator goes through it rather than assuming one. `as_str()` returns the actual characters as a static string, written as explicit unicode escapes rather than literal characters — because a literal carriage return in source is invisible and a literal line separator is worse.

What the user sees

None of this is a feature anyone asks for. It is the layer that makes a file open as itself and save as itself, and the only time it becomes visible is when it fails: a file full of question marks, a diff with every line changed, a stray control character at the end of each line. The status-line elements of Appendix A include file encoding and line ending precisely so that when something looks wrong, the two values that explain it are already on screen.

The design principle running through both modules is the same: detect nothing that can be recorded, record everything that was detected, change nothing that was not asked for, and refuse rather than guess when a conversion cannot round-trip.

Chapter 91 – Sessions, Snapshots, and Swap Files

Three separate mechanisms in `zmax` exist to answer three separate versions of the question “what if I lose this?” They come from three traditions, they store different things in different places, and they are deliberately not unified — because a session, a version, and a crash-recovery buffer are different objects with different lifetimes.

The session

`zmax-core/src/desktop.rs` backs the Emacs `desktop` family — `desktop-save`, `desktop-read`, `desktop-clear`, `desktop-change-dir`, `desktop-revert`. The module's scope statement is precise about what it owns: “the **on-disk desktop-file format** and nothing else,” with the commands enumerating buffers and opening files, and the `serialize`/`parse` round trip here as pure functions.

The format is four tab-separated fields per line:

```
flag<TAB>line<TAB>column<TAB>path
```

`flag` is an asterisk for the buffer that was current when the desktop was saved and a dash otherwise; `line` and `column` are the zero-based point; and the `path` comes last. That ordering is not aesthetic. The header says why: “`path` is the file path and comes last so it may itself contain TABs.” Every other field has a bounded, known grammar, so parsing is three splits and the remainder is the path, whatever it contains. A path-first format would need escaping. This one needs none.

The `parse` is tolerant in a specific, useful way: “a data line with no TABs is treated as a bare path (flag `-`, point `0,0`), so hand-written desktop files listing one path per line work.” That turns the format into something a shell script can generate — a list of paths, one per line, is a valid desktop file.

Lines beginning with a double semicolon are comments, matching the Lisp convention of the file this is replacing, and one more decision is recorded in a comment on the default basename: “Emacs uses `.emacs.desktop`; `zmax` keeps its own name so the two never collide.” A user running both editors in the same directory gets two sessions, not one corrupted one.

The version

`zmax-term/src/local_history.rs` is short and ports a JetBrains feature rather than an Emacs one: on every save, snapshot the file's contents, independent of git.

```
-/.zmax/projects/<proj>/local-history/<relpath>/<unix-ts>.snap
```

The path encoding is worth noting. `dir_for` strips the workspace root from the path and then replaces every path separator with a percent sign, so a nested source file becomes a single flat directory name. That avoids creating a mirror of the project tree inside the state directory, and it means listing a file's snapshots is one directory read rather than a walk.

`record` has two guards, both of which are the difference between a useful feature and a disk-filling one:

```

if let Some((_, latest)) = existing.first() {
    if std::fs::read_to_string(latest).is_ok_and(|s| s == content) {
        return; // unchanged since the last snapshot
    }
}

```

Saving a file you did not change writes nothing. This matters more than it sounds: save-on-focus-loss, a format-on-save that is a no-op, and habitual manual saves all produce writes with identical content, and without this check a day of editing produces hundreds of identical snapshots.

The second guard is the prune, capped at `MAX_SNAPSHOTS`, executed after every write by re-listing and deleting everything past the cap. A bounded number of versions per file, unbounded in file count but bounded per file.

`snapshots(path)` returns timestamp-and-path pairs newest-first, parsing the timestamp out of the file stem and sorting in reverse. Every error path in the module is swallowed — a failed directory creation returns early, and the write and delete results are discarded. That is correct for this feature and would be wrong for most: a snapshot is a convenience, and a filesystem problem while taking one must never interrupt the save that triggered it.

The crash

`zmax-term/src/vim_swap.rs` ports vim’s swapfile: a hidden `.swp` file written periodically while a buffer has unsaved changes, removed on a clean save, and detected on open — vim’s `E325` — to warn that a previous session may have died with unsaved work.

The write policy is the part that has to be gotten right, and it is one pure function:

```

fn swap_write_due(count: usize, updatecount: usize) -> bool {
    updatecount != 0 && count.is_multiple_of(updatecount)
}

```

`updatecount` is vim’s option, documented in the code with vim’s own words: “After typing this many characters the swap file will be written to disk. When zero, no swap file will be produced at all.” The default is 200. The zero case short-circuits before the modulo, which is both a division-by-zero guard and the option’s documented meaning — zero disables the feature entirely, not “write every time.” The function is marked pure and unit tested, which is possible precisely because the policy was separated from the writing.

Two pieces of state sit in thread-locals rather than in the editor. A per-document change counter means the buffer is not rewritten on every keystroke — only every `updatecount` changes. A no-swap set records the documents opened under vim’s `:noswapfile` modifier, and the comment explains the subtlety: “the modifier says ‘this command doesn’t touch the swap file’, and a buffer that was opened that way must not grow one behind the user’s back either.” A literal reading of the modifier would only suppress the swap check at open time. Vim’s actual behaviour, and this port’s, is that the exemption persists for the life of the buffer.

The swapfile-enabled flag and the swap directory are process-level statics, cached from the config, “so the change hook (which only gets the document) can act without the editor config.” That is a real constraint of the hook system of Chapter 26 — hooks receive immutable references and cannot reach arbitrary editor state — and the cache is the standard answer to it.

Why three

Each mechanism answers a question the others cannot.

The desktop answers “what was I working on?” It survives a clean quit, stores no content, and restores a working set. Losing it costs you the arrangement, not the work.

Local history answers “what did this file look like an hour ago?” It survives everything, stores content, and is independent of version control — which is the entire point, because the changes you most want back are the ones you made between commits, and git has nothing to say about those.

The swap file answers “did I lose unsaved changes when that crashed?” It is short-lived by design, deleted on a clean save, and its presence at open time is itself the signal.

A single unified “recovery system” would have to be all three at once: durable like local history, cheap like the desktop, and self-deleting like the swap file. Those requirements contradict. Three mechanisms, each matched to its own lifetime, is not duplication — it is the correct number.

Chapter 92 – Watchers: the Tree, the Theme, and the Build

An editor that only reacts to keystrokes is out of date the moment you switch to another terminal. `zmax` runs several watchers that react to the world changing underneath it, and they share a structure worth extracting, because the same threading problem shows up in each one.

The shape

`zmax-term/src/file_watcher.rs` states it:

A dedicated OS thread owns the `notify` watcher and a receive loop. On a relevant event it coalesces a short burst, then hops onto the main thread via `job::dispatch_blocking` to rebuild the tree; the event loop renders right after each dispatched callback, so the change shows up immediately.

Four steps: own the watcher on its own thread, coalesce, hop to the main thread, and get a render for free because the loop renders after every dispatched callback. That last point is what makes these features feel instant — a change on disk repaints the screen without the user touching the keyboard, because the render is not waiting on an input event.

`job::dispatch_blocking` is the seam, and it is worth looking at what it actually is:

```
pub fn dispatch_blocking(
    job: impl FnOnce(&mut Editor, &mut Compositor) + Send + 'static,
) {
    let jobs = JOB_QUEUE.wait();
    send_blocking(jobs, Callback::EditorCompositor(Box::new(job)))
}
```

A boxed `FnOnce` taking mutable references to the editor and the compositor, sent down a channel. Any thread can produce one; only the main loop runs one; and inside it the closure has exclusive mutable access to the entire editor. That is the whole concurrency model for background work — not locks around editor state, but a single-consumer queue of closures that run where the state lives. The `wait()` on the queue blocks until it exists, so a watcher that starts before the loop is up does not need a readiness protocol.

Two disjoint event classes

The file watcher's most interesting content is not the threading; it is an observation about git:

Two disjoint classes of event are handled, because a commit made in another terminal writes *only* inside the git directory — the working tree is left byte-for-byte identical, so no ordinary file event ever fires for it.

This is the bug that a naive watcher has and never notices. You commit in another pane. The file contents did not change, so no watch event fires for any tracked file, so the editor's diff gutters still show every line of that commit as modified against a HEAD that has moved. The gutters are wrong and nothing tells you.

The fix is a second event class. Worktree paths rebuild the tree and reload changed buffers; git ref paths — matched by `is_head_move` — trigger a refresh of every open buffer's diff base, because "HEAD moved, so every open buffer's diff base (HEAD's blob) is stale." Two categories, two responses, derived from one fact about how git writes.

`is_ignored` filters the noise, and the list is concrete: the git directory, the build output directory, the node modules directory, a cache directory, `dist`, `build`, and two environment directories. A build generates thousands of events under its output directory and none of them mean anything to the file tree. Note that the git directory appears in the ignore list *and* is handled by the ref watcher — the general recursive watch ignores it, and the ref-move detection is a separate, targeted concern.

`spawn(root)` is idempotent, guarded by a static atomic boolean with a swap. Reopening the IDE does not accumulate watchers. And the registration itself is deliberately off the startup path — the comment notes that "startup must not block on registering the watches (a recursive add over a large tree is not free)," which is the correct instinct for a project whose stated bar is that no startup trick should be necessary.

The theme watcher

`zmax-term/src/zwire.rs` uses the same pattern for something entirely different: following the terminal host's colour scheme. `zwire` records its active theme in a TOML file under its own config directory, and `zmax` ships a `zgui-<scheme>` theme for every `zwire` scheme, plus a light variant.

The mapping rule is stated as a design constraint rather than an implementation detail: "The scheme name is used verbatim (no hardcoded scheme list), so a new `zwire` scheme with a matching `zgui-<name>` theme works with no code change; an unknown scheme simply fails to load and the caller keeps the current theme." No table to keep in sync, and the failure mode is "nothing happens" rather than "no theme."

Then there is the case the simple mapping cannot cover. If the user *edited* a built-in scheme, or built a fully custom palette, there is no matching theme file to load. `zwire` also writes a resolved palette — the variable-to-hex colours — and `zmax` grafts those onto the scheme's face structure through a variable-to-palette map, falling back to a known scheme's face structure for a wholly custom one. The distinction being made is between a theme's *structure* (which faces exist and which palette slot each uses) and its *colours* (what each slot is), and keeping them separable is what lets an arbitrary palette reproduce.

The sync is bidirectional, and that is where a naive implementation breaks. `write_back` reverse-maps a `zmax` theme to a `zwire` scheme-and-light pair and rewrites just those two keys when the user changes theme inside the editor. Two watchers now point at each other. The header states both guards: themes outside the app-shell prefix are ignored, and "a write that matches what's already on disk is skipped — so the two watchers can't loop." The second guard is the load-bearing one. Without it, every theme change ping-pongs forever.

The build watcher

`zmax-term/src/ci.rs` is a poller rather than a watcher, and the difference is forced by the medium: workflow status arrives over a REST API, so there is nothing to watch. The threading note in the header is blunt about the consequence — the fetch is "over `ureq`, blocking — call from a `spawn_blocking` task, never the UI thread."

Results live in a process-global mutex-guarded state so "both the IDE CI panel and the statusline segment can read them without threading state through the editor." That state carries four fields — runs, error, loading, fetched — and all four are needed for correct display. `loading` distinguishes "fetching" from "idle"; `fetched` distinguishes "no runs" from "have not asked yet"; and `error` is separate from an empty run list so a failed request does not render as a repo with no CI.

`CIRun::icon()` returns a glyph and a theme scope in one call, and the mapping handles the case people forget:

```
if self.status != "completed" {  
    return ("●", "warning"); // running / queued  
}
```

A run in progress has no conclusion, and matching on the conclusion first would classify it with the unknowns. Status is checked before conclusion, so in-flight runs get their own glyph. Below that, success maps to the diff-plus scope, the failure family — failed, timed out, startup failure — to the error scope, and the non-events — cancelled, skipped, neutral — to the comment scope. Returning the theme scope rather than a colour means the CI panel recolours with the theme, and with the `zwire` sync above, with the terminal.

The common lesson

Three watchers, one structure: a thread that owns the source of events, a coalescing step, a hop to the main thread through a closure queue, and a render that comes for free. What differs between them is entirely the event source and the response.

The reason this matters architecturally is the one the file watcher's git observation illustrates. Every one of these features is a place where the editor's model of the world can silently diverge from the world. The gutters can be stale. The theme can be wrong. The build status can be from ten minutes ago. None of those throws an error; all of them just quietly lie. Watchers are how an editor stops lying, and the cost of having them is one thread each and a queue that already existed for the language server.

Chapter 93 – The Property Store

Every editor eventually discovers that some of what it draws is not a function of the text. Syntax highlighting is a function of the text — feed the same bytes to the same grammar and you get the same colors. But the moment a user selects a region and says “make this bold”, or a file arrives carrying its own formatting, or a mode decides some characters should not be displayed at all, the editor needs somewhere to put information that belongs to *characters* rather than to the syntax tree.

Emacs solved this in the seventies with text properties: attributes stored on the characters themselves, surviving edits, saved through format converters, read by redisplay. `zmax` did not have that, and the module header of `zmax-core/src/text_props.rs` is unusually direct about the consequence:

`zmax` had no such store: faces could only come from tree-sitter or a theme scope, which is why `facemenu`, `enriched-mode` and `hide-ifdef-mode` had nothing to write to.

That is the shape of a whole class of porting problem. Three user-visible features were stuck not because their logic was hard but because the substrate underneath them did not exist. `enriched-mode` in particular had been ported as far as changing the mode line and no further, because a decoder with nowhere to write its output is not a decoder.

What a run carries

The store is a list of runs. A run is a half-open char range plus a `Props` value, and `Props` carries two things: a `Face` and an `invisible` flag. Two, not one, and the module says why:

The two Emacs removal commands are genuinely different operations here, which is why `Props` has more than one field: `facemenu-remove-face-props` clears only the face and leaves `invisible` alone, while `facemenu-remove-all` drops the whole run.

This is a good example of a data-structure decision falling directly out of a command-set decision. If Emacs had only one removal command, `Props` could have been a bare `Face` and the store would be simpler. It has two, they mean different things, and so the run needs at least two independently clearable compartments.

`Face` itself has six fields: an optional named face, three boolean attribute toggles (bold, italic, underline), and two optional colors. The doc comment frames it as a delta rather than a value:

A face is a *delta*: only the attributes that are set are applied, so `facemenu-set-bold` over a region that is already red produces bold red text.

The merge rule follows: an optional field that the delta sets wins, an optional field it leaves `None` is inherited, and the three booleans are OR-ed. There is no way to express “explicitly not bold” as a delta, which is correct for `add-face-text-property` — Emacs’ way of turning bold off is to remove the face, not to layer an anti-bold on top.

The empty face gets a name of its own, `Face::is_default`, and it does real work. `add_face` checks it first and reroutes:

```
pub fn add_face(&mut self, range: Range<usize>, delta: &Face) {
    if delta.is_default() {
        // `facemenu-set-default` - a face with nothing set means "no face".
        self.remove_face(range);
        return;
    }
    self.update_range(range, |props| props.face.merge(delta));
}
```

Without that branch, `facemenu-set-default` would merge a delta that sets nothing and change nothing, which is exactly the wrong behavior for a command whose whole job is to strip formatting back to plain.

The invariant, and why it is worth paying for

`TextProps` documents a five-part invariant on its span list: sorted by start, no overlaps, no empty ranges, no run carrying empty props, and no two adjacent runs carrying equal props. The last one is the interesting one, and the payoff is stated where you would want it:

`TextProps::spans` therefore is a canonical form: two `TextProps` describing the same properties compare equal.

Canonical form is what makes the tests readable. Without coalescing, “bold from 2 to 5” and “bold from 2 to 3 plus bold from 3 to 5” are different values that mean the same thing, and every assertion has to normalize before comparing. With coalescing enforced by every mutator, `assert_eq!` on the derived `PartialEq` is a complete check. It also keeps the list from fragmenting: a user who bolds a paragraph one word at a time ends up with one run, not one run per word.

Everything that mutates the store goes through a single private primitive, `update_range`, described in the source as “the single primitive every public mutator is built from”. It takes a range and a closure over `Props`, and it does the three-way split: the part of each existing run before the range keeps its props, the overlapping part gets the closure applied to a clone, the part after keeps its props. Gaps inside the range that no existing run covers get a fresh run built by applying the closure to a default `Props`. A cursor walks the range so the gap-filling happens in one pass rather than as a separate scan.

The five public mutators are then thin wrappers. `add_face` merges. `set_face` assigns. `remove_face` assigns the default face. `remove_all` assigns default props wholesale. `set_invisible` sets the flag. Each ends with the same retain-sort-coalesce tail because `update_range` owns it. This is the correct factoring for a data structure with a nontrivial invariant: exactly one function is allowed to break it, and that function restores it before returning.

Surviving edits

A property store that does not track edits is worse than no store, because it silently mislabels text. If you bold characters 10 through 20 and then insert five characters at position 0, the bold has to move to 15 through 25.

`zmax` already had the machinery: a `ChangeSet` knows how to map positions through an edit. So `TextProps` exposes its boundaries as raw mutable references:

```
pub fn positions_mut(&mut self) -> impl Iterator<Item = &mut usize> {
    self.spans
        .iter_mut()
        .flat_map(|s| [&mut s.start, &mut s.end])
}
```

The caller — `Document::apply` — runs `ChangeSet::update_positions` over that iterator, which rewrites each boundary in place. That leaves the invariant broken in every way it can be broken: a deletion can collapse a run to zero width, can push a boundary past the document end, and can reorder two runs relative to each

other if they were adjacent to a deleted region. So there is a second call, `repair`, which clamps to the new length, drops collapsed and empty runs, re-sorts, and coalesces.

The split into two calls is deliberate and it is the only sane arrangement. The change set does not know about spans; the span list does not know about change sets. The seam is a mutable iterator over integers, which is the smallest possible contract between them. It also means any other position-mapping algorithm — a different edit representation, a replay from a journal — can drive the same store without either side learning about the other.

Invisibility as an overlay

The `invisible` flag does not reach the renderer as a flag. It reaches it as a list of character indices:

```
pub fn invisible_chars(&self) -> impl Iterator<Item = usize> + '_' {
    self.spans
        .iter()
        .filter(|s| s.props.invisible)
        .flat_map(|s| s.start..s.end)
}
```

Those indices become grapheme overlays with empty replacement text — the same mechanism `conceallevel` uses, which is the same mechanism this book's chapter on prettify-symbols will show being used for a third purpose. One rendering primitive, three features. The renderer never learns what an `invisible` text property is; it learns that grapheme *n* draws as the empty string, which it already knew how to do.

That is the recurring shape in this part of the codebase. The expensive thing is the substrate — a run list with a canonical form and an edit-mapping seam. Once it exists, `facemenu`, `enriched-mode`, `hide-ifdef-mode`, and anything else that wants to decorate characters are each a small amount of glue on top of it, and none of them touch the renderer.

Chapter 94 — Faces, the Face Menu, and Enriched Text

With a property store in place, three Emacs features become implementable in a few hundred lines each. They are worth reading together because they form a complete loop: a table of things you can apply, a way to apply them, and a file format that survives a round trip through disk.

Two static tables

`zmax-core/src/facemenu.rs` is almost entirely data. It holds the color table that `list-colors-display` browses and the face table that `list-faces-display` browses, and it is careful to be nothing else — the interactive overlay lives in the terminal crate, so this file has no I/O and is unit-tested directly.

The color table holds forty-eight entries. Its doc comment says “~48 X11 / Emacs color names”; the table is exactly forty-eight, and the comment’s approximation is the kind of thing that goes stale in either direction, so treat the code as the count. The entries are `const fn`-constructed, so the whole table is built at compile time and lives in the binary as static data.

The comment on the table records a property that matters for lookup:

Names are unique; RGB values may repeat (e.g. `cyan` and `aqua`).

Both halves are load-bearing. Unique names mean name-to-RGB is a function, which is what `find_color` needs. Repeating RGB means RGB-to-name is not a function, which is why there is no reverse lookup — asking “what is this color called” has no single answer for `(0, 255, 255)`, and rather than pick arbitrarily the module simply does not offer the question. There is a test that pins the uniqueness of names so a later addition cannot quietly break the lookup direction that does work.

The face table holds twenty-six entries, each a name plus a short prose description of what the face does. It covers the structural faces (`default`, `region`, `highlight`, `shadow`, `error`, `warning`, `success`), the link faces, the `font-lock-*` family, and the UI chrome faces (`mode-line`, `cursor`, `fringe`, `minibuffer-prompt`).

The bridge to a theme

A name like `font-lock-keyword-face` is meaningless on its own. It has to become a color, and in `zmax` colors come from the active theme, not from the face table. `theme_scope` is the translation, and it is a plain match with twenty-one arms:

```
"font-lock-keyword-face" => "keyword",
"font-lock-string-face"  => "string",
"font-lock-comment-face" => "comment",
"region"                => "ui.selection",
"secondary-selection"    => "ui.selection",
"shadow"                 => "comment",
```

Twenty-one arms against twenty-six faces leaves five faces unmapped, and they are exactly the five attribute faces: `default`, `bold`, `italic`, `underline`, `bold-italic`. Those do not need a theme scope because they

are not colors — they map onto the boolean toggles in `text_props::Face` directly. The module header says as much: the attribute faces “map straight onto the attribute toggles; the rest name a *theme scope*”. The partition of the table into mapped and unmapped is not an oversight, it is the design, and the arithmetic works out cleanly enough that a future addition to either table which breaks the partition would be visible.

Two of the mappings collapse: `region` and `secondary-selection` both resolve to `ui.selection`, and `shadow` resolves to `comment`. That is a real loss of fidelity — Emacs draws the secondary selection differently from the primary one — and it is the honest thing to do when the target theme format has no separate scope for it. Inventing a scope would put a key in every theme file that no theme author has ever heard of.

The payoff of the indirection is stated in the header: `font-lock-keyword-face` “picks up whatever the active zmax theme paints keywords with”. Apply that face to a region, switch themes, and the region follows the new theme. A face menu that resolved names to RGB at apply time would freeze the old theme’s colors into the buffer, which is wrong in a way that only shows up weeks later.

text/enriched, and what was deliberately skipped

`zmax-core/src/enriched.rs` is the format converter behind `enriched-mode`. The file on disk carries face information as SGML-ish annotations; decoding strips the markup and produces text plus face runs; encoding does the reverse.

```
Content-Type: text/enriched
Text-Width: 70

plain <bold>bold</bold> and <x-color><param>red</param>red</x-color>
```

The header detection is more careful than it first looks. `header_len` lowercases and trims the first line before comparing against `content-type: text/enriched`, then walks forward through `split_inclusive('\n')` accumulating character counts until it hits a blank line. Character counts, not byte counts, because everything downstream indexes the text by character. And there is a fall-through for a file that is nothing but a header with no blank line after it, which returns the whole length rather than `None` — a degenerate but legal file decodes to empty text rather than being misread as plain.

The interesting part of this module is what it refuses to do:

Not implemented: RFC 1896’s soft-newline folding (a lone newline is a soft break, `n` newlines mean `n-1` hard ones). That is the job of Emacs’ separate `use-hard-newlines`, which this port does not have, so the decoder and encoder pass newlines through verbatim and a file round-trips byte for byte apart from the annotations.

This is the right call, and the reasoning is the interesting half. Soft-newline folding is not a property of the enriched format in isolation; it is a property of the format *plus* a buffer-local notion of which newlines the user typed and which the filler inserted. Emacs tracks that in `use-hard-newlines`, which is itself a text property. Implementing the folding without the tracking would mean guessing, and guessing here corrupts documents: a decode-encode cycle would silently reflow the user’s paragraphs.

The chosen alternative has a property worth having. Passing newlines through verbatim means the round trip is exact apart from markup, so opening an enriched file and saving it without editing produces the same bytes. An editor that silently rewrites files it merely opened is a data-loss hazard, and “we implemented less of the spec and therefore change nothing” beats “we implemented more of the spec and reflow your text on open.”

Note also that the escape rule is the format’s, not an invention: a literal `<` is written `<<`, per RFC 1896. There is exactly one escape, which is why the decoder needs no state machine to speak of.

The loop, closed

Put the three modules together and the loop closes. `facemenu` supplies the names and the theme bridge. `text_props` stores what was applied, keeps it canonical, and moves it through edits. `enriched` serializes it to disk and reads it back. Remove any one and the other two have nothing to do: a face table with no store is a list, a store with no converter is session-local, a converter with no store has nowhere to write.

Chapter 95 – Abbreviations: Two Traditions, Two Modules

zmax carries two abbreviation systems that do not share a line of code, and the reason is not neglect. Vim abbreviations and Emacs abbrevs are different features that happen to share a name.

Vim's version is a classifier

`zmax-core/src/abbrev.rs` is the Vim side. What makes it substantial is not the storage — three hash maps — but the trigger rules, which are one of the more baroque corners of Vim's documented behavior.

Vim recognizes an abbreviation only when a non-keyword character is typed after it, and only for three shapes of left-hand side. The enum has exactly three variants, matching `:help abbreviations: FullId` where every character is a keyword character (`foo, g3`), `EndId` where the last character is a keyword character and every other character is not (`#i, ..f`), and `NonId` where the last character is a non-keyword character and the rest may be anything (`def#, 4/7$`). Anything else — `a.b, #def, a b` — is not a valid abbreviation at all, and `add` rejects it rather than storing something that could never fire.

The three shapes exist because each carries a different rule about the character *in front of* the match, and the module header says so plainly:

full-id and end-id require a non-keyword char (or start-of-line) before them, while non-id requires a keyword char (or start-of-line). That distinction is the whole reason the three shapes exist and is what `should_trigger` encodes.

Read that against the classifier and it makes sense. A full-id abbreviation like `foo` must not fire inside `xfoo`, because `xfoo` is one word and the user is typing it. A non-id abbreviation like `def#` is designed to fire at the end of a word — the `#` is the terminator — so it *wants* a keyword character in front. The two rules are opposite, and which one applies is determined entirely by the shape of the left-hand side. `classify` returns the shape and `should_trigger` switches on it; between them they are the whole of Vim's rule.

`classify` reads the shape off the last character first, which collapses the logic considerably:

```
let (last, rest) = chars.split_last()?; // None on empty
if !is_keyword_char(last) {
    // Ends in a non-keyword char: non-id regardless of the prefix.
    return Some(AbbrevKind::NonId);
}
if rest.iter().all(|&c| is_keyword_char(c)) {
    Some(AbbrevKind::FullId)
} else if rest.iter().all(|&c| !is_keyword_char(c)) {
    Some(AbbrevKind::EndId)
} else {
    None // mixed keyword/non-keyword prefix before a keyword tail
}
```

The `?` on `split_last` is how the empty string is rejected, which is neater than a length check and cannot be forgotten.

Lookup at typing time is `find_expansion`, which takes the line text up to but not including the just-typed

trigger character and finds the longest left-hand side that is a suffix of it *and* whose preceding-character rule holds. Longest wins, which is what lets `abcdef#` and `def#` coexist. Both are suffixes of `abcdef#`; the longer one is the one the user meant.

Three maps, and the mode arithmetic

`AbbrevMode` also has three variants — `Insert`, `Command`, `Both` — and `Both` is a real third map rather than a synonym for “in the other two”. That choice propagates into every operation, and each operation resolves it differently.

`add` removes the key from all three maps before inserting, so a word lives in exactly one map at a time. Redefining `teh` as command-only after defining it as insert-only moves it rather than shadowing it, which is what Vim does.

`remove` is asymmetric on purpose. `:iunabbrev` on a word defined with plain `:abbreviate` drops it entirely, including for command mode, because a both-mode abbreviation genuinely is active in insert mode and there is no way to be active in only half of it. `:cunabbrev` on an insert-only word removes nothing and reports so. The tests for this are the clearest documentation of the behavior in the file.

`clear` follows the same rule at table scale: clearing insert also clears the shared map, and clearing `Both` clears everything.

`lookup` chains in priority order — mode-specific map first, shared map second — so a mode-specific definition shadows a both-mode one for that mode. `entries` tags each row with which map it came from, because `:abbreviate` with no arguments lists the mode alongside the word and the user needs to know whether they are looking at an `i`, a `c`, or a plain entry.

Emacs abbrevs are a different feature

`zmax-term/src/emacs_abbrev.rs` shares nothing with the above, and should not. Emacs abbrevs are persistent by default, expand on an explicit command, and live in a global table plus per-major-mode tables.

The store is a file at `<config-dir>/abbrevs`, one `name\texpansion` row per line, with tabs and newlines in the expansion escaped so the row format survives. The escaping is the whole reason the file has a `parse_line/format_line` pair rather than a `split('\t')`, and the tests cover the awkward cases: a trailing lone backslash stays a backslash, an unknown escape passes through intact, a row with no tab or an empty name parses as nothing.

Every operation reloads the file, mutates a `Vec`, and writes it back. That is $O(\text{file})$ per define, and for a table of abbrevs it is the correct trade — the file is small, there is no cache to invalidate, and two `zmax` instances editing abbrevs do not corrupt each other’s view any worse than last-write-wins.

The mode-local tables are the interesting half, because they made a different persistence decision inside the same module. They live in a process-global `Mutex<HashMap<String, HashMap<String, String>>>`, not on disk, and the comment justifies the split:

the global table above stays file-backed, matching the `abbrevs` file emacs persists by default while mode abbrevs are typically (re)defined per session.

`get_effective` is the lookup Emacs’ `expand-abbrev` performs: the buffer’s mode-local table first, the global table second. That ordering is the entire point of having two tables, and it is one function.

What is not implemented is stated up front:

This ports emacs’s *explicit* expansion (`expand-abbrev`, `C-x '`) plus a define command; auto-expansion as you type (full `abbrev-mode`) would need an insert-keypress hook and is left for later — explicit expansion is the non-invasive core.

Which is a precise statement of the remaining work: not “abbrev-mode is hard” but “abbrev-mode needs one specific hook that does not exist yet.” The Vim side has the same gap on the other end — its header notes that “wiring the actual Insert-mode expansion into the typing path is a separate editor hook.” Two independent ports, both complete as state machines, both waiting on the same missing seam. That is a useful thing to notice about a port in progress: the remaining work is often one shared piece of plumbing rather than N separate features.

Chapter 96 – Cross-References Without an Index

`zmax-core/src/xref.rs` backs the Xref overlay, and it opens by telling you exactly how much it is not doing:

Prior art: GNU Emacs `xref` (find-definitions / find-references). Since `zmax` has no semantic index, this is a grep-style match rather than a tags lookup.

That sentence is doing more work than it looks like. It sets the expectation for every result the feature produces, and it explains why the module can be tested without a project on disk. Whole-word grep over a set of files is a function; a semantic index is a service.

The seam

The division of labor is spelled out: the terminal layer walks the directory tree, reads each text file into a `(path, contents)` pair, and hands the whole set to `find_matches`. The core does the scanning. Nothing here touches the file system, which is why the tests build their corpus as a literal:

```
vec![
  (
    "a.rs".to_string(),
    "fn foo() {} \n      foo(); \nlet foobar = 1; \n".to_string(),
  ),
  (
    "b.rs".to_string(),
    "// call foo here \nbar(foo, baz); \n".to_string(),
  ),
]
```

Four lines of that fixture match `foo` as a whole word and one does not, and the test asserts exactly four hits — the `foobar` line is the negative case sitting inside the positive fixture rather than in a separate test. That is a good pattern for boundary logic: the thing that must not match is adjacent to the things that must.

Note also that `find_matches` reports at most one hit per line, the first. That is a display decision, not a limitation — the overlay renders a line of context per hit, and two hits on the same line would render the same line twice.

The word-boundary check

`find_word_col` searches with `str::find` and then validates both edges:

```
let before_ok = line[..i]
  .chars()
  .next_back()
  .is_none_or(|c| !is_word_char(c));
let after_ok = line[end..].chars().next().is_none_or(|c| !is_word_char(c));
```

`next_back()` on the prefix and `next()` on the suffix, each defaulting to “OK” when there is no such character, which is how start-of-line and end-of-line are handled without a special case. On failure the scan restarts one

byte later rather than one match later, so `foofoo` does not hide a valid match at offset three.

The returned column is deliberately not the byte offset:

```
return Some(line[..i].chars().count());
```

Bytes for searching, characters for reporting. The test that pins this uses a line with a multibyte character before the match and asserts the column counts characters, which is the only way this bug ever gets caught — with ASCII input the two are equal and the mistake is invisible.

A heuristic that admits it is one

`looks_like_definition` is how the overlay distinguishes a probable definition from a plain reference so it can present them differently. It has two strategies. The first is a keyword list — nine keywords, drawn from several languages at once (`fn`, `let`, `struct`, `enum`, `def`, `class`, `const`, `type`, `trait`) — each tested as `keyword symbol` with word boundaries on both ends. The second is a bare assignment: the symbol followed by optional whitespace and a single `=`, excluding `==` so a comparison is not mistaken for a binding.

Reading it as an adversary is instructive. The keyword branch checks boundaries on both sides of the `keyword symbol` phrase. The assignment branch checks the boundary only *before* the symbol, and then looks at what follows. That asymmetry has a consequence: the line `a.foo = 1` is reported as a definition of `foo`, because the character before `foo` is `.`, which is not a word character, and what follows is `= 1`. Structurally that is a field assignment, not a definition.

The module does not claim otherwise. It calls the function “a cheap heuristic (no parser)” and the tests exercise a set of positive and negative cases without ever implying completeness. The important thing about a heuristic in a codebase is not whether it is right in every case but whether it is labeled. This one is labeled, it is confined to a display decision — getting it wrong changes how a line is styled in an overlay, not what the search returns — and the cost of being wrong is one mis-styled row.

`group_by_file` is the last piece, and it is a short loop with one deliberate property: it preserves first-seen order of both files and hits rather than sorting. Since `find_matches` visits files in the caller's order and lines top to bottom, the grouped result is already in display order and the UI does no sorting of its own. Sorting here would be a bug, not an improvement, because the caller's file order is the tree-walk order the user expects.

Chapter 97 — Filtering, Sorting, and Counting Lines

Three Emacs commands — `keep-lines`, `flush-lines`, `how-many` — are in-buffer `grep`, `grep -v`, and `grep -c`. Their `zmax` substrate is `zmax-core/src/line_filter.rs`, and it is a small file that makes several decisions worth copying.

The predicate is injected

There is no regex in this module. Every function takes a closure:

```
pub fn flush_lines_ranges(
    text: &str,
    matches: impl Fn(&str) -> bool,
) -> Vec<(usize, usize)>
```

The header explains the pattern and names its sibling: “the caller injects the actual match test as a closure (exactly as `crate::region_ops::occur` does), so the engine stays dependency-free and separately unit tested.” The tests then use predicates like “contains a substring” and never construct a `Regex`, which means a change in regex crate or syntax cannot break them, and a bug in line splitting cannot hide behind a bug in the pattern.

Ranges, not strings

The transforming functions return char ranges to delete, not new text. The reason is that the caller is not holding a `String`, it is holding a rope, and the ranges feed straight into `Transaction::delete`. Returning text would mean serializing the whole buffer, deleting from the copy, and diffing it back, which is both slower and loses the undo granularity.

Two properties of the returned ranges are stated and both matter. They are ascending and non-overlapping, which is what a transaction requires. And each range covers a whole physical line *including* its trailing newline, “so deleting a line leaves no empty remnant.” That second one is the classic off-by-one in this family of commands: delete the line content but not the newline and a hundred-line flush leaves a hundred blank lines behind.

The line splitter earns its own paragraph in the header:

Line boundaries are found with `split_inclusive('\n')`, which yields no spurious trailing empty element, so the ranges tile the input exactly and a full-buffer keep/flush round-trips cleanly.

`split_inclusive` versus `split` is exactly the kind of choice that is invisible until the buffer ends without a trailing newline, or ends with one, and the two cases behave differently. `split('\n')` on `"a\nb\n"` yields three items, the last empty; `split_inclusive` yields two. Since these ranges are meant to tile the input, a phantom empty final line would produce a zero-width range that either does nothing or, worse, shifts a subsequent range.

There is a related detail in `line_spans`: the range covers the newline but the `content` handed to the predicate does not. The comment gives the reason — “matching an Emacs pattern that is anchored with `$` behaves as

expected because the newline is not part of `content`". Range semantics and predicate semantics differ by exactly one character, deliberately.

`keep_lines_ranges` is described as "the exact complement of `flush_lines_ranges`", and reading the two, it is: identical bodies but for a `!` on the predicate. Writing it as a complement rather than as a call to the other with a negated closure keeps the two symmetrical in the source, which is worth more than the two lines it saves.

Counting is two independent questions

`how_many` returns a pair and takes two closures, which looks redundant until you read why:

`count_total` is applied once to the whole span (so patterns that span newlines are counted, matching Emacs's `re-search-forward` loop); `line_matches` is the per-line predicate used to tally how many distinct lines contain a match. The two are independent because a single line may hold several matches.

Emacs' `how-many` reports both numbers, and they are genuinely different computations, not one computation reported two ways. Total matches must run against the whole text because a regex can span a newline; matching lines must run per line because a line with three matches is still one line. Trying to derive either from the other gives wrong answers on real input.

Applying the ranges without a rope

`delete_ranges` exists so tests and string-holding callers can see the result, and it is defensive in a specific way:

```
for &(from, to) in ranges {
    if from < cursor || to > chars.len() || from > to {
        continue;
    }
    out.extend(&chars[cursor..from]);
    cursor = to;
}
```

Out-of-order, out-of-bounds, and inverted ranges are skipped rather than panicking. The doc comment states the precondition — sorted and disjoint — and then the code declines to enforce it with a panic. For a helper whose main job is verifying ranges in tests, a panic-free skip is the better failure mode: a test that produced bad ranges gets a wrong string it can print and inspect, not a stack trace that hides the ranges.

A different sorter for a different record

`zmax-core/src/sort_subr.rs` is the generic engine behind Emacs' `sort-subr`, and its header identifies the subtlety that makes it worth porting rather than reimplementing:

The key subtlety (`sort-reorder-buffer`): records are permuted into their ORIGINAL slots while the inter-record gaps stay fixed in place.

This is not how anyone would write a sort from scratch. The obvious approach — sort the records, join them with the separators — is wrong for Emacs' model, because the gaps between records are not necessarily uniform. What Emacs does is walk the original record order and, for each original slot, emit the gap that preceded it followed by whichever record sorted into that position. Gaps stay put; contents move.

`reorder` is a direct transcription of that. It builds an index permutation, sorts the permutation by key, and then iterates the *original* records, emitting `chars[last..rec.start]` — the preceding gap — and then the

contents of `records[order[i]]`. Every index is defensively `.min(n)`-clamped so a caller that hands in ranges past the end of the text produces truncated output rather than a panic. The sort is stable and the comparison is by code point, matching Emacs with `sort-fold-case` nil.

`page_records` is one record builder for that engine, cutting the text into form-feed-delimited pages and treating the newlines between them as gaps. The comment maps it back to the Emacs original — “Mirrors `sort-build-lists` driven by `nextrecfun = skip \n` and `endrecfun = forward-page`” — which is the right level of citation for a port: name the function you are mirroring and the arguments it was called with, so the next reader can diff against the original.

Chapter 98 – Addresses in Text

Two Emacs features exist because plain text keeps containing things that are really references: a URL in a comment, a stack-trace path in a log, a file name in a README. `goto-address-mode` finds the first kind, `ffap` finds the second, and both are in `zmax` as pure scanners with the resolution left to the caller.

goto-address

`zmax-core/src/goto_address.rs` recognizes a fixed array of six prefixes: `https://`, `http://`, `ftp://`, `file://`, `mailto:`, and the bare `www.` that Emacs also accepts. That last one is the reason the module has an `Address::target` method rather than just returning the matched text — a `www.` host is not a URL until something prepends a scheme, and an e-mail address is not one until something prepends `mailto:`. Doing that at the match site would mean rewriting the buffer text; doing it in `target` keeps the range pointing at real characters while still handing the browser something it can open.

The scanner is line-at-a-time, and the header says why: “pure and line-at-a-time so the renderer can run it over the visible lines only.” A buffer-wide scan on every redraw would be the naive implementation and would cost the whole file per frame. Restricting the unit of work to a line makes the cost proportional to the viewport.

Two character classes drive the scan, and they are byte predicates rather than char predicates:

```
fn is_url_byte(b: u8) -> bool {
    b.is_ascii_alphanumeric() || b"-._~:/?#[ ]@!$&'()*+.,;=%".contains(b)
}
```

That set is RFC 3986’s unreserved plus sub-delims plus the generic delimiters, which is to say: every byte that can legally appear in a URI. Working in bytes is safe here precisely because every character in that set is ASCII — a multibyte character can never be confused for one of them, so the scan cannot split a character.

The trimming rule is where the module earns its keep. The header states it as a reader’s intuition rather than a spec:

The URL is trimmed the way a reader would: trailing sentence punctuation and a closing bracket that has no opener inside the URL are not part of it.

Both halves are needed and neither is derivable from the URI grammar. `.` and `,` are legal URL characters, so `See https://example.com/x.` has a technically valid URL ending in a period — and every human reader knows the period is the sentence’s. Parentheses are worse, because a URL genuinely may contain balanced ones (Wikipedia article titles are the standard example), so the rule cannot simply strip closers. Counting openers inside the candidate and dropping only the unmatched trailing closer handles `(see https://example.com/a(b))` correctly in both directions.

ffap

`zmax-core/src/ffap.rs` is the pure half of “find file at point”. The split is stated: “Emacs decides with `ffap-next-regexp` what is worth trying, then keeps only the guesses that name a file that exists. This module is the first half — no file system.”

The scan runs over a set of path characters and a set of wrapper characters, and notably the wrappers are excluded from the path set at the loop condition rather than trimmed afterward, so a name inside quotes or

parens terminates at the delimiter instead of swallowing it.

What makes a run a *guess* rather than a word is a three-way test stated in the doc comment: it contains a `/`, or starts with `~`, or has a file extension. “Bare words, plain numbers and lone dots are not guesses.” Without that filter, every word in every prose file would be a candidate and the existence check — which is a stat per candidate — would be the dominant cost of the feature. The filter is what makes the expensive half affordable.

Trailing punctuation is peeled the same way `goto_address` peels it, and then there is one more step that shows the module knows what it is for:

```
/// The line number that followed the name (`src/main.rs:42`), if any.  
pub line: Option<usize>,
```

A trailing `:LINE` or `:LINE:COL` is split out of the name and reported separately. That is not part of any file-name syntax; it is the format compilers and `grep` use, and it is routinely the thing a reader actually wants to follow. Note that `:` is in the path character set — it has to be, for `file://` and for Windows drive letters — so the line-number suffix cannot be handled by the character classes and needs its own pass. Recognizing it turns “open the file” into “open the file at the line the error was on,” which is the difference between a feature that saves a keystroke and one that saves a search.

Chapter 99 – The Bookkeeping Modes

Three Emacs features do clerical work on files: stamp a modification time, maintain a ChangeLog, track hours. They are unglamorous and they are also the kind of thing that is quietly wrong in most reimplementations, because the edge cases are all in the parsing.

time-stamp

`zmax-core/src/time_stamp.rs` rewrites the text between delimiters after a `Time-stamp:` marker near the top of a file. Two constants define the search space: `LINE_LIMIT` is 8, matching Emacs' `time-stamp-line-limit` default, and `DELIMS` holds two delimiter pairs, `<...>` and `"..."`.

The line limit is not an optimization. The header gives the reason: “Emacs only looks at the first `time-stamp-line-limit` lines (8 by default) so a stamp mentioned in the body of the file is not clobbered.” A file that documents its own stamp format, or a test fixture containing an example stamp, would otherwise be corrupted on every save. The limit is a correctness bound that happens to also be fast.

`find` walks lines and tracks the character offset of each line start manually:

```
line_start += line.chars().count() + 1; // +1 for the '\n'
```

Characters again, because the returned `Template` carries char offsets that the caller will hand to a rope. The `+ 1` for the newline is correct because the split is on `'\n'` and the separator is not in the pieces.

One small piece of Emacs fidelity is easy to miss:

```
// Emacs allows '\\' before the delimiter (for stamps inside string literals).
while i < chars.len() && (chars[i].is_whitespace() || chars[i] == '\\') {
    i += 1;
}
```

A time stamp embedded in a C string literal is written `"Time-stamp: \"...\""`, so the character before the delimiter is a backslash. Skipping backslashes alongside whitespace is a one-token change that makes the feature work in generated source files, and it is exactly the sort of thing that only appears in a port if someone read the original.

The `Template` type returns the range of the *content* between delimiters plus the old content. Not the whole match, not the delimiters. That means the rewrite is a replacement of a range that never includes a delimiter, so a malformed replacement cannot destroy the template and break every subsequent stamp.

ChangeLog

`zmax-core/src/changelog.rs` handles the GNU ChangeLog format: dated entries newest first, each holding tab-indented file lines naming a source file and, parenthesized, the function that changed.

The write side is two small formatters. The read side, `source_at`, is the one that has to be careful, because it backs `change-log-goto-source` — the command that opens the file a line names:

```
let rest = line.trim_start().strip_prefix("* ")?;
// The file name runs up to the first `(`, `:` or `,` - whichever comes first.
let end = rest.find(['(', ':', ',']).unwrap_or(rest.len());
```

Three terminators because the format admits three shapes: `* file (sym): text`, `* file: text`, and `* file, other-file: text`. A parser that only knew about `(` would put the colon in the file name.

The genuinely interesting function is `entries_from_diff`, which builds ChangeLog file lines straight from a unified diff. It reads `+++` lines for the post-image path — skipping `/dev/null`, since a deletion’s name came from the `---` line — and then reads the funcname context git appends after the second `@@`:

```
@@ -1,7 +1,9 @@ fn parse(input: &str)
```

That trailing text is git’s own idea of which function the hunk is inside, which means `zmax` gets function attribution for free from a tool that already computed it. The alternative would be parsing every changed file to find enclosing definitions.

`defun_name` extracts a name from that context, and it is a heuristic with two stated examples: `pub fn parse_status(porcelain: &str)` yields `parse_status`, and `impl Component for MagitStatus {` yields `MagitStatus`. It works by cutting at the first `(`, `{`, `<`, or `=` and taking the last identifier of what remains. Then it rejects a short list of bare keywords, because a context line that is just `else` or `impl` names nothing and would otherwise produce a ChangeLog entry claiming a function called `else` changed.

The `impl Component for MagitStatus` case is worth pausing on, because “last identifier before the argument list” gets it right for the wrong-looking reason: the trait name comes first, the type name last, and the type is the thing you want in a ChangeLog. That the same rule handles both a function declaration and a trait implementation is why the rule is stated as “last identifier” rather than as a per-language pattern.

`entries_for_files` closes the loop with Emacs’ workflow: you write the ChangeLog first, then seed the commit message from it. Only the newest matching entry per file is taken, with the stated reason that “an older one describes an older change” — an obvious constraint that a naive grep for the file name would get wrong on any repository older than a week.

timeclock

`zmax-core/src/timeclock.rs` models Emacs’ timelog: a sequence of clock-in and clock-out events with whole-second Unix timestamps.

The design decision that makes it testable is stated in the header: all timing is integer arithmetic on Unix seconds, and “today” is the UTC calendar day, computed as `secs.div_euclid(SECS_PER_DAY)`. No timezone database, no clock. The command layer supplies the current time as an argument, so every function in the module is pure and every test is a fixed integer in and a fixed integer out.

This is a real trade and the module does not hide it. Using the UTC day rather than the local day means someone working across their local midnight in a non-UTC zone gets their hours split at UTC midnight instead. What is bought is a model with no dependency on the host’s timezone configuration, no DST discontinuities, and tests that pass identically on every machine. For a feature whose output is a rounded hours figure, that is the right side of the trade, and the alternative — pulling in a timezone database to make `elapsed_on_day` correct — would be a heavy dependency for a clerical feature.

`elapsed_on_day` is the arithmetic core, and it handles three cases in one loop. A clock-in followed by a clock-out is a closed interval and the loop advances by two. A clock-in at the end of the log is still open and closes at `now_secs`, advancing by one. A stray clock-out with no matching clock-in is skipped, which is what makes a partially written or hand-edited log usable rather than fatal.

Every interval is then clipped to the day window before being summed:

```
let s = start.max(day_start);
let e = end.min(day_end);
if e > s {
    total += e - s;
}
```

So a session that started yesterday evening and is still open contributes only today's portion. That is three lines and it is the difference between a feature that reports plausible numbers and one that reports twelve hours worked at nine in the morning.

`parse` is deliberately tolerant — lines that do not start with a valid code or lack a numeric time are skipped, “mirroring emacs’s tolerance of a partially written log.” A timelog is appended to over months; being strict about it would mean one bad line makes the whole history unreadable.

Chapter 100 – The Grid Modes

Emacs has a set of modes that treat the buffer as something other than a stream of lines. They are old, they are odd, and each one is a small self-contained geometry problem. `zmax` ports three of them as pure transforms with the display concerns left to the terminal layer.

picture-mode

`zmax-core/src/picture.rs` turns the buffer into a fixed grid you paint on. Typing overwrites the cell under point and then advances in the current drawing direction, which is one of the eight compass points. The `Dir` enum has eight variants — the four cardinals and the four diagonals — and three methods that are each a match: `delta` gives the `(row, col)` step, `reverse` gives the opposite direction for `picture-motion-reverse`, and `arrow` gives a glyph for the status display.

The core of the mode is small enough to quote whole:

```
pub fn put_char(&mut self, ch: char) {
    let (r, c) = self.cursor;
    self.grid[r][c] = ch;
    self.move_step();
}
```

Overwrite, then step. That is the entire difference between `picture-mode` and normal editing, and having it be this small is the payoff of modeling the buffer as a bounded grid rather than as text with a position. `move_step` clamps to the grid rather than wrapping or growing, so drawing off the edge stops at the edge.

`draw_rectangle` is the other thing the mode exists for. It normalizes the corner pair — `min/max` on each axis, clamped to the grid — so the two corners can be given in any order, and then draws in three passes: horizontals, verticals, and corners last. Corners last is what makes the `+` win over the `-` and `|` that the first two passes wrote there. Doing it in the other order, or trying to special-case the corners during the edge passes, is how this function normally acquires bugs.

The `Display` impl trims each row's trailing blanks. A grid is rectangular by construction; a text file should not be padded to the width of its longest line. Trimming on the way out means the grid model never leaks into the saved file.

two-column

`zmax-core/src/two_column.rs` is smaller and is honest about which part it is:

The *live* linkage (a buffer-local pointer to the partner buffer, synchronized scrolling, `2C-newline` splitting a newline across both) is a display/association concern handled in the terminal layer. What lives here is the reversible text geometry.

Two functions. `split_columns` cuts each line from a starting row at a column, returning left and right halves. `merge_columns` pads each left line to a separator column and appends the matching right line.

The header states the invariant that makes them a pair: “The two are inverses when `merge_columns`’s separator equals `split_columns`’s cut column and every left line reaches that column.” Both conditions are necessary and the second one is the interesting one. A left line shorter than the cut column comes back

from `split_columns` un-truncated (the cut is `col.min(chars.len())`), and merging pads it back out to the separator, which adds spaces that were not there before. So the round trip is exact only for a rectangular region — which is precisely the situation two-column mode is for.

`merge_columns` also handles the ragged cases rather than assuming equal lengths: left lines with no right counterpart are padded and kept, and right lines past the end of the left column are emitted on their own, indented to the separator. That matters because the two buffers are edited independently between the split and the merge; requiring them to stay the same length would make the mode unusable.

selective-display

`zmax-core/src/selective_display.rs` implements `C-x $`: hide every line indented past a column threshold. The port's decision is to not implement hiding at all, but to express the result as fold ranges and reuse the existing fold machinery.

`leading_indent_columns` measures indentation in display columns, expanding tabs to the next tab stop:

```
'\t' => col += tw - (col % tw),
```

Not “add `tab_width`” — advance to the next multiple. A tab after three spaces with `tab_width` 4 advances one column, not four. Getting this wrong makes the feature behave differently on tab-indented and space-indented files, which is the one thing it must not do.

The doc comment also pins the blank-line behavior against the original: a blank or all-whitespace line reports the full width of its whitespace, “matching Emacs, which measures indentation up to the first non-whitespace character or the end of the line.” That is a decision, not a fallout — a blank line could reasonably report zero — and it is recorded as fidelity to Emacs rather than as a preference.

`selective_display_folds` groups consecutive over-indented lines into runs and emits `(start, end)` pairs where `start` is the *visible* line above the run. That off-by-one is not incidental; it is the shape `Folds::create` expects, because a fold keeps its header line visible. And it forces one honest edge case:

A run of over-indented lines with no preceding visible anchor (i.e. one starting at line 0) is left visible, since a fold needs a visible header line.

A buffer whose first line is indented past the threshold cannot have that line folded, because there is nothing above it to fold under. Emacs, which hides lines directly rather than folding them, has no such constraint. This is a visible behavioral difference and the module names it in the same breath as the design choice that caused it, which is the right way to record a divergence — next to the decision, not in a separate list nobody reads.

Chapter 101 – Two Consumers of the Overlay Mechanism

Chapter 93 established that grapheme overlays are how `zmax` draws something other than the character that is in the buffer. Two features consume that mechanism in ways worth comparing, because they sit at opposite ends of the static-versus-dynamic axis.

prettyfy-symbols and glyphless-display

`zmax-core/src/prettyfy.rs` holds both `prettyfy-symbols-mode` (draw `->` as `→`) and `glyphless-display-mode` (draw an invisible control character as something you can see). Its header states the property that makes them safe:

Emacs implements both with a `display` text property / composition: the characters are still there, `point` still walks them, the file on disk is untouched — only the glyphs drawn on screen change.

That is the contract these modes have to honor. A user who prettifies `->` and then deletes the arrow must delete two characters, because two characters are what is there. Anything that made the substitution real would be a feature that silently rewrites source files.

The mechanism handles multi-character symbols in a way that falls out of the overlay model: an overlay replaces one grapheme, so a two-character symbol is two overlays — the first carrying the glyph, the second carrying nothing.

```
out.push(Substitution { char_idx: i, text: to });
// The rest of the token is swallowed by the glyph drawn at its start.
for offset in 1..width {
    out.push(Substitution { char_idx: i + offset, text: "" });
}
```

The renderer needs no concept of a multi-character substitution; it needs the concept of an empty replacement, which it already had for concealment and for invisible text properties.

Matching is longest-first at each position, which the source explains with the case that motivates it: `<=` must beat a hypothetical `<`. And symbols made entirely of word characters get a word-boundary check on both sides that punctuation symbols skip:

```
let wordish = from.chars().all(is_word);
if !wordish {
    return true;
}
```

Without that, `lambda` `→` `λ` would fire inside `lambdas` and inside `my_lambda_helper`. Punctuation symbols do not need the check because `->` cannot be a substring of an identifier.

The symbol tables are per-language and small. The operator table shared by the C-family and ML-family languages has eight entries — the comparison and arrow operators. The Lisp table has one, `lambda` `→` `λ`. Python's has five. Haskell's has eight. The Haskell table's doc comment calls it “the richest of the standard ones”; by entry count it ties the operator table at eight, so read that as a comment about which symbols are

covered rather than how many. `symbols_for` maps language names to tables and returns `None` for anything unlisted, which is how a language opts out — silence rather than an empty table.

`glyphless` is a different kind of table: a match over character ranges rather than a list of strings. C0 controls map to their Unicode Control Pictures via a thirty-two-entry array indexed directly by code point, `DEL` maps to its own picture, C1 controls map to `¿`, and the zero-width and bidi format characters map to visible marks. The rationale for that last group is given directly:

Zero-width and bidi format characters: invisible by design, which is exactly why Emacs shows them.

This is the security-relevant half of the feature. A right-to-left override in a source file can make code display in an order different from how it executes; that is a real class of supply-chain attack, and the defense is exactly this — render the invisible character as something visible.

Three characters are deliberately excluded: tab, newline, and carriage return. The reason is not that they are harmless but that the renderer already handles them, and the comment says so: hiding whitespace rendering “behind a box would fight it.” Two features drawing the same character is worse than one feature drawing it imperfectly.

Hi-Lock

`zmax-term/src/hi_lock.rs` is the dynamic counterpart. Where `prettify`’s tables are compile-time constants, Hi-Lock’s patterns are user regexes added at runtime by `highlight-regex` and friends.

The state is a process-global `Mutex<Vec<Pattern>>`, and the module labels the divergence rather than presenting it as the design: it is “a simplification of emacs’s buffer-local `hi-lock-interactive-patterns`”. In Emacs, highlighting a regex in one buffer does not highlight it in another. Here it does. That is a real difference, it is stated in the first paragraph, and the reader can decide whether it matters for their workflow.

Three things about the implementation are worth noting. Patterns are compiled once on add, not per render, so a bad regex is rejected at the command with an error message rather than failing silently every frame. A duplicate source string is ignored rather than added again, which keeps `unhighlight-regex` a well-defined inverse — remove by source, and there is exactly one thing to remove. And color assignment is by index into the pattern list rather than stored per pattern, so the palette cycles automatically and adding a pattern never has to pick a color that is not already taken.

The render integration is the same discipline as `goto-address`: the loop calls `viewport_matches` with the visible slice. Running user-supplied regexes over the whole buffer every frame is the obvious implementation and the one that makes a large file unusable; scoping to the viewport bounds the cost by screen size regardless of file size.

Between them, these two modules show the two shapes the overlay mechanism supports. `Prettify` is static, pure, exhaustively unit-testable, and needs no state. `Hi-Lock` is dynamic, global, and needs a mutex. Both produce the same thing — character indices and replacement text — and the renderer cannot tell them apart.

Chapter 102 – The Files You Were Just In

Three small modules in `zmax-term` answer versions of the same question: what was I working on? They store their answers in three different places, and the differences are the point.

Recent files, scored like `z`

`zmax-term/src/recent_files.rs` is the persistent one. It backs the start screen, the IDE “RECENT” tab, and the recent-files picker — one store, three consumers, which is why it is worth more than a list.

The store is at `<config-dir>/recent_files`, one line per entry as `path\t<rank>\t<unix-time>`. There is a compatibility path for older stores that held a bare path per line, read as rank 1 and time 0. That is a little extra `split('\t')` handling and it is the difference between an upgrade that keeps your history and one that silently drops it.

Ranking is frecency, and the module names its source: the `z/autojump` algorithm, specifically `rupa/z`. The scoring function is four buckets:

```
fn frecency(rank: f64, age_secs: u64) -> f64 {
    if age_secs < 3600 {
        rank * 4.0
    } else if age_secs < 86_400 {
        rank * 2.0
    } else if age_secs < 604_800 {
        rank / 2.0
    } else {
        rank / 4.0
    }
}
```

Within an hour, quadruple the frequency count. Within a day, double it. Within a week, halve it. Older than that, quarter it. The buckets are the reason frecency works better than either pure recency or pure frequency: a file you have opened fifty times over a year still ranks, but a file you opened four times this morning ranks higher.

The aging rule is the part people leave out when they reimplement `z` from memory. Once the summed rank passes 9000, every rank is multiplied by 0.99 and entries that fall below 1.0 are dropped. Without it, `rank` is monotonically increasing and a file you used heavily two years ago outranks everything forever. With it, the store self-limits and old habits decay. The threshold and the decay factor are both `z`’s, and the module says so — “exactly like `z`” — which is the right kind of citation because it tells a reader where to check the behavior.

Two more properties matter for a store that feeds a picker. Entries are capped at fifty. And `load_entries` drops entries whose files no longer exist, with the reason given: “so no consumer ever offers a dead path.” Filtering at load rather than at each consumer means all three consumers get the guarantee for free and none of them has to stat anything.

The `recentf-mode` toggle is a single `AtomicBool` guarding the one write path, which is `record`, driven by the `DocumentDidOpen` hook. One write path is what makes a single boolean sufficient; if recording happened in three places, the toggle would need to be checked in three places and one of them would eventually be missed.

Closed files, and why they are not persistent

`zmax-term/src/closed_files.rs` is a `Mutex<Vec<PathBuf>>` and nothing else. Push on `DocumentDidClose`, pop for `reopen_last_closed` — the browser’s `Ctrl-Shift-T` gesture. Capped at fifty. Explicitly session-scoped, not persisted.

That last decision is correct and worth stating. “Reopen the file I just closed” is an undo of an action taken seconds ago. A persisted version would offer to reopen something you closed last Tuesday, which is not undo, it is the recent files list — and that already exists, next door, with better ranking.

Push dedupes by removing any existing copy before pushing, so re-closing the same file moves it to the top rather than stacking. The test drains the stack before asserting, with a comment noting it is draining anything left by other code paths — an honest acknowledgment that a process-global mutable static makes tests order-dependent unless they clean up first.

Local history

`zmax-term/src/local_history.rs` is the third answer: a snapshot of the file’s contents on every save, stored under the project’s state directory, capped at fifty snapshots per file. The header describes it as JetBrains-style Local History and notes that it is “independent of git”, which is the entire justification for the feature — the changes you most want back are the ones you made between commits, which is exactly the window git does not cover.

The path derivation is worth reading:

```
let rel = path.strip_prefix(&root).unwrap_or(path);
let key = rel.to_string_lossy().replace(['/', '\\'], "%");
```

The project-relative path is flattened into a single directory name by replacing separators with `%`. That trades a directory tree for a flat one, which means listing a file’s snapshots is a single `read_dir` with no recursion, and it means a file that moves within the project gets a new key rather than inheriting the old file’s history. Both are the right side of the trade for a feature whose primary operation is “show me this file’s versions.”

Snapshots are named by Unix timestamp, so `snapshots()` recovers the ordering by parsing the file stem — no index file, no metadata, and no possibility of the index disagreeing with the directory. The list is sorted newest-first. And `record` skips the write when the content matches the most recent snapshot, so saving an unmodified buffer repeatedly does not consume the fifty-snapshot budget with identical copies.

Three modules, three storage decisions — a scored file, an in-memory stack, and a directory of timestamped blobs — and each one follows from how far back the question reaches. That is the useful generalization: the persistence model is downstream of the time horizon, not a separate choice.

Chapter 103 — Two Lists: Arguments and Processes

Two more modules that are pure state machines with their UI elsewhere. They have little to do with each other, and reading them together shows how consistently the same split is applied.

The argument list

`zmax-core/src/arglist.rs` is Vim's `:args` list: an ordered list of file names with a current index, navigated independently of the buffer list. The module is a `Vec<String>` plus a `usize`, and everything interesting is in how the index behaves when the list changes underneath it.

`next` refuses rather than clamping:

```
pub fn next(&mut self, count: usize) -> Option<String> {
    let count = count.max(1);
    if self.files.is_empty() || self.current + count >= self.files.len() {
        return None;
    }
    self.current += count;
    self.current_file()
}
```

The doc comment names the error this reproduces — Vim's “E165: Cannot go beyond last file” — and the behavior is that the index does not move at all on failure. A clamping implementation would silently move to the last file and report success, which is different in a way that matters when `:next` is inside a macro: Vim's version terminates the macro at the end of the list, a clamping version loops forever on the last file.

`goto` has the same shape for `:argument N`, returning `None` and leaving the index unchanged for an out-of-range `N`. `prev` refuses below zero. `first` and `last` cannot fail and use `saturating_sub` for the empty case.

`delete_matching` is where the index arithmetic gets real. Deleting entries from a list with a cursor in it requires knowing how many of the deleted entries were before the cursor:

```
if patterns.iter().any(|p| glob_match(p, f)) {
    if i < self.current {
        removed_before_current += 1;
    }
} else {
    kept.push(f.clone());
}
```

Then the index is decremented by that count and clamped to the new length. The effect is that `:argdelete` on files you have already passed keeps you pointed at the same file rather than jumping. Without the count, deleting three earlier entries would shift you three files forward.

`dedupe` solves the same problem differently and better suited to its case: it captures the current file *name* before deduping and then re-finds it afterward, because removing later duplicates may remove the very entry the index pointed at and the right answer is “point at the first occurrence of the same file.”

The glob matcher is written out rather than pulled from a crate — `*`, `?`, and literals, anchored to the whole string, with backtracking on `*`:

```
} else if let Some(sp) = star {  
    pi = sp + 1;  
    star_ti += 1;  
    ti = star_ti;  
}
```

The classic iterative wildcard matcher: remember where the last `*` was, and on mismatch rewind the pattern to just after it while advancing the text by one. The comment states the complexity as $O(n \cdot m)$ worst case, which is the honest bound for this algorithm and is fine for a list of file names. A regex crate would be correct too and would drag a dependency into a module whose entire point is being dependency-free.

The trailing loop that skips remaining `*` characters after the text is exhausted is what makes `foo*` match `foo`, and it is the line most hand-written wildcard matchers are missing.

The process list

`zmax-core/src/proced.rs` is the pure half of a `top`-like process viewer. The split is by now familiar: this module parses `ps` output, sorts, and filters; the overlay in `zmax-term/src/ui/proced.rs` runs `ps` and `kill`, renders, and handles keys.

Parsing the output of a specific command line is a design decision with a real benefit. The header names the exact invocation — `ps -axo pid,ppid,user,pcpu,pmem,comm` — which means the parser has a fixed, known column set rather than a header-detection problem. It also means the module works identically on any platform whose `ps` accepts that option string, and that a platform where it does not is a one-line change in the caller rather than a rewrite here.

The `Sort` enum has five variants, one per column, and the direction is baked into the key rather than carried as a separate flag:

`Cpu/Mem` sort descending (the busiest first, like `top`); `Pid` ascending; `User/Comm` lexically.

That is the right encoding. Nobody wants the least busy process at the top, so a direction toggle for CPU would be a control that exists to be left alone. Baking the direction into the column removes a piece of state and a keybinding, and the `next()` method cycles columns, which is the only navigation the feature needs.

The filter is a case-insensitive substring test returning borrowed references rather than clones, so filtering a large process table allocates a vector of pointers rather than copying every row's strings. For a view that re-filters on every keystroke of the search box, that is the difference between smooth and not.

Chapter 104 — Packages: an ELPA Client Without a Byte Compiler

`zmax-core/src/package.rs` is the port of Emacs' `package.el`, and it opens by answering the question a reader of a non-Emacs editor would ask first: what, here, is a package?

Emacs packages are Emacs Lisp libraries, and zmax runs Emacs Lisp: the embedded `elisp` interpreter is a real evaluator with the editor bound to it as a host. So a zmax package is exactly what an Emacs package is — **a directory of `.el` files**.

That is not a small claim and it is the reason this module can talk to the real archives rather than a zmax-specific one. Packages install to `~/.zmax/elpa/<name>--<version>/`, go on the elisp load path, and are activated by evaluating their autoloader file — the same lifecycle, the same directory layout.

Two divergences are stated immediately, and the module says it is stating them “once here so every command that relies on them can point at it.” The first is that there is no byte compiler, because `elisp` is an interpreter and there is no `.elc`. The consequence is drawn out rather than left implicit: what Emacs gets from byte-compiling at install time is a parse-and-eval check of every form, and zmax gets that by evaluating the file, which is what `package-recompile` does here. The second is that a package's effect is whatever its elisp does — a package that is really a C module, a GUI widget, or a font installs and loads and then does nothing, “a property of the elisp surface, not of the package system.”

That distinction is the one worth internalizing. If a package fails, the question is whether an elisp function it called is missing, not whether the package manager is broken. Putting that at the top of the module means every bug report about a non-working package starts in the right place.

Reading the archive

An ELPA archive publishes `archive-contents`, which is an Emacs Lisp s-expression. So the module contains an s-expression reader — restricted, per its own section comment, to “what the file uses.” `Sexp` has five variants: integer, string, symbol, list, and vector. The list variant carries an optional `dot` field for improper lists, because `archive-contents` is built almost entirely out of dotted pairs:

```
List {
  items: Vec<Sexp>,
  dot: Option<Box<Sexp>>,
},
```

Modeling the dotted tail as an optional field on the list, rather than as cons cells all the way down, is the right shape for a reader whose consumer wants `(name . [vector])` and not a linked list. It gives up the ability to represent arbitrary cons structure and buys direct indexing of the parts that matter.

The parser test is pinned against reality rather than against a mock:

The exact bytes GNU ELPA serves (the head of the real <https://elpa.gnu.org/packages/archive-contents>), so the parser is pinned to the wire format rather than to a guess about it.

The fixture includes three entries covering the shapes that matter: a tar package with dependencies and full metadata, a tar package with a shorter metadata alist, and a single-file package whose dependency field

is `nil` rather than a list. Three entries, three shapes, and the assertion checks all three parse. Testing a wire-format parser against handwritten input is how you end up with a parser that handles your imagination rather than the server.

Versions

Versions are `Vec<u64>`, parsed by splitting on `.` with `map_while` so a non-numeric component truncates the version rather than failing. Comparison is element-wise with a missing element counting as zero:

```
pub fn version_cmp(a: &[u64], b: &[u64]) -> Ordering {
    let n = a.len().max(b.len());
    for i in 0..n {
        let x = a.get(i).copied().unwrap_or(0);
        let y = b.get(i).copied().unwrap_or(0);
        ...
    }
}
```

The doc comment names the Emacs function this reproduces, `version-list-<`, and gives the two cases that pin it: `1.2 < 1.2.1` and `1.10 > 1.9`. Those are the two mistakes a naive implementation makes — string comparison gets the second wrong, and length-first comparison gets the first wrong.

The menu is a model

Everything the package menu does is in this file as data plus methods, with no rendering. `Status` has four variants and they are not the obvious two. `Installed` and `Available` are expected; `Obsolete` means installed with a newer version in the archive, and `External` means installed but present in no archive at all — a package installed from a file or from version control. Without `External`, such a package would either be misreported as `Installed` (implying the archive agrees it is current) or vanish from the menu entirely.

`Mark` has two variants, `Install` and `Delete`, each with its Emacs display character. `Filter` has nine variants matching the Emacs package menu's `/` commands — name, description, name-or-description, keyword, status, archive, version, marked, and upgradable — and each knows how to label itself for the header line, which is what makes stacked filters legible.

The menu type carries the rows, the selection, the filters, and the hidden set, and exposes the operations as methods: move the selection, mark, unmark, mark all upgrades, mark obsolete for deletion, hide, toggle hiding, add and clear filters. `visible()` applies filters and hiding and returns the rows to draw. A UI built on this holds no state of its own beyond the viewport.

Dependency order

`install_order` is the last piece, and it is the one graph algorithm here. It resolves a package's dependency closure against the archive listing, deepest first, “so installing in order never leaves a package loaded before something it requires.”

It is a depth-first walk with a `seen` list for cycle protection, pushing each package *after* recursing into its dependencies — the standard post-order topological sort. Two things are dropped along the way. A dependency named `emacs` is skipped, because it is a host-version requirement rather than a package; installing “emacs” from an ELPA archive is not a thing. And a dependency already installed at a version that compares not-less than the requirement is skipped, using the same `version_cmp` as everything else.

The `seen` list is a `Vec<String>` with `contains`, which is linear per lookup. For a dependency closure — realistically a handful of packages, occasionally a few dozen — that is faster than a `HashSet` and it keeps the order deterministic. Reaching for a hash set here would be a reflex, not an optimization.

Chapter 105 — Small Operations That Are Not Small

Two of the smallest modules in `zmax-core` are worth a chapter because both contain a decision that looks arbitrary until you find the case it exists for.

Case conversion is two different features

`zmax-core/src/case_conversion.rs` holds the case transforms, and the first thing to notice is that it draws a line between converting an *identifier* and converting *prose*. They are not the same operation and conflating them is the usual bug.

`to_pascal_case_with` walks characters, upper-cases the first alphanumeric of each word, and drops everything that is not alphanumeric. The dropping is the point, and there is a comment sitting on the exact line that does it:

```
// we don't count _ as a word char here so case conversions work well
if !c.is_alphanumeric() {
    at_word_start = true;
    continue;
}
```

Treating `_` as a separator rather than a word character is what makes `foo_bar_baz` become `FooBarBaz` rather than `Foo_bar_baz`. That is the whole reason to have a Pascal-case function — snake to Pascal is the conversion people actually run — and it only works if the underscore is a boundary that vanishes.

`capitalize_words_with` is the prose version and it does the opposite with non-alphanumerics:

```
if !c.is_alphanumeric() {
    buf.push(c);
    at_word_start = true;
}
```

Pushed, not dropped. Its doc comment states the difference against its neighbor directly — every non-alphanumeric character is “preserved verbatim (unlike `to_pascal_case_with`, which drops them for identifier conversion)”. Two functions with nearly identical control flow and one inverted branch, and the branch is the whole semantic difference between an identifier transform and Emacs’ `capitalize-region`.

There is a third difference in the same function, easy to miss: `capitalize_words` lower-cases the non-initial characters, while `to_pascal_case` pushes them through unchanged. So `HTTPServer` capitalizes to `Httpserver` but Pascal-cases to `HTTPServer`. Again correct for both: Emacs’ `capitalize-word` genuinely does down-case the rest of the word, and an identifier converter that down-cased would destroy every acronym in the codebase.

The case functions are also written in pairs — an allocating `to_x` and a `to_x_with` that appends into a caller-supplied `Tendril`. The `_with` form is the real one and the allocating form is a thin wrapper over it. That matters because these run per selection in multi-cursor editing, where one buffer reused across a hundred ranges is a hundred allocations avoided.

At the top of the file there is one honest open question, left as a comment rather than as a claim:


```
// todo: should this be grapheme aware?
```

The functions iterate `char`, and Rust's `char::to_uppercase` is one-to-many-correct (it yields the two-character `SS` for `ß`, which is why the loops write `buf.extend(c.to_uppercase())` rather than pushing a single char). What they are not is grapheme-cluster correct: a base character followed by a combining mark is two `chars`, and “first alphanumeric of the word” could in principle land between them. Leaving that as a marked todo is better than either pretending it is handled or writing grapheme-segmentation code for a case that has not come up.

Auto-pairs, and the closing-brace problem

`zmax-core/src/auto_pairs.rs` inserts the closing character when you type an opening one. The default table holds six pairs — the three bracket kinds and the three quote kinds — and the module credits its prior art in a comment, pointing at CodeMirror's `closebrackets`.

The data structure is the first decision worth noting:

```
/// The type that represents the collection of auto pairs,
/// keyed by both opener and closer.
pub struct AutoPairs(HashMap<char, Pair>);
```

Keyed by both ends. The map holds two entries per distinct pair, so looking up a typed character finds the pair whether the user typed the opener or the closer. That is required because typing a closer is a real operation with real behavior — if the closer is already there, typing it should move past it rather than insert a second one — and finding the pair from either side means one lookup handles both cases.

The three quote pairs collapse to one entry each, since opener and closer are the same character, which is why `Pair::same()` exists and why there is a whole separate insertion path for it.

The condition for auto-closing is where the module earns its complexity:

```
pub fn should_close(&self, doc: &Rope, range: &Range) -> bool {
    let mut should_close = Self::next_is_not_alpha(doc, range);

    if self.same() {
        should_close &= Self::prev_is_not_alpha(doc, range);
    }

    should_close
}
```

Two rules, and the second applies only to same-character pairs. The first rule — do not auto-close when the next character is alphanumeric — is what stops typing `(` before an existing word from wrapping it in a paren the user did not ask for. Both checks default to `true` at the buffer edges, via `.map(...).unwrap_or(true)`, so typing a quote at the very start or end of a file does the helpful thing rather than the edge-case thing.

The extra rule for same-character pairs is the interesting one, and the case it exists for is the apostrophe. Type `'` after `don` in prose and a naive implementation gives you `don'`. The character before is alphanumeric, so this is not the start of a quoted string, so do not pair. Bracket pairs do not need the rule — `foo(` is a function call and pairing is exactly right — which is why the check is gated on `same()` rather than applied to everything.

There is a further wrinkle the module names in a comment: `handle_insert_same` exists to “handle cases where open and close is the same, or in triples (`"""docstring"""`)”. Triple-quoted strings mean the same-character path has to count what is already adjacent rather than just check one character, because the correct behavior for the third quote of a Python docstring is different from the correct behavior for the first.

The last thing worth pointing out is a limitation stated where it will be found:

```
// Since auto pairs are currently limited to single chars, we're either  
// inserting exactly one or two chars. When arbitrary length pairs are  
// added, these will need to be changed.
```

The selection-adjustment arithmetic downstream assumes an insertion of one or two characters. That assumption is not enforced by a type, so the comment is the enforcement — and it is placed on the code that would break rather than in a design document. Multi-character pairs (`<% / %>`, `# { / }`) are a plausible future feature, and whoever adds them will read this comment before they read anything else, because it sits on the function they have to change.

Appendix A – Configuration Schema

zmax is configured in TOML, at `~/.config/zmax/config.toml` (and a workspace-local `.zmax/config.toml` merged over it). This appendix is a tour of the `[editor]` schema as it stands in `zmax-view/src/editor.rs`, grouped by area, with defaults shown. It is not exhaustive of every leaf, but it covers what you will actually reach for. Unknown keys are an error, so the spelling here is the spelling that parses.

Top level and core editing

```
theme = "onedark"           # color scheme; or [theme] dark/light/fallback

[editor]
scrolloff = 5               # lines kept above/below the cursor
scroll-lines = 3            # lines per scroll step
mouse = true                # mouse support
line-number = "absolute"    # or "relative"
cursorline = false          # highlight the cursor's line
cursorcolumn = false        # highlight the cursor's column
text-width = 80             # used by reflow and wrap-at-text-width
auto-format = true          # format on save when a formatter exists
auto-completion = true      # pop completion as you type
path-completion = true      # complete filesystem paths
default-yank-register = ""
rulers = []                 # columns to draw vertical rulers at
true-color = false          # force 24-bit color
rainbow-brackets = false    # depth-colored brackets (needs rainbows.scm)
```

Cursor, gutters, and the status line

```
[editor.cursor-shape]
normal = "block"           # block | bar | underline
insert = "bar"
select = "underline"

[editor.gutters]
layout = ["diagnostics", "spacer", "line-numbers", "spacer", "diff"]
# gutter components: diagnostics, line-numbers, spacer, diff, code-action-hint, marks

[editor.statusline]
left   = ["mode", "spinner", "file-name", "read-only-indicator",
  ↵ "file-modification-indicator"]
center = []
right  = ["diagnostics", "selections", "register", "position", "file-encoding"]
separator = "|"

[editor.statusline.mode]
normal = "NOR"
insert = "INS"
select = "SEL"
```

The status-line element vocabulary is broad — mode, file name and path variants, diagnostics and workspace-diagnostics, selection counts, position and percentage, file type and encoding and line-ending, version control, register, trailing-whitespace and mixed-indent warnings, and spacers — so the three slots can show as much or as little as you like.

Whitespace, indent guides, soft wrap

```
[editor.whitespace]
render = "none"           # none | all | a per-kind table

[editor.whitespace.characters]
space = " ."
tab   = "→"
newline = "↵"

[editor.indent-guides]
render = false
character = "|"
skip-levels = 0

[editor.soft-wrap]
enable = false
max-wrap = 20             # max chars wrapped past the edge
max-indent-retain = 40
wrap-indicator = "↵"
wrap-at-text-width = false
```

Language servers, search, saving

```
[editor.lsp]
enable = true
display-messages = true
auto-signature-help = true
display-inlay-hints = false
auto-document-highlight = false
snippets = true
goto-reference-include-declaration = true

[editor.search]
smart-case = true           # case-insensitive unless the query has a capital
wrap-around = true

[editor.auto-save]
focus-lost = true          # save when the editor loses focus
after-delay = { enable = false, timeout = 3000 }

[editor.workspace-trust]
level = "servers"           # none | servers | insecure
prompt = true
```

The file picker (`[editor.file-picker]`) and file explorer (`[editor.file-explorer]`) each take the `hidden`, `follow-symlinks`, `git-ignore`, `git-global`, `git-exclude`, and `parents` toggles of Chapter 12, with the picker defaulting to respect git ignores and the explorer defaulting to show more. Keybindings are overridden under `[keys.normal]`, `[keys.insert]`, and `[keys.select]`, mapping a key to a command name or a list of them — the same command vocabulary Movement II is built from.

Appendix B – Keymap Cheat Sheet

The default keymap is `spacemacs`, which is the vim base of `zmax-term/src/keymap/vim.rs` with the `SPC` leader and the emacs `C-x` prefixes over it (Chapter 66); the keys below are that vim base and work under either preset. This appendix is the quick reference: the keys you will use most, by mode. It is a digest, not the whole file — for the complete, evidence-backed enumeration, the reference companion (and the `keybinding_report` of Chapter 29) is the exhaustive source.

Normal mode – motion

```
h j k l      left, down, up, right (j/k by visual line)
w b e        next word start / prev word start / word end
W B E        same, by WORD (whitespace-delimited)
0 ^ $        line start / first non-blank / line end
gg G         file start / file end
f F t T      find char fwd/back, till fwd/back
; ,          repeat last find / reversed
/ ? n N      search fwd/back, next/prev match
* #          search word under cursor fwd/back
%            matching bracket
```

Normal mode – edit

```
i I a A o O  insert: before / line-start / after / line-end / open below / above
d c y        delete / change / yank operators (d{motion}, c{motion}, y{motion})
dd cc yy     whole-line operator
x X          delete char under / before cursor
D C Y        to-end-of-line delete / change / yank
J ~ r R      join / toggle-case / replace char / Replace mode
p P          paste after / before
u C-r        undo / redo
> < =        indent / outdent / reindent operators
gU gu g~     uppercase / lowercase / toggle-case operators
<ret> <bs>   wildfire: grow / shrink selection to closest text object
tab          expand snippet trigger / next tab stop / smart-tab
```

Normal mode — navigate and command

```
gd gD gy gr gi goto definition / declaration / type / references / implementation
K          hover documentation
gt gT      next / previous buffer
[d ]d      prev / next diagnostic      [c ]c  prev / next change
[n ]n      prev / next merge conflict
C-o C-i    jumplist back / forward
C-w s|v    split horizontal / vertical
C-w h|j|k|l focus window left/down/up/right
m` '       set mark / goto mark / goto mark line
" q @      select register / record macro / replay macro
:          command line              SPC leader menu
```

Select mode and Insert mode

```
SELECT (v; V linewise; C-v block-via-multicursor):
  motions extend the selection; d|x delete, c change, y yank,
  > < indent/outdent, ~ switch case, o O flip ends, esc collapse.

INSERT (esc | C-c | C-[ to leave):
  C-w delete word back    C-u kill to line start    C-h backspace
  A-f A-b word fwd/back   A-d kill word fwd
  C-n C-p completion      C-x ... completion submode (C-x C-o omni/LSP)
  C-r insert register     C-g u undo checkpoint
```

The emacs layer rides the Meta keys in Normal mode — **A-x** command palette, **A-</A->** buffer ends, **A-w** copy, **A-;** comment, **C-space** set mark — and never collides with the vim letters, which use no Meta chord.

The emacs preset, deeper

Switch to the emacs preset with **:keymap emacs** (Chapter 66) — it starts in Insert mode, effectively modeless — and the deep emacs machinery of Chapter 60 comes forward:

```
C-y / M-y      yank from kill-ring / yank-pop
C-SPC          set mark          C-x C-SPC pop-to-mark    C-x C-x exchange point/mark
C-x r SPC / j   point register: store / jump
C-x r n / + / i number register: store / increment / insert
C-x r k/d/c/y   rectangle: kill / delete / clear / yank    (C-x r M-w copy)
C-x r m/b/l     persistent bookmark: set / jump / list
C-x a g / C-x ' abbrev: define / expand
```

The Spacemacs leader essentials

The branches you reach for most, from Normal mode behind **SPC** (the full tree is Appendix G):

```
SPC SPC palette    SPC f files      SPC b buffers    SPC w windows
SPC g git          SPC s search    SPC e errors     SPC p project
SPC l layouts      SPC n narrowing SPC k paredit    SPC x text ops
SPC h help         SPC a apps/REPL SPC m major mode SPC D ediff
SPC W workbench    SPC R run cfgs   SPC , preferences SPC z IDE toggle
```

The JetBrains commands

Mostly palette-reached (**SPC SPC**, then the name); the bound few carry **SPC** chords:

```
extract_function / _variable / _constant / _field / _parameter refactorings
inline_refactor  rewrite_refactor  refactor_this             more refactorings
column_selection C-v visual_block_mode                       rectangular block
toggle_line_numbers / _indent_guides / _whitespace_render / _soft_wrap / _inlay_hints
fold_open_recursive / _close_recursive                       recursive folds
SPC d C dap_run_to_cursor  SPC W q/TAB/b tool-window management
```


Appendix C – Languages and Themes

zmax ships broad language and theme support out of the box — on the order of three hundred and fifty languages and two hundred and thirty themes, with `lang-support.md` and `runtime/themes/` as the authoritative counts — configured by data files rather than code, which is why the breadth grows without the editor's source changing. This appendix is the format of both.

languages.toml

A language is a `[[language]]` block in `languages.toml`:

```
[[language]]
name = "rust"
scope = "source.rust"
file-types = ["rs"]
roots = ["Cargo.toml", "Cargo.lock"]
comment-token = "//"
block-comment-tokens = [{ start = "/*", end = "*/" }]
auto-format = true
language-servers = ["rust-analyzer"]

[language.formatter]
command = "rustfmt"

[language.indent]
tab-width = 4
unit = "    "
```

The fields you will touch most are `file-types` and `roots` (how the language is detected and where its project root is), `language-servers` (which servers apply, with optional `only-features/except-features` per server), `auto-format` and `[language.formatter]` (formatting), and `[language.indent]`. Servers are declared separately in `[language-server.NAME]` blocks with `command`, `args`, `environment`, `config`, `timeout`, and `required-root-patterns`; debuggers in `[language.debugger]` with `templates` (Chapter 15). Adding a language is, in the usual case, three things: a tree-sitter grammar, a set of `.scm` queries under `runtime/queries/<name>/`, and a `[[language]]` entry here — no editor recompile to teach it a new language.

theme.toml

A theme maps highlight scopes to styles. The simple form assigns a color; the full form assigns a table:

```
"keyword" = "#c678dd"
"function" = { fg = "#61afef", modifiers = ["bold"] }
"ui.cursor.primary" = { fg = "#282c34", bg = "#61afef" }
"diagnostic.error" = { underline = { color = "#e86c75", style = "curl" } }

[palette]
background = "#282c34"
foreground = "#abb2bf"
```

Two families of scope exist. *Syntax* scopes name tree-sitter captures — `keyword`, `function`, `function.macro`, `type`, `constant.numeric`, `string`, `string.regexp`, `comment`, `variable.builtin`, `punctuation.delimiter`, `markup.heading`, `diff.plus/diff.minus`, and so on, dot-separated from general to specific, with the most specific match winning. *UI* scopes name the editor's own chrome — `ui.background`, `ui.cursor`, `ui.cursor.primary`, `ui.selection`, `ui.linenr`, `ui.statusline` and its mode variants, `ui.menu`, `ui.popup`, `ui.virtual.whitespace`, `ui.virtual.inlay-hint`, the `diagnostic.*` and `error/warning/info/hint` scopes. Styles take `fg`, `bg`, a `modifiers` list (`bold`, `italic`, `dim`, `underlined`, `reversed`, and more), and an `underline` with one of several styles (`line`, `curl`, `dashed`, `dotted`, `double_line`). A `[palette]` lets you name colors once and reuse them. The `theme` key in `config.toml` selects one, or a `[theme]` table picks separate `dark` and `light` schemes with a `fallback`. Everything lives under `runtime/` — themes in `runtime/themes/`, queries in `runtime/queries/`, grammars in `runtime/grammars/`, and the `tutor` lesson file alongside them.

Appendix D — Glossary

Terms used throughout this book, defined once.

- **anchor** — the fixed end of a **Range**; the end that stays put while the head moves.
- **AOT (ahead-of-time)** — compiled to native code before running. `zmax` and all five embedded interpreters are AOT-compiled Rust.
- **compositor** — the stack of UI layers (editor view, pickers, popups) rendered bottom to top; transient overlays live here.
- **defsubr** — `elisp`'s mechanism for registering a native Rust function as an `elisp` builtin; how editor operations become callable from `elisp`.
- **DAP** — Debug Adapter Protocol; the standard an editor uses to drive a debugger.
- **fusevm** — the shared, language-agnostic bytecode VM (224 opcodes, fused superinstructions, 3-stage Cranelift JIT) that all five embedded tongues compile to.
- **gap index** — a position counted between characters rather than on them; position 0 is before the first character.
- **head** — the moving end of a **Range**; the end a motion drags.
- **host callback** — a native function an embedded language calls to reach the editor; in `zmax` a bare `fn` pointer reaching the editor through a thread-local.
- **hunk** — a contiguous run of changed lines in a diff; navigated with `[c/]c`.
- **injection** — a region of one language embedded in another (SQL in a string, Rust in a Markdown fence), parsed with the inner grammar.
- **JIT (just-in-time)** — compiled to native code at runtime. `fusevm`'s JIT fires for dynamic `eval`, not for the AOT engine itself.
- **KeyTrie** — the trie structure that holds the keymap; a prefix key descends into a submap rather than running a command.
- **leader** — the `SPC` key opening the Spacemacs-style discoverable command tree.
- **LSP** — Language Server Protocol; how the editor gets semantic knowledge of code.
- **operator-pending** — vim's verb-then-motion state (`d` awaiting a motion); emulated in `zmax` as `[collapse, extend, operate]` command sequences.
- **picker** — a fuzzy-finding list overlay (files, buffers, symbols), powered by `nucleo`.
- **primary selection** — the one range in a multi-range **Selection** whose position the status line reports and whose viewport the screen follows.
- **Range** — a span of text with an anchor and a head; the irreducible unit of editing.
- **rope** — the tree-structured text representation (via `ropey`) giving cheap edits, structural sharing, and char-indexed access.
- **Selection** — a non-empty set of **Ranges**; every motion and edit acts on all of them.
- **textobject** — a syntax-defined selection (a function, an argument, a comment) from a grammar's `textobjects.scm`.
- **Transaction** — a composed set of changes applied to the rope, carrying its own position-remapping logic.
- **tree-house** — the tree-sitter wrapper `zmax` uses for incremental parsing, query iteration, and injection handling.
- **kill-ring** — the emacs ring of cut/copy entries; `C-y` yanks the head, `M-y` (yank-pop) cycles to older entries.
- **mark-ring** — the emacs per-buffer stack of recent mark positions; `C-x C-SPC` (pop-to-mark) walks back through it.
- **register (emacs)** — a one-character-keyed slot under `C-x r` holding a position, a number, or a region of text.

- **rectangle** — a column-spanning region between two corners; operated on by the emacs `C-x r` rectangle commands and by the `C-v` block mode.
- **narrowing** — restricting the editor to a region so it behaves as the whole buffer; `narrow_to_region` (`SPC n r`), reversed by `widen` (`SPC n w`).
- **indirect buffer** — a second view sharing one buffer's text, narrowable independently, so a region and the full file can be edited side by side.
- **paredit** — the structural-editing suite (`SPC k`) that edits the tree of an s-expression while keeping every parenthesis balanced.
- **layout** — a named arrangement of windows and buffers (`SPC l`); layouts group into a workspace, the Spacemacs persistent-workspace model.
- **winner mode** — undo/redo for the window configuration (`SPC w u` / `SPC w . U`), a history separate from the text undo history.
- **workbench** — the optional IDE chrome (`:ide`, `F2`) of tool windows — project tree, structure outline, problems, run console, minimap — around the same view.
- **keymap preset** — one of the three switchable base keymaps (`vim`, `helix`, `emacs`), chosen with `:keymap`; Spacemacs and JetBrains layer on top of the choice.
- **JetBrains layer** — the IntelliJ-modeled command vocabulary (refactorings, view toggles, tool windows) reached by name from the palette and the `SPC` leader.
- **typable command** — a `:` command (e.g. `:write`, `:json-to-csv`); over twelve hundred of them, each with aliases, a completer, and a signature (Appendix J).
- **static command** — a named keymap primitive (e.g. `move_char_left`, `paredit_slurp_forward`); around two thousand of them, bound to keys per mode (Appendix K).
- **leap** — a motion that labels every visible target and jumps on the label you type (`goto_word`, `SPC j l`); flat in distance.
- **harpoon** — a small, manually pinned, order-preserving set of files jumped to by number (`SPC H`).
- **freccy** — a frequency-and-recency ranking of recent files (`SPC f r`, the start screen) that surfaces the file you most likely want.
- **snippet** — a trigger word that expands on `Tab` to a parameterized body with `${1:...}/$0` tab stops; managed in the `:snippets` library.
- **emmet** — markup-abbreviation expansion (`ul>li*3` → a nested list), on `Tab` after user triggers, via `emmet_expand`.
- **drag-stuff** — sliding the current line or selection up/down past its neighbors (`SPC x . j/k`).
- **auto-reload** — vim `autoread`: reloading a buffer when its file changes on disk, keeping local edits on conflict.
- **rx form** — a regex written as a nested s-expression (`((one-or-more digit))`), convertible to/from PCRE and Emacs syntax under `SPC x r`.
- **black-hole register** — `"_`, which discards on write and returns nothing on read; used to delete without touching the yank.

Appendix E – From Helix and From Vim

zmax has two natural origin points — a vim user crossing over, or a Helix user staying on the same engine but flipping the keymap. This appendix is the short orientation for each.

Coming from Vim

Most of what you know transfers unchanged: the motions, the operators, the single-key edits, search, marks, registers, the `g/z/[ ]` prefixes, `C-w` windows, the ex commands. The genuine surprises are two. First, the engine is selection-first, so `n` after a search *selects* the match (not just moves to it), and multi-cursor editing is the default shape rather than a plugin — both gifts once you expect them. Second, Select mode *is* vim's visual mode, and block selection (`C-v`) is a real rectangle — the `BlockSelect` region of `zmax-view/src/editor.rs`, not the multi-cursor imitation earlier editions of this appendix warned about. The operator edits an earlier edition deferred — operator-plus-find (`df,`), operator-plus-text-object (`ciw, di()`), the `.` repeat, `q/@` macros, named marks, and a true Replace mode — are all wired now (Chapter 5), so the grammar in your fingers transfers more completely than this appendix once warned.

Coming from Helix

The engine is the one you know — rope, selections, tree-sitter, LSP, DAP, pickers, themes, registers, the undo tree — so nothing about how editing *works* has changed. What changed is the keymap: it is now vim, verb-before-noun, so `dw` deletes a word where Helix's `wd` did, and the operator-pending grammar of Movement II replaces the selection-first key order. If you prefer the original, the Helix selection-first keymap still exists in the source (`zmax-term/src/keymap/default.rs`) as the fallback — zmax flipped the default, it did not delete the alternative. Configuration is the familiar `config.toml` and `languages.toml` and `theme.toml`, with the same `runtime/` layout, so your Helix muscle memory for *configuring* the editor carries over even as your muscle memory for *driving* it switches to vim. And then there is the thing Helix never had at all: five languages embedded in the editor, the subject of Movement IV — the reason to make the move beyond the keymap.

Appendix F – Selected Typable Commands

The command line (Chapter 11) holds more than 340 typable commands — well over half of them the selection transforms of Chapter 57 — and the reference companion enumerates all of them with evidence (Chapter 29). This appendix is the working subset — the `:` commands you actually type — with their aliases, so the quick ones are at hand without the full catalog; for the transform shelf in particular, Chapter 57 is the map and the reference is the index.

Files, buffers, windows

<code>:write</code>	<code>:w</code>	write the buffer to disk
<code>:write-quit</code>	<code>:wq</code>	write and close the window
<code>:write-quit-all</code>	<code>:wqa :xa</code>	write all and quit
<code>:quit</code>	<code>:q</code>	close the window
<code>:quit!</code>	<code>:q!</code>	close, discarding changes
<code>:exit</code>	<code>:x</code>	write-if-modified, then quit
<code>:open <path></code>	<code>:e :o</code>	open a file
<code>:new</code>	<code>:n</code>	new scratch buffer
<code>:buffer-close</code>	<code>:bc</code>	close the current buffer
<code>:buffer-next</code>	<code>:bn</code>	next buffer
<code>:buffer-previous</code>	<code>:bp</code>	previous buffer
<code>:vsplit <path></code>	<code>:vs</code>	open in a vertical split
<code>:hsplit <path></code>	<code>:sp</code>	open in a horizontal split

Editing, search, and the ex commands

<code>:s/pat/rep/flags</code>	substitute (vim syntax; %s for whole file)
<code>:g/pat/cmd</code>	run cmd on every matching line
<code>:42 / :42:10</code>	jump to line 42 (and column 10)
<code>:sort</code>	sort the selected lines
<code>:reflow [width]</code>	reflow the selection to a width
<code>:format :fmt</code>	format via the language server
<code>!:cmd</code>	run a shell command
<code>:pipe cmd</code>	pipe the selection through a shell command
<code>:earlier / :later <n></code>	step through the undo history by time

Configuration and inspection

```
:set-option <k> <v>      set an editor option live
:toggle-option <k>       flip a boolean option
:theme <name>            load a theme
:config-open             open config.toml for editing
:config-reload           reload configuration
:character-info           describe the character under the cursor
:log-open                open the editor's log
:tutor                   open the interactive tutorial
```

Version control, merge, and tools

```
:magit      :git :gst  open the Magit-style status porcelain
:diff       :gdif     side-by-side diff of the buffer vs. git HEAD
:merge      :resolve  3-pane (ours/result/theirs) conflict resolver
:conflict-ours :conflict-theirs / :conflict-both keep a side at the cursor
:snippets   :snip     edit the user snippet library
:hex        :hexview  open the byte-faithful hex editor
```

Where two spellings share a line, either works — the long, self-documenting name while you are learning and the one-letter vim form once your hands are fast — from the same prompt, with the same completer guiding the arguments. For the complete, evidence-backed enumeration of every typable command, every static command, and every key, the reference companion (generated from the [port/](#) machinery of Chapter 29) is the exhaustive source; this appendix is the handful you reach for hourly.

The newer command families

The recent work added whole families of typable commands; these are the ones the new movements lean on, by area:

```
WORKBENCH and PANELS
:ide :workbench      toggle the IDE workbench (also F2, SPC z)
:terminal :term      a PTY shell in a pane (SPC p ')
:repl [lang]         the five-tongue REPL panel (SPC a r)
:preferences         the tabbed Preferences window (SPC ,)
:keymap <vim|helix|emacs> switch the active keymap preset

NARROWING and STRUCTURE
:narrow / narrow_to_region (SPC n r)  widen (SPC n w)
:hex :hexview :hexedit  the byte-faithful hex editor

ORG and SCRIPTING
:org-agenda :agenda     the date-aware agenda (SPC a o a)
:org-capture :capture   append a TODO to the inbox (SPC a o c)
:elisp :vim :awk :zsh :stryke  evaluate an embedded tongue over the buffer

EMBEDDED-LANGUAGE aliases
:el :eval-expression    elisp      :viml :vimscript  vimscript
:awk-filter             awk        :zshell      zsh        :st stryke
```

The transform library of Chapter 57 contributes well over two hundred more — the `:json-*`, `:csv-*`, `:to-snake/:to-camel`, `:align`, `:reflow`, `:sort-by-field`, `:extract-*`, `:base32/:caesar/:morse`, `:uuid/:lorem/:seq` families and the rest — each running on the selection or the whole buffer as one undoable change. They are too many to list here and are the reason the typable-command count crossed several

hundred; Chapter 57 is the map and the generated reference is the index.

Appendix G – The Leader Map

The Spacemacs `SPC` leader (Chapter 10) is the discoverable surface, and because `auto-info` shows each branch as you reach it you rarely need this table — but a map of the whole tree is useful once, to see the shape. These are the top-level branches and their most-used entries, grounded in `zmax-term/src/keymap/vim.rs`; the which-key popup is always the live, authoritative version.

The top-level branches

```
SPC SPC  command palette (run a command by name)
SPC f    files          SPC b    buffers          SPC w    windows
SPC p    project        SPC s    search          SPC j    jumps / leap
SPC g    git/magit      SPC e    errors          SPC c    comments
SPC l    layouts        SPC n    narrowing    SPC k    paredit (sexp)
SPC x    text ops       SPC m    major mode    SPC a    applications
SPC h    help           SPC D    ediff          SPC d    debug
SPC t    toggles        SPC K    macro ring    SPC R    run configs
SPC W    workbench      SPC H    harpoon       SPC y/p   clipboard
SPC i    generators     SPC T    themes        SPC F    frames/layouts
SPC z    IDE toggle     SPC ,    preferences   SPC S    settings
```

Selected branch contents

```
SPC f    r recent (frecency)  c copy-file  L locate  l find-literally  u reopen
          e ... project/shell env config
SPC b    n/p next/prev  d kill  N indirect buffer  M kill-by-regex  W goto by buffer
SPC w    s/v split  h/j/k/l focus  u winner-undo  . g golden  . a ace-window
SPC g    s status  b blame  m resolve-conflicts  c conflict menu  f file dispatch
SPC s    project search (seeded with symbol at point)  SPC * search word at point
SPC j    l leap word  w extend-to-word  s split-sexp
SPC x    t transpose (e/s/p)  . j/k drag line  a r align-at-regex  r rx regex forms
SPC m    <tab> org-cycle  t org-todo  p org-priority  e elisp eval  h h describe at point
SPC a    r REPL  f file explorer  o a org-agenda  o c org-capture
SPC t    g golden-ratio  s diagnostics  h s syntax  h a symbol-highlight  k k which-key
SPC d    C run-to-cursor  (and the DAP controls of Chapter 15)
SPC K    r n/p macro-ring next/prev  c a/c counter add/insert  e r to-register
```

The leader is wide on purpose: it is the home of the operations you reach for weekly rather than hourly, where discovery beats memorization and the half-second which-key popup is a bargain. The fast vim grammar of Chapters 6 and 7 is the other half — the operations you do constantly, where a popup would only slow you down — and the two surfaces are live at once, which is the Spacemacs wager Chapter 10 explained.

Appendix H – Embedded Scripting Reference

The five embedded tongues (Movement IV) are reached the same way — a typable command that evaluates source against the live buffer, or the REPL that fronts all five. This appendix is the working reference for both.

The evaluation commands

LANGUAGE	COMMAND (aliases)	PLATFORMS	INTERPRETER
Emacs Lisp	:elisp (:eval-expression, :el)	all	elisprs
Vimscript	:vim (:viml, :vimscript)	all	vimlrs
AwK	:awk (:awk-filter)	all	awkrs
zsh	:zsh (:zshell)	unix only	zshrs
stryke	:stryke (:st)	unix only	strykelang

:elisp, :vim, and :stryke evaluate source and keep state across calls; :awk filters the selection (or the whole buffer) through a program and replaces it as one undo step; :zsh runs a command in the embedded shell and shows captured output, with shell state persisting across calls. All five are gated behind the `scripting` Cargo feature, on by default; a `--no-default-features` build omits them and the commands report that scripting was not compiled in.

The REPL panel

SPC a r (or :repl [lang]) opens a full-screen REPL fronting all five tongues:

Enter	evaluate	Alt-Enter	insert a newline
Tab / S-Tab	cycle the active language		
Up / Down	or C-p / C-n		per-language history
C-l	clear the transcript		
PgUp / PgDn	scroll	Esc	close

:repl awk (and so on) opens directly on a chosen language; per-language input history persists to `~/.zmax/repl-history.toml`.

Startup scripts

At launch, zmax sources two files from `~/.zmax/` if present, best-effort, with errors surfaced on the status line:

```
init.el    evaluated as Emacs Lisp
init.vim   evaluated as Vimscript
```

These are the door to configuring the editor in the language of the keymap your hands already know — and, as Chapter 71 recorded, the vimscript side now runs real control flow and the bulk of the builtin library, with the editor-mutating ex-commands the remaining frontier.

What each tongue reaches today

The five tongues are at different stages of *driving* the editor (Chapters 21 and 71), and the honest one-line status of each, by crate, is worth keeping in the reference:

```
elisp    elisprs    22 defsubrs (message/command/buffer/mark/kill-ring); buffer TEXT is
                                mirrored in+out per eval, not shared. No keymap bridge
vimscript vimlrs    eval + source; 100% of neovim builtins; :let/:if/:for/:try/:echon
↳ run;

stryke    strykelang    host hooks for :set, :map, :command, :highlight, :colorscheme + an
                                ex-cmd passthrough. Only :autocmd lacks a hook
awk        awkrs        parse_and_run_string + VMHelper; native host-fn registration still
                                pending, so no api_* access; print suppressed (fd layer)
zsh        zshrs        run_program; filter a selection through a program; fusevm AWK opcodes
↳ persists    ShellExecutor + execute_script behind with_captured_fds; state
```

The throughline is the shared machine of Chapter 19: all five lower to one fusevm, so the work of maturing each embedding is front-end work — giving a tongue more host functions, or a library-shaped entry point — rather than building a new runtime. elisp is furthest, with a clean `defsubr` surface the editor is already wired to; vimscript moved most in the recent work; the others run their language and are growing the host surface that turns “runs in the editor” into “drives the editor.” The book tracks the progress rather than freezing a snapshot, and the per-crate line above is the current state to update against.

Appendix I – Capability Index

The port report’s primary measure (Chapter 29) is *functionality coverage* — distinct editor capabilities, deduplicated, regardless of how many ancestor editors expose them — and it tracks 186 of them across the areas below, of which 185 are implemented (99.5%). This appendix is that table read as an index: each area, its coverage, and the representative commands or keys this book has cited for it, so you can find “does zmax do X, and how” by area rather than by tradition. The percentages are the report’s at the time of writing and move as the source does.

Areas at full or near-full coverage

AREA	%	representative commands / keys
clipboard	100	kill-ring (C-y/M-y), registers, :clipboard-*, SPC y/p
debug	100	DAP (Chapter 15), dap_run_to_cursor (SPC d C)
diagnostics	100	[d /]d, the PROBLEMS panel (SPC W e), SPC e
diff-merge	100	:diff, ediff (SPC D), :merge,]n/[n, conflict-*
files	100	:open, :w, file picker (SPC f), :move/:delete-file
folding	100	za/zo/zc/zf, fold_open_recursive/_close_recursive
macros	100	q/@ record-replay, the SPC K ring + counter
marks	100	m{a-z}/^{a-z}, the jumplist (C-o/C-i), mark-ring
motion	100	h j k l, w/b/e, f/F/t/T, gg/G, leap (SPC j l)
org	100	SPC m outline, SPC a o agenda/capture
project	100	:grep/:rg, :todos, the PROJECT tree (SPC W t)
scripting	100	:elisp/:vim/:awk/:zsh/:stryke, the REPL (SPC a r)
snippets	100	:snippets, Tab expand, goto_next/prev_tabstop
spell	100	z= suggest, zg/zw good/bad,]s/[s
structural	100	paredit (SPC k), wildfire (<ret>/<bs>)
terminal	100	:terminal, the C-\ window leader
text-objects	100	ciw/di(/ya{, textobjects.scm, wildfire
transforms	100	the transform shelf of Chapter 57
ui	100	themes, gutters, rulers, inlay hints, minimap
vcs	100	magit (SPC g), blame, hunks, branches/stash
lsp	100	gd/gr/gy/gi, rename, code actions
search	100	/, *, :s, :g, project grep
windows	100	C-w splits, SPC w, layouts
selection	100	set-of-ranges native, multi-cursor
refactoring	100	extract/introduce/inline, Refactor This (Chapter 59)

The one area not at full coverage

AREA	%	note
editing	95.2	20 of 21; one editing capability uses a different model

The shape of the table is the honest picture the per-source percentages of Chapter 29 obscure: by *capability*, zmax is a complete editor bar one row. Refactoring, which earlier editions of this book named as the frontier, is at 100% in both its *functionality* and *jetbrains* rows. Read this index for “can it do X”; read the per-source tables for “does my muscle memory from editor Y transfer.” They answer different questions, and

the project publishes both. As always, [docs/port_report.md](#) is the live version of this table and this page is a snapshot of it.

Appendix J – The Typable Command Catalogue

Appendix F was the handful of `:` commands you reach for hourly; this is the wider tour. The command line registers **1,241 typable commands** (2,448 names once aliases are counted), generated into [book/src/generated/typable-cmd.md](#) and looked up by name or alias when you press Return. This appendix walks them by family — not every leaf, but the shape of each shelf, so you know what is there to reach for. Every name below is the real registered spelling; where two or three share a slash, they are aliases for one command.

Files, buffers, and views

```
:open :o :edit :e      open a file into the current view
:new :n                new scratch buffer
:reopen :reopen-closed reopen the most recently closed file
:buffer-close :bc      close the current buffer ( ! to force )
:buffer-close-others :bco close all but the focused buffer
:buffer-close-all :bca close every buffer without quitting
:buffer-next/-previous :bn :bp step between buffers
:vsplit :vs :hsplit :sp split the current view (optionally on a file)
:cd :pushd :popd :dirs change / stack the working directory ( :pwd shows it )
:move :delete-file     rename / delete the file with the buffer
:mkdir :make-executable make a directory / chmod +x the file
:read                  read another file's contents into the buffer
```

Writing and quitting – the full matrix

vim's write/quit verbs are present in their entirety, each with a `!` force variant that also creates missing parent directories on write:

```
:write :w              write ( :write! forces + mkdirs )
:write-quit :wq         write and close the view
:write-buffer-close :wbc write and close the buffer (keep the view)
:write-all :wa         write every modified buffer
:write-quit-all :wqa :xa write all and close all views
:quit :q :quit-all :qa close the view / all views
:exit :x :xit           write-if-modified, then quit
:cquit :cq             quit with an exit code (default 1)
```

History, formatting, and inspection

```
:earlier :ear :later :lat    step through edit history by count or time span
:format :fmt                format via formatter or language server
:indent-style               set tabs ('t') or 1-16 spaces
:line-ending                set lf / crlf for the document
:config-open :config-reload open / reload config.toml ( :config-open-workspace )
:set-option :toggle-option  change an editor option live
:theme                      load a color scheme (current if no name)
:character-info :wc :words :count inspect the character / count lines-words-chars
:log-open :tutor            the editor log / the interactive tutorial
```

Search, substitution, and the ex-commands

The command line special-cases vim's ex syntax (Chapter 11) rather than tokenizing it:

```
:s/pat/rep/flags            substitute ( %s for the whole file )
:g/pat/cmd                  run a command on every matching line
:42 :42:10                  jump to a line ( and column )
:goto-offset                jump to a character offset
:!cmd                       run a shell command :pipe cmd pipe the selection through
                             ~ one
:count-matches :count-regex count pattern hits ( non-destructive )
:rename-word :rename-local  textual / scope-limited rename
:find-references             textual references into the Run console
```

LSP, diagnostics, and version control

```
:lsp-restart :lsp-stop      restart / stop the language servers
:lsp-workspace-command      run a server-provided workspace command
:yank-diagnostic             copy the diagnostic at the cursor
:magit :git :gst            the Magit-style status porcelain
:blame                       git blame for the current line
:diff :gdiff                read-only side-by-side vs git HEAD
:merge :resolve              the 3-pane conflict resolver
:conflict-ours / -theirs / -both keep a side at the cursor
```

The transform library – structured data

This is a deep family, though not the largest one — that distinction belongs to the embedded-hardware commands, where `pio-*` alone runs to 170 names and `arduino-*` to 88, dwarfing the transform shelf's couple of dozen. The typable surface as a whole is well past a thousand: the generated `typable-cmd.md` lists 1,241 primary names carrying a further 1,207 aliases. The JSON shelf alone is deep:

```
:json-query :json-get        pull a value at a dot-path
:json-flatten / :json-unflatten path = value lines and back
:json-pick :json-omit        keep / drop keys
:json-sort :json-unique :json-group-by reshape arrays
:json-keys :json-paths :json-values-of introspect structure
:json-to-csv / -toml / -lines / -jsonl / -env / -kv convert out
:csv-to-json / :toml-to-json / :jsonl-to-json / :kv-to-json convert in
:json-table :json-describe :json-validate render / summarize / check
```

CSV, TSV, and tables

```
:csv-column :csv-col      keep the Nth column
:csv-to-tsv :tsv-to-csv   convert delimiters
:csv-to-html-table :md-table  render a grid / align a Markdown table
:transpose-grid           rows become columns
:csv-validate            every row has the same field count
:lines-to-csv-row :csv-row-to-lines  reshape a single row
```

Numbers and statistics

```
:stats :sum                count / sum / mean / min / max ( non-destructive )
:running-total :running-max/-min  cumulative values per line
:percent-of-total :scale :offset :clamp :to-fixed :abs  per-line arithmetic
:sum-fields :sum-column :row-sum  reduce each row to one number
:bases :base-info           an integer in dec / hex / oct / bin
:calc                    evaluate an arithmetic expression
:increment-numbers :pad-numbers :zero-pad  number-in-text helpers
:human-bytes :ordinal :dec-to-hex :hex-to-dec  format conversions
:count-unique :uniq-count :distinct-count     frequency counts
```

Identifiers, lines, and text

```
:to-snake :to-kebab :to-camel :to-pascal :to-constant  identifier case
:cycle-case :change-case :swapcase :invert-case      rotate / flip case
:align :tabularize           align on a delimiter
:reflow :unhardwrap :unwrap-paragraphs  rewrap text to a width
:sort :sort-by-field :sort-by-length :natural-sort :sort-words  the sort family
:dedup :dedup-adjacent :dedup-words     remove duplicates
:shuffle :sample :rotate-lines :reverse-lines  reorder
:number-lines :numbered-list :strip-line-numbers  line numbering
:trim-lines :squeeze-blank-lines :pad-left :pad-right  tidy
:head :tail                  keep the first / last N lines
```

Extraction, encoding, and generators

```
:extract <regex>           one match per line
:extract-urls / -emails / -ips / -numbers / -quoted / -between  targeted pulls
:base32-encode / -decode :crc32 :checksum           hashing / encoding
:caesar :rot47 :morse-encode / -decode :nato :hexdump  classic encoders
:to-html-list :html-list-to-lines :wrap-tag :wrap-with  markup
:uuid :lorem :lipsum :date :datetime :seq :sequence  generators
```


Org, workbench, and the embedded tongues

```
:org-cycle / -todo / -priority / -promote / -demote      outline editing
:org-agenda :agenda      :org-capture :capture          plan and capture
:ide :workbench :zen      the IDE workbench ( Chapter 55 )
:terminal :term          a PTY shell      :repl [lang]    the five-tongue REPL
:hex :hexview :hexedit    the byte-faithful hex editor
:snippets :snip           the snippet library editor
:emmet :zencode           expand an emmet abbreviation
:elisp :vim :awk :zsh :stryke    evaluate an embedded language over the buffer
```

The list is long because the philosophy of Chapter 57 is to make the common chore a command rather than a program — and the moment a chore needs *logic* the pre-built command lacks, you drop to one of those five embedded tongues on the last line. The full, evidence-backed enumeration lives in the generated reference; this appendix is the map of its neighborhoods.

Aliases, completion, and signatures

Two properties of the typable system make the long catalogue usable rather than overwhelming. The first is **aliases**: nearly every command has a long, self-documenting name and one or more short vim-style forms, and they are interchangeable from the same prompt — `:write` and `:w`, `:buffer-next` and `:bn`, `:format` and `:fmt`. You type the long name while you are learning a command and the short one once your hands are fast, and the editor never makes you choose between discoverability and speed. The second is that each command carries a **completer and a signature** (Chapter 11): the command line completes filenames where a path is expected, theme names where a theme is expected, option keys where `:set-option` wants one, and it knows how many positional arguments a command will accept. So the prompt is not a bare text box — it guides you toward the valid spelling as you type, which is what lets a catalogue this size stay navigable: you do not have to *remember* the command, only enough of it to start the completion. This is the same discoverability ethic the leader and the help browser embody (Chapter 64), applied to the typed surface.

Appendix K – The Static Command Surface

Typable commands are what you type after `:`; **static commands** are the named primitives the keymap binds to keys — `move_char_left`, `delete_selection`, `paredit_slurp_forward`. There are **2,010** of them, generated into `book/src/generated/static-cmd.md` with their descriptions and their default bindings per mode, and they are the vocabulary every keymap (vim, helix, emacs) and every leader menu ultimately calls. This appendix walks the families; the generated table is the per-key authority (and the source of the port report’s numerator, Chapter 29).

Motion and selection

The motions you met in Chapter 6 are static commands with vim bindings attached:

```
move_char_left/right  move_line_up/down      h j k l (and visual-line variants)
move_next_word_start  move_prev_word_start  move_next_word_end    w b e
goto_line_start       goto_line_end         goto_first_nonwhitespace 0 $ ^
goto_file_start       goto_last_line        gg G
find_next_char        find_prev_char        till_next_char          f F t T
goto_column           extend_to_column       |
```

Selection manipulation

This is where the engine’s set-of-ranges nature surfaces as commands, many unbound by default and reached through the leader or a remap:

```
rotate_selections_forward/backward  cycle which range is primary
rotate_selection_contents_forward/backward rotate the *text* among ranges
rotate_selections_first/last        make the first/last range primary
trim_selections                     tidy the selected spans
align_selections                     align ranges in a column ( SPC x a {delim} )
split_selection                     split_selection_on_newline keep_primary_selection remove_primary
```

The `align_selections` binding is a small lesson in the leader’s density: it is bound to `SPC x a` followed by any of a set of delimiter keys — `=`, `:`, `,`, `(`, `{`, and more — so “align on the equals sign” is `SPC x a =` and “align on the colon” is `SPC x a :`, one command parameterized by its final key.

View, scrolling, and windows

```
align_view_top/center/bottom  zt zz zb      ( zz also on C-l )
scroll_column_left/right      zh zL          scroll_half_column_left/right  zH zL
resize_view_narrower/wider    C-w [ / C-w ] and the SPC w mirrors
rotate_view                   next / previous window ( C-w w / C-w W )
```

Registers, marks, and macros

```
select_register      "      insert_register      C-r
register_picker      browse and paste a register ( SPC r e/r/y )
copy_between_registers  move text from one register to another
vim_record_macro    vim_replay_macro      q{reg} / @{reg}
```

Structural editing: paredit, narrowing, folds

The paredit suite of Chapter 62 is a clean family, each command carrying its `SPC k` binding in its own description:

```
paredit_slurp_forward/backward  SPC k s / S      paredit_barf_forward/backward  SPC k b /
~ B
paredit_splice                  SPC k W          paredit_raise                  SPC k r
paredit_transpose               SPC k t          paredit_absorb / _convolute    SPC k a /
~ c
paredit_splice_kill_forward/backward SPC k e / E  paredit_insert_sexp_after/before SPC k
~ ) / (
narrow_to_region                SPC n r          ( with the SPC n family of Chapter 61 )
fold_toggle / fold_open / fold_close ( and _all variants )  za zo zc
```

The transform statics

Many transforms exist as *static* commands too, distinct from their `:` spellings, so the leader's `SPC x` menu can reach them directly:

```
json_pretty_selection  json_minify_selection  json_escape_selection / _unescape
to_json_string_selection  csv_to_json_selection  transpose_csv_selection
format_table_selection    align the selected Markdown table
jwt_decode_selection      decode a JWT into header + payload JSON
```

Workbench and tool windows

```
toggle_idc ( F2, :ide )      toggle_drawer_mid ( SPC W f )
hide_active_tool_window  jump_to_last_tool_window  focus_bookmarks
document_outline  reveal_in_tree  the STRUCTURE and PROJECT surfaces
toggle_soft_wrap  toggle_line_numbers  toggle_indent_guides  toggle_whitespace_render
toggle_inlay_hints  fold_open_recursive  fold_close_recursive
```

How a binding proves itself

The reason this catalogue is trustworthy is the mechanism of Chapter 29: every default binding shown in the generated table is read out of the keymap source, and every port-report claim that “zmax covers vim's `dw`” carries a `static:<command>` or `key:<mode>:<chord>` reference that must resolve to real code or the report fails. So the static command surface is not a hand-maintained list that can rot — it is re-derived from `commands.rs` and the keymap on every run. When this appendix says a command exists, the generated reference can point at the line that registers it.

Commands without a default key

A large share of the static commands ship with *no* default binding — their entry in the generated table has an empty “Default keybinds” column — and that is by design rather than omission. The keyboard has only so many ergonomic chords, and binding every primitive would mean a keymap no one could learn; so the common operations get keys, and the rest wait to be reached. There are three ways to reach an unbound command. You run it by name through the command palette (**M-x**, **SPC SPC**), which is how the JetBrains refactorings of Chapter 59 are invoked. You bind it yourself in **[keys]** (Appendix N), the one-line TOML that maps a key to any static command. Or you reach it through a leader menu that already routes to it, the way **SPC x a =** reaches **align_selections**. The unbound commands are therefore not hidden — they are *available*, just not pre-assigned, which keeps the default keymap teachable while leaving the full primitive set one palette search or one config line away. This is the same trade the whole editor makes: a small, learnable default surface over a large, reachable total surface, with the gap bridged by discovery rather than memorization.

Appendix L – The Language Support Matrix

Appendix C covered the *format* of `languages.toml`; this appendix covers the *breadth*. zmax ships syntax and tooling support for **around 350 languages** out of the box, tracked in the generated `lang-support.md`, and the support is not uniform — a language can have any subset of five capabilities, each driven by a different `.scm` query or a configured server. Knowing which language has which is the difference between expecting `gd` to work and being surprised it does not.

The five capability columns

Each language's row records five independent capabilities, every one a separate piece of data under `runtime/` rather than editor code:

Syntax Highlighting	a <code>highlights.scm</code> query – the table stakes; nearly every language
Treesitter Textobjects	a <code>textobjects.scm</code> query – function/class/parameter selections, the structural editing of Chapter 13
Auto Indent	an <code>indents.scm</code> query – syntax-driven indentation on a new line
Code Navigation Tags	a <code>tags</code> query – symbol outline and the structure panel
Rainbow Brackets	a <code>rainbows.scm</code> query – depth-colored brackets (off by default)
Default language servers	a <code>[language-server]</code> config – the LSP that powers <code>gd/gr/hover</code>

A language with all five plus a server is a first-class IDE experience; a language with only the first is a syntax-highlighted buffer with no semantic tooling. The honest reading of the matrix is that the long tail is highlight-only, while the languages people actually build in cluster at the full-capability end.

The full-stack languages

These carry all five queries *and* a default language server — the complete experience, where every chapter of this book applies unchanged:

amber	bash	c	c-sharp	clojure	cpp	crystal	dart	docker-bake	elixir	
erlang	glsl	go	java	javascript	jsx	julia	kotlin	lua	nix	perl
php	python	r	ruby	rust	scala	svelte	swift	toml	tsx	typescript
wgsl	woodpecker-ci	zig	gitlab-ci	github-action						

A handful more have all five queries but no bundled server — `kdl`, `picat`, `scheme`, `unison`, `ron` — so they get structural editing and rainbow brackets and a structure outline, just without LSP semantics until you point one at them in `languages.toml`.

A representative slice

To see the shape of a row, here is the matrix for a spread of common languages (✓ = present):

language	syntax	textobj	indent	tags	rainbow	server
rust	✓	✓	✓	✓	✓	rust-analyzer
python	✓	✓	✓	✓	✓	ty / ruff / pylsp / ...
go	✓	✓	✓	✓	✓	gopls
typescript	✓	✓	✓	✓	✓	typescript-language-server
c	✓	✓	✓	✓	✓	clangd
java	✓	✓	✓	✓	✓	jdtls
ruby	✓	✓	✓	✓	✓	ruby-lsp / solargraph
haskell	✓	✓		✓	✓	haskell-language-server
markdown	✓			✓		marksman / markdown-oxide
json	✓	✓	✓		✓	vscode-json-language-server
yaml	✓	✓	✓		✓	yaml-language-server / ansible
toml	✓	✓	✓	✓	✓	taplo / tomls
elisp	✓			✓		

Note the last row: `elisp` ships syntax and navigation tags but no server — which is fair, because the editor's *own* elisp interpreter (Movement IV) is how you run it, not a foreign language server. Several config and markup formats — `git-config`, `dockerfile`, `css`, `html`, `sql` — sit in the middle, with highlighting and some queries but uneven indent or server coverage, exactly as you would expect from formats no one writes a full IDE for.

The five embedded tongues in the matrix

Four of the five embedded languages of Movement IV appear here as ordinary rows — `awk` (with `awk-language-server`), `bash`/zsh-adjacent shells, `vim` (highlighting plus rainbow brackets), `perl` (the full stack, since stryke is Perl-compatible) — because being *embedded as an interpreter* and being *supported as a language to edit* are two different things, and `zmax` does both. You can edit an AWK script with tree-sitter highlighting and the `awk` language server, and you can *run* that AWK over a selection with the embedded `awkr`, and the two facts are independent rows of capability.

Coverage at a glance

Reading down the 336-row matrix, a rough shape emerges that is worth stating because it sets honest expectations. Syntax highlighting is nearly universal — it is the entry price, and all but a few rows have it. Tree-sitter textobjects and auto-indent are present for the large majority of languages people actively program in and thin out across the config and markup formats. Code-navigation tags and a bundled language server are the scarcer columns, concentrated on the few dozen languages with mature server ecosystems — `rust-analyzer`, `gopls`, `clangd`, `pylsp`, `typescript-language-server`, `jdtls`, and their kin. Rainbow brackets, off by default, are wired for the bracket-heavy languages where nesting depth is hardest to read by eye. The practical takeaway: if you work in a mainstream language, expect the full experience — highlighting, structural editing, indent, outline, LSP — and if you open something exotic, expect at least correct highlighting, with the richer capabilities arriving as someone writes the `.scm` queries for it, which they can do without touching the binary. The matrix is a living document precisely because its cells are data files, and the empty cells are invitations rather than verdicts.

Adding to the matrix

Because every capability is a data file under `runtime/`, extending the matrix never touches the editor's source. Adding a language is, in the usual case, three things: a tree-sitter grammar under `runtime/grammars/`, a set of `.scm` queries under `runtime/queries/<name>/` (any subset of `highlights`, `textobjects`, `indents`, `tags`, `rainbows`), and a `[[language]]` entry in `languages.toml` naming the file types, roots, and any servers. Improving an existing language's row is the same move at finer grain — write the missing

`textobjects.scm`, and that language gains the structural editing of Chapter 13 with no recompile. The matrix grows by data, which is why 336 is a number that keeps climbing between releases without the binary changing shape.

Appendix M – The Configuration Reference

Appendix A was the configuration *overview* — the keys you reach for. This appendix is the deeper reference: the `[editor]` sub-section tables as they stand in `zmax-view/src/editor.rs` and `book/src/editor.md`, with real defaults, for the times you want to know exactly which key parses and what it does. Unknown keys are an error, so the spelling here is the spelling that works. Everything lives at `~/.config/zmax/config.toml`, merged over by a workspace-local `.zmax/config.toml`.

More of the top-level `[editor]` table

Beyond the keys Appendix A showed, the `[editor]` table carries the editing-behavior and display knobs you tune once and forget:

```
[editor]
scrolloff = 5           # padding lines kept around the cursor
scroll-lines = 3        # lines per scroll-wheel step
mouse = true            # mouse support
middle-click-paste = true # X11 primary-selection paste on middle click
shell = ["sh", "-c"]    # shell for :! and the filter operator
continue-comments = true # extend the comment token onto a new line
idle-timeout = 250      # ms before idle timers (hover, etc.) fire
completion-timeout = 250 # ms after a word char before completion shows
completion-trigger-len = 2 # word length that triggers autocompletion
completion-replace = false # replace the whole word, not just the prefix
auto-info = true        # the which-key / auto-info popups
bufferline = "never"    # "always" | "never" | "multiple"
color-modes = false     # tint the mode indicator per mode
text-width = 80         # used by :reflow and wrap-at-text-width
insert-final-newline = true
trim-trailing-whitespace = false
trim-final-newlines = false
atomic-save = true      # write via temp-file + rename
indent-heuristic = "hybrid" # "simple" | "tree-sitter" | "hybrid"
jump-label-alphabet = "abcdefghijklmnopqrstuvwxyz" # leap labels
end-of-line-diagnostics = "hint"
editor-config = true     # honor .editorconfig files
kitty-keyboard-protocol = "auto"
```

`[editor.statusline]`

The status line is divided into left, center, and right zones, each a list of named elements:


```
[editor.statusline]
left = ["mode", "spinner", "file-name", "read-only-indicator",
  ~ "file-modification-indicator"]
center = []
right = ["diagnostics", "selections", "register", "position", "file-encoding"]
separator = "|"
mode.normal = "NOR"           # the text shown for each mode
mode.insert = "INS"
mode.select = "SEL"
```

The elements you can place include `mode`, `spinner` (LSP activity), `file-name` and its `file-absolute-path` / `file-base-name` variants, `file-modification-indicator` (the [+]), `read-only-indicator`, `diagnostics` and `workspace-diagnostics`, `selections`, `primary-selection-length`, `position` and `position-percentage`, `version-control` (the branch), `register`, `code-action-hint`, and `spacer` to pad between them.

[editor.lsp]

```
[editor.lsp]
enable = true
display-messages = true           # window/showMessage under the statusline
display-progress-messages = false
auto-signature-help = true        # parameter hints popup
auto-document-highlight = false  # highlight references at the cursor
display-inlay-hints = false       # the toggle_inlay_hints default state
display-color-swatches = true
display-signature-help-docs = true
snippets = true                   # snippet completions (needs :lsp-restart)
goto-reference-include-declaration = true
```

[editor.cursor-shape]

```
[editor.cursor-shape]
normal = "block"                 # block | bar | underline | hidden
insert = "bar"
select = "underline"
```

The pickers: file, explorer, buffer

The file *picker* defaults to a developer's instincts in a git repo — skip the noise; the file *explorer* defaults the other way, showing most files, because a tree is for seeing the directory as it is:

The gutter components are `diagnostics` (also breakpoints and code-action hints), `line-numbers`, `diff` (the change bars of Chapter 65, themed by `diff.plus/diff.minus/diff.delta`), `spacer`, and `code-action-hint`.

Soft wrap, smart tab, and inline diagnostics

```
[editor.soft-wrap]
enable = false
max-wrap = 20
max-indent-retain = 40
wrap-indicator = "↵"
wrap-at-text-width = false # wrap at text-width rather than the viewport

[editor.smart-tab]
enable = true # Tab runs move_parent_node_end when text is to the left
supersede-menu = false # let Tab cycle menus rather than always smart-tab

[editor.inline-diagnostics]
cursor-line = "warning" # min severity shown inline on the cursor's line
other-lines = "disable" # ... and on other lines
prefix-len = 1
max-diagnostics = 10
```

Word completion and workspace trust

```
[editor.word-completion]
enable = true
trigger-length = 7 # word chars before completing from open buffers

[editor.workspace-trust]
level = "servers" # "none" | "servers" | "insecure"
prompt = true # modal when opening an untrusted workspace
trusted = [] # glob patterns trusted without a grant
```

The `workspace-trust` table is the gate Chapter 45 described: `servers` (the default) trusts LSP and DAP launches but still requires `:workspace-trust` before honoring a workspace-local `.zmax/config.toml` or `languages.toml`, `none` prompts for everything, and `insecure` trusts anything under a matching path — discouraged, and the statusline keeps its `[⚠]` indicator either way. This is the whole of the `[editor]` schema you will reach for; the rest is leaves, and the generated `editor.md` is the exhaustive list.

Putting the sub-sections together

To see how the tables compose, here is a coherent configuration that touches the sub-sections this appendix covered — not the exhaustive setup of Chapter 47, but a focused example of the display-and-behavior knobs working together:

```

theme = "onedark"

[editor]
line-number = "relative"
cursorline = true
rulers = [80, 120]
bufferline = "multiple"
color-modes = true
text-width = 100

[editor.statusline]
left = ["mode", "spinner", "version-control", "file-name"]
right = ["diagnostics", "selections", "position", "file-type"]
mode.normal = "NORMAL"
mode.insert = "INSERT"
mode.select = "SELECT"

[editor.lsp]
display-inlay-hints = true
auto-document-highlight = true

[editor.indent-guides]
render = true
character = "|"

[editor.soft-wrap]
enable = true
wrap-at-text-width = true

[editor.inline-diagnostics]
cursor-line = "hint"
other-lines = "warning"

[editor.cursor-shape]
normal = "block"
insert = "bar"
select = "underline"

```

Read it top to bottom and it is a legible portrait of a setup: relative line numbers and a highlighted cursor line, rulers at the column limits, the buffer line shown when more than one file is open, a mode-colored status line carrying the git branch, inlay hints and reference highlighting on, dotted indent guides, soft wrap at the 100-column text width, and inline diagnostics tuned to show hints on the cursor line and warnings elsewhere. Every key is from the tables above, every value parses, and the whole thing is optional — the editor behaves well with none of it, and this is just bending the defaults to a taste.

Appendix N – Remapping and the [keys] Config

The book has argued throughout that zmax needs no configuration to be useful — but “needs none” is not “allows none,” and when you do want to bend a key, the mechanism is a TOML [keys] table. This appendix is that mechanism: one-way key remapping, the three kinds of thing a key can run, minor modes, and the modifier and special-key spellings that parse. It is deliberately small, because the design goal is that you reach for it rarely.

Three things a key can run

A binding's right-hand side is one of three kinds, and knowing which is which is the whole of the syntax:

```
static command  a primitive like move_char_right ( Appendix K )
typable command a : command like :write!         ( Appendix J )
macro          a key sequence prefixed with @, run in order, e.g. @miw
```

Static commands are the movement-and-edit primitives the keymap normally binds; typable commands are everything you can run from `:`; and a macro is literally a string of keys replayed, so `@miw` selects the surrounding word by pressing `m`, `i`, `w`. Macros may not appear inside a binding *sequence* (a list), a limitation of how command sequences run, and modifier keys inside a macro are wrapped in angle brackets — `@miw<A-o>`.

The three mode tables

You bind under `[keys.normal]`, `[keys.insert]`, or `[keys.select]`, one section each:

```
[keys.normal]
C-s = ":w"                # Ctrl-s -> the :write alias
C-o = ":open ~/.zmax/config.toml" # jump to your config
a = "move_char_left"      # rebind a plain letter
"[C-S-esc]" = "extend_line" # Ctrl-Shift-Escape
"ret" = ["open_below", "normal_mode"] # a sequence: run both in order
"A-x" = "@x<A-d>"         # a macro: select the line, delete without yanking

[keys.insert]
"A-x" = "normal_mode"
j = { k = "normal_mode" } # a nested table = a minor mode (jk to exit insert)
```

A binding can be a single command, a *list* of commands run in order (the `ret` line), or a *nested table*, which creates a minor mode.

Minor modes

A nested table under a key turns that key into a prefix — a minor mode with its own bindings, exactly the mechanism the leader and the `g/z` prefixes use:

```
[keys.normal.g]
a = "code_action"          # ga -> code actions

[keys.normal.z]             # invert j/k inside the z view-mode
j = "scroll_up"
k = "scroll_down"

[keys.normal."+" ]         # a brand-new minor mode on '+'
m = ":run-shell-command make"
c = ":run-shell-command cargo build"
t = ":run-shell-command cargo test"
```

The last block is the idiom worth remembering: a single unused key — `+` here — becomes a personal command menu, `+m/+c/+t` running your build and test commands, with no plugin and no Lua, just three lines of TOML reaching the same `:run-shell-command` the rest of the editor uses.

Modifiers, the super key, and special keys

Modifiers are prefixes: `C-` for Ctrl, `S-` for Shift, `A-` for Alt, combined with a dash (`C-S-esc`). The super key — Windows/Command — is `Meta-`, `Cmd-`, or `Win-`, all synonyms, when the terminal supports the enhanced keyboard protocol:

```
[keys.normal]
C-s = ":write"
Cmd-s = ":write" # Cmd / Win / Meta + s
```

Named keys are spelled as strings: `"ret"` (Enter), `"space"`, `"backspace"`, `"tab"`, `"esc"`, `"del"`, `"ins"`, `"home"`, `"end"`, `"pageup"`, `"pagedown"`, the arrows (`"left"/"right"/"up"/"down"`), and `"lt"/"gt"` for the angle brackets. Punctuation keys bind literally — `"?"`, `"!"`, `"-"` — except that a modifier plus dash must be written `A-minus` rather than `A--`. A key is disabled by binding it to `no_op`, the do-nothing command. That is the whole surface: one-way, TOML, three kinds of target, nested for minor modes — enough to bend the editor to your hands without a configuration language, which is exactly the amount of power the zero-configuration design wants to expose.

Appendix O – Registers in Full

Chapter 17 introduced registers and Chapter 60 added the emacs `C-x r` family; this appendix is the complete model — user-defined registers, the four defaults, and the special registers whose read or write side does something other than store text. Registers are storage slots for text and search results, selected with `"` exactly as in vim, and they are one of the places the editor's set-of-ranges nature shows through.

User-defined registers

Any letter is a register you fill and read with `"`:

```
"ay      yank the selection into register a
"op      paste register o after the selection
"hc      store the selection in register h, then change it
"md      store the selection in register m, then delete it
```

The pattern on the last two lines is the one worth internalizing: selecting a register *before* a change or delete stashes the selection into that register and *then* performs the operation, so `"md` is “delete this, but keep a copy in `m`” — the named-register cut that vim users reach for to avoid clobbering the default yank.

The default registers

When a command that uses a register is given none, it falls back to a default, and a few registers are filled automatically by the editor:

```
/      the last search pattern
:      the last executed command
"      the last yanked text ( the unnamed default )
@      the last recorded macro
```

These are read the same way — `"/p` pastes your last search pattern, `":p` your last command — which makes them a quiet source of text you did not have to re-type.

The special registers

Six registers do something special on read or write, and they are where registers stop being mere storage and become an interface to the editor's state:

```
_      read: nothing      write: discards everything ( the black hole )
#      read: selection indices ( first is 1, second 2, ... )      write: not allowed
.      read: the contents of the current selections      write: not allowed
%      read: the current file's name      write: not allowed
+      read: the system clipboard      write: joins + yanks to the system clipboard
*      read: the primary clipboard      write: joins + yanks to the primary clipboard
```

The black-hole register `_` is the one you reach for constantly without thinking — `_"d` deletes without touching the yank, the same job `"md` did but throwing the copy away rather than keeping it. The `#`, `.`, and `%` registers are read-only windows onto editor state: `#` gives you the selection indices (useful when a multi-cursor edit

needs to know which range is which), `.` gives the current selections' text, and `%` the filename. And `+` and `*` are the two OS clipboards of Chapter 60 — the system clipboard and the X11 primary selection — reached as registers, so `"+y` yanks to the system clipboard and `"+p` pastes from it.

How multi-selection interacts with the clipboard

One behavior falls directly out of the engine being multi-range. When you yank several selections to a clipboard register (`+` or `*`), the selections are *joined with newlines* into the clipboard. Pasting them back pastes multiple selections — one per line — if the clipboard was last written by this zmax session; otherwise the clipboard's contents come in as a single selection. So a multi-cursor yank-and-paste round-trips as multiple cursors within zmax, and degrades gracefully to one block of text when the clipboard came from another application. It is a small rule, but it is the reason multi-cursor copy-paste “just works” across panes and survives a trip through the OS clipboard.

Appendix P – The fusevm Machine, in Reference

Chapter 19 argued for the shared bytecode machine and Chapters 20 through 22 showed the tongues riding it; this appendix collects the machine's own facts in the reference register — the shape of its opcode space, its acceleration features, and the value model the five front ends share — so that a claim the narrative made in passing can be checked against a single table. Every number here is the machine's as of this writing and moves with the code, in the same way the rest of the book's numbers do.

The opcode space

```
fusevm – a stack-based, language-agnostic bytecode machine
  224 opcodes, organized across 17 sections
  11 fused superinstructions (one dispatch for a common opcode pair)
  slot-indexed locals (fast-path local variable access)
  3-stage Cranelift JIT, behind a feature flag

The 21 sections, by kind
  general machinery      arithmetic · comparison · control flow (jumps/branches)
                        calls and returns · locals (slots) · heap/objects
  language-shaped        shell operations          29 first-class opcodes
                        AWK operations             87 first-class opcodes
  ( remaining sections carry the strings, collections, closures, and
    constant/stack plumbing a front end lowers the rest of a language onto )
```

The two counted rows — twenty-nine shell opcodes, 87 AWK opcodes — are the ones to remember, because they are the machine confessing its purpose. A general VM would make zshrs and awkrs build those behaviors out of primitives; fusevm answers a pipeline or a field reference with one native opcode, which is why the two most peculiar of the five tongues embed as cleanly as arithmetic does.

The acceleration features

FUSED SUPERINSTRUCTIONS (11)

A single opcode that performs the work of a common opcode pair, so the hot path dispatches once where a naive bytecode dispatches twice. The eleven are the pairs that measurement showed dominate real lowered code.

SLOT-INDEXED LOCALS

Local variables are reached by slot index, not by name lookup, so reading or writing a local is an array index rather than a map probe.

THREE-STAGE CRANELIFT JIT (feature-flagged)

linear	straight-line native translation of a bytecode run; cheapest, fires first, all that cold-ish code ever earns
block	recompiles a warmer region a basic block at a time; folds constants, keeps values in registers across a block
tracing	records the path a hot loop actually takes and compiles that one path as guarded straight-line native code

The escalation is unconditional-win by construction: code that runs once is never compiled past linear and pays nothing for machinery it will not use, while the loop that runs a million times earns the tracing compiler. Because all three stages live in fusevm and not in any front end, one improvement to the tracing stage speeds every tongue's hot loop at once.

The shared value model

ONE value representation underlies all five tongues

a string is the same string whether elisp or stryke made it
garbage-collected objects live behind HANDLES, not raw pointers, so a moving/compacting collector can relocate them transparently
(strykelang's own VM NaN-boxes its values; on fusevm the tongues share the one representation the machine defines)

WHY IT MATTERS

one representation	the tongues share one value model instead of translating between five, so a value crossing tongues needs no conversion
one JIT	a speed improvement to the machine is a speed improvement to all
↳ five	
one host seam	the embedding of Appendix R is defined against fusevm, once, not
↳ per language	

Appendix Q – The Five Tongues at a Glance

Chapters 20 and 21 walked the five embedded languages one at a time, each at a different stage of readiness; this appendix is that walk as a table, so the crate, the embedding API, the purpose, and the honest gap for each tongue sit in one place. The book’s standing rule holds here in the sharpest form: the “gap” column is not a roadmap wish but the named piece of host-side wiring that stands between “runs the language” and “runs the language *driving the editor*.”

The readiness table

TONGUE	CRATE	EMBEDDING SURFACE TODAY	THE NAMED GAP
elisp	elisprs	eval_str + persistent host; 22 defsubrs (message, command, buffer name, mark/region, kill-ring); buffer TEXT via a mirror synced in+out per eval	no keymap bridge (elisprs's define-key drives its own keymaps, not zmax's KeyTrie); buffer text costs a full copy each way, per eval
vimscript	vimlrs	eval_expr, eval_source, persistent globals; host hooks for :set, :map, :command, :highlight, :colorscheme + an ex-cmd passthrough; EditorHost for getline/setline/cursor/bufname	:autocmd – the one ex-command with no vimlrs host hook. Native `vim_autocmd` covers the surface independently
stryke	strykelang	VMHelper, parse_and_run_string; persistent globals and subs	native host-fn registration – the api_* surface is not exposed; reaches native code only through rustc-compiled rust{} blocks. Also: print is suppressed (its fd layer is outside the editor's capture)
AWK	awkrs	run_program(program, input); wrapped by run_awk_filter, which reads a region and replaces it	– (nothing outstanding; the filter shape is the whole role AWK needs)
Zsh	zshrs	ShellExecutor + execute_script behind with_captured_fds; state persists across calls	– (nothing outstanding)

What the table says, read across

The elisp row is the deepest and the most nuanced. eval_str, a host that persists definitions between calls, and twenty-two defsubr-registered editor operations make it the tongue that can do the most inside a buffer — but its buffer-text builtins run against a mirrored copy rather than the live rope, and it cannot bind a key. The vimscript row is the one that changed most since earlier editions of this book: what was described as “stubs awaiting wiring” is wired, and vimscript is now the only tongue that can define a command *and* bind it, which makes it the practical choice for configuration. stryke is the row with a real outstanding gap, and it is upstream VM work rather than editor work. AWK and Zsh have no outstanding gaps because their roles — text filter and

shell pipeline — are fully served by what they already expose; a tongue does not need a `defsubr` equivalent to do a job that is text in, text out. Different distances, each measured against what the tongue is *for*, none rounded either way.

One machine, one version

The cross-cutting constraint of Chapter 21 belongs in the reference too, because it is the detail that decides whether “five embedded languages” compiles at all:

```
All five tongues ride the ONE fusevm – so they must ride the SAME fusevm.
  all five vendor/*/Cargo.toml pin the IDENTICAL fusevm version; they differ only in
    which features they request (awkrs and vimlrs add `jit`; all five take
      `jit-disk-cache` and `aot`)
  none of the five treats fusevm as optional – it is the machine, not an add-on
  the shared machine is a shared machine only if everyone links the same one, so a
    single crate drifting to another fusevm major would put two bytecode machines in
    one binary and end the claim this movement rests on
  ( the version number itself moves; check the five manifests rather than this page )
```

Appendix R – The Embedding Seam, in Reference

Chapter 22 defended the host-callback mechanism as sound-though-unsafe; this appendix is the same mechanism as a checklist, for the reader who wants the invariant stated as conditions to verify rather than prose to trust. The seam is a thread-local raw pointer to the editor's command context, installed by an RAI guard for exactly the extent of a synchronous eval, and read by stateless host callbacks that reconstitute a `&mut compositor::Context` from it. Everything below is the discipline that makes that safe.

The lifecycle, as steps

1. a command handler holds the one `&mut compositor::Context` the main loop lent it
2. `eval_elisp` builds a `CxGuard`:
 - saves the slot's previous value, then writes the context as a raw pointer into `CX_PTR` (a `thread_local Cell<*mut ()>`, type-erased)
3. it calls the interpreter's eval entry (e.g. `elisprs::eval_str`)
4. the machine runs the lowered bytecode; on a host builtin it calls the registered callback – a bare fn for elisp defsubrs, a captureless `Box<dyn Fn>` for vimlrs
5. that callback calls an `api_*` helper, which calls `with_cx`: reads `CX_PTR`, errors if null, else reconstitutes `&mut compositor::Context` and calls the real method
 - while the command's original `&mut` sits idle, blocked in the eval call
6. the eval returns (OR unwinds on error/panic)
7. the guard drops; its destructor `RESTORES` the previous pointer (not null – the saved value), which makes nested evals sound and leaves the outermost level back at null. Restored on unwind too, so a blown-up eval leaves nothing dangling

For elisp, steps 3 and 6 are bracketed by a buffer mirror: `load_buffer_into_host` before the eval and `flush_host_into_buffer` after, because elisp's buffer-text builtins run against `elisprs`'s own `EditBuffer` rather than the live rope (Chapter 22). Editor operations take the pointer path above with no copy; buffer text is copied both ways, once per eval.

The three soundness conditions

The unsafe seam is sound if and only if all three hold, and the editor is built to hold all three:

SINGLE-THREADED	eval runs synchronously, on the editor's own thread, inside a command handler – exactly one <code>&mut</code> context exists; no second thread to race, no second mutable borrow to alias in practice
BOUNDED VALIDITY	the RAII guard ties the pointer's life to the eval's stack frame – the slot is never read while stale, and <code>with_cx</code> 's null check means a read outside any eval is an error, not a dereference
NESTING-SAFE	the guard saves and restores rather than clearing, so a nested eval leaves the outer eval's context intact underneath it and unwinding one level restores one level
NO RE-ENTRANT <code>&mut</code>	an <code>api_*</code> helper must not trigger another eval while it holds the <code>&mut</code> from <code>with_cx</code> – two live <code>&mut</code> borrows of one context would alias. This is a stated contract in the module docs, not a checked invariant, and it is the seam's sharpest edge

Under those the pointer is always valid when read, never aliased in the act, never stale. It is unsafe in the language's eyes and correct in the program's, and the gap between the two is the price of an integration with no process boundary — paid in a small, bounded patch of `unsafe` at one seam, in exchange for no latency, no protocol, and no process to keep alive.

The editor-side anchors

The seam plugs into hook points the editor already had, not surfaces invented for the languages:

<code>TypableCommand</code> registry	where an eval command lives (<code>:elisp</code> , <code>:stryke</code>) – the typed front door that funnels its argument through the guard
<code>main.rs</code> <code>config-load</code>	the natural place to run an init script – the <code>.zmax</code> answer to emacs's <code>init.el</code> – before the main loop starts
runtime <code>KeyTrie</code> keymaps	a script can bind keys, reaching the same structure the vim keymap is built from
shell-out machinery	the precedent that "run something, get a result, update the buffer" already exists and works synchronously

Appendix S – The Operator-Pending Table

Chapter 5 opened the keystone seam: vim's verb-first operators, emulated on a selection-first engine by keymap entries that run *lists* of native selection commands. This appendix tabulates the pattern, because seeing a column of operators reduced to the same three-beat shape is the fastest way to believe the trick is real and repetitive rather than a special case per key. The command spellings below are quoted from `zmax-term/src/keymap/vim.rs`; that file is the ground truth, and where an entry's exact command names are not quoted here the file has them. Note that “three-beat” is the shape rather than a literal length — several operators need a fourth command to match vim's cursor placement, and the table marks those.

The three-beat shape

Every operator is `collapse · extend · verb`: collapse whatever is selected down to a bare cursor, extend a selection to encode the motion, then run the verb on that selection.

OPERATOR	THE COMMAND LIST IT RUNS
<code>dw</code>	<code>[collapse_selection, subword_extend_w, delete_selection]</code>
<code>dW</code>	<code>[collapse_selection, extend_next_long_word_start, delete_selection]</code>
<code>dd</code>	<code>[collapse_selection, extend_to_line_bounds, delete_selection_linewise,</code> <code>goto_first_nonwhitespace]</code> ← four, not three
<code>cw</code>	<code>[collapse_selection, subword_extend_e, change_selection]</code> → Insert mode ↑ <code>_e</code> , not <code>_w</code> : vim's <code>cw</code> acts like <code>ce</code>
<code>yy</code>	<code>[collapse_selection, extend_to_line_bounds, yank, collapse_selection]</code>
<code>gUw</code>	<code>[collapse_selection, subword_extend_w, switch_to_uppercase, collapse_selection</code> <code>]</code>

Two entries in that table are the interesting ones. `dd` runs four commands because vim's `dd` is linewise and lands the cursor on the first non-blank of the next line; `delete_selection_linewise` and `goto_first_nonwhitespace` are those two facts spelled out. And `cw` binds `subword_extend_e` where `dw` binds `subword_extend_w`, because vim's `cw` stops at the end of the word rather than at the start of the next one — the same key under two operators takes two different motions. Neither is derivable from the pattern; both are ports of observed vim behavior.

The `change_selection` at the end of `cw` deletes *and* drops you into Insert mode; the trailing `collapse_selection` on `yy` and `gUw` tidies the selection back to a cursor after the verb runs. The indent operator (`>j`), the reformat operator (`=G`), and the shell-filter operator (`!!`) are all the same shape — a collapse, an extend that encodes the motion, and a verb.

Counts ride along for free

```
3dw  the count is consumed by the keymap engine BEFORE the list runs, and it
      multiplies the motion inside the list – extend across three words, then delete
2dd  extend across two lines, then delete
      ( no submap entry mentions counts; the engine's repeat machinery does it )
```

Where a static list learned one moment of memory

A fixed list cannot pause to read an interactive key, so the edits that need one are built on the editor's *next-key* callback — a command that, instead of finishing, registers a closure receiving the very next keystroke.

```
OPERATOR + FIND      d f ,    the f entry is delete_find_char_forward: arms the
                        next-key callback, waits for the target char, extends
                        to it, deletes. Also c t ), c f ", and the F/T backward
↳ variants.
OPERATOR + TEXT-OBJECT c i w    the i / a entries are delete_textobject_inner /
                        change_textobject_around and siblings: read the following
                        w ( p " as an object specifier, select inner/around,
↳ operate.
                        d i ( d i p y a { - the whole operator + i/a + object
↳ matrix.
```

What remains genuinely emulated

```
q / @      macros record and replay keystrokes to a named register
m{a-z}     named marks remember positions, carried along as the buffer is edited
`{a-z} '{a-z}  jump to a mark's position / its line
R          a true Replace mode that overtypes
.          repeats the last change as a replayable unit - solid for insert-and-change
↳ edits
C-v        block-visual: a REAL RECTANGLE (BlockSelect in zmax-view/src/editor.rs),
           re-derived by block_reproject; `to_eol` models vim's CTRL-V $ ragged right edge
```


Appendix T – The Crate Workspace, in Reference

Chapter 28 mapped the workspace and argued its boundaries; this appendix is the map as a reference — each crate, what it owns, and the direction its dependencies point — because “dependencies point one way” is a claim best verified by seeing the arrows in a column. The workspace is a Cargo workspace of fourteen crates; the ones the narrative names are below, and the embedded-language crates of Appendix Q link in at the top.

The spine, pure to concrete

```
zmax-core    the foundation, no screen and no terminal in it:
              the rope · Selection and Range · Transaction / ChangeSet · the syntax layer
              ↑ depends on
zmax-view    editor state, still no terminal:
              the Editor · the Document (text + history + selections + diagnostics + LSP)
              ↓
              the View (viewport + gutters) · the Theme
              ↑ depends on
zmax-term    the running terminal program:
              the Application · the main loop · the command implementations · the UI
              components · the command-line – and where the five tongues plug in
zmax-tui     the drawing layer, generic over its backend, below term
```

Dependencies point one way — `term` → `view` → `core`, never the reverse — and the payoff is testability: because `core` has no UI and `view` has no terminal, the editor logic runs under a `TestBackend` through the real `view` and `core`, with no screen. A boundary that points one way is a boundary you can test against.

The services and utilities

```
zmax-lsp / zmax-lsp-types  the language-server client (Chapter 14) and its types
zmax-dap / zmax-dap-types  the debug-adapter client (Chapter 15) and its types
zmax-vcs                   the git integration (Chapter 16)
zmax-loader                runtime, grammar, theme, and config discovery (Chapter 2)
zmax-event                 the hook / dispatch system (Chapter 26)
zmax-parserc               a parser-combinator library, used for snippet syntax
zmax-stdx                  stdlib extensions: path normalization, rope helpers,
                           URL and range types – shared everywhere
zmax-plugin                the native plugin SDK: a versioned C ABI (repr(C) structs,
                           extern "C" fn pointers) shared by the editor and any
                           cdylib plugin loaded with `:plugin load` over dlopen
```

Each service crate owns one external-facing concern behind an explicit seam, so it is replaceable and testable on its own. The embedded languages plug into `zmax-term`, at its command, config, and keymap surfaces — because those are the surfaces a language drives, and `term` is the crate that owns them. The crate map is not bureaucracy; it is what keeps a fourteen-crate editor with five embedded languages legible enough to reason about.

Appendix U – The Port Report, by the Numbers

Chapter 29 described the `port/` machinery — a coverage fraction whose numerator is re-derived from `zmax`’s source at report time and whose denominator is parsed from the upstream editors’ own documentation. This appendix is that chapter’s figures in one place.

Read them as a snapshot and nothing more. Every number below is regenerated by `scripts/gen_port_report.py` into `docs/port_report.md` and `docs/keybinding_report.md`, those files are the authority, and a reader who wants the current answer should run the generator rather than trust this page. Any figure printed in a book that tracks a live tree is stale by construction; the reason to print these at all is to show the *shape* of the measurement, not to pin the values.

The two halves, and the surface they measure

```
NUMERATOR – re-derived from zmax source at report time
2,103 static commands      (read out of commands.rs)
2,483 typable command spellings (aliases included, out of typed.rs)
4,109 default keybindings  (the keymap sources under keymap/)
every claimed mapping carries EVIDENCE: static:<cmd> · typable:<cmd> ·
  ~ key:<mode>:<chord>
marked ported / partial / absent. A "broken" mapping – evidence that does not resolve
  to real code – is COUNTED AS ABSENT, listed in the report, and announced with a
  printed WARNING. The generator does not fail on it; see Chapter 29

DENOMINATOR – parsed from upstream, not hand-listed
Vim / Neovim normal · insert · visual · command-line · ex commands
the GNU Emacs command set      the Spacemacs bindings
the fzf.vim command list        the JetBrains IntelliJ IDEA default keymap
the Cursor command set
```

The per-source measure

```
6,847 distinct cited feature items (across all sources above)
5,957 ported      (87.0%)
  373 partial
    0 broken

KEYBINDING-ONLY SLICE (excludes ex-commands, options, functions, M-x commands)
3,113 cited keys
2,692 ported      (86.5%)
  179 partial
```

The per-source fraction overlaps heavily and flatters or punishes by scope: `go-to-line` is counted once per tradition (four rows for one capability), and the Emacs denominator is the *entire* GNU Emacs manual — `Dired`, `Gnus`, `Calc`, `TeX-mode`, the games — so a low Emacs score measures how much of that encyclopedia a modal

editor declines to be, not a hole in the editor. Read alone, these numbers are honest but easy to misread, which is why the report leads with the second measure instead.

The capability measure – the headline

```
186 distinct editor capabilities, deduplicated (one row each, regardless of ancestry)
185 implemented (99.5%)
  1 partial or a different model
  0 genuine gaps

100% areas every tracked area except one
LOWEST ROW editing, 20 of 21 – the sole row not at 100%
```

The two measures answer different questions and the project publishes both rather than picking the flattering one: the per-source fraction tells a vim refugee how much of their reflexes survive the move (Neovim normal mode and visual mode both above 90%); the capability table tells anyone how complete the editor is as an editor. That refusal to choose the kinder number is the ethic of the [port/](#) directory restated as a pair of tables — a counting mechanism a reader can re-run, rather than a claim a reader must take on trust.

Chapter 106 — Rmail: Reading Mail Out of a Flat File

The most surprising directory in `zmax-core/sic` is the one that reads mail. `rmail.rs` is a port of the GNU Emacs mail reader, and it is not a token gesture — it parses mbox, tracks per-message attributes, decodes MIME, splits digests, unwraps forwards, and serialises back to disk. The module opens by saying exactly what it will and will not do:

This module is the pure, tested core of that reader — it parses an mbox into an ordered list of messages, tracks the current message plus per-message Deleted/labels/seen attributes, navigates (next/previous, undeleted-only, first/last, by label), marks and expunges deletions, and serialises the mailbox back to mbox text for saving. ... No I/O, no network — the command layer supplies the file bytes and decides where a reply draft goes.

That last sentence is the load-bearing one. Rmail in Emacs is entangled with `movemail`, with `rmail-file-name`, with the babyl-versus-mbox format history, and with a display buffer that is also the storage buffer. The zmax port keeps none of that. It takes a `6str` and gives back a `Mailbox`, which makes the whole reader unit-testable without a mailbox on disk — and the module carries 27 `#[test]` functions to show for it.

The message and its six attributes

A message is a struct with an envelope, headers, body, and three pieces of Rmail state:

```
Msg { envelope, headers, body, deleted, seen, labels }
```

`envelope` is the mbox `From` line minus the literal `From`, so it holds the sender address followed by an asctime date. `deleted` is Rmail's Deleted attribute — the thing `d` sets and `x` acts on. `seen` is the inverse of Rmail's Unseen attribute: the port stores the positive form and sets it when a message is displayed. `labels` is the free-form keyword list that `a` adds to and `k` removes from.

The interesting design choice is that attributes are struct fields rather than synthesised headers. Real Rmail stores them in the message text itself, in an `X-MAIL-ATTRIBUTES` style header that survives a save. The zmax port does not serialise them at all: `to_mbox` writes `envelope`, then the headers, then the body, and the three attribute fields are dropped on the floor. Deleted messages are gone because expunge removed them, which is the only attribute that survives a round trip by construction. Labels and seen state are session-local. Nothing in the module claims otherwise, but nothing calls it out either, and it is the first thing a reader should check before trusting the module with a real mailbox.

Where a message begins

The mbox format's original sin is that a message separator — a line beginning `From` — is also a legal line of English prose. `parse_mbox` handles this with a rule stated in a comment inside the function rather than in the doc header:

mboxrd rule: a `From` line is a message separator only at the start of the file or immediately after a blank line. This disambiguates it from a body line that merely begins with “From” (which real mailers `>-quote`, but lenient readers must still not split on).

The implementation is a single carried boolean, `prev_blank`, initialised to `true` so that the first line of the file qualifies. That is a good rule and it is tested: the module's fixture mailbox contains the line `From is a tricky body line.` in the middle of the first message's body, and `parses_all_messages` asserts the mailbox has three messages and that the tricky line survived inside the body of the first.

Note that the function's own doc comment states the simpler, wrong rule — “A message begins at a line starting with `From`” — and the blank-line qualification appears only in the inline comment further down. The code is right; the header sentence is a summary that lost a clause.

The unstuffing asymmetry

Having accepted that bodies contain `From` lines, the reader has to deal with the escaping real mailers apply. The flush closure does this:

```
if l.trim_start_matches('>').starts_with("From ")
    && l.starts_with('>')
{
    l[1..]
} else {
    l.as_str()
}
```

Strip one `>` from any line that is a run of `>` followed by `From`. That is the mboxrd unstuffing rule, and it is correct.

The writer is not its inverse. `to_mbox` re-quotes with:

```
if line.starts_with("From ") {
    out.push('>');
}
```

Only a *bare* `From` line gets a `>`. So trace `>>From x` through a load and a save. The reader sees a `>`-run followed by `From`, strips one, and stores `>From x` in the body. The writer looks at `>From x`, sees it does not start with `From`, and writes it out unchanged. The file now contains `>From x` where it contained `>>From x`. Load and save again and it becomes `From x` — which the *next* load will treat as a message separator if it happens to follow a blank line.

This is a genuine, verifiable data-loss path in a save/load cycle, and it is narrow enough to be worth stating precisely: it only bites bodies that contain `>`-prefixed `From` lines, which in practice means quoted replies that themselves quote an mbox separator. It is a two-character fix — the writer's condition should be the reader's condition — and the reason it is worth a page here is that the reader's comment cites mboxrd explicitly while the writer's comment says only “Re-quote any body line beginning with `From`.” The comment describes what the code does. The code does not do what the format requires.

Seven sort keys and one sorting engine

Rmail has a family of `rmail-sort-by-*` commands, and the port implements them as seven key functions on `Msg` feeding one generic engine. The engine is worth reading because it does something most sort implementations skip:

```
fn sort_by_key<K: Ord>(&mut self, key: impl Fn(&Msg) -> K) {
    let cur = self.current;
    let mut order: Vec<usize> = (0..self.msgs.len()).collect();
    order.sort_by_key(|&i| key(&self.msgs[i]));
    self.current = order.iter().position(|&i| i == cur)...;
    // permute msgs by `order`
}
```

It sorts an index permutation, uses that permutation to find where the current message ended up, and only then applies it to the message vector. The cursor follows the message rather than the slot. Sorting a mailbox in Emacs and having the display jump to a different message is exactly the annoyance this avoids, and the port pays a `Vec<Option<Msg>>` scratch allocation to avoid it. The comment in the body — “sort_by_key is stable, so equal keys keep their original order” — is doing real work: for `sort-by-subject` on a thread, stability is what keeps the replies in arrival order under their root.

The keys themselves are where the mail-format knowledge lives. `sortable_date` is the one to read. Emacs has `rmail-sortable-date-string`; the port reimplements it as a token scan that produces a lexically comparable `YYYYMMDDhhmmss`:

- any token that starts with a three-letter month abbreviation sets the month;
- any token containing `:` is split into `hh:mm[:ss]`;
- a four-digit number in `1000..=9999` is the year;
- a number in `1..=31`, if the day is still unset, is the day.

The order-independence is deliberate: it parses RFC 5322 (`Mon, 2 Jan 2026 15:04:05 +0000`) and `asctime` (`Mon Jan 2 15:04:05 2026`) with the same code, because it never assumes a position. The cost is stated honestly in the doc: “The zone offset is ignored (times are compared as written).” A message sent at 09:00+0900 sorts before one sent at 23:00-0800 on the same date even though it arrived later. For a local mail reader ordering a personal mailbox that is usually fine; for anything auditing delivery order it is wrong, and the module says so rather than pretending.

The day heuristic has a quieter failure. `if (1..=31).contains(&n) && day == 0` takes the *first* small number as the day. In `Mon, 2 Jan 2026 15:04:05 +0000` the time token contains `:` and is consumed by the time branch first, so the day is unambiguous. But a `Date:` header with a two-digit year — legal in RFC 822, still emitted by antique mailers — would put a second small number in play, and whichever came first wins. Undated and unparseable messages produce all zeros, which sorts them first; that is the documented behaviour.

Filters, and the regexp that is also a substring

The four `rmail-summary-by-*` commands are four filter functions over the same matcher builder, and the builder is the interesting part:

```
regex::Regex::new(&format!("{}",(?){}pattern"))
  .unwrap_or_else(|_| Regex::new(
    &format!("{}",(?){}", regex::escape(pattern))
  ).unwrap())
```

Treat the user’s input as a case-insensitive regexp; if it does not compile, treat it as a literal. The doc explains why — “so a plain substring typed at the summary prompt always works” — and this is the right trade for an interactive prompt where the user typed `++` or `(no subject)` and does not want a syntax error. The escape-and-retry can never itself fail, hence the `unwrap`.

The four filters differ only in which fields they inspect: `by-recipients` scans `To/From/Cc`, `by-senders` scans `From`, `by-topic` scans `Subject`, and `by-regexp` scans every header key, every header value, and the body. Each returns `Vec<usize>` of message indices rather than a filtered mailbox, which keeps the summary buffer a view over the real message list — the same relationship Rmail’s summary buffer has to the Rmail buffer.

MIME in one recursive function

`parse_mime` is a deliberately small MIME implementation with one shape guarantee stated up front:

A non-MIME message yields exactly one entity holding its whole body, so the display code never needs a special case.

That single invariant is what makes the rest of the module simple. There is no `Option<Vec<MimeEntity>>`, no “is this MIME” predicate at the call site; the display code walks a list that always has at least one element.

The walk is `walk_part`, recursive on `multipart/*`, bounded by a depth cap:

```
const MAX_MIME_DEPTH: usize = 8;
```

Eight levels, and past that the multipart is treated as a leaf and its raw body becomes the entity text. This is the correct failure mode — a malformed or hostile message produces ugly output, not a stack overflow.

Each leaf is content-transfer-decoded (base64, quoted-printable, or verbatim), and then, if its media type starts with `text/` or `message/`, charset-decoded. Everything else gets `text: None` and `hidden: true`, which the display layer renders as an attachment line. The `hidden` flag is what `mail-mime-toggle-hidden` flips.

The base64 decoder is a bit-accumulator with a six-bit alphabet table and one notable behaviour: `if c == b'=' { break; }`. It stops at the first `=`, not just at padding, and it silently skips any byte outside the alphabet, which is how it tolerates the line breaks base64 bodies are wrapped at without a separate un-wrap pass. The quoted-printable decoder handles `=XX` hex escapes and treats a trailing `=` as a soft line break that deletes the newline. Both are short enough to read in one sitting and both are exercised by the module’s tests.

Charset handling delegates to `encoding_rs`, with a normalisation step first, and the mapping table is Emacs-specific knowledge encoded as three rules: drop an `-unix/-dos/-mac` end-of-line suffix, rewrite `latin-N` as `iso-8859-N`, then ask `encoding_rs` for the label. Emacs coding-system names are not WHATWG labels and this is the smallest bridge between them. An unknown name yields `None`, which the caller turns into a lossy UTF-8 decode — the doc calls this “Emacs’s `undecided` default.”

The parameter parser stops early

`content_param` extracts `; name=value` parameters from a `Content-Type`-style header, and it has a defect worth naming because everything above depends on it — the boundary, the charset, and the filename all come out of this function.

```
for part in header.split(';').skip(1) {
    let (k, v) = part.split_once('=')?;
    if k.trim().eq_ignore_ascii_case(name) { ... }
}
```

The `?` is on the `split_once`, inside the loop, in a function returning `Option<String>`. A parameter segment with no `=` does not get skipped — it returns `None` from the whole function, abandoning every parameter after it. So `text/plain; charset=utf-8` works, and `text/plain; format=flowed; charset=utf-8` works, but a header with a valueless attribute before the one you want does not. The fix is `else { continue }` instead of `?`. As written, the function’s contract is “the value of the named parameter, unless an earlier parameter lacks an `=`, in which case nothing.”

A discriminant that is computed and discarded

`MimeEntity::label` builds the one-line header shown above an entity:

```
match (&self.filename, self.text.is_some()) {
    (Some(name), _) => format!("[{}] ({} bytes)", ...),
    (None, _)       => format!("[{}] ({} bytes)", ...),
}
```

Both arms bind the second tuple element to `_`. The `self.text.is_some()` call is evaluated and thrown away; the match is a two-arm `Option` match wearing a tuple. This is harmless and it is the visible residue of a version where the label differed for text and binary parts. It is the kind of thing that survives review forever because it compiles and produces correct output.

Digests and forwards: two unwrapping problems

`undigestify` and `unforward` solve the same shape of problem — a message carrying another message — in two different eras' encodings, and the port implements both.

For digests, the MIME path is easy: a `multipart/digest` splits on its boundary and each part is a `message/rfc822` wrapper whose body is the embedded message, so the code parses the part, then parses the part's body. The classic path is RFC 1153, where messages are separated by a run of hyphens. The port's separator test is:

```
t.len() >= 27 && t.bytes().all(|b| b == b'-')
```

Twenty-seven, not thirty. The doc explains the slack — “RFC 1153 specifies exactly 30; real digests vary, so any long run counts” — and names it as Emacs's sloppy digest parser. The number is not arbitrary in the sense of being undocumented, but it is arbitrary in the sense that nothing distinguishes 27 from 26.

The chunk filter is the elegant part. Both digest paths funnel through a `build` closure that returns `None` when the parsed chunk has no headers at all, with the comment: “A digest chunk with no headers at all is the trailer (‘End of Digest’), not a message.” One predicate handles both the leading table of contents (dropped because it precedes the first separator and so is never in a chunk) and the trailing sign-off (dropped because it has no headers). No special-casing by position.

For forwards, the MIME path looks for the first `message/rfc822` entity in the parsed MIME tree. The text path is RFC 934, which requires *stuffing*: the forwarder prefixes every line beginning with `-` with `-`, so that the `----- End of forwarded message` marker cannot be forged by the payload. The port unstuffs with `l.strip_prefix("- ")` and finds the markers with a normaliser that trims hyphens and whitespace and lower-cases, then compares against three accepted opening phrases:

```
const FORWARD_START: [&str; 3] = [
    "start of forwarded message",
    "forwarded message",
    "original message",
];
```

The third is the Outlook/Gmail variant, and the doc says so: “Emacs's `mail-forward-separator` shapes plus the widely used Gmail/MUA variants.” A missing end marker is not an error — `unwrap_or(lines.len())` takes the rest of the message — which is the right call for a format nobody validates.

Finally `pgp_armor_range` returns the byte span from `-----BEGIN PGP MESSAGE-----` through the end of `-----END PGP MESSAGE-----`, so that `mail-epa-decrypt` can splice plaintext back in place. The port does no cryptography; it locates the block and hands the range up. That is the correct division: an editor should not be a PGP implementation.

What the reply builders do and do not synthesise

Three functions build message-mode drafts, each returning `(to, subject, body)`. Reply prefers `Reply-To` over `From`, prefixes the normalised subject with `Re:`, and cites the body. Forward leaves the recipient empty, prefixes `Fwd:`, and cites. Resend leaves the recipient empty, keeps the subject verbatim, and puts the *entire original message* — headers included — in the body.

Resend carries the port's most explicit self-limitation:

This is a partial port: it hands message-mode an editable resend draft but does not synthesise the `Resent-From/Resent-To` headers real Emacs adds, because `zmax` has no send transport beyond queuing to a local outbox.

That is the right way to write down an incomplete feature: what is missing, and the structural reason it is missing. The `Resent-*` headers exist to record a re-injection into the mail system; with no transport there is

no re-injection to record.

`normalize_subject` is shared by the reply subject, the forward subject, the subject sort key, and same-subject navigation. It strips a *chain* of `Re:/Fwd:/Fw:` prefixes in a loop, not just one, which is what stops a mailbox from accumulating `Re: Re: Fwd: Re:`. Because the same function backs navigation, `[-c [-n` groups a thread whose subjects drifted through different mailers' prefix conventions.

Cursor discipline, and where it slips

The `Mailbox` cursor is moved by a private `goto`, and `goto` does two things: clamp the index, and set `seen = true`. Every navigation command routes through it — `next`, `prev`, `first`, `last`, `show`, `search`, `next_labeled`, `next_same_subject`, and their backward twins.

Two paths do not. `undelete` assigns `self.current = i` directly when it walks back to find a deleted message, and `expunge` assigns `self.current` directly after retaining. Both are defensible: undeleting a message is not the same as reading it, and expunge is a bookkeeping operation rather than a navigation. But the effect is that a message the cursor lands on via `u` or `x` stays Unseen until the display layer marks it — and since `seen` is not persisted anyway, the practical consequence is confined to a session's summary rendering. It is worth knowing that the invariant “the current message is seen” holds for every navigation command and for neither of these two.

Expunge's index arithmetic is the part to check:

```
let deleted_before = self.msgs[..self.current.min(len)]
    .iter().filter(|m| m.deleted).count();
self.msgs.retain(|m| !m.deleted);
self.current = self.current
    .saturating_sub(deleted_before)
    .min(self.msgs.len() - 1);
```

The slice is `..current`, exclusive, so a deleted *current* message is not counted. Subtracting only the deletions strictly before it means the cursor lands on whatever now occupies the current message's old slot — which, if the current message was itself deleted, is the message that followed it. That is exactly the documented behaviour, achieved without a search, and it degrades correctly at the end of the mailbox via the `min`.

Chapter 107 — Diff-Mode as a Text Transformation Suite

Chapter 106's mail reader treats a file as a container of messages. `zmax-core/src/diffmode.rs` treats a file as a container of edits, and then — this is the part that distinguishes diff-mode from a diff viewer — lets you edit the container. Emacs' `diff-mode` is a major mode for buffers whose contents are patches, and its commands are text transformations on patch syntax: split this hunk, reverse this patch, convert this to a context diff, kill this file's section, refresh these line numbers.

The module is pure and it says so in the first paragraph: “No I/O and no terminal types live here, so every bit of it is unit-tested below.” It carries 33 `#[test]` functions.

The model, and the flat rendering beside it

There are five types. `LineKind` classifies a displayed line into six roles (context, added, removed, secondary header, file banner, hunk header). `DiffLine` pairs a kind with the full text — and the doc is specific that body lines “keep their leading `+/-`/space so the overlay can show the glyph,” which means the renderer never has to reconstruct a prefix it stripped. `Hunk` holds four parsed numbers plus the raw header text plus its lines. `FileDiff` holds two paths and its hunks. `Diff` holds files.

`Hunk` keeping both `old_start/old_len/new_start/new_len` and the original `header` string looks redundant until you reach the reverse and conversion commands, which need to reproduce a header's exact numeric spelling. Storing the raw text alongside the parsed values is what lets a reverse round-trip byte-for-byte rather than canonicalising `@@ -1 +1 @@` into `@@ -1,1 +1,1 @@`.

Alongside the tree there is `flatten`, which renders the whole `Diff` back into a `Vec<DiffLine>`. The interactive overlay navigates the flat list — `next_hunk_line` and `prev_hunk_line` take `&[LineKind]` and an index and find the next `HunkHeader`. Passing only the kinds, not the lines, is a small discipline that makes navigation impossible to accidentally couple to text.

Editing the patch: split, reverse, convert

The transformation commands are the substance. Each takes the buffer text (and, where it matters, the line point is on) and returns a new `String`.

`diff_split_hunk` splits the hunk containing point into two at point. The hard part is not the split; it is the arithmetic. A unified hunk header declares how many old and new lines the hunk covers, so splitting one hunk into two means recomputing four numbers from the body. The port does this with a helper that walks the body lines counting `-` toward old, `+` toward new, and context toward both, then emits:

```
@@ -{o_s},{fo} +{n_s},{fnn} @@
@@ -{o_s+fo},{so} +{n_s+fnn},{sn} @@
```

The second header's starts are the first header's starts advanced by the first half's lengths. This is correct and it is more than Emacs does — the doc says so plainly:

Both resulting `@@` headers get correct start and length numbers (zmax recomputes both halves; stock Emacs leaves the second header's lengths as a `,1` placeholder to be fixed up by hand).

That is a deliberate divergence from the reference implementation, documented at the point of divergence, in the direction of doing more work. It is worth flagging as a pattern: this port's divergences are not uniformly simplifications. Some are corrections.

The guard conditions are strict: `at <= hs || at >= he` returns `None`, so point must be strictly inside the hunk body, and both halves must be non-empty. A split that would produce a zero-line hunk is refused rather than emitted.

`diff_reverse_direction` swaps the patch's direction. Three things change: the `---/+++` path pair swaps, each hunk header's two ranges swap, and every `+` body line becomes `-` and vice versa. The doc claims "Reversing twice restores the original text exactly," and the implementation is built to earn that. The hunk header is not re-parsed into numbers and re-formatted; it goes through `split_hunk_header_raw`, which returns the two range strings as *written* plus the trailing suffix after the closing `@@`:

preserving the exact number format so a reverse round-trips byte-for-byte.

So `@@ -1 +1,3 @@ fn main` reverses to `@@ -1,3 +1 @@ fn main`, keeping the abbreviated `-1` spelling and the function-context suffix that `git diff -p` appends. A parse-and-reformat implementation would silently normalise both and break the round trip.

The file-header case is handled before the body case and consumes two lines at once, requiring `---` to be immediately followed by `+++`. That ordering matters: a `--- a/f` line reaching the body arm would be treated as a removed line and gain a `+`.

`diff_unified_to_context` and `diff_context_to_unified` are the pair that converts between the two patch dialects. The unified-to-context direction has to solve a representational mismatch: unified counts lines (`start, len`), context names ranges (`start, end` inclusive). The port's `ctx_range` handles the degenerate case explicitly:

```
if len == 0 { format!("{}",0", start.saturating_sub(1)) }
else      { format!("{}",{"", start, start + len - 1) }
```

A zero-length side in a context diff is spelled `N,0` where `N` is the line *before* the insertion point — GNU diff's convention, reproduced rather than invented. `saturating_sub` keeps an insertion at line 1 from underflowing.

The body rewriting is `ctxify`, which turns a one-character prefix into a two-character one by inserting a space: `x` becomes `x`, `-x` becomes `- x`. That is the whole difference in body syntax between the dialects, and isolating it into a three-line function is why the conversion is readable.

The other seven commands

Beyond the marquee transformations there are `diff_hunk_kill` and `diff_file_kill` (remove the hunk or the whole file section point is in), `diff_delete_trailing_whitespace`, `diff_ignore_whitespace_hunk`, `diff_refresh_hunk` (recompute a hunk's header numbers from its current body, for after you have hand-edited it), and `apply_hunk` / `apply_file_diff`, which actually apply a parsed hunk to target text and return `Result<String, String>`.

All of them are reachable from the editor. Grepping the tree for each of the eleven public commands finds a `commands.rs` registration entry, a wrapper function, and the call — three references apiece. `apply_hunk` and `apply_file_diff` carry more, because `commands.rs` reaches for them from three separate commands between them. That is a fully wired subsystem, which is worth establishing because the next chapter's module is not.

Chapter 108 — merge_ops: A Module That Is Almost Entirely Tests

`zmax-core/src/merge_ops.rs` is the most interesting module in the tree to audit, because reading it and grepping for its callers produce two different pictures.

The module header is a claim about the state of the art:

`diff.rs` only exposes [`compare_ropes`], which produces a ropey `Transaction` for the undo engine; it has no *renderable* line or word diff, no three-way merge, no unified-patch application, and no conflict reconciliation. VCS-aware editors (GNU Emacs `smerge/ediff`, VS Code, Neovim, Sublime, JetBrains, Zed and Helix) all ship these; this module adds them as plain, editor-type-free functions over `&str`.

Seven public functions follow: `lcs_pairs`, `line_diff`, `word_diff`, `three_way_merge`, `parse_unified_diff`, `apply_unified_diff`, and `reconcile_conflict`. Fifteen `#[test]` functions cover them.

Grep the non-vendor tree for each name. `three_way_merge` has three call sites in `commands.rs`. The other six have none outside `merge_ops.rs` itself and its test module. Six of seven public functions are, in the shipped binary, dead — including the one the module marks with a star.

That is not a scandal, and it is not padding either. It is a real fact about the shape of the codebase that a reader should have before deciding how much to trust the module, and it changes how you read the header paragraph. The claim “this module adds them” is true of the module and not yet true of the editor.

The shared primitive

Everything rests on one function:

```
pub fn lcs_pairs<T: PartialEq>(a: &[T], b: &[T])
    -> Vec<(usize, usize)>
```

A longest common subsequence over any comparable token type, returning matched index pairs in increasing order on both sides. The doc is honest about the cost: “Runs in $O(n*m)$ time and space.” The implementation allocates `vec![vec![0u32; m + 1]; n + 1]` — a full dynamic-programming table, with a `Vec<Vec<u32>>` layout rather than a flat buffer.

That quadratic *space* is the thing to weigh. Merging two thousand-line files allocates roughly a million `u32` in a thousand separate heap allocations. Ten thousand lines a side is a hundred million cells — four hundred megabytes, plus ten thousand allocation headers. Real diff implementations use Myers, or Hunt–Szymanski proper with a candidate list, or Patience, precisely to avoid this. The module’s header namechecks “Hunt–Szymanski / dynamic-programming longest-common-subsequence,” and what is implemented is the dynamic-programming half of that phrase.

For the one function that is actually wired — `three_way_merge`, used by the merge UI on files a human is reconciling by hand — this is likely fine, and the generic token type is what lets the same routine serve the line diff, the word diff, and the merge without a second implementation. The tradeoff is stated in the doc rather than hidden, which is the correct handling. It is simply a tradeoff that gets more expensive the more the module gets used.

Three-way merge by sync points

`three_way_merge` implements diff3, and its structure is worth reading because it derives the alignment rather than assuming it. Two LCS runs give base-to-ours and base-to-theirs index maps. Sync points are base lines matched in *both*:

Because each LCS is monotonic, the base indices common to both maps are monotonic in *a* and *b* too, giving aligned `(bi, ai, ti)` triples to anchor the merge.

That is the whole insight. Everything between two consecutive sync triples is a region where the three versions may disagree, and it can be resolved independently. The resolver has four cases in a fixed order:

1. ours equals base → take theirs (only theirs changed);
2. theirs equals base → take ours;
3. ours equals theirs → take either (both made the same change);
4. otherwise → emit diff3 conflict markers and increment the count.

The order matters for the degenerate case where all three are equal: case 1 fires and takes theirs, which equals ours, which equals base. Correct by accident of ordering, but correct.

The markers are git's diff3 style, with the base section included:

```
<<<<<< OURS
||||| BASE
=====
>>>>>> THEIRS
```

The doc comment writes the third marker as `=====` with a trailing space; the code pushes `"=====`" with none. The code is right — a trailing space after the marker would be a real interoperability problem for any tool that matches the line exactly — and the doc has a stray character. It is the kind of mismatch that only surfaces when someone builds a marker parser from the documentation instead of from the code.

`MergeResult` carries the text and a conflict *count*, and `0` means clean. The count is what the three `commands.rs` call sites use to decide whether to tell the user the merge needs attention.

The token reconciler

`reconcile_conflict` is the module's original contribution, and it is marked as such:

☆ zmax original — beyond GNU Emacs `smerge`, `git merge`, VS Code, Neovim, Sublime, Jet-Brains, Zed and Helix, all of which conflict at *line* (or at best *hunk*) granularity: attempt to auto-resolve a diff3 conflict at *token* granularity.

The motivating case is the one every developer has hit: two branches touch the same line, one renaming an identifier and the other adjusting a comment, and git hands you a conflict for edits that do not actually interact.

The algorithm tokenises base, ours and theirs; computes each side's edits against base as `(start, end, replacement)` spans; concatenates and sorts them; dedups; then replays them onto the base token stream, bailing out on two conditions. First, overlap: if a span starts before the previous span ended, the two sides touched the same tokens and there is a genuine conflict. Second, co-located insertion: two zero-width edits at the same base offset with different replacements are ambiguous in ordering, so the function refuses rather than picking one.

The dedup deserves a note. `edits.sort_by_key(|x| (x.0, x.1))` sorts on span only, then `edits.dedup()` removes adjacent *fully* equal elements. So two sides making the identical edit collapse to one — the intended behaviour — while two sides making different edits to the same span sort adjacently, survive the dedup, and are then caught by the overlap check. The two-step is load-bearing; neither step alone gets both cases.

The guarantee at the end of the doc is the important sentence: “never a silent wrong merge.” The function returns `Option<String>` and `None` means “I could not do this safely,” which the caller is expected to turn into a normal conflict. A token-granularity auto-merger that guessed would be worse than useless; one that refuses cleanly is a strict improvement over conflicting always.

It has four tests and no callers. The idea is sound and implemented; the wiring is the missing piece.

Chapter 109 — cc-mode Without a Parser

`zmax-core/src/cmode.rs` ports the parts of GNU Emacs' `cc-cmds.el` that can be done on lines and character indices, and it is upfront about the boundary:

where a construct's detection is deliberately restricted to the common forms (e.g. statement motion does not parse string/comment contents) that is called out on the relevant function.

Thirty-four `#[test]` functions cover it. Four distinct C-editing subsystems live in the file: preprocessor conditional motion, dead-branch analysis, the `cwarn-mode` lint checks, and macro/comment/backslash formatting.

Depth counting, in both directions

The conditional motion commands (`C-c C-n`, `C-c C-p`, `C-c C-u`) are ports of the depth-counting core of `c-scan-conditionals`. `classify_directive` reduces a line to one of four values by trimming leading whitespace, requiring a `#`, trimming again, and reading the alphabetic run:

```
"if" | "ifdef" | "ifndef"      -> If
"elif" | "elifdef" | "elifndef" | "else" -> Else
"endif"                        -> Endif
```

The two-stage trim is what accepts `# ifdef FOO`, which is legal C and appears in real headers. `elifdef` and `elifndef` are the C23 spellings; their presence is a small signal that the port was written against a current directive list rather than a remembered one.

`forward_conditional` starts a depth counter at zero and returns the first `#endif` that drives it to zero or below. Because the counter starts at zero rather than one, a single scan handles both cases the doc names: an `#endif` closing a block that opened after point (depth goes 1, then 0) and an `#endif` closing the block point is already inside (depth goes straight to -1). One loop, two behaviours, no positional special-case. `backward_conditional` is the exact mirror with the roles of `#if` and `#endif` swapped.

Deciding which branches are dead

`conditional_branches` is the engine behind `hide-ifdef-mode` and `cpp-highlight-buffer`, and it contains the port's most carefully reasoned divergence. A `Branch` records its body line span, its condition as written, and `taken: Option<bool>` — where `None` means “cannot be decided from the file alone.”

That tri-state is the divergence. The doc explains:

GNU Emacs' `hide-ifdef-mode` instead evaluates against `hide-ifdef-env` and hides every branch that is not true, so with the default (empty) env it hides the body of every `#ifdef`. `zmax` has no `hide-ifdef-define` to populate such an env, so hiding on “cannot tell” would blank out most of a real C file. The file's own `#defines` are used instead, and undecidable branches stay visible.

This is the right reasoning written down at the point of decision. Emacs' design assumes the user will populate an environment interactively; `zmax` has no command to do that, so inheriting Emacs' “unknown means false”

rule would make the feature actively destructive. Changing the semantics is the smaller error than shipping a feature that hides working code.

The definition environment is the file itself, and `defined_macros` returns `Vec<(usize, String)>` — line number and macro name. The line number is not decoration:

The *line* matters: a macro counts as defined only for the directives that follow its `#define`, exactly as the preprocessor sees it. Ignoring the order would break the include-guard idiom — `#ifndef FOO_H` immediately followed by `#define FOO_H` would evaluate false and hide the whole header.

`is_defined` enforces this with `*line < before`. Trace the include guard: the `#ifndef FOO_H` is at line 0, the `#define FOO_H` at line 1, and at the moment the `#ifndef` is evaluated `1 < 0` is false, so `FOO_H` is not defined, so `ifndef` yields true, so the header body is live. Without the line comparison the guard would evaluate false and the entire header would be marked dead — which is the single most common shape in C and would have made the feature unusable on its first real file.

`eval_condition` is deliberately narrow. It decides exactly four shapes:

- `#ifdef X / #ifndef X / #elifdef / #elifndef` — a definedness query;
- a bare integer literal, so `#if 0` and `#if 1` work;
- `defined(X)` or `defined X`;
- a single leading `!` negating the above.

Anything else — arithmetic, comparisons, a macro's *value* rather than its existence — is `None`, and `None` is never hidden. The name check (`chars().all(|c| c.is_alphanumeric() || c == '_')`) rejects anything that is not a plain identifier, so `defined(A) && defined(B)` falls through to `None` rather than being misread as a macro named `A) && defined(B)`.

There is one behaviour the doc does not mention: `defined_macros` scans every line, including lines inside branches that are themselves dead. A `#define FOO` sitting inside a `#if 0` block still registers as a definition. The line-order rule is preserved but the *reachability* rule is not, so a file that defines a macro only in an unreachable branch will have later `#ifdef FOO` blocks marked live. Getting this right requires evaluating the file top to bottom with a live definition set rather than pre-scanning, which is a different and larger design; the pre-scan is the reasonable ninety-percent implementation, and the gap is worth knowing rather than worth fixing blindly.

cwarn: two checks out of three

`cwarn-mode` highlights suspicious C constructs. The port implements two of the three and says which one it dropped and why:

`cwarn.el`'s third check — a `&` in a C++ function call, warning that the argument is passed by reference — is not ported: it needs the callee's declaration, which a line scan does not have.

That is a real reason, not an excuse. Detecting pass-by-reference at a call site requires knowing the callee's signature, which requires either a parser with a symbol table or a language server. Neither is available to a `&str` scanner.

The two implemented checks share one scan. For each of three keywords — `if`, `while`, `for` — the scanner finds each occurrence, rejects it if the preceding byte is a word character (so `elif` does not match `if` and `notwhile` does not match `while`), skips whitespace, requires a `(`, and finds the matching `)` with a depth counter.

Inside that condition, `lone_assignment` looks for a bare `=`. The exclusion list is the interesting part: it skips `=` when the next byte is `=` (so `==` is not flagged) and when the previous byte is any of `= ! < > + - * / % & | ^`. That covers `!=`, `<=`, `>=`, every compound assignment, and — because it looks one byte back — the `= of` `<=<` and `>=>` via the `<` and `>` entries. The doc records a deliberate faithfulness decision here:

A compound assignment in a condition is just as suspicious, but `cwarn.el` only flags plain `=`, so this does too.

The port declines to improve on the reference implementation and says so. That is the opposite decision from `diff-split-hunk` in Chapter 107, which does improve on it, and the difference is instructive: `diff-split-hunk`'s Emacs behaviour produces a *wrong patch* that the user must repair by hand, so correcting it is unambiguously better; `cwarn`'s narrower rule produces fewer highlights, which is a taste question, and taste questions go to the reference.

The `for` keyword is excluded from the assignment check, because `for (i = 0; ...)` is the idiom rather than the bug. It is not excluded from the second check: `for (...);` is flagged as an empty body just like `if (...);`.

Both checks operate on raw line text with no string or comment awareness, which the module header declares. A `printf("if (x = 1);")` will produce two spurious warnings. For a highlighting mode whose output is a face rather than a build failure, that is an acceptable false-positive rate.

Macros, comments, and backslashes

The last group is formatting. `in_cpp_macro` decides whether a line is inside a multi-line `#define` body, by walking backward over the unbroken chain of backslash-continued lines feeding into it and checking whether the chain's first line is a directive. `c-context-line-break` uses this to decide whether a newly opened line needs a trailing `\`.

`comment_continuation_prefix`, `strip_comment_markers` and `fill_c_comment` handle refilling a block comment at a fill column while preserving its leading `*` column. `backslash_column`, `align_backslashes` and `remove_backslashes` implement `c-backslash-region` — line up every continuation backslash in a macro at a common column, or strip them.

`related_file_names` is `ff-find-other-file`: given a filename, produce the candidate names of its counterpart, driven by a static extension table (`OTHER_FILE_EXTS`) mapping each extension to the list of extensions to try. Returning candidate *names* rather than resolving a path keeps the function pure and leaves the existence check to the command layer, which is the same substrate-versus-command split the whole file follows.

Chapter 110 – Four Rings and a Store

Four small modules in `zmax-term/src` implement Emacs' position-and-text memory: `emacs_kill.rs`, `emacs_mark.rs`, `emacs_register.rs`, and `emacs_bookmark.rs`. They are worth reading together because they make the same architectural bet four times, and each one writes it down.

Chapter 60 described what these four give a user: the chords, the gestures, and which other tradition each one parallels. This chapter is about what is behind them, and it contains one correction to the picture a reader would form from the gesture description alone ♦ Emacs scopes the mark ring per buffer, and this port does not.

The bet

Emacs scopes these structures per-buffer or per-frame. The kill ring is global; the mark ring is per-buffer; registers are per-frame; bookmarks are global and persistent. All four zmax ports use a process-global `Lazy<Mutex<...>>`, and each says so:

- kill ring: “zmax stores the latest kill in a register, which overwrites on every kill — so there is no *ring*. This module keeps a bounded ring.”
- mark ring: “Simplification: the ring is process-global rather than per-buffer (like the kill-ring port).”
- registers: “Process-global, like the kill/mark rings — a simplification of emacs's per-frame registers.”

The mark-ring module also states the safety property that makes the simplification survivable:

Positions are character offsets; the command layer clamps to the current document length before jumping, so a stale offset from another buffer lands safely rather than panicking.

That is the crux. A global ring of raw character offsets is only sane if every consumer clamps, because offset 8000 from a large buffer will be applied to a small one. Both `emacs_mark` and `emacs_register` push the clamping responsibility to `commands.rs` and document it rather than clamping internally — which they could not do, since they hold no reference to any document.

The user-visible consequence of going global is that popping the mark can move you to a position in the buffer you are in that was recorded while you were in a different buffer. In Emacs, `C-u C-SPC` walks your history in *this* buffer. In zmax it walks your history everywhere, projected onto this buffer. For a single file that is identical; across files it is a different feature that happens to share a keybinding.

The kill ring's three pieces of state

`emacs_kill.rs` is the largest of the four and the only one that has to model a *command sequence* rather than just data. Its struct carries the entries, a cycling index, the selection ranges the last yank left behind, and an `append_next` flag. The bound is `MAX_ENTRIES: usize = 60`.

The hard problem is `yank-pop`. In Emacs, `M-y` is only valid immediately after a yank, and Emacs knows this by checking `last-command`. zmax's compositor has no equivalent, so the port substitutes a different signal, stated in the header:

Cycling: after `yank-from-kill-ring` selects the inserted text, the selection is remembered; `yank-pop` only fires while that exact selection is still in place (our stand-in for emacs's “last command was a yank” check), replacing it with the next-older entry and re-remembering.

Using the live selection as the freshness token is a genuinely good substitution for `last-command`, because it fails in the right direction. Any command that moves point or changes the selection invalidates the token, which is very nearly the set of commands that should invalidate a yank sequence. The one behavioural difference is that a command which leaves the selection exactly intact — a no-op, or a failed search — would let a subsequent `M-y` still fire, where Emacs would refuse. That is a strictly more permissive failure and not a destructive one.

The `append_next` flag models `append-next-kill` (`C-M-w`), and the port implements it as two entry points rather than one with a direction parameter: `record` appends the new text to the most recent entry, `record_prepend` prepends it. The reason is directional kills — killing backward should join the text on the correct side of what is already there. The flag is consumed by whichever fires first.

De-duplication is documented as matching Emacs: “Empty kills and exact-duplicate consecutive kills are ignored.” Only *consecutive* duplicates, so a ring can still hold the same text twice with something between.

The mark ring rotates rather than pops

`emacs_mark.rs` is the smallest and the most precisely specified. `MAX_MARKS: usize = 16`. `push` drops a consecutive duplicate by comparing against `m.first()` and returns early, then inserts at the front and truncates. Front insertion into a `Vec` is $O(n)$, which at sixteen elements is not worth avoiding.

`pop_to` is the one that repays reading:

the front mark is removed and `current` is pushed to the back, so repeated pops walk through the ring (and eventually back to where you started).

It takes the `current` position as an argument and returns the target. That signature is what makes the rotation work: the ring cannot know where you are, so the caller supplies it and the ring absorbs it at the tail. Repeatedly invoking `C-u C-SPC` therefore tours the ring and returns to the origin, which is Emacs’ behaviour and is the reason the mark ring is useful for navigation rather than just for undoing a mark.

Registers: one map, three value types

`emacs_register.rs` is a `HashMap<char, RegVal>` where `RegVal` is `Pos(usize) | Num(i64) | Rect(Vec<String>)` — point registers (`C-x r SPC`), number registers (`C-x r n`), and rectangle registers.

Every getter is type-checked at read time:

```
pub fn get_pos(ch: char) -> Option<usize> {
    match REGS.lock().unwrap().get(&ch) {
        Some(RegVal::Pos(p)) => Some(*p),
        _ => None,
    }
}
```

Reading a number register with `jump-to-register` yields `None` rather than a coerced value or an error. The register namespace is shared across the three kinds — storing a number in `a` overwrites a position in `a` — which matches Emacs, where registers are one namespace holding heterogeneous values.

`incr` implements `C-x r + (increment-register)` and returns the new value, so the command layer can echo it without a second lock acquisition.

Bookmarks: the one that persists

`emacs_bookmark.rs` is different in kind from the other three, and its header positions it precisely: “Named, persistent positions — the string-named, cross-session cousin of the char-keyed point registers.” Where

registers are a char to a position for this session, bookmarks are a string to a file, line and optional column, forever.

The split is the notable part. The store logic lives in `zmax_core::bookmark::BookmarkStore`, and this module is described as “the thin IO wrapper that loads/saves it at `<config-dir>/bookmarks`.” The format is one tab-separated record per line:

```
name<TAB>file<TAB>line[<TAB>column]
```

A line-oriented, tab-delimited text file, with the parser in the dependency-free core crate and the filesystem access in the terminal crate. That division means the format is testable without touching disk, and it means the format is readable and repairable by hand — which for a file that accumulates across years of use is worth more than a compact binary encoding.

`Load` returns an empty store on any error: “empty if the file is missing or unreadable.” A missing bookmarks file is the normal first-run state and a corrupt one should not prevent the editor from starting. The cost is that a permissions problem is indistinguishable from a fresh install, and the user’s bookmarks silently appear to be gone rather than producing a message.

Chapter 111 — Major-Mode Overlays: Making One Keymap Behave Like Many

Emacs' keymaps are a stack, and the layer that does the most work is the major mode's. `zmax-term/src/keymap/major_mode.rs` ports that layer, and its header opens with the problem rather than the solution:

Emacs reuses the same chords in every major mode and disambiguates them by the buffer's mode: `C-c C-t` is `org-todo` in Org, `sgml-tag` in HTML, `c-toggle-hungry-state` territory in C, ... A single global keymap cannot hold them all — they collide with each other and with the global map.

Two kinds of mode

The port's first move is to establish that a major mode and a language are usually the same thing in `zmax`:

the Emacs `M-x <lang>-mode` commands are ported as exactly that: `org-mode`, `latex-mode`, `sgml-mode`, `html-mode`, `fortran-mode`, `f90-mode` all set the current document's language. So a language-scoped overlay is not an approximation of Emacs's major-mode map; it is the same mechanism under `zmax`'s name for it.

That is a real argument, not a rationalisation, and it is followed immediately by the case where it fails:

Emacs's major modes are a *superset* of `zmax`'s languages, though. `outline-mode`, `text-mode`, `enriched-mode`, `view-mode` and `nroff-mode` all have keymaps, but they name no grammar and have no `languages.toml` entry, so a document's language can never be `outline` and those maps could never be reached.

The resolution is an explicit `major_mode: Option<String>` on `Document`, consulted in preference to the language. The overlay key is a string either way — a language name for a file mode, a bare mode name for a language-less one — so the lookup path does not branch. Counting the distinct first-field values in the binding table confirms both kinds are present: seventeen mode names, of which `c`, `cpp`, `elisp`, `commonlisp`, `scheme`, `fortran`, `html`, `xml`, `latex`, `org` and `mail` are languages, and `text`, `outline`, `enriched`, `view`, `nroff` and `cmdline-window` are the language-less modes the design paragraph was written to accommodate.

The table

`MAJOR_MODE_KEYS` is a flat `&[(&str, &str, &str, &str, &str)]` — languages, modes, chord, label, command. The first field is space-separated, so one row can bind a chord across several modes; the second is a subset of the characters `n`, `s`, `i` for normal, select and insert.

Flat tuple tables are usually a smell, but here the shape is doing something: every row is uniform, so a single test can validate every row, and the module has exactly that test. `every_major_mode_chord_resolves` checks four properties per row, and the doc comment above it explains why each is needed:

Every row must name a real command, parse as a chord, and still resolve to a leaf inside its own overlay — a typo'd command name compiles (these are strings resolved at runtime) and a chord sitting on another chord's prefix key silently swallows it.

Both hazards are real and neither is caught by the compiler. Command names are strings resolved through `MappableCommand`'s `FromStr`, so a typo is a runtime lookup failure at the moment the user presses the key. And a binding on `C-c t` makes `C-c t x` unreachable, silently, because the trie resolves to a leaf and stops. Testing that every chord still resolves to a leaf *after* the whole overlay is built catches shadowing that inspecting rows individually cannot.

The two guard rails

The overlay is consulted before the base keymap, which is what makes a major-mode chord shadow a global one. That is also what makes it dangerous, and the header names two constraints that keep it from breaking the modal presets.

The first is about prefixes:

A major-mode prefix never opens on a base leaf. In the `vim` preset `C-c` is `normal_mode` (escape); in `helix` it is `toggle_comments`. The overlay only turns `C-c` into a prefix where the base map already has a prefix there (the shipped `spacemacs` preset, and `emacs`). Enforced at lookup time, tested by `vim_preset_keeps_ctrl_c_escape`.

The named test exists in the module's test block, which matters because a doc comment naming a test is not evidence the test exists — this one is real. The rule itself is the minimal thing that preserves modal muscle memory: if the base map has already committed `C-c` to a command, the overlay is not allowed to turn it into a prefix, because doing so would make `C-c` wait for a second key instead of escaping. A vim user's escape key is not negotiable and this is the rule that protects it.

The second is about typing:

Each row names the modes it is live in (`normal` / `select` / `insert`). Bare keys that mean something while you type keep their base meaning in Insert (Org's `TAB` does not swallow indentation).

Org mode binds `TAB` to cycle visibility. If that binding were live in insert mode, typing in an Org buffer would cycle headings instead of indenting. Rather than a global “no bare keys in insert” rule, the table lets each row declare its live modes, so `TAB` can be Org's in normal mode and the editor's in insert mode — which is what Emacs achieves through the fact that it has no insert mode and `zmax` has to achieve deliberately.

Building the lookup

`overlay(language, mode)` is a two-level map lookup: mode to language to `KeyTrie`. The tries are built once, lazily, by iterating the table and, for each row, for each live mode, for each space-separated language, entering the chord into that trie via `add_chord`.

The nesting order — mode outermost, language inner — matches the query. Lookup happens on every keystroke with a known mode and a known language, and the structure means a keystroke in a mode with no overlays fails at the first `get` with no allocation and no scan of the table. The table is read once at startup and never again.

Chapter 112 – The Word Index: Debouncing an Eventually-Consistent Set

`zmax-view/src/handlers/word_index.rs` maintains “an eventually consistent set of words used in any open buffers” for lexical completion. It is the module in the tree that has thought hardest about doing expensive work without blocking the editor, and it is worth reading as a piece of concurrency design independent of what it indexes.

Reference counting instead of per-document sets

The storage is one map:

```
struct WordIndexInner {  
    words: HashMap<Word, u32>,  
}
```

Word to occurrence count, where `Word` is a `kstring::KString` — a small-string type that inlines short strings rather than heap-allocating them, which for a corpus of identifiers is most of them. The comment states the reasoning:

Words are very likely to be reused many times. Instead of storing duplicates we keep a reference count of times a word is used. When the reference count drops to zero the word is removed from the index.

The alternative — a set per document, unioned at query time — would need a scan per completion and would duplicate every common identifier once per open file. The refcount collapses all documents into one map at the cost of never knowing *which* document a word came from, which the completion feature does not need.

`insert` uses `saturating_add`; `remove` matches on `Some(1)` to remove and `Some(n)` to decrement. The asymmetry is real but unreachable in practice: a word would need `u32::MAX` occurrences to saturate, after which the count could never return to zero and the word would be permanently resident.

Three debounce policies in one function

The hook implements `AsyncHook`, whose `handle_event` returns an optional deadline. The policy has three branches and each is a different bet.

If a change is already pending for this document, coalesce and extend the deadline. If not, and the change is *insignificant*, store it and set a deadline one second out. If not, and the change is significant, forward it immediately and leave the existing deadline alone.

```
const DEBOUNCE: Duration = Duration::from_secs(1);
```

The significance test is what makes this work. Typing produces a stream of tiny changesets, and debouncing them is pure win — a burst of keystrokes should cost one index update, not thirty. But a paste, a multi-cursor

operation, or a formatter run produces one large changeset, and delaying that by a second means completion is stale for a second on exactly the edit that changed the most words. So large changes skip the queue.

The coalescing hazard, and how it is handled

The subtle part is what happens when two changes arrive for the same document inside one debounce window. The obvious move is to compose the two changesets. The module refuses to, and the comment explains at length:

```
We deliberately do NOT chain the observed changesets here. The index skips ghost transactions (see register_hooks), so a ghost edit may have altered the document between these two observed changes, which may leave them non-contiguous in ways a cheap check can't catch. Instead, mark the change dirty and recompute the changeset from the real ropes at finish_debounce, i.e. once per debounce window rather than once per keystroke.
```

This is a correctness argument with a performance conclusion attached. Because the index deliberately ignores some transactions, the changesets it *does* see are not guaranteed to compose — there may be an unobserved edit between them, so chaining them would produce a changeset describing a document state that never existed. The `Change` struct therefore carries a `dirty` flag and holds `old_text` and `text` as full ropes, and `finish_debounce` recomputes:

```
change.changes = compare_ropes(&change.old_text, &change.text)
    .changes().clone();
```

A fresh diff between the real endpoints, valid regardless of what happened in between. The cost is one rope diff per document per debounce window; the benefit is that the index cannot drift from the buffer through an unobservable path.

Holding both ropes is cheap because ropery ropes are structurally shared — `old_text` and `text` share their unchanged spans — so the memory cost is proportional to the edit, not the document.

The delete path handles the same race in the other direction. If a document closes with a pending change, the pending change's `old_text` is what the index currently reflects, so that is what must be removed — not the document's final text, which was never indexed. Two lines of code, and getting it backwards would leak every word of every edited-then-closed buffer into the index forever.

Cancellation at a coarse granularity

Indexing runs on a blocking task with a `TaskHandle` that can be cancelled, and the cancellation checks are counted rather than timed:

```
const CANCEL_CHECK_INTERVAL: usize = 4096;
```

`add_document` and `remove_document` check `i % CANCEL_CHECK_INTERVAL == 0` on the word index. `update_document` cannot, because it iterates changed *windows* and one window can be the whole document, so it counts words processed across windows instead — the comment says exactly that: “A single changed window can span the whole document, so the check is driven by a count of the words processed rather than the number of windows.”

Getting that right required noticing that the outer loop's iteration count is not proportional to the work. It is the kind of detail that only shows up as a symptom — an unresponsive editor when one paste replaces an entire file — and only if you go looking for why.

The consequence of cancellation is that all three functions `return` from the middle of a mutation while holding the write lock, leaving the index partially updated. For `add_document` that means some of a document's words are counted; for `remove_document`, some are decremented. Since the structure is a bag of refcounts with no per-document ledger, there is no way to reconcile afterwards. The module's title says “eventually

consistent,” and the honest reading is that cancellation makes it eventually *approximate* — a few stale or missing completion candidates, which is a cost the feature can absorb and a stronger guarantee could not be had without a per-document word list, which is the design the refcount exists to avoid.

The word filters are two constants: `MIN_WORD_GRAPHemes: usize = 3` and `MAX_WORD_LEN: usize = 50`. Three graphemes rather than three chars, so a word made of combining sequences is measured as a reader sees it; fifty chars as a cap on base64 blobs and minified lines.

The sort that goes the wrong way

The query path is four lines:

```
pub fn matches(&self, pattern: &str) -> Vec<String> {
    let inner = self.inner.read();
    let mut matches = fuzzy_match(pattern, inner.words(), false);
    matches.sort_unstable_by_key(|(_, score)| *score);
    matches.into_iter().map(|(word, _)| word.to_string())
        .collect()
}
```

`fuzzy_match` wraps `nucleo` and returns `Vec<(T, u16)>` of items and scores. Grepping the non-vendor tree for `fuzzy_match` finds it defined once in `zmax-core/src/fuzzy.rs` and called from `zmax-term/src/ui/mod.rs`, `zmax-term/src/commands.rs`, `zmax-term/src/commands/typed.rs`, and here. There are nineteen call sites in `zmax-term`. Scanning the lines following every one of them for a `sort`, a `reverse`, or a `max_by` turns up exactly one hit — the line below. Every other consumer deconstructs the result as `|(name, _)|` and preserves the order the matcher returned. This is the only place in the tree that re-sorts.

And it sorts ascending on the score. `sort_unstable_by_key` with the score as the key orders lowest first. If `nucleo`’s score is a quality metric where higher is better — which is the convention that makes the other nineteen call sites correct as written — then the word-index completion list is ordered worst match first.

Either the re-sort is inverted, or it is a leftover from an older matcher whose score was a distance rather than a quality. Either way it is the sole divergence from a convention every other consumer follows, and it is one `Reverse(...)` away from matching them.

Chapter 113 – Amusements, Corpora, and a Module That Cannot Compile

Two things in this tree look like jokes and are not: the ported Emacs amusements, and the platform-fallback module in `zmax-stdx`. Both reward a closer look, for opposite reasons.

The amusements are a fidelity argument

Emacs ships `doctor`, `dunnet`, `dissociated-press`, `spook`, `hanoi`, `life`, `gomoku`, `blackbox`, `5x5`, `mpuz`, and more. They are not there because anyone needs them; they are there because Emacs' distribution model was “the whole thing, including the parts you will never use.” Porting them is a claim about what kind of port this is.

The `ui/` directory carries 120 modules, and reading the list makes the scope plain: alongside `picker`, `prompt`, `magit`, `dired` and `lsp` sit `asteroids`, `battleship`, `blackbox`, `blackjack`, `bombberman`, `breakout`, `centipede`, `chess`, `connectfour`, `digdug`, `donkeykong`, `dunnet`, `fifteen`, `fivex5`, `flappy`, `frogger`, `galaga`, `gomoku`, `hangman`, `hanoi`, `invaders`, `klondike`, `lunarlander`, `mancala`, `mastermind`, `minesweeper`, `missilecommand`, `mpuz`, `nonogram`, `pacman`, `pong`, `reversi`, `simon`, `snake`, `sokoban`, `solitaire`, `sudoku`, `tetris`, `tictactoe`, `tron`, `videopoker` and `yahtzee`. Several of those have no Emacs ancestor at all; they are the terminal-game tradition rather than the Emacs one.

What makes the group technically interesting is that three of them are *data* ports rather than *code* ports, and the data was extracted rather than retyped.

Dunnet: the world, dumped from Emacs

`zmax-term/src/ui/dunnet_data.rs` carries the world of Emacs' text adventure, and its header states the extraction method:

```
transcribed faithfully from GNU Emacs 30.2 lisp/play/dunnet.el by loading it in Emacs
batch and dumping the resolved constants (so obj-* symbols are real indices). Generated
by scripts/gen_dunnet_data.py; do not edit by hand.
```

The generator script exists in the repository, and its docstring makes the same point from the other side:

```
Parse the Emacs-dumped dunnet data (canonical Lisp from prin1) into a Rust data module. The
data was resolved by Emacs itself (obj-* constants -> numbers), so this is a faithful transcription of
dunnet.el's world, not a reconstruction.
```

The distinction matters. `dunnet.el` defines its object constants as symbols and uses them throughout; a human transcribing the tables would have to resolve every `obj-shovel` to an index by hand, and would get some of them wrong. Having Emacs evaluate the file and `prin1` the resolved values makes the reference implementation the source of truth for the data, not the transcriber's reading of it.

The dimensions line up as a world should. `ROOMS` has 105 entries, each a (description, short name) pair. `MAP` has 105 rows of exactly 12 entries — one row per room, one column per direction, `i16` because a negative value encodes something other than a destination. `ROOM_OBJECTS` has 105 entries, the initial contents of each room. `OBJECTS` has 28 entries, and both `OBJ_LBS` (weights) and `OBJ_PTS` (score values) have exactly 28

elements, so every object has a weight and a point value with no ragged edges. `OBJNAMES` has 88 entries mapping a typed noun to an object index — more names than objects, because a text adventure has to accept synonyms.

The `ROOM_OBJECTS` doc records the encoding: “Negative = a fixed scenery object (see `PERM_OBJECTS` by absolute value); 255 = a special.” Sign as a discriminant and a sentinel for the rest — a 1983 Lisp encoding preserved rather than modernised, which is the correct choice when the game logic that consumes it is also being ported.

Spook: a corpus with a parsing rule

`ui/spook_data.rs` holds the NSA line-eater keyword list, and its header specifies not just the source but the parse:

the NSA line-eater keyword corpus from GNU Emacs 30.2 `etc/spook.lines`, parsed with the faithful `cookie-snarf` rule (discard the header before the first NUL, split remaining entries on NUL, strip surrounding whitespace).

`etc/spook.lines` is a NUL-delimited file with a header, and Emacs’ `cookie.el` reads it with `cookie-snarf`. Naming the rule rather than the outcome is what makes the extraction reproducible.

The corpus has 1227 phrases, and this is one of the few numbers in the tree that is pinned by a test. In `commands.rs`:

```
fn spook_corpus_loaded() {
    let p = crate::ui::spook_data::PHRASES;
    assert_eq!(p.len(), 1227);
    assert_eq!(p[0], "$400 million in gold bullion");
    assert!(p.iter().all(|s| !s.is_empty()));
}
```

Length, first element, and no-empties. The comment above it says why the test exists: “spook_data must parse to the full `etc/spook.lines` corpus.” A generated data file with no assertion about its size is a file that can silently regenerate to a fraction of itself.

The `spook` command itself is a small faithfulness exercise. It shuffles indices with a Fisher–Yates prefix — `j = i + fastrand::usize(0..(n - i))`, swap, for the first `count` positions only — because it needs 15 distinct phrases out of 1227 and shuffling the whole vector to take a prefix would be wasteful. The comment claims this “matches cookie-shuffle-vector’s algorithm,” and the count is `15.min(n.saturating_sub(1))`, described as Emacs’ `spook-phrases-default-count`.

The chosen phrases are then filled into a paragraph by `fill_words` at width 70, a greedy word-wrap the doc positions as emulating “emacs `fill-region-as-paragraph` at the default `fill-column` (70).” And the command sets the status line to `Checking authorization...Approved`, which is what Emacs prints. Porting the joke as well as the algorithm is the point.

Doctor: ELIZA in one pure function

`ui/doctor.rs` is the ELIZA port, and its structure is the reason it can be tested at all. The response engine is one pure function:

```
pub fn respond(input: &str, turn: usize) -> String
```

No state, no `self`, no random number generator. The `turn` parameter is how variety is achieved without either: it indexes into a fallback list modulo its length, so consecutive non-keyword remarks get different replies deterministically. Six fallbacks, cycled.

`reflect` is the pronoun swapper — eleven mappings, first person to second and back, so `I am sad` becomes `you are sad` and can be echoed as `How long have you been sad?`. The keyword rules are a fixed cascade: four `strip`-based patterns that capture and reflect a remainder (`i need`, `i am/i'm`, `i want`, `i feel`), then substring triggers (`because`, family words, `dream`, `computer/emacs`, `sorry`), then prefix triggers (`yes`, `no`), then a question-mark catch-all, then the fallbacks.

The order is the design. `strip` uses `lower.find(kw)` rather than `starts_with`, so `I think I need help` matches the `i need` rule and reflects only the tail. Placing the capture rules before the substring rules means `I feel sad because of work` produces `Tell me more about feeling sad because of work`. rather than `Is that the real reason?` — the more specific response wins because it is tested first. That is the whole of ELIZA's craft and the port preserves it.

Including `emacs` in the same trigger as `computer` — producing “Do machines worry you?” — is the port editorialising by one word.

Dissociated press, and a reproducible RNG

`ui/dissociate.rs` ports the travesty generator, and its header describes the algorithm precisely enough to reimplement from:

copy a short run of source text to the output, take the last `arg` characters as an “overlap”, jump to a random point in the source, search forward (wrapping) for that overlap, and continue copying from just after it.

The port's divergence is in presentation, not algorithm:

Emacs redisplay the `*Dissociation*` buffer as text is added and asks whether to continue every screenful; this overlay generates a bounded block up front (pure, unit-tested) and reveals it progressively, closing on any key.

Generating up front and revealing progressively is what buys the unit test. `dissociate(source, arg, max_chars, rng)` takes the RNG state by mutable reference and is deterministic given a seed, so a test can assert on exact output. Emacs' version, which interleaves generation with redisplay and user prompts, cannot be tested that way at all.

The generator is an inlined LCG:

```
*state = state
    .wrapping_mul(6364136223846793005)
    .wrapping_add(1442695040888963407);
*state >> 33
```

Those are Knuth's MMIX constants, and the `>> 33` discards the low bits, which in a linear congruential generator have short periods. This is the right amount of random number generator for scrambling prose: reproducible, dependency-free, and no pretence of quality it does not need. `OUTPUT_CHARS: usize = 3000` bounds one run.

`search_forward` is written to match Emacs' semantics rather than Rust's — it returns the index *just past* the match, like `search-forward` leaves point after the match rather than before it. Matching the reference function's convention, including its off-by-one, is what keeps the port's arithmetic identical to the original's.

A module that cannot compile

`zmax-stdx/src/faccess.rs` wraps file access checks — vendored from an external crate, as its header says — and splits into three `cfg`-gated `imp` modules: one for `unix`, one for `windows`, and one for `not(any(unix, windows))`.

The Unix implementation is the interesting one. It builds a `rustix` access mask from the `AccessMode` bitflags, and on Linux it short-circuits first:

```
if capability_is_in_ambient_set(CapabilitySet::DAC_OVERRIDE)
    .unwrap_or(false) {
    return Ok(());
}
```

With `CAP_DAC_OVERRIDE` in the ambient set, permission bits do not apply, so reporting a file read-only because its mode says so would be wrong. Checking a capability before checking permissions is a detail most access wrappers omit.

The fallback module is a different story. It is what compiles on a target that is neither Unix nor Windows, and it contains this signature:

```
pub fn copy_metadata(from: &path, to: &Path) -> io::Result<>
```

Lowercase `path`. There is no type named `path` in scope; the type is `std::path::Path`, which is imported and used correctly two characters later in the same signature. That module does not compile.

It has a second defect. The top-level wrappers are:

```
pub fn readonly(p: &Path) -> bool { imp::access(...) }
pub fn copy_metadata(from, to) { imp::copy_metadata(from, to) }
pub fn hardlink_count(p: &Path) { imp::hardlink_count(p) }
```

Three functions delegating to `imp`. The Unix module defines all three. The Windows module defines all three. The fallback module defines `access` and `copy_metadata` and stops — there is no `hardlink_count`. So even after fixing the lowercase `path`, the fallback build fails on a missing function.

Neither defect is a bug in the shipped editor, because `zmax` targets Unix and Windows and the fallback module is never compiled. That is exactly why both survived: `cfg`-gated code that no CI target selects is not type-checked, not linted, and not read. The lesson is not that this code should be fixed — it may never matter — but that a `cfg` gate on a module is also a gate on the compiler, and any branch no target selects should be assumed broken until something selects it. A `cargo check --target` on one exotic platform, or moving the fallback behind a feature flag that CI enables, converts an assumption into a fact. The file also carries zero `#[test]` functions, which is consistent: it is the least-exercised file in the crate, and it shows.

Chapter 114 – Vim Display Options Inside a Grapheme Layer

`zmax-core/src/graphemes.rs` began life, as its header records, as a grapheme-cursor utility adapted from an external editor project. What has been added to it since is a complete implementation of vim's character-display options, and the placement is not obvious until you see why nothing else could own them.

Six options live here as thread-locals:

```
AMBIWIDTH_DOUBLE    bool
VARTABSTOP           Vec<usize>
BREAKAT              Vec<char>
EMOJI                bool
ISPRINT              Option<[bool; 256]>
DISPLAY_UHEX         bool
```

The comment explains both the choice of storage and the default:

Each is off/empty until `:set` opts in, so the default rendering is untouched. Thread-local, like the other `:set`-driven character tables (`chars::set_extra_keyword_chars`): the options are set and read on the editor thread.

Thread-locals are the unfashionable choice and the correct one here. These values are read inside the width calculation for every grapheme on every rendered line — the hottest loop in the editor. A `Mutex` would put a lock acquisition in that loop; an `RwLock` would put an atomic in it; passing them as parameters would thread a config struct through every function in the module and every caller of `grapheme_width`. A thread-local read compiles to a register- relative load, and the invariant that makes it sound is stated: set and read on the editor thread.

The `Option<[bool; 256]>` for `ISPRINT` is the sharpest of the six. `None` is not “empty table”; it is “the user never `:set isprint`,” which is semantically distinct from “the user set isprint to something restrictive.” The distinction is used: `unprintable_repr` returns `None` — meaning *print it normally* — when `ISPRINT` is `None` and `display=uhex` is off, so an untouched editor renders every character as itself and none of this machinery runs.

Parsing an option syntax from 1991

`parse_isprint` implements vim's `isprint` value syntax, which is shared with `isfname` and is the sort of thing that exists because it was convenient in C. A comma-separated list where each item may be:

- a literal character (`a`);
- a decimal code (`128`);
- a range of either (`a-b`, `128-140`);
- `@`, meaning every alphabetic character;
- any of the above prefixed with `^`, meaning exclude rather than include.

The port handles each in order, with the ambiguities resolved the way vim resolves them. A `code` helper accepts either a parsed integer below 256 or a single character below 256, so `65` and `A` both work and `AB` is rejected. The range branch requires both endpoints non-empty precisely so that a lone `-` falls through to the literal-character branch — a comment says so: “a lone `-` is the literal hyphen.”

The `^` prefix is handled by rewriting the item and flipping the value written into the table, rather than by a second code path, so exclusion works uniformly for characters, codes, ranges and `@`.

Then, unconditionally, after everything the user asked for:

```
for slot in table.iter_mut().take(127).skip(32) {
    *slot = true;
}
```

with the vim documentation quoted directly above it: “The characters from space (ASCII 32) to ‘~’ (ASCII 126) are always displayed directly, even when they are not included in ‘isprint’.” Printable ASCII is forced on last, so it overrides an explicit `^` exclusion. That is vim’s behaviour and it is the difference between an option and a footgun — no `isprint` value can make your source code invisible.

The unprintable table, reproduced exactly

`unprintable_repr` maps a character to how vim shows it when it is not printable. The doc carries vim’s table verbatim:

```
0 - 31    "^@" - "^_"    128 - 159  "~@" - "~_"
127       "^?"          160 - 254  "| " - "|~"
255       "??"          255       "~?"
```

and the implementation reproduces it with four arithmetic arms. Control characters add 64 to reach the caret range. 128–159 subtract 64, which maps 128 to `@` and 159 to `_`, giving the `~@~_` row. 160–254 subtract 128, mapping 160 to space and 254 to `~`, giving `|` through `|~`. 127 and 255 are named constants. Every boundary in the documented table is a boundary in the code, and the two agree — worth stating because this chapter has now found four places where a comment and its code did not.

The `display=uhex` path short-circuits all of it with `format!("{}", <code:02x>)"`, and it has one further subtlety. When `uhex` is on but `isprint` was never set, the function falls back to `default_isprint()`, which parses vim’s own default value `@,161-255`. Without that fallback, turning on `uhex` alone would do nothing, because with no `isprint` table every character is printable. The comment names it: “the table `display=uhex` falls back on when `isprint` itself was never `:set`.”

The guard `if code >= 256` returns `None` for everything above Latin-1, with the reason given: “vim only classifies the first 256 codepoints.” Emoji, CJK and combining marks are never rendered as `<xx>`, which is correct — vim’s unprintable machinery predates Unicode and applying it above U+00FF would be an extrapolation, not a port.

Variable tab stops

`varstabstop` replaces the fixed `tabstop` with a list of widths: the first tab stop is `stops[0]` columns in, the next `stops[1]` further, and the final width repeats forever. `varstab_width_at` walks the list accumulating positions, and its last-element branch is the one to read:

```
if i + 1 == stops.len() {
    let step = w;
    let past = visual_x - pos;
    let n = past / step + 1;
    return pos + n * step - visual_x;
}
```

Rather than looping until it passes `visual_x`, it computes how many repetitions of the final width are needed in one division. A cursor at column 10000 with a two-element stop list costs the same as a cursor at column 3.

`tab_width_at` is the entry point the renderer calls, and its structure is the zero-cost-when-unused pattern the whole module follows:

```

if let Some(width) = VARTABSTOP.with(|v| {
    let stops = v.borrow();
    (!stops.is_empty()).then(|| vartab_width_at(visual_x, &stops))
}) { return width; }
tab_width as usize - (visual_x % tab_width as usize)

```

One thread-local read and an emptiness check, then the ordinary modulo. The `.then(|| ...)` inside the `with` closure means the borrow is released before the function returns in either branch — a small discipline that matters because `RefCell` borrow panics are the failure mode of this pattern.

`breakat_allows` follows the same shape for soft-wrap break points, returning `Option<bool>` where `None` means “the option is empty, use the editor’s own rule” and `Some(b)` means “the option is set and this is the answer.” Three states in an `Option<bool>` is usually a smell; here the outer layer is *whether the user configured anything* and the inner is the answer, which is the one case where the type is honest.

The through-line across all six options is the same: the editor’s own rendering is the default, each vim option is opt-in, each is stored where the hot loop can read it without synchronisation, and the check for “is this option even on” is cheaper than the option’s behaviour. That is what lets a compatibility layer for a thirty-year-old option set sit inside the per-grapheme path without costing anything to the user who never types `:set`.